

Nasdanika SDK Runtime

More Structure with Less Effort

Models, capabilities and telemetry on the JDK

Pavel Vlasov

nasdanika.com



Nasdanika SDK Runtime

More Structure with Less Effort

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1
Preface . . . 2
- Who this book is for . . . 3
- What this book assumes . . . 4
- How this book is organized . . . 4
- The Nasdanika Zephyr Books . . . 5
- Conventions . . . 8
- Versions . . . 9
- Feedback . . . 9

Where the opinions come from . . . 10
Prior art, first-hand . . . 14
- Before EMF . . . 14
- Models of what already exists . . . 15
- Models of how work is done . . . 16
- The organization as a model . . . 17
- What carried over . . . 17

I: The Why . . . 19
- **1. More structure with less effort** . . . 20
  - 1.1. What structure is for . . . 20
  - 1.2. Structure has been expensive . . . 21
  - 1.3. What changed . . . 22
  - 1.4. Small, formal, composable . . . 23
  - 1.5. What the runtime is . . . 23
  - 1.6. What it is not . . . 24
  - 1.7. Who gets what . . . 25
  - 1.8. A short tour . . . 26
  - 1.9. Where this is going . . . 27
- **2. Principles** . . . 28
  - 2.1. Pure Nasdanika: the JDK, EMF core and the OpenTelemetry API . . . 28
  - 2.2. If the JDK can do it, use the JDK . . . 28
  - 2.3. If Xcore can model it, model it . . . 30
  - 2.4. Telemetry is part of the runtime . . . 30
  - 2.5. Services before capabilities . . . 31
  - 2.6. Declaration is registration . . . 31

2.7. Micro-models and nano-models . . . 32
2.8. One identity per module . . . 33
2.9. Pull, don't push . . . 33
2.10. Local first, federated, no server . . . 34
2.11. Say what is stuck . . . 34
2.12. Humans at both ends . . . 35
2.13. Lineage . . . 35
3. Built with an AI assistant . . . 37
3.1. Binding decisions . . . 37
3.2. Who does what . . . 38
3.3. One level up . . . 39
3.4. The mainstream filter . . . 40
3.5. When fluency is not scarce . . . 41
3.6. Fall back to the keep . . . 42
3.7. How the work flows . . . 43
3.8. Gradient ascent . . . 44
3.9. What the assistant does not do . . . 45
3.10. This book . . . 46
4. Against the grain . . . 48
4.1. Faster horses . . . 48
4.2. Owning the result . . . 48
4.3. Stories are not source code . . . 50
4.4. When a question becomes answerable . . . 51
4.5. Failing conventionally . . . 53
5. The runtime at a glance . . . 54
5.1. The modules . . . 54
5.2. Three planes . . . 58
5.3. How the pieces find each other . . . 63
5.4. One command, end to end . . . 63
5.5. Where things are in the repository . . . 65
5.6. What is available today . . . 66
II: Foundations . . . 67
6. Getting started . . . 68
6.1. What you need . . . 68
6.2. Building the runtime . . . 68
6.3. A first project . . . 69

6.4. A first model . . . 70
6.5. Watching it run . . . 73
6.6. The examples in this book . . . 75
7. Models: EMF without Eclipse . . . 77
7.1. Why EMF . . . 77
7.2. Why not Eclipse . . . 78
7.3. Ecore in one page . . . 78
7.4. A model in Xcore . . . 79
7.5. Generating with Maven . . . 81
7.6. Registering the model . . . 84
7.7. Resource sets configured from services . . . 85
7.8. Contributing to resource sets . . . 86
7.9. Formats . . . 87
7.10. Loading from modules . . . 88
7.11. One model, several files . . . 89
7.12. Operations . . . 89
7.13. What you have now . . . 90
8. Generic types at run time . . . 91
8.1. Matching and wiring . . . 91
8.2. What survives erasure . . . 92
8.3. Types . . . 92
8.4. Adaptation . . . 95
8.5. Asking before there is an instance . . . 98
8.6. Access . . . 98
8.7. Where this is used . . . 99
8.8. A trained eye . . . 99
8.9. The JVM, not Java . . . 99
9. Flows, pumps and transformations . . . 101
9.1. Flow, not Reactor . . . 101
9.2. Flows . . . 101
9.3. The pump . . . 102
9.4. The transformer . . . 105
9.5. Mapping methods . . . 108
9.6. One engine . . . 110
10. The capability framework . . . 111
10.1. Concepts . . . 111

10.2. Requirements . . . 112
10.3. Factories with capability methods . . . 113
10.4. Factories against the interface . . . 116
10.5. Registering and loading . . . 117
10.6. Composing sources and loaders . . . 118
10.7. When nothing comes back . . . 120
10.8. A case study: file names as pipelines . . . 121
10.9. When to use which . . . 124
III: Observability . . . **125**
11. Telemetry in the runtime . . . **126**
11.1. Two kinds of telemetry . . . 126
11.2. The conventions . . . 126
11.3. The Telemetry class . . . 127
11.4. Instrumenting your code . . . 128
11.5. Units of work . . . 130
11.6. What the runtime instruments . . . 131
11.7. Crossing threads and processes . . . 132
11.8. Where telemetry goes . . . 134
12. Watching work live . . . **135**
12.1. Publishing OpenTelemetry . . . 135
12.2. Span events . . . 136
12.3. A live tree . . . 137
12.4. Following one operation . . . 138
12.5. Testing with telemetry . . . 139
12.6. What comes next . . . 139
13. Telemetry as a model . . . **140**
13.1. Why keep telemetry as a model . . . 140
13.2. The telemetry model . . . 141
13.3. Exporting to the model . . . 141
13.4. Recording changes . . . 143
13.5. Reading telemetry back . . . 144
13.6. Recording a unit of work . . . 146
13.7. Formats and where files go . . . 147
13.8. Every signal is precious . . . 147
IV: Files and Serving . . . **148**
14. Volumes and content . . . **149**

14.1. Why not NIO . . . 149
14.2. The volume SPI . . . 149
14.3. Capabilities . . . 152
14.4. Volumes over directories . . . 153
14.5. Content . . . 154
14.6. Models in volumes . . . 156
14.7. Telemetry of transfers . . . 157
14.8. What comes next . . . 158
15. Serving over HTTP and WebDAV . . . 159
15.1. Serving a volume . . . 159
15.2. Reading over HTTP . . . 161
15.3. One trace across client and server . . . 161
15.4. A server for the length of a test . . . 163
15.5. What comes next . . . 163
V: The Command Line . . . 164
16. Commands as capabilities . . . 165
16.1. Picocli . . . 165
16.2. A first command . . . 166
16.3. How the tree is built . . . 168
16.4. Groups . . . 169
16.5. Mix-ins . . . 171
16.6. Overriding . . . 172
16.7. Commands in the runtime . . . 172
16.8. Commands and telemetry . . . 173
17. Wiring commands with generic types . . . 174
17.1. What a command needs from its parent . . . 174
17.2. ExecutionSupplier . . . 176
17.3. The runtime’s save command . . . 176
17.4. What a command accepts as sub-commands . . . 177
17.5. Why export hides its library . . . 179
17.6. When wiring happens . . . 179
17.7. Rules . . . 180
17.8. Why declarations . . . 180
18. Distributions and launching . . . 181
18.1. What a distribution is . . . 181
18.2. Module path and class path . . . 182

18.3. Starting from any directory . . . 182
18.4. A launch is a unit of work . . . 183
18.5. Building a distribution . . . 184
18.6. Running it . . . 185
VI: Models in the Runtime . . . 186
19. Authoring models with the Groovy DSL . . . 187
19.1. A library as a script . . . 187
19.2. The rules of the language . . . 189
19.3. Metamodels too . . . 190
19.4. Binding the DSL to a metamodel . . . 191
19.5. Scripts that derive models . . . 193
19.6. Errors and telemetry . . . 193
19.7. What comes next . . . 195
20. The core model . . . 196
20.1. The element hierarchy . . . 196
20.2. Identity and references . . . 197
20.3. Documentation . . . 198
20.4. Provenance . . . 200
20.5. Attributes . . . 200
20.6. Evaluators . . . 201
20.7. Why a shared core matters . . . 202
21. Meta: a language-neutral type system . . . 203
21.1. What it models . . . 203
21.2. What it leaves out . . . 204
21.3. Documentation as structure . . . 204
21.4. A vocabulary . . . 205
21.5. The text syntax . . . 206
21.6. Reading and writing text . . . 210
21.7. Projections . . . 212
21.8. Sources and targets . . . 215
21.9. AI-assisted domain design . . . 216
21.10. Instances whose type is a reference . . . 217
21.11. Federated by design . . . 218
22. The JVM model . . . 219
22.1. What your assembly became . . . 219
22.2. Identity . . . 220

22.3. What it models 220
22.4. A snapshot of a running JVM 221
22.5. Loading 224
22.6. Applications 225
23. From models to the web 226
23.1. Pages are models too 226
23.2. A catalog site 227
23.3. Navigation 229
23.4. Applications without npm 229
23.5. Static or served 234
23.6. The TypeScript and React models 235
23.7. Walking through an application 236
23.8. JavaScript as another bytecode 237
23.9. What comes next 238
24. Polymorphic UI 239
24.1. Where the idea comes from 239
24.2. Dispatch on the class of an element 240
24.3. Views are capabilities 241
24.4. The resolver 242
24.5. Views of the library 243
24.6. The reflective view of last resort 245
24.7. Contributing views 246
24.8. Putting it together 247
24.9. Mix-ins and sections 248
24.10. Static pages, and the browser 249
24.11. What comes next 249
25. Diagrams 250
25.1. A continuum, not a choice 250
25.2. What a good diagram is worth 251
25.3. Two layers, seven modules 251
25.4. A diagram generated from a model 252
25.5. Read back, and checked 254
25.6. Drawn as SVG 256
25.7. The Zephyr Books 258
25.8. What comes next 258
VII: Looking Ahead 260

26. Roadmap and vision **261**
26.1. The North Star 261
26.2. Three planes 262
26.3. Next: browsing and serving 263
26.4. Next: more volumes 264
26.5. Next: the command line 264
26.6. The agentic increments 265
26.7. Humans at both ends of the line 266
26.8. An architecture method on Git 267
26.9. The model tower 267
26.10. Following along 267
Appendix A: Module reference **268**
A.1. Runtime modules 268
A.2. Models 271
A.3. Book examples 274
Appendix B: Telemetry reference **275**
B.1. Spans 275
B.2. Attributes 276
B.3. Events 277
B.4. Configuration 278
Appendix C: Rules of thumb for people and assistants **279**
C.1. Foundations 279
C.2. Models 279
C.3. Types, flows and capabilities 280
C.4. Telemetry 281
C.5. Files and serving 281
C.6. Commands 282
C.7. User interfaces 282
C.8. The Groovy DSL 283
C.9. Working with an assistant 283
C.10. For assistants 283
Appendix D: Glossary **285**
Appendix E: Building and publishing this book **293**
E.1. Layout 293
E.2. Building 293
E.3. Conventions of the manuscript 294

E.4. The cover . . . 294
E.5. Publishing . . . 295
E.6. Building a Zephyr Book . . . 295
Appendix F: The Nasdanika Body of Knowledge . . . 296
F.1. What it holds . . . 296
F.2. How it is laid out . . . 297
F.3. How it is published . . . 297
F.4. How to read it . . . 298
F.5. Why it can be trusted, and for which version . . . 298
About the author . . . 300

Colophon

Nasdanika SDK Runtime: More Structure with Less Effort

Edition 2026.11.0, built on 2026-11-07. DOI: doi.org/10.5281/zenodo.23740338.

This edition describes Nasdanika SDK Runtime 2026.11.0. Every code listing in it is included from an example module that is compiled and tested against that version of the runtime when the book is built.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Nasdanika SDK Runtime: More Structure with Less Effort*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code examples are licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Eclipse, EMF and Xtext are trademarks of the Eclipse Foundation. Java and OpenJDK are trademarks or registered trademarks of Oracle and its affiliates. IBM and Redbooks are trademarks of International Business Machines Corporation. OpenTelemetry is a project of the Cloud Native Computing Foundation. Other names may be trademarks of their owners. Their use here identifies the technologies the runtime builds on and implies no endorsement.

The source of this book, its examples, and the runtime it describes are at github.com/Nasdanika-SDK/runtime.

Typeset with Asciidoctor PDF in Noto Serif, Noto Sans and M+ 1mn. Diagrams drawn with PlantUML.

Preface

The books that shaped how I build software have one thing in common: they are a body of knowledge you can read from the first page to the last. *EMF: Eclipse Modeling Framework* explained not only how to generate code from a model, but why a model is worth having and what it buys you once you have one. *The C4 model for visualising software architecture* gave boxes and lines a vocabulary small enough to remember, a stark contrast to no vocabulary at all or UML, BPMN and other complex notations, which are so huge that it is almost the same as having no vocabulary - the point at infinity. Neither is a reference manual. Both are arguments, made one chapter at a time, with enough "how" to make the "why" concrete.

Two books about programming languages shaped how I build software too, from the side of the code. Bjarne Stroustrup's *The C++ Programming Language*, in its third and special editions, teaches its key features together with the design techniques they exist to support, gives its last part to design, and ends each chapter with numbered advice. Bruce Eckel's *Thinking in Java* teaches a way of thinking rather than a syntax; Eckel put it on the web while he was writing it, gave its early editions away, and let what readers sent him shape the editions that followed.

This book tries to do the same for the Nasdanika SDK Runtime - thinking in models and JVM (not just Java).

The runtime is the part of the Nasdanika SDK you install first and build everything else on. It gives you models written as text in a language called Xcore and handled by the Eclipse Modeling Framework (EMF), a capability framework through which code finds what it needs by asking for its type, telemetry that is part of the code rather than bolted on, volumes that present anything as files, and a command line assembled from whatever modules happen to be present. It is influenced by EMF but has nothing to do with the Eclipse IDE. It speaks OpenTelemetry but keeps telemetry local, as a model, next to the models it describes. It is built on the JDK and adds as little as it can. Those choices have reasons, and the reasons are most of what this book is about.

Who this book is for

Architects who want more structure with less effort. If you have ever tried to keep a domain vocabulary, an architecture or a portfolio in office documents, a wiki and a drawing tool, you know the problem: structure is expensive, so it decays. The runtime is my answer to making structure cheap enough to keep. You do not need to write Java to get value from the first part of this book, and the parts that are about code explain what the code makes possible before they show it.

Java developers who want an observable, testable foundation with few dependencies, and who are curious what modeling looks like when it does not require an IDE.

Researchers and R&D teams, in a company, a lab or in the open, who need to try an idea before anyone will provision infrastructure for it. The runtime needs a JDK and nothing else: a solution built on it runs in one JVM, on a laptop, a single virtual machine or the free tier of a CI service, with no cluster, database server or message broker to set up and ask permission for. It also keeps the record a researcher wants: listings that are tested code, telemetry kept as a model next to the results (Chapter 13), and a snapshot of exactly which modules ran (Chapter 22), so that an experiment can be repeated and its result explained. Academic research is mostly done in Python, and this book does not try to change that; it is for the research and development that happens where the JVM already is, or where one machine has to be enough.



When releases are archived on Zenodo, add the concept DOI here and in the colophon, so that a result can cite the version of the runtime it ran on.

Collaborators. If you want to contribute to Nasdanika, or build on it, this is the map. It says what exists today, what is a sketch, and what is planned.

Me. "The faintest ink is better than the strongest memory." A design that is not written down is re-derived, slightly differently, every time it is needed.

AI assistants. A coding assistant working on a Nasdanika project is grounded in what it is given. This book is written to be given: the terms are used consistently and defined in Appendix D, the rules of thumb are collected in Appendix C, and every listing is code that compiles. If you are an assistant reading this, start with Appendix C.

What this book assumes

This book does not start from zero, and it is not an introduction to modeling or to Java. It assumes you have seen the problems it is about: a definition that lives in a schema, a wiki and three spreadsheets, a diagram nobody trusts, a question about your own systems that nobody can answer. You do not need to write Java to read the first part, but you will meet terms like EMF and metamodel there. Appendix D defines them, and an assistant can take you further than I could here.

My sister, who knows modeling a bit, says of this material *человек с мороза не поймёт*: someone walking in from the cold will not get it. She is right. If the problems are not yet yours, the book will keep until they are.

How this book is organized

Part I explains the problem the runtime addresses, the principles it follows, how it is built with an AI assistant, why some organizations will resist it, and how its parts fit together. Part II covers the `common` module: models with EMF, generic types at run time, reactive flows on the JDK, and the capability framework. Part III is about telemetry: how the runtime instruments itself, how to watch work while it runs, and how telemetry becomes a model. Part IV introduces volumes, the runtime's file abstraction, and serving them over HTTP and WebDAV. Part V shows how commands are contributed by modules and wired by generic types. Part VI tours the models that ship with the runtime: the Groovy DSL for authoring, the core model, Meta, the JVM model, the models that generate the web, user interfaces assembled from views chosen by the class of what they show, and diagrams: drawn in Draw.io, laid out with ELK, and drawn as SVG. Part VII is the roadmap and the vision. The appendices are references: modules, telemetry names, rules of thumb, a glossary, and how this book is built.

Like "Beyond Diagrams", this book is more a menu than a recipe book. Architects can read Part I and the first section of each later chapter, which says what a capability is for, and skip the listings. Developers should read the parts in order, because each builds on the previous one, and run the examples as they go.

In a martial art, techniques are learned one at a time and then put together in forms, as words are put together in sentences. Most chapters of this book teach techniques: a

volume, a flow, a capability, a model. The examples that combine them, from the command followed end to end in Chapter 5 to the stories of the Nasdanika Body of Knowledge (Appendix F), are forms. Both differ from the mainstream the way one school of kung fu differs from another.

My kung fu teacher (Where the opinions come from) says: "I teach you my kung fu so that it becomes your kung fu." Students differ, heavier or lighter, taller or shorter, and each practices what they are taught in the way that fits them. This book is offered the same way. Its techniques have worked for a practitioner working alone or in a small team; take the ones that fit, adapt them, and make them your own.

The Nasdanika Zephyr Books

This book is the why, the what and a high-level how of the runtime. It shows enough of each module to make its purpose concrete, and stops there. The depth is in the Nasdanika Zephyr Books: in-depth technical manuals, one per module, or per small group of modules that are used together, for the modules that need more than a chapter. If this book is the menu, the Zephyr Books are the recipes.

The model is IBM Redbooks. They are technical guides written by practitioners, IBM specialists working with partners and customers, and they are free. They cover what a product is for, the scenarios it serves and, step by step, how to put it to work, which is more than a reference manual does.

A Zephyr Book does the same for one module of the runtime. It goes deeper into the how: every form and option, the extension points, and the errors and how to read them. Where the module calls for it, it goes deeper into the why and the what as well, with the use cases the module was built for, which a chapter of this book can only name. It stands on its own: a reader who starts with a Zephyr Book does not need this one, so where the two touch, the Zephyr Book repeats what it needs. The chapter that introduces a module points to its Zephyr Book.

These are available with this edition:

- **Matching and Wiring**, for the `common` module of Part II. Its why is that most of software development is matching parts and checking that they fit, and that the runtime

matches at run time on the generic declarations the compiler has checked, with the compiler's rules. Its how is a chapter per package: types, adaptation, capabilities, flows, resource sets and telemetry.

- **Telemetry Model**, for the telemetry model of Chapter 13: traces, log records and metrics exported from the OpenTelemetry SDK as EMF models, kept next to the models they describe, with the changes a unit of work made to models recorded in the spans that made them.
- **Volumes and Content**, for the `volume` module of Chapter 14 and Chapter 15: a small SPI for trees of files wherever they live and a re-readable abstraction of bytes, models stored in volumes through EMF, and anything presented as files served over HTTP and WebDAV, in one trace with its clients.
- **Harness**, for the `harness` module of Chapter 15: an HTTP server that is up while code runs against it, the code's requests and the server's work in one trace, recorded as a model.
- **Declarative Command Lines**, for the `cli` module of Chapter 16 and Chapter 17: command lines assembled from contributed picocli commands that find their parents and sub-commands by generic types, mix-ins, overriding, the runtime's commands and the launcher, and the pipelines the approach makes possible.
- **Groovy DSL**, for the `groovy` module of Chapter 19. The user's side is the language rule by rule: call forms, references and name resolution, loading other resources, prototypes, saving, diagnostics and telemetry. The developer's side is binding the DSL to a metamodel of your own, with handlers, capability factories and resolvers. Its use cases are models kept as scripts, models derived from diagrams and spreadsheets, and scripts that write themselves back when they are saved.
- **Meta Model**, for Meta of Chapter 21: a language-neutral type system, a vocabulary written down once in a text syntax with documentation as structure, drafted with an AI assistant for the people who own the words, and projected to glossaries, Ecore, TypeScript and the tool schemas of agents.
- **JVM Model**, for the JVM model of Chapter 22: what a Java virtual machine can name, from layers and modules down to methods and parameters, identified the JVM's own way so that builds, coverage, profiles and traces join on it without a mapping.
- **HTML and Bootstrap Models**, for the web models of Chapter 23: HTML as an EMF model with a generator that writes it, and Bootstrap 5 with Bootswatch themes as a

model of what components mean, expanded to HTML for static pages and used in React components.

- **TypeScript and React Models**, for the web models of Chapter 23: TypeScript's declaration surface and React's components, hooks and JSX as EMF models, with generators that manage imports and write code a browser runs with no npm and no build step.
- **Walk-throughs with Playwright**, for the `playwright` module of Chapter 23: browser sessions in the server's trace, with steps as spans, the server's work as their children, screenshots and narration recorded in the steps, and an optional video of the session that the trace explains.
- **Diagram Model**, for the diagram model: diagrams as data, built, stored as YAML, converted from Draw.io and mapped to the models they depict. With the Draw.io Model book, it is the mapping of diagram elements to model elements that makes a diagram the source of a model.
- **Diagram Layout with ELK**, for automatic layout of the diagram model with the Eclipse Layout Kernel, with diagrams of Ecore models generated, laid out and rendered with React Flow and Draw.io.
- **Draw.io**, for the `drawio` and `drawio-playwright` modules of Chapter 25: diagrams that people draw and programs read. Its why is the visual communication continuum, from a picture to a diagram that is documentation, configuration or the source of a model. Its how is the Java API for Draw.io documents, pages, layers, nodes, connections, properties, links and styles, and rendering pages with Draw.io's own viewer as images and videos. Its diagrams are PNG images with the diagram embedded: a reader sees a picture, Draw.io opens the same file for editing, and the book's tests load it as the document the next listing works on.
- **Draw.io Model**, for Draw.io documents as an EMF model on the diagram and presentation models: loaded into a resource set from `.drawio` files and exported images, read, changed, generated and saved, with a diagram of the running JVM generated, laid out with ELK and drawn by Draw.io's viewer.
- **SVG Model**, for SVG as an EMF model: the subset of SVG that generated diagrams need, with a diagram laid out with ELK and written as SVG, every shape a link.
- **Presentation Model**, for the presentation model of Chapter 25: decks, slides, masters, shapes, text, tables, connectors and speaker notes as an EMF model on the diagram

model, because a slide is a diagram.

The rule of this book holds for them. Every listing in a Zephyr Book is included from the module's own tests, and a warning fails the build, so a Zephyr Book that no longer matches its module does not build.

Each Zephyr Book is on its module's page on the runtime site, runtime.sdk.nasdanika.org, as a PDF and as a single HTML page. It is also on Maven Central, attached to its module's artifact with the classifier `book`: `org.nasdanika.sdk.runtime:groovy:pdf:book:2026.11.0` is the PDF of the Groovy DSL book, and `html` in place of `pdf` is its HTML. The test sources that the listings come from are attached the same way, with the classifier `test-sources`, so the whole test behind a listing is one download away. The Zephyr Books are also on Leanpub, in a bundle with this book. Printed versions of this book and its Zephyr Book can be ordered on Lulu. An assistant can follow the same division as a person: this book to find a module and understand why it exists, its Zephyr Book for how to use it, and its test sources for the complete code.

The name is borrowed, not invented. Redbooks are named after the color of their first covers. Nasdanika's color is blue, and a blue book is taken: by Kelley Blue Book for cars and, in programming, by the Smalltalk-80 "Blue Book" of 1983. So the books are named after the Bootswatch theme they are styled with, Zephyr, which is blue and is also the theme of the runtime's site. A name that describes a look promises nothing else, which is what a series name should do: a reader who has one Zephyr Book knows how the next one will look and how it will read. The books have nothing to do with the Zephyr real-time operating system, a project of the Linux Foundation.

Conventions

I write in the first person. "The author" is ten times more letters than "I", and a book about doing more with less effort should not waste them. "You" is the reader, whether you write the code or decide what gets built.

Code listings are included from the book's example modules, in the runtime repository under `doc/book/examples`. Each module is a small Maven project, built and tested before the book is rendered, so a listing that no longer compiles stops the build instead of misleading you. The examples use the `org.example` namespace for Java packages and modules, as your

own code would use yours. Numbered markers in a listing, such as `<1>`, are explained in the list right below it.

Status labels say how far along something is: **AVAILABLE** means it is in Nasdanika SDK Runtime 2026.11.0 and the book shows it working; **SKETCH** means a model or an interface exists without the behavior around it; **PLANNED** means it is designed, on the roadmap, and not built yet.

Versions

This edition describes Nasdanika SDK Runtime 2026.11.0, built with Java 25 and Maven 3.9.16 or later. The runtime's version is a calendar version: year, month, patch.

Feedback

The book is developed with the runtime, in github.com/Nasdanika-SDK/runtime. Issues and pull requests against the manuscript in `doc/book/manuscript` are welcome, and so are questions: a question is a section that is missing.

Where the opinions come from

The runtime is opinionated, and you are entitled to know where its opinions come from.

I trained as a physicist, with a master's degree in plasma physics. I was taught Fortran, and Quasic, a small interactive language for programming laboratory hardware. The rest I taught myself: Pascal, assembler and machine code, Basic, Object Pascal in Delphi, C++, Java and JavaScript, and XML with XPath and XSLT. Between them they cover most of the distance from the machine to the model, and working with an assistant, as Chapter 3 describes it, is one more level up.

I learned to think in shapes and connections, and in signals moving along them, long before I learned UML. As a teenager I built electronic devices: amplifiers, radio receivers and my first computer. I read and drew schematics, and laid out my own printed circuit boards, by hand and later in OrCAD, and etched them with ferric chloride and zapon varnish. As a physicist I designed scientific devices in AutoCAD, and a workshop built them as drawn. A schematic, a board layout and a CAD drawing are not illustrations: they are what the thing is made from. So ERwin for databases and UML for software came easily; they were drawings to build from too.

Software gave me more drawings that run. I programmed Lego Mindstorms robots, first in the visual language that comes with the kit, where a program is blocks wired together, and then in Java with leJOS. A robot is event-driven by nature: it waits for a sensor to fire and reacts while its motors are still turning, and that shaped how I build software. At work I built integrations in TIBCO BusinessWorks, where a process is drawn as activities and transitions and deployed as drawn, and modeled business processes in TIBCO Business Studio. In all of them the drawing was the program, not a picture of it. The North Star of this runtime, work drawn as a graph and executed as drawn (Chapter 26), is the same idea once more. When I wrote about the executable graphs of earlier Nasdanika releases, the metaphor I reached for was a printed circuit board with processors soldered to it (*Executable (computational) graphs & diagrams*, *Concurrent Executable Diagrams*).

From 1989 to 1992 I was also trained as a platoon commander, in the military department of my university, and commissioned as a lieutenant in the reserve. Two things from that training stayed with me. One is maps: reading them, and drawing a plan of defense on

one, another drawing to act from. The other is Article 17 of the *Combat Regulations of the Ground Forces*, Part III, for the platoon, the squad and the tank (Voenizdat, 1990), which was drilled into our heads. On receiving a combat mission, a platoon commander must:

1. understand the mission: the task of the company as well as the platoon's own, which targets on the platoon's front the senior commanders' weapons will engage, the tasks of the neighbors and how to coordinate with them, and the time to be ready;
2. assess the situation: the enemy, the platoon and the units attached to it, the neighbors, the terrain, the weather, the season and the time of day;
3. make a decision: how to accomplish the mission, the tasks of the squads and of the attached units, and how the platoon will be commanded;
4. reconnoiter the ground;
5. issue the combat order;
6. organize coordination, support and command, and prepare the people, weapons and equipment for combat;
7. check that the platoon is ready, and report it to the company commander at the set time.

It works well beyond a platoon: I have explained it to my children, and they know my version of it by heart. I later recognized it in project management and in the TOGAF Architecture Development Method, and the loop of Chapter 3 has the same shape.

The training met its first real test in a computer game. In the late 1990s I played a lot of Command & Conquer, a real-time strategy game; I never cared for Doom. At first I lost badly, and it stung: I had been trained for this. A short while later I could not understand how I had ever lost so badly. Doom puts you in one body and asks for reflexes. A real-time strategy game gives you a map and units that act on their own: you set them a task, watch, correct and decide again. That is the work of directing an assistant, decades before there were assistants to direct. Against the usual advice, I later had my children play it, for the strategic and spatial thinking it trains.

Looking back, it is one path. As a child I spent hours with Lego, long before Mindstorms; a cousin of mine holds that it trains spatial thinking, and I agree. Then came schematics and circuit boards, maps and Command & Conquer. Now I play Go, which is older than chess and has far more possible games. At home we play Harmonies, a board game in which a

landscape is built tile by tile. Chinese has a word for a lifelong practice like this, *dào*, the way, which Japanese reads as *dō*, as in judo, kendo and bushido. Mine is a way of seeing structure: things, the connections between them, and how they will change; modeling is where it led.

The other way I have kept is a martial one, since about 1989. Wushu as a student, aikido in the 1990s, and after ten years without practice in the United States, Kenpo and Modern Arnis: I had taken my son to the classes, realized how much I had missed it, and joined the adult group, where the oldest student after me, past forty, was fifteen. Then came ninjutsu, and now Praying Mantis kung fu, tai chi, and the broadsword and the long sword.

My teacher explains every technique he teaches and shows its applications. Some techniques are counter-intuitive at first and take as much effort of the mind as of the body, and once the application is understood, repeating a drill for the hundredth time is not boring: each repetition is an exercise in seeing what the technique is for. He also talks about small moves. A master wins with moves that a casual observer does not see: no more motion than the situation needs, and all of it to the point. That is efficiency and effectiveness together, more with less, and it is what I look for in software: a loop over an array instead of a database (Chapter 4), a few thousand lines of Xcore instead of hundreds of thousands of lines of Java (Chapter 3), the JDK before a library (Chapter 2). And he says that kung fu is "eating bitter", *chī k*□: hours of practice that nothing replaces, ten thousand of them, he says, to master the long sword.

Ten thousand hours is also the number Malcolm Gladwell made famous. In *Outliers* he tells how Bill Gates got time on a computer as a schoolboy, when almost nobody could, and spent every hour on it that he could find. Opportunity and hours, at the moment a technology is new. The PC was such a moment, and Chapter 3 tells what my mother made of it. Assistants are another, and this book is what my hours with one have produced so far.

I taught myself from books, and in Khabarovsk, in the Far East of the Soviet Union and then of Russia, books about computers were scarce. A few bookstores had a few of them; on business trips to Moscow I found more. The first computer store I visited in the United States was a Micro Center, and I went straight to its books section, which held more computer books than I had ever seen in one place. A colleague who came with me was surprised: "Oh, you are still reading…" He had read his computer books in college and did

not expect to need more.

Scarcity leaves habits behind. Mine is to assume that I do not know enough yet and that somebody has done this before, and to find out who, and what they learned. That is the first step of every design in this runtime.

Physics taught the same habit, with a story that I was told when I was in the university. A student wrote his diploma thesis on computing an integral by numerical methods. An old academician who happened to attend the defense opened a table of integrals and asked why the student had computed numerically an integral that has a closed form. The student got a C, the lowest passing grade, out of pity. The computation was correct; the work was not needed, and an hour in the library would have shown it.

Later I learned the name lawyers give to what that student skipped. I prepared two patent applications myself, for the automated build and deployment system my team had built, and a patent application is judged against the prior art: everything published before it that already does what it claims. The examiner searches for it, so you search first, and you learn to read what others have done as the measure of what you have. The applications were abandoned when the team left the company; the habit stayed. It is the reason for the next section, which is the prior art of this runtime in my own work, and for the table in Chapter 2 of what the runtime borrows from the work of others.

Plenty of books did not mean plenty of fluency, though. When I came to the United States I assumed that every developer knew UML, and I was wrong. Chapter 3 argues that fluency in modeling, and in technologies such as EMF, was scarce for twenty years and is not anymore. I take the end of a scarcity seriously, because I have watched one end already, in the aisles of a computer store.

Prior art, first-hand

The runtime is new; most of its ideas are not. I have built nearly every one of them at least once before, in large organizations, for a real problem and under harder constraints than the runtime has now: 32-bit address spaces, no assistant, and teammates who had never heard of EMF. This section is the record, so that you can judge the claims the book makes against what they come from. The organizations, the commercial products and the people are left unnamed; the problems, the numbers and what I built are as they were.

Before EMF

One of my first projects in the United States integrated a mainframe through a screen-integration product. It was meant to be used by mapping the input and output fields of every screen in its user interface and then writing Java against its API, one integration at a time. There were dozens of screens with hundreds of fields. Instead, I worked out the product's file formats and described the screens, and the flows between them, in XML. Then I wrote an interpreter for that XML: it received a request from the message bus, navigated the screens, entered and collected the data, and sent the result back, with XSLT for transformations and Java handlers for what XSLT could not do. The same XML generated an HTML view of every screen, which people who knew the mainframe could read without knowing the product. They and the developers wrote flows, and I tested and deployed them. It was an executable model, documentation as code and inner source, years before I had a name for any of them.

At that time people could log in to application servers and change them by hand, and source control labeled files one at a time instead of committing a set of files together. Environments drifted apart: a logging library's jar had the same name on every server and a different size, timestamp and digest on many of them. I wrote a scanner that walked folders, and the archives inside them, computed a digest of every file and of every directory from the digests of its contents, and compared two scans as an HTML report. It was a Merkle tree; I learned the name some twenty years later.

The team had also bought a commercial code-review tool. It was expensive and hard to use, and I was responsible for it. So I matched the part of it we used with an open source project of my own, Hammurapi. It parsed Java sources with an ANTLR grammar into a

hand-made model of the language, with cross-references resolved down to method calls and bytecode consulted where it was available, and its inspectors navigated the model rather than matching patterns in the text. On a 32-bit JVM a large code base did not fit in memory, so the model was held with soft references. The reviews ran in the automated build, which checked the code delivered by outside contractors before anybody had to read it.

Later, as the integration architect of a call-center modernization, I had to talk to a mainframe in a binary format that carried no metadata: two bytes of element identifier, two bytes of length, then the bytes of the value. The client code had to know what each identifier meant. There were about 250 message tables with 18,000 element types, and they changed every month. What we were given was XML generated from the mainframe's C header files: a metamodel of the mainframe, although nobody called it that. I wrote a generator that turned it into Java codecs streaming between the binary format and XML, and regenerated them when the tables changed.

Models of what already exists

My path with EMF started in 2008, in an R&D team. The first model was of decisions: which technology to use, and when, decided with a formal method by several experts (Chapter 4 tells the rest of that story).

The second kind of model I kept building was of things that already existed and that nobody could see whole. A customer-interaction framework drew processes as activities mapped to Java methods, with transitions between them and processes calling processes. After several years one project had about 800 of them, and nobody knew which were still used. I loaded them into a model, found the entry points in the configuration, resolved the references and reported the processes and the Java code that nothing reached.

I have modeled the code base of a visual integration platform three times, in two organizations, over fifteen years. The first time the model generated documentation for the whole code base, with a diagram of every process, and ran automated reviews in the manner of Hammurapi. The second time it was an inventory, to estimate a modernization. The third time it was an engine that executed the processes from the model directly (*Direct semantic execution*).

In 2012 the task was a monolith: about fifteen thousand classes in a thousand packages and two and a half million lines of code, with two hundred packages in a single dependency cycle. On our diagrams those packages were drawn red, and we called it the black hole. A commercial dependency analyzer gave a first look and stopped there. We built an Ecore model of Java that went beyond reflection to call trees, field references and inheritance in both directions, populated it from bytecode with ASM, and browsed it in a web application with class and sequence diagrams generated by PlantUML; it held about two million objects. An instrumentation tool recorded call trees at run time, to catch what reflection hid from static analysis, and an Eclipse plug-in extracted into a new module only the code traceable from chosen entry points, down to the individual members of a class. It all ran on a 32-bit JVM.

In the same R&D years I exposed a banking domain model as JavaScript APIs generated from an EMF model, because writing them by hand would have been slow and error-prone. The generated code loaded classes, data and templates on demand, worked synchronously or with promises, and measurably reduced network traffic compared to the approach then in use.

Models of how work is done

For a cloud migration, the software delivery process was a drawing of about a hundred activities. I built a diagram editor backed by a model of what is done, by whom, in which phase and with what resources, and migrated the drawing into it. For the same effort I built a generator of starter solutions. A developer filled out a form in the browser, the form was committed to a repository as YAML, and the commit started a build that generated the code, pushed it to the developer's repository, started its first build and wrote the identifiers of the cloud resources it created back into the YAML. It was a Git-driven workflow before I called it that. It also taught me a lesson that Chapter 3 returns to: the generator's specifications were EMF models saved as XMI, and my teammates, who did not know EMF, edited them in a text editor.

The developer-experience work that followed modeled the delivery process as journeys: directed graphs of activities, nested without limit, performed by roles using tools and consuming and producing deliverables, kept as YAML and Markdown in source control and generated into documentation with clickable diagrams. Next to it, documentation as

code generated and federated the sites of more than sixty components from their own repositories.

The organization as a model

The largest model I have built joined three sources in a bank: the people directory, with tens of thousands of people and their reporting lines; the inventory of thousands of applications, with their owners, tiers and dependencies; and tens of thousands of source repositories, with their commits, files and Maven dependencies. Joining them was most of the work, because the same person had different identities in each. Once joined, the model answered questions that had needed a project each: who owns what technology, rolled up the reporting hierarchy; which organizations contribute to the same repositories; which languages the code is written in; and, from an organization-wide dependency graph, where code could be reused and which vulnerable libraries sit closest to the attack surface.

What carried over

None of these is the runtime. Each was built for one organization and one problem, on the platform of its day, and each ended when its project did. What they share is a shape, and the runtime is that shape built once, in the open, with the organization-specific parts left out.

Built before	Where it is now
An executable XML model of mainframe screens and flows	Work drawn as a graph and executed as drawn, the North Star (Chapter 26)
HTML views generated from the screen model; documentation sites generated from sources	Documentation as code, the web models (Chapter 23) and the Zephyr Books
Digests of directories computed from the digests of their contents	Snapshots of what ran (Chapter 22)
Hammurapi's model of Java; the 2012 model loaded from bytecode	The JVM model (Chapter 22)

Built before	Where it is now
Codecs generated from the mainframe's metadata	Models in Xcore and the code generated from them (Chapter 7)
Processes and flows loaded from the files of the tools that drew them	Diagrams as sources of models (Chapter 25)
JavaScript APIs generated from a banking model	TypeScript and React generated from models (Chapter 23)
Generator specifications in XMI, edited as text	Models kept as YAML and as Groovy scripts (Chapter 19)
A form committed as YAML, and a build that delivered the result	An architecture method on Git (Chapter 26)
A model of decisions with criteria, experts and pairwise comparisons	Decisions bound and recorded before the code (Chapter 3)
One model of people, applications and repositories	Micro-models joined by identity, local and federated (Chapter 2)

Part I: The Why

Why the runtime exists, the principles it follows, how it is built with an AI assistant, why most organizations will resist it, and how its parts fit together. No code is needed to read this part; it is the argument the rest of the book makes concrete.

Chapter 1. More structure with less effort

Glass is a curious material. It is a solid, it holds its shape, and you can build with it. Look closely, though, and there is no crystal inside: the atoms have short-range order, each one sitting sensibly next to its neighbors, and no long-range structure at all. It is a liquid that stopped moving before it got organized.

Most enterprise knowledge is glass. Every team has its documents, its diagrams, its spreadsheets and its wiki pages, and each of them makes sense locally. Step back and there is no structure that connects them. The definition of a customer lives in a database schema, a JSON Schema, a partner's XML Schema, a Java class, a Confluence table and three spreadsheets, and nothing relates them. Each is a projection of a model that nobody ever wrote down.

This chapter is about what structure is for, why there has been so little of it, what changed recently that makes it fixable, what kind of structure lasts, and what the Nasdanika SDK Runtime is in that picture.

1.1. What structure is for

"Life, at its core, is a self-sustaining, autocatalytic process," writes the biologist Alexander Markov in *Birth of Complexity* (*Рождение сложности*, 2010; my translation). An autocatalytic process makes the things that speed it up, so the more it has grown, the faster it grows. Business works the same way. A company does some things better, faster or cheaper than others, sells the difference as a margin, and puts the margin into doing them better still. An enterprise is a group of stakeholders and resources devoted to a mission, and structure is worth exactly what it adds to the effectiveness and efficiency with which the mission is pursued.

Life also shows what happens without structure. About three and a half billion years ago, microbial mats divided labor between layers of bacteria and archaea, which exchanged the products of their work and signaled to each other with chemicals. They did well for billions of years, then lost the lead to organisms with nervous systems, which carry a

signal across a whole body in a fraction of a second. Enterprise IT is still a mat: each group works from its own playbook, often a tribal one, nobody has the whole picture, and improvement stops at the edge of a group. Glass and the mat are one observation, made by physics and by biology: order that is only local. Structure that connects the groups is the nervous system.

1.2. Structure has been expensive

We are managing twenty-first century complexity with twentieth century tools. Office documents were designed to be printed. Wikis were designed to be edited by anyone, which is exactly why nobody owns what they say. Drawing tools produce pictures: a box on a diagram looks like a system, and nothing about it says which system, who owns it, or whether it still exists.

None of this is news, and the industry has tried to fix it more than once. I learned UML in the 1990s, when it was going to be how software was designed. By the 2010s I had concluded that it was dead in the enterprise, and I was not alone. Simon Brown observed in the C4 model book that many teams "have already thrown them out in favour of much simpler 'boxes and lines' diagrams". The reason was not that structure is useless. The reason was cost.

Modeling came with a tax. The tools were desktop applications with their own repositories, their own licenses and their own learning curves. The open source alternative, the Eclipse Modeling Framework, was excellent and lived inside an IDE: to use a model you installed Eclipse, configured a workspace and generated code with a wizard. Catalogs and ontology platforms moved the tax to a server you rent, with an administrator who owns the whole graph. Every one of these asks people to adopt something before they get anything back.

So people rationally chose less structure. When a model costs more to keep than it pays, a spreadsheet wins, and the spreadsheet is the real competitor of every modeling tool. It is free, everybody can use it, and it never asks anyone to adopt anything.

The price of that choice is paid later, and by somebody else. Ask three people what the third measure on a list in some system prompt means and you get three answers. Ask who owns the definition and you get silence, then a suggestion that some other team may

know. The knowledge exists. It lives in people's heads, and it leaves when they do. An assistant cannot be fed it either.

Volume is not what makes structure expensive; complexity is. When I joined a bank's people directory, application inventory and source repositories into one model (Prior art, first-hand), a conventional web application already reported on much of the same data, from about fifty relational tables. It handled the volume well. It could not keep up with the questions, because every new question needed a new join, a new table or a new screen, and the reports it could not produce were produced from the model instead.

1.3. What changed

Four things changed, and together they make structure cheap enough to keep.

The JDK grew up. Virtual threads became final in Java 21, so blocking code no longer costs a platform thread, and the main reason for reactive libraries disappeared. `java.util.concurrent.Flow` has been in the JDK since Java 9, the HTTP client since 11, `ScopedValue` since 25, and a class file API since 24. Many of the libraries an enterprise application drags along were written before the JDK had the feature. The reasons to take those dependencies expired; the dependencies did not. A runtime that starts today can say "if the JDK can do it, use the JDK" and mean it.

EMF works without Eclipse. EMF itself never needed the IDE; the tooling around it did. Today a model is a text file in Xcore, generated by a Maven plugin in any build, edited in any editor, and checked into Git next to the code. The twenty years of engineering in EMF (reflection, serialization, notifications, change recording, cross-resource references) come without the workspace.

OpenTelemetry standardized signals. Spans, log records and metrics have one API, independent of where they end up. Code can be instrumented once, and whether the signals go to an operational backend, to a console, or into a model that is kept forever is decided by whoever runs it.

A new coworker arrived. AI assistants are capable, and they see only what they are given: meeting notes, wiki pages, a deck from three years ago. They cannot absorb tribal knowledge at the coffee machine. An assistant grounded in a declared, versioned, owned

model answers from it and can cite it; an assistant given a database schema and a wiki export infers, and inference is not a chain of custody. At the same time, assistants are good at the mechanical part of modeling: they draft a model from a description, a human corrects it, and correcting is much cheaper than authoring.

There is an old camping joke about two friends who meet a bear. One of them starts lacing up his running shoes. "You cannot outrun a bear!" "I do not have to outrun the bear. I only have to outrun you." Businesses do not have to outrun AI either. They have to harness it: ground it in their own terms, systems and rules, so that people and assistants speak the same language. The harness is structure.

1.4. Small, formal, composable

Cheap structure is necessary but not sufficient: structure also has to last. It lasts when it has the three properties that keep code and its libraries alive while the documents around them rot: it is small, formal and composable. Take away any one of them and the failure is familiar. Small and formal but not composable is the monolithic repository of an enterprise architecture tool. Small and composable but not formal is the wiki that drifts the day it is written. Formal and composable but not small is the canonical enterprise data model that nobody updates.

The runtime is built to make all three cheap. Micro-models keep structure small (Chapter 2), Xcore keeps it formal (Chapter 7), and models that reference each other across repositories, with capabilities and commands that find each other by type, keep it composable (Chapter 10, Chapter 17).

1.5. What the runtime is

The Nasdanika SDK Runtime is the set of modules everything else in the Nasdanika SDK runs on, in one repository, released as one unit. The analogy I use is the Java runtime, `rt.jar` and its successor, the JDK's module image, or the runtime of an integration platform: the part you install first and build on.

It is built on three things, which I call the "pure Nasdanika" foundation: the JDK, EMF core and the OpenTelemetry API. All three are small, stable and standard, and any module may

assume them. Anything beyond them has to justify itself.

On that foundation the runtime provides:

- **Models as the API.** Data and operations are defined as text in Xcore, the build generates Java from it, and EMF does the rest: models are typed and documented, can be saved as XML (XMI), JSON, YAML or binary, and can be called by name, so an operation is also a tool a command line or an agent can call (Chapter 7).
- **A capability framework.** A piece of code says what it needs, and the framework finds it among the modules that are present. It asks by type, down to the generics: `Supplier<Library>`, something that supplies a library, and `Supplier<Catalog>` are different things to ask for (Chapter 10).
- **Telemetry as part of the code.** The runtime's own operations report what they do through the OpenTelemetry API, as spans and log records, with a separate record for each unit of work, such as a command run. You can watch them live with no OpenTelemetry SDK and no backend, or save them as a model next to the models they changed (Chapter 11).
- **Volumes**, a small abstraction of files, so that a model, a running JVM or a remote system can be presented as files, loaded as models, served over HTTP or mounted as a drive over WebDAV (Chapter 14).
- **A command line assembled from contributions.** Modules contribute commands, and each command finds its parent by type, so adding a dependency adds commands (Chapter 16).
- **Models that ship with it**: a core model for identity, documentation and provenance, Meta as a language-neutral type system, a JVM model, a telemetry model, and models of HTML, Bootstrap, TypeScript and React that make the web a generation target (Part VI).

1.6. What it is not

It is easier to place something by what it refuses to be.

Not Eclipse. There is no workspace, no plug-in registry, no update site, no OSGi. Modules are standard Java modules (JPMS) and Maven artifacts. The influence of EMF is

everywhere; the IDE is nowhere.

Not Spring. There is no container that owns object lifecycles, no classpath scanning, no XML or annotation configuration of beans. Services are registered in `module-info.java`, and capabilities are resolved on request.

Not a modeling tool. There is no diagram editor in the runtime. Models are authored as text (Xcore, a Groovy DSL, YAML), harvested from what already exists, or drawn in tools people already use, and the runtime loads them.

Not an observability backend. It does not compete with Grafana or Datadog. Operational telemetry is sampled, retained for days and forgotten; the runtime is for the small fraction of signals worth keeping, resolved against a model, for years.

Not a platform you rent. There is no server. Models live in Git and in Maven repositories, reference each other by URI, and are owned by whoever publishes them. Everything runs locally, in a container, or in a CI pipeline, as the same artifact.

Not all or nothing. You take the modules you need. A Java program that wants a volume over a directory depends on the `volume` module and nothing else above the foundation.

1.7. Who gets what

Architects get structure at the cost of a text file. A vocabulary in Meta is one source for a generated glossary, a JSON Schema for a tool call, a TypeScript package for a user interface, and a lookup an agent can be grounded in (Chapter 21). A running system can describe itself: a JVM snapshot shows what an assembly became after its modules were resolved, which no build file says (Chapter 22). Telemetry kept as a model is evidence of who changed what, in the course of what work, years after the fact (Chapter 13). Because models are files in Git and artifacts in Maven repositories, teams own their parts and reference each other's, and nobody administers the whole.

Developers get a foundation that is small, observable from the first line, and tested. The examples in this book depend on the runtime and on JUnit; the runtime depends on the JDK, EMF, the OpenTelemetry API and a handful of libraries that do something the JDK does not.

Subject-matter experts get their words written down precisely without learning a notation: an assistant drafts a vocabulary from what they say and hand over, the build checks its structure, and they correct its meaning (Section 21.9).

Agents get typed models instead of prose, operations that are tools by construction, and file systems they already know how to use.

1.8. A short tour

The rest of the book explains how each piece works. Here is what a few of them look like from the outside, using the output of the book's examples.

The first program in Chapter 6 builds a one-class metamodel at run time, saves an instance as XMI, JSON and YAML, and does nothing else. Run under a publishing OpenTelemetry instance, with no SDK and no backend, this is what it prints:

```
> Save a book
  > ResourceSetContributor.configure
    Contributed uri-handler ...ClassPathURIHandlerFactory$1
    Contributed contributor ...XMIResourceFactoryContributor
    Contributed contributor ...JsonResourceFactoryContributor
    ...
  < ResourceSetContributor.configure
  > Resource.save
    Saved file:/tmp/watch-it/book.xmi: 230 bytes
  < Resource.save
  ...
< Save a book
```

Nobody instrumented that program. The runtime's resource set configuration, resource factories and URI handlers report what they do through the OpenTelemetry API, and whoever runs the program decides where it goes.

The command line example in Chapter 16 contributes a `library` command and a few sub-commands. The runtime's own `save` command was written long before that example and knows nothing about libraries, yet it appears under `library`, because the library command offers what `save` declares it needs: something that supplies a collection of EMF objects.

```
nsdk
  launcher
```

```
  hello
  library
    books
    export
      csv
      markdown
    save
```

And the recording example in Chapter 13 edits a library inside a span, with the OpenTelemetry SDK exporting to the telemetry model. Reading the saved traces back as a model, the change is in the span that made it:

```
Curate the shelf
  * created Library 'A Modeler's Shelf'
  Resource.save (2059 bytes)
  Correct the year
    * Book 'Domain-Driven Design'.year = 2004
    Resource.save (2059 bytes)
```

That last output is the point of the whole runtime in miniature. "Somebody changed the year of a book" is a log line. "In the course of the work named 'Correct the year', this attribute of this object changed to this value", with the work, the object and the attribute all being model elements you can navigate and query, is structure. It cost one line of code: `Changes.record(⋯)` around the edit.

1.9. Where this is going

The runtime is a seed. Its North Star, explained in Chapter 26, is agentic work on models: executable graphs whose activities include AI agents, operating on bounded, typed models, with transactional safety and state saved to Git, runnable on a laptop, in a container or in a CI pipeline as the same artifact. Several capabilities on the way there are useful long before it, and they are what this book is mostly about: models without an IDE, telemetry without a backend, files without a file system, and a command line without configuration.

The next chapter states the principles the runtime follows. They are short, and every later design decision traces back to one of them.

Chapter 2. Principles

A runtime is a long series of small decisions. Most of them are made under time pressure, by whoever happens to be writing the code, and the result is consistent only if the decisions follow from a few rules that everybody knows. This chapter states the rules the Nasdanika SDK Runtime follows, with the reasoning behind each one. Later chapters refer back to them by name.

2.1. Pure Nasdanika: the JDK, EMF core and the OpenTelemetry API

The foundation of the runtime is three things:

- **The JDK**, currently Java 25.
- **EMF core**: `org.eclipse.emf.common`, `org.eclipse.emf.ecore` and `org.eclipse.emf.ecore.xmi`.
- **The OpenTelemetry API**: `io.opentelemetry.api` and `io.opentelemetry.context`, not the SDK.

I call this "pure Nasdanika". Any module may depend on all three, `common` does, and code that uses only them counts as foundational. All three are small, stable, and standard in their space, which matters more than any feature they have: a platform team can approve "Java, EMF and the OpenTelemetry API" in a meeting, while "Java, EMF, Reactor, Netty, Jackson and two vendor SDKs" is a review.

Everything beyond the foundation is justified per module. `emf-json` brings Jackson because EMF JSON is implemented on it. `groovy` brings Groovy because it is a Groovy DSL. The command line module brings picocli and JLine because they are what a command line is made of. None of them is allowed into `common`. The one exception made so far is geantyref, a tiny library with no dependencies that `common` uses to match generic types, wrapped behind the runtime's own `Types` class so that no public signature mentions it (Chapter 8).

2.2. If the JDK can do it, use the JDK

Occam's razor, applied to dependencies: a third-party library earns its place by doing something the JDK does not do.

Concern	The usual choice	The runtime's choice
HTTP client	Netty, vendor SDKs	`java.net.http.HttpClient`
HTTP server	Netty, servlet containers	`com.sun.net.httpserver.HttpServer` on virtual threads
Asynchrony	Reactor `Mono` and `Flux`	Blocking code on virtual threads; `java.util.concurrent.Flow`
Ambient context	Thread locals, frameworks	`ScopedValue`, and the OpenTelemetry `Context`
JSON	Jackson data binding	org.json where JSON is a wire format; EMF JSON where it is a model format
Logging API	SLF4J and a backend	The OpenTelemetry Logs API, correlated with spans; where a log record goes is decided by whoever runs the unit of work

Most of the libraries in the middle column were written before the JDK had the feature. Reactor and Netty solved a real problem: a blocked platform thread was expensive, so non-blocking I/O and reactive composition were the only way to serve many concurrent requests. Virtual threads removed the cost. The reasons to take the dependency expired; the dependencies did not.

Blocking code is also better code for the work the runtime is heading toward. An agentic loop is sequential: call the model, run a tool, call again. On virtual threads that reads as it runs, the OpenTelemetry context propagates through straight-line code with no hooks, spans nest the way the code nests, and stack traces are readable. Reactive operators pay off for fan-in and fan-out pipelines, and the runtime keeps `Flow` for those (Chapter 9).

There is one refinement. Where the runtime *consumes* something, it uses the JDK's API. Where it *implements* something many times, it prefers a small interface of its own. File systems are the example: NIO is how the runtime reads existing file systems, and `Volume`, an interface with five methods in the shape of Go's `io/fs`, is what it implements over a directory, an in-memory tree, a running JVM or a remote system (Chapter 14). Implementing an NIO file system provider is a large job; implementing a volume is an afternoon.

2.3. If Xcore can model it, model it

Data and APIs are Xcore classes and operations, not hand-written Java beans and interfaces.

A model is more than a data structure. It is documented, so a generated site explains it to people. It is serializable, to XMI, JSON, YAML or binary, so a snapshot is a file you can diff and commit. It is reflective, so a viewer can show any object without code written for its class, and a query language can traverse it. And with operation reflection turned on, every operation can be invoked by name with arguments, which means an operation is already a tool: it has a name, parameters, types and documentation, which is everything a tool declaration for an agent needs.

The cost is a code generator in the build. Chapter 7 shows that this cost is now one Maven plugin and a text file, which is the change that made the principle affordable.

2.4. Telemetry is part of the runtime

Instrumentation is not an add-on that somebody remembers to do later. Every runtime module that does something worth knowing about reports it through the OpenTelemetry API, and modules wrap the JDK APIs they use so that instrumentation is the default. Four conventions make that consistent:

Spans for operations, log records for what happens in them. A span has a duration; a log line, a measurement or the progress of a long-running operation is a log record emitted through the OpenTelemetry Logs API, correlated with the current span. Span events are not used, because OpenTelemetry is moving events onto the Logs API.

One `OpenTelemetry` per unit of work. A unit of work is a command, a request or a test. It builds or is handed an `OpenTelemetry` instance and makes it current, the way it would make a span current, and everything instrumented underneath reports to it. `GlobalOpenTelemetry` is never used. This is what lets one command save its own telemetry next to its own output, while another, running in the same JVM, sends its telemetry somewhere else.

Console output is a subscriber, not a second API. Progress bars and live trees subscribe to the same span and log events that a model exporter or a backend receives (Chapter 12).

Every signal is precious. Mainstream observability collects everything cheaply, samples, and forgets within days. The runtime is for the opposite end: deliberate collection, indefinite retention, and every signal resolved against a model, so that "who changed what, in the course of what work" can be answered years later (Chapter 13). The two are complementary: the same instrumentation can feed an operational backend and a model at once.

2.5. Services before capabilities

The runtime has two ways for code to find other code, and the rule for choosing between them is simple:

- When the lookup is by type alone, such as every `EPackage` or every resource set contributor, it is a Java service, registered in `module-info.java` and found with `java.util.ServiceLoader`.
- When there is a requirement object for providers to interpret ("a renderer for this format", "a contents handler for this file name"), or when resolving one thing requires resolving others, it is the capability framework (Chapter 10).

`ServiceLoader` is a dozen lines of JDK and costs nothing. The capability framework is more powerful and has a cost: a resolution, a pump, memoization, telemetry. Using it where a service would do is the kind of generality that makes a codebase hard to read.

2.6. Declaration is registration

The runtime avoids strings that name types and configuration files that list classes. What a module contributes is declared in its `module-info.java` (`provides CapabilityFactory with …`), and what a contribution provides and needs is declared in Java types the compiler checks.

A capability method's generic return type is what it provides, and its parameters are what it needs. A command's `@Parent` field says, by its generic type, which parents it can work under. An adapter method's return type says what an object can be adapted to. One class can declare many such members behind a single entry in `module-info`, and a refactoring tool renames them like any other code.

The honest caveat goes with it: objects carry no type arguments at run time, so what is matched is always a declaration, never an instance. A method declared to return `Supplier<Library>` is matched as one, whatever it actually returns.

2.7. Micro-models and nano-models

Models rot the way applications do: they accrete concepts until every consumer pays for every concept. The answer is the one the industry found for applications and for code: decompose along concerns, and make the dependencies explicit. Microservices decompose deployment, Java modules decompose code, and micro-models decompose meaning.

A **micro-model** is a small model with a single concern, explicit dependencies on other models, and no knowledge of its consumers, in a module of its own, so that it is versioned, released and adopted on its own. Extension happens at the edge: a model extends the models below it, and they never learn about it. Meta and the JVM model are micro-models.

A **nano-model** is smaller still: one of several models in one module, because they are reused and changed together. The core model and its `kind` package are two nano-models in one module; telemetry, traces, logs and metrics are four in another. The Xtext Maven plugin generates every `.xcore` file under its source root in one go, so a nano-model costs one file. One model per project was an Eclipse convention, and nothing in the runtime needs it.

A new module, rather than a new file, is justified by a dependency the others do not want, a different set of consumers, or a different maturity. The telemetry exporters need the OpenTelemetry SDK, so they are a module of their own, while the four telemetry models share one.

The size of a model has a budget, and the budget is attention. Code hides its implementation behind an interface; a model hides nothing, because its concepts are its interface. So a model should fit in the head of the team that maintains it, and its core concepts should fit in one diagram that a newcomer understands in a quarter of an hour. A model whose diagram outgrows a page is two models. The same budget makes a model fit in the context of an AI assistant (Chapter 3).

2.8. One identity per module

Every module has one path in the repository, and every name it has is derived from that path.

Identity	Rule
JPMS module and root package	`org.nasdanika.sdk.runtime.` followed by the path with `/` as `.`, dropping a trailing `model` segment: `models/telemetry/model` is `org.nasdanika.sdk.runtime.models.telemetry`
Maven coordinates	Group `org.nasdanika.sdk.runtime` followed by the parent path, artifact the last segment: `org.nasdanika.sdk.runtime.models.telemetry:model`
Namespace URI of a model	The model's page: `runtime.sdk.nasdanika.org/models/telemetry`, and a child path for each further model in the module, such as `…/models/telemetry/traces`
Documentation page	`runtime.sdk.nasdanika.org/` followed by the path

One rule means any identity can be computed from the others, and someone who has seen one module knows where to find the next. It also removes name and version clashes with the older Nasdanika organizations by construction: nothing in the runtime shares a Maven group, a module name or a package with them, so the old and the new line can sit on one machine during a migration.

2.9. Pull, don't push

The runtime grows from a seed, one increment at a time, and modules from the older Nasdanika code base come in when an increment needs them, not before. Each increment has to state the capability it adds, the demonstration that shows it, what it pulls in, and that a clean clone builds.

Pulling a module is not moving it. Its core comes in as a small library, and each integration becomes a separate adapter that is pulled only when something needs it. `common` started empty: a helper enters it when a second module needs it and no JDK feature replaces it. That keeps `common` from becoming the place things go because they have nowhere else to go.

The same rule shapes how documentation and sites are produced. A site is not pushed out by a generator that knows every page; it is pulled from a model presented as files, by whatever reads it: a static copy, an HTTP server, a WebDAV mount, an agent (Chapter 15).

2.10. Local first, federated, no server

Everything the runtime does runs on a laptop, in a container and in a CI pipeline, as the same artifact. There is no central server and no administrator of the whole.

Models are files in Git and artifacts in Maven repositories. They reference each other by URI, the way a class references a class in another jar, and URI handlers load them transparently from wherever they are. A team owns its vocabulary and publishes it; another team references it without either coordinating a release. A published version is what it was, so "what did this mean last quarter" is an install, not an archaeology project.

This matters most where data cannot leave the building. Local telemetry, local models and local agents are the configuration that a regulated organization can approve.

2.11. Say what is stuck

A failure that does not say what failed is a mystery, and mysteries are expensive. The runtime prefers reports to guesses:

- The pump that drives capability resolution and transformations fails at once when nothing can progress, with a list of what was waiting for what, including the path of a dependency cycle (Chapter 9).
- A capability loader can be given a listener that explains, line by line, which factories were asked about a requirement, which declined, and what each produced (Chapter 10).
- Command wiring fails with the types a command needed and what its parent offered, and asking for help never fails on wiring (Chapter 17).

2.12. Humans at both ends

Draw software development as a line from intent to operation. AI is taking the middle of that line: writing code, writing tests, translating between formats. It pushes people out to both ends.

To the left is intent: deciding what things are, what words mean, who owns which fact. That is modeling, and it is why the runtime makes models cheap.

To the right is assurance: knowing what the system actually did. Assurance means checking what ran against what was intended, and that check is only as good as the evidence. Telemetry per unit of work answers "what did this command or this agent turn do?". A JVM snapshot answers "what actually runs?". Live observation answers "what is it doing right now?", for the person who has to decide whether to stop it.

When intent and evidence are both models, assurance is a query instead of a meeting.

2.13. Lineage

The runtime borrows freely, and saying from where helps to read it.

Idea	Borrowed from
Models as the API, reflection, change recording	EMF, without the IDE
Contributions found by declaration	Eclipse extension points, `java.util.ServiceLoader`, JPMS `provides`
Documentation contributed by modules	Eclipse Help, without the workbench
A file system interface with optional capabilities	Go's `io/fs`
Re-readable content	Guava's `ByteSource`; the JDK HTTP client's `BodyPublishers.ofInputStream`
Content written on demand	JAX-RS `StreamingOutput`
Blocking by pumping	`ForkJoinTask.join`, SWT's `readAndDispatch`, Kotlin's `runBlocking`

Idea	Borrowed from
A Published Language	Domain-Driven Design
Diagrams as views of a model	The C4 model, and "Beyond Diagrams"
Micro-models	Microservices and Java modules, applied to meaning
User interfaces chosen by the class of what they show	JFace content and label providers, EMF.Edit item providers, Naked Objects
Documentation anchored to model elements	Wikis, with the model as the site map

With the principles stated, the next chapter tells how the runtime is built, by one engineer and an AI assistant, and why that changes which technologies are practical.

Chapter 3. Built with an AI assistant

The runtime's repository was created on 29 September 2026. This edition describes it twenty-six days and 59 commits later. By then it held about 40,000 lines of Java and 24,000 lines of tests in 395 test cases, 8,800 lines of Xcore from which EMF generates about 449,000 lines of Java, and 2,600 lines of Groovy. Part of it was migrated from earlier Nasdanika repositories rather than written from scratch, and the migration was work too: new packages, new identities, new conventions.

One person did this work, with an AI coding assistant. This chapter is about how, because the how is part of the argument of the book. The runtime is built on technologies that most organizations have kept at arm's length for twenty years, for a reason that no longer holds.

3.1. Binding decisions

My favorite definition of software development comes from a talk by Ivar Jacobson that I heard around 2006. This is how I remember what he said: software development is the process of binding decisions to make them executable. A design starts as a space of open questions, and development binds them one by one, at design time, at build time, at deployment and at run time, until what is left can run.

The definition does not say who binds. For most of the history of software, people bound every decision, and compilers and builds checked that the bindings were consistent. An assistant now binds some of them: a name, a data structure, a library, a way of handling errors, the shape of an interface, and now and then one that matters a great deal. That is the real change, more than the speed, and it makes the traceability of binding paramount: which decision was bound, by whom or by what, in the course of what work, and on what evidence.

The runtime makes binding traceable at both times. During development, a decision is written down before the code that binds it, in the loop described later in this chapter; changes to models are recorded in the spans that made them (Chapter 13), and markers record where an element came from (Chapter 20). During execution, the runtime binds late and says how: a capability loader can explain which factories it asked about a requirement, which declined and what each produced (Chapter 10), command wiring that

fails names the types a command needed and what its parent offered (Chapter 17), and a JVM snapshot records how modules and services were bound when an assembly started (Chapter 22). Late binding without a record is magic; with a record, it is a decision that can be reviewed.

3.2. Who does what

The division of labor settled quickly, and it is worth stating plainly, because it is easy to get wrong in both directions: an assistant is neither an autocomplete nor a colleague who can be left alone.

Intent comes from experience. I decide what the runtime is for, which problems are worth solving, and in what order. That judgment comes from decades of building software and of watching what works and what does not, in projects of every size. The principles of Chapter 2 are such lessons, written down. Documentation is part of the model because I have watched documentation drift away from what it describes wherever the two were kept apart. The JVM model snapshots what runs because I have seen dependency trees that bore little relation to it. Telemetry is a model kept with the work because telemetry kept for a week cannot answer the question asked a year later.

Research comes from the assistant. Every design starts with a question: who has done this before, and what did they learn? The volume interface of Chapter 14 was compared with Go's `io/fs`, Python's fsspec, Visual Studio Code's file system provider, Hadoop's `FileSystem`, Apache Commons VFS and Eclipse EFS before a line of it was written. The polymorphic user interface of Chapter 24 was set against Naked Objects, EMF.Edit, JSON Forms and Django's admin. When I believed an idea was new, the research often found that it was not, which is better news than it sounds: an idea with twenty years of precedent is easier to adopt and to defend than a novelty, and the precedent shows where the pitfalls are. The assistant checks claims as well as designs: a claim about what a regulation requires was checked against the text of the regulation and narrowed in the draft.

Implementation comes from the assistant. Changes that would take a person weeks take hours. The pumped publisher, the transformer and the capability loader were rebuilt as one engine, with tests, in a day (Chapter 9). Telemetry per unit of work went into the

runtime's core classes, with tests, in another (Chapter 11). Mechanical work that is tedious and error-prone for people, such as renaming packages across modules, aligning identities, migrating a module from an older code base and regenerating models, is fast and consistent for an assistant.

Assurance stays with people, and with the build. I review designs, read diffs and decide. The build checks the rest: every module is tested, every listing in this book is compiled and tested, and a warning in the manuscript fails the build. Writing this book was also a review of the runtime: describing what the code does exposed where it did not do what it should. A command tree was assembled only in part, two families of dependencies were out of step, and names had drifted. Each became an editor's note in the first draft, and each was fixed in the next session.

The division is older than assistants: when I designed scientific devices, I drew them, my boss reviewed the drawings and a workshop built the devices as drawn; an assistant does the workshop's part, from intent and models instead of drawings.

The routing of work is older than software, too. When I bought my first PC, in 1996, my mother thought I was crazy. Then she saw an opportunity in it and built a writing business that fed our extended family for several years. Each writer in the family specialized in a field; my mother took the orders and routed each one to the writer whose field it was. She never called it orchestration, but that is what it was: one who understands the request and knows who is good at what, and specialists who do the work. Multi-agent systems are built the same way today, and the part my mother played is the part that stays with people.

3.3. One level up

Development with an assistant, as this chapter describes it, is the same activity as development without one, a level of abstraction higher: further to the left on the line from intent to operation (Chapter 2), closer to the why and the what than to the how.

Without an assistant I write code, and the compiler tells me where I got it wrong: a typo, a reference to a type that does not exist, a generic type that does not fit. In my C++ years I loved templates, and they were one of my nightmares: a small flaw in a design, and I spent hours sifting through pages of compiler errors to find it.

With an assistant I write intent, and the feedback is about the intent. The assistant tells me the prior art of an idea, how my idea differs from it, what the difference is worth, and what to watch for and improve. Like the compiler's, the feedback comes in minutes and costs little, so I can afford to be wrong early, at the level where being wrong is cheapest.

The feedback is not a proof, though. A compiler that accepts a program has checked it; an assistant that agrees with an idea has only not found a reason to disagree. So the decision stays with me, and the compiler and the tests still check everything below it.

3.4. The mainstream filter

For two decades EMF was hard to propose in most large organizations, and not because it did not work. Technology choices there pass through a filter that is rarely written down: will an average developer on an average team be able to work with it? A technology that fails the filter is "too complex", however much it would save. The mainstream framework, which in the Java world means Spring, passes by definition, so it wins by default. It is a fine framework; the problem was never the choice, but a filter that left room for only one.

The filter was rational. The cost of a technology includes the cost of everyone who has to understand it, and people fluent in EMF were few. The price of the mainstream choice was paid elsewhere, and quietly: in the effort of explaining what needed to be done, in reviewing whether it had been done right, and often in doing it yourself. It also produced judgments that sound odd side by side. Two hundred lines of Xcore were too complex; fifty thousand lines of AI-generated TypeScript were a great achievement. The first measured the cost of understanding; the second measured output.

This is not a criticism of developers. Fluency follows exposure, and organizations exposed their people to what the market used most. It is a description of an economy in which fluency was scarce.

I paid that price myself. The specifications of a code generator I built for a cloud migration (Prior art, first-hand) were EMF models saved as XMI. My teammates did not know EMF, so they edited the XMI in a text editor, and the generator worked anyway. The obstacle was not the model but the format it asked people to read. If the mountain will not come to Mohammed, Mohammed must go to the mountain: since then I keep models in formats people already read and write, YAML, JSON and Groovy scripts (Chapter 19), with EMF

behind them.

3.5. When fluency is not scarce

An assistant is fluent in Xcore, EMF, OpenTelemetry, the Java module system, picocli and Groovy, and in Spring and React as well. It does not tire of explaining, and it does not need things explained to it twice. The premise of the filter, that a technology must be within reach of every developer working unaided, loses its force when nobody works unaided.

Two questions take its place.

Is it the right abstraction? Mendeleev is said to have compared burning petroleum as fuel to heating a stove with banknotes: a valuable resource spent on low-value work. An assistant writing the boilerplate of a framework is the same trade. Its leverage compounds when it works a level or two higher: on models that generate the routine code, on languages that remove the need for it, on user interfaces that render themselves from the model. In this runtime, 8,800 lines of Xcore generate about 449,000 lines of Java, and people review the 8,800. Xcore is the concise, high-level representation; the generated Java is the verbose, low-level implementation. Xcore is declarative with occasional operations - much smaller human review surface and "attack surface" for errors.

Can the result be checked? What an assistant produces must be checkable, cheaply, by people and by machines. That is a design constraint, and it explains much of this book: typed models instead of maps of strings, generic types that the compiler checks (Chapter 8), declarations instead of registration code, tests for every module, listings that compile, telemetry kept as models (Chapter 13), and changes recorded in the spans that made them, so that what an agent did can be reviewed and reverted.

Understanding still matters; it moves. It moves from the mechanics of a framework, which the assistant carries, to the meaning of the model, which only the people who own the domain can judge. That is where the experienced engineer, the architect and the domain expert spend their time, and it is a better use of it.

Fluency can be borrowed from an assistant; judgment cannot. Judgment grows the way any expertise does, by what psychologists call deliberate practice: practice aimed at what one cannot do yet, with feedback, and often with a teacher who explains (Ericsson,

Krampe and Tesch-Römer, 1993). My kung fu teacher calls it eating bitter (Where the opinions come from), and nobody can eat it for you. What an assistant changes is the price of the practice, not the need for it. Its feedback comes in minutes (Section 3.3), and the hours it saves on the mechanics of a framework can go to practice at the level that matters: the model, and whether it is right.

3.6. Fall back to the keep

In *The Two Towers*, when the outer wall of Helm's Deep is breached, the defenders fall back to the keep of the Hornburg, the innermost and strongest part of the fortress; in the film the order is shouted along the wall: "Fall back to the keep!"

The analogy is not literal. Assistants are not the enemy; they are allies, and very capable ones. But the shape of the situation is similar. Assistants now hold most of the outer wall: they write code, tests and documentation faster than people can read them. What people keep is the place from which they can still check what was built, and the stronger that place, the more of the work can safely be given to the assistants.

I think the JVM is a better keep for people than Node.js. Its declarations are checked by a compiler and kept in the class files, so a person or a program can read what a part claims to provide and rely on the claim, and the runtime matches parts on those declarations with the compiler's rules (Chapter 8). And with JavaScript as a generation target (Chapter 23), falling back to the keep gives up nothing in the browser.

There is also the matter of where the systems already are. In my experience, in the enterprises I have worked in and with, Java has been at least twice as common as Node.js for systems in production, and about an order of magnitude as common as Python. I measured it once. Scanning the tens of thousands of source repositories of one bank (Prior art, first-hand), I counted about twice as many Java files as JavaScript files, with all of JavaScript's extensions counted, and about ten times as many as Python files. That is one organization, and it counts files rather than systems in production, so it is evidence, not research: I know of no public study that measures it, and developer surveys measure something else, the languages developers use, by which JavaScript and Python are ahead of Java. Both can be true at once, since many developers write some JavaScript and some Python, while the business logic (e.g. microservices) of large organizations, in my

experience, run disproportionately on the JVM - comparing to JavaScript and Python and excluding mainframes.

3.7. How the work flows

The work runs as a loop, and every turn of it leaves something written down.

1. **Intent.** A few paragraphs of what I want and why, often ending with a request for an assessment.
2. **Assessment.** The assistant agrees or pushes back, finds prior art, and names risks and alternatives. I decide.
3. **Design.** The decision is recorded: a design document, or a section of the roadmap saying where the work lands (Section 3.8).
4. **Increment.** Code, tests and documentation change together, and the build is green from a clean clone.
5. **Observation.** Using the result, and reading its telemetry, produces the next intent.

A few rules keep the loop honest, and they serve people and assistants alike:

- **One source of truth per fact**: one identity rule, one model, and generated code that is never edited by hand.
- **Everything is compiled and tested**, the listings of this book included.
- **Failures say what is stuck**, which helps an assistant find a cause as much as it helps a person.
- **Small models.** A micro-model, one concern in one module (Chapter 2), fits in an assistant's context as well as in a person's head.
- **Documentation anchored to the model.** A micro-wiki (Chapter 20) gives every model element a documentation file of its own, so people and assistants know which file to edit when they learn something about an element.
- **Status labels.** What is available, sketched or planned is marked, so that neither people nor assistants mistake a design for a feature.

3.8. Gradient ascent

The loop turns on a roadmap, and keeping the roadmap is one of the most valuable things the assistant does.

I have used Jira, and it is good at what it was made for: a unit of work with an owner, a status and a history. It is not good at what a project like this one needs most, which is ideas that depend on each other. A new idea rarely stands alone: it revises two earlier decisions, makes a third unnecessary and changes the order of the next five steps. A tracker records such links as "relates to", keeps the reasoning in comments, and orders the work by a rank field. What I need to know is not the rank of an item but the best next action: the step that brings the most value for the time and effort it costs, given everything already decided.

The runtime’s roadmap is a single Markdown document, started on 19 September 2026. A month later it held about 72,000 words, more than this book, in 32 dated sections, each written in answer to an intent of mine. A section has a regular shape: the decision, the assistant’s assessment of it, the prior art, what it revises in earlier sections, and where it lands in the order of the work. Nothing is rewritten silently. When a decision supersedes an earlier one, the earlier section stays and the later one says which of them wins, so the roadmap is also the history of how the plan changed and why.

In that sense the roadmap is a log, as a database keeps one: entries are only appended, a later entry overrides an earlier one, and what is true now is what the log says once all of it has been read. And like a log, it has to be compacted from time to time, for the same reason an assistant compacts a long conversation: the reader has a limited context, and the more the log grows, the more of that context goes to finding out what still holds. The first compaction came on 25 October, when the roadmap had reached about 89,000 words. A new document became the plan of record: the decisions still standing, one line each with a link to the section that argued it, the ranked list of next steps, and new dated sections after them. Unlike a conversation’s summary, this one lost nothing: the old document stays as it was, as the record of why each decision was made, and the new one starts where it ends.

The assistant maintains it. When I bring an idea, it reads the roadmap, places the idea among the decisions already made, says what the idea revises and what it makes unnecessary, and proposes where it lands and what to do next. I decide, and the decision

becomes a section. Kept by hand, a document like this would be a job of its own, and it would drift within weeks. Kept by an assistant, it costs minutes per idea and stays consistent, because the assistant reads all of it each time it is asked.

The order that comes out of it is a gradient ascent, the mirror image of the gradient descent that trains neural networks: from where the work stands, take the step that climbs steepest, the most value per unit of effort, then measure the slope again. Value includes what a step makes cheaper later, not only what it delivers now, and the slope changes after every step, because every step changes what the next one costs and what it is worth.

For its first nine days, the plan of record was a migration in one piece: move the models of the older Nasdanika code base into a new core, then build on it. On 28 September it became a seed crystal instead: start from one small working core and grow it one increment at a time, each adding a capability that a demonstration shows. An epic that would have delivered nothing until it ended, a slope of zero for weeks, became a climb that delivers something at every step, and code that no increment needed was never migrated, which settled the size of the migration better than any analysis would have. Four days later the route turned again: the North Star stayed, and the capabilities useful without language models, such as telemetry, volumes and the JVM model, moved ahead of the ones that need them, because they gave developers something to use now for a fraction of the effort.

Gradient methods have a known weakness: they climb the nearest hill, which need not be the highest. The North Star is the guard against it (Chapter 26): a step that climbs steeply in another direction is not admitted, and the rule for an increment is that test, written down.

The roadmap is the least formal artifact of the project, and deliberately so. It is where decisions are bound first, in prose, while binding them is still cheap to undo; designs, models and code bind them further and become formal as they do. What keeps the prose useful is not a schema but a reader that reads all of it every time.

3.9. What the assistant does not do

- **Decide what matters.** The North Star, the order of the work and the trade-offs are mine (Chapter 26).

- **Know what is not written down**: the constraints of an organization, the history behind a decision, the people involved. An assistant sees what it is given, which is why grounding it in models matters (Chapter 1).
- **Own the consequences.** A person signs off on what ships.
- **Be right every time.** An assistant can be confidently wrong, as people can. Types, tests, reviews and failing builds catch it, and the same habit of research that finds prior art also catches my mistakes: claims of novelty, a misremembered fact, a requirement read too broadly.

3.10. This book

This book was written the same way. I described what it should be and for whom; the assistant read the runtime's code, my stories and the roadmap, wrote the examples first and the prose around them, built the PDF and looked at its pages. I review, correct and decide. The examples are modules of the runtime's build, so the book cannot claim what the code does not do without the build saying so. In that sense the book is code: if an example does not build, neither does the book.

It is also small, measured the way an assistant measures text. This edition is about 56,000 words, listings included, or roughly 100,000 tokens: about a tenth of a context window of a million tokens. An assistant will rarely load all of it, and does not need to. It looks things up the way an engineer does: a term in the glossary, a convention in the rules of thumb, the chapter about the module it is changing, and the example module a listing comes from, which the manuscript names in each include. A book grounds an assistant well when it is small enough to hold whole and structured enough not to have to. The same holds for the books around it: this book is one of the Nasdanika Body of Knowledge, whose layout and reading order are the subject of Appendix F.

Writing the book reviewed the runtime, and reading it reviewed my thinking. Graham Wallas recorded a child's question in 1926, and E. M. Forster attributed it to an old lady a year later: "How can I tell what I think till I see what I say?" Working with an assistant gives the question a second reading: how can I tell what I think till I see what it makes of what I have said? Reading the drafts of this book, I kept having the same reaction: this is what I meant, and I never quite put it like that. For years I had written about what makes structure last, a story at a time, and an assistant put it in three words: small, formal,

composable (Chapter 1).

Recognition is a check, but a weak one, because a fluent summary can feel right without being right. A distillation earns its place when it can be tested, and this one can: take away any one of the three words and you get a failure I have seen many times.

The next chapter is about the organizations the runtime is built for: what an assistant amplifies in them, and why many of them will resist an approach like this one.

Chapter 4. Against the grain

The previous chapter is about how the runtime was built. This one is about the organizations it is built for, and why many of them will resist it. An assistant amplifies the way of working it is given: it makes a faster horse faster, and it lets people who never owned their code produce more of it. The sections below describe five patterns I have watched for years, with the names others have given them, and what the runtime does about each.

4.1. Faster horses

Steve Jobs liked to quote a line he credited to Henry Ford, although nobody has found it in anything Ford said or wrote: if he had asked his customers what they wanted, they would have said a faster horse. Developers given AI tools often ask for a faster horse. They speed up the tasks they know, such as boilerplate, mappings between formats and tests for code that should not exist, when the tasks themselves should be rethought. Modern trains do not have stokers with ergonomic shovels. They do not have stokers.

A smaller count is not a weaker system either. In the third presidential debate of 2012, answering the charge that the Navy had fewer ships than in 1916, Barack Obama said: "Well, Governor, we also have fewer horses and bayonets, because the nature of our military's changed." Lines of code, services and boxes on a diagram count the stokers, not the train. The question to ask before giving a task to an assistant is not how to do it faster but whether it should be done at all; in this runtime the answer is often a model and a generator (Chapter 7, Chapter 23).

4.2. Owning the result

An assistant raises the stakes of ownership. Somebody has to see the big picture and own the why, the how and the assurance of what ships, and with an assistant that somebody did not type most of it. Owning what you did not type takes evidence: what changed, in the course of what work, at whose request, and how to undo it. That is one of the reasons the runtime keeps telemetry as a model and records changes down to single attributes, in the spans that made them (Chapter 13): "the assistant did something" becomes a list of

changes to review and, if need be, to revert.

Tools do not create ownership where the organization of work does not. For decades much of the industry has separated deciding from doing: the people who write the code are not the people who decide why it is written, and neither answers for how it runs. Teams measured by tickets closed and staffed by the hour learn to stay inside their tickets. Given AI tools, they produce more code, faster, that they do not understand and do not want to own.

The same separation produces a pattern I have watched more than once. An architect with better AI tools than the team generates tens of thousands of lines in a language of their own choosing, with no regard for what the team knows or for what the organization uses for that kind of component: TypeScript, so that the user interface and the business logic share one code base, for a team of Spring developers in an organization that builds its server side in Java and Spring. There is a demo, a round of applause from the business,the code is handed over, and the architect moves on to the next team. The developers are left to work out what they now own, and to take the blame if it fails. The same pattern deploys a vector database for a similarity search over fifty vectors, which a loop over an array answers in microseconds. The database costs money and does nothing a loop would not, and the person who chose it does not pay for it; what they get is another box on a diagram and a line in the resume.

These patterns have names, which is some comfort: it is not only me who sees them. Nassim Taleb calls what the architect lacks skin in the game: making decisions without bearing their consequences. Choosing a technology for how it looks on a résumé, or on a diagram, rather than for the problem has been defined and measured as résumé-driven development (Fritzsch and others, 2021).

Where people do not contradict those above them, which students of culture call high power distance, an organization learns only what its leaders already believe. John Maxwell calls this the law of the lid: an organization rises no higher than the ability of the person who leads it (Maxwell, 1998). If the boss has mastered addition, whoever computes 1 + 2 + … + 99 + 100 as n(n + 1) / 2 will be flogged. Legend has the young Carl Friedrich Gauss do exactly that in a Brunswick schoolroom in the 1780s, writing 5050 on his slate while his classmates were still adding, under a schoolmaster known for his cane; that schoolmaster, at least, recognized the answer. To my surprise, I have crossed paths with

several people in the twenty-first century who epitomize the law. Nokia is a studied case: interviews with 76 of its managers and engineers found middle managers who feared their superiors and peers and kept bad news from the top, and top managers who, with a rosier view of the company's technology than the facts supported, underinvested in what came next (Vuori and Huy, 2016). Aviation met the problem in the cockpit and answered it with crew resource management, which trains first officers to challenge captains, and with checklists that ignore rank. The software equivalent is evidence anyone can query: a junior engineer who can point to the decision record and the trace does not have to contradict a superior. When intent and evidence are both models, assurance is a query instead of a meeting (Chapter 2).

None of this is a failing of the tools, or of people who work the way they were taught to. It is what happens when the people who bind decisions are not the people who live with them. The runtime cannot reorganize a company, but it can keep the binding visible and keep ownership where it belongs. Decisions are recorded before the code, changes are recorded in the spans that made them, and generated code is the output of models the owning team works with: the web models of Chapter 23 let a team of Java developers own a React application as Java code and models, without a second code base in TypeScript.

4.3. Stories are not source code

This section is speculation, and I label it as such.

Agile, as many large organizations practice it, works like a virtual machine. Stories are its instructions: small, uniform, prioritized, and executed by teams sprint after sprint. The machine compiles intent into running software, which is what an AI assistant does as well. What it does not keep is the source. Ron Jeffries described the card of a user story as "a token representing the requirement", with the requirement itself carried by conversation; in the machine, the card is all there is. The why, the vocabulary and the whole that the stories are parts of are in nobody's repository.

Armies, which nobody accuses of loose hierarchy, settled this long ago. In the combat regulations I was trained on (Where the opinions come from), the first duty of a platoon commander who receives a mission is to understand it: the task of the company as well as the platoon's own, the tasks of the neighbors and how to coordinate with them, and the

time to be ready. The US Army asks the same of its platoon leaders, who must understand the situation and the commander's intent one and two levels above their own (Field Manual 3-21.8, 2007). A story card carries the platoon's task and leaves out the level above and the neighbors.

That has always been the tension between architecture and agile as practiced: architecture is about the source, and the machine ships only bytecode. I am not alone in seeing the machine: Martin Fowler named it the Agile Industrial Complex, which imposes process on teams in the name of agility, and John Cutler described teams that crank out features without measuring their effect as a feature factory.

AI assistance as this book describes it pulls toward the source: intent and models at the left end of the line (Chapter 2), compiled by an assistant and checked by the build. A machine built to turn stories into code competes with the assistant at what the assistant does best, and I expect the tension to grow. None of this is an argument against agility. The Agile Manifesto values individuals and interactions over processes and tools; much of what is practiced in its name inverts that.

4.4. When a question becomes answerable

Most of what I describe in Prior art, first-hand made a question answerable that had not been: which processes are dead, which packages tie a monolith into a knot, who owns an application, on what grounds a technology was chosen. While a question cannot be answered, nobody is accountable for the answer. Once it can be, somebody is. An answer arrives with three costs that have little to do with the technology that produced it, and knowing them in advance saves disappointment.

It moves decisions. In 2008 I built a model for choosing technologies with the Analytic Hierarchy Process: a hierarchy of criteria, several experts weighted by their expertise in each branch of it, alternatives compared in pairs, and the consistency of each expert computed. It worked, and it went no further, because using it would have changed who decides and how. That is an old pattern. Simple formal methods have been matching or beating unaided expert judgment since Paul Meehl reviewed the evidence in 1954, and experts have resisted them for as long; Kahneman, Sibony and Sunstein retell the history in *Noise* (2021). Organizations also gather information they do not use, and use

information to justify decisions already made (Feldman and March, 1981), and a formal procedure can be kept as a ceremony, apart from the work it describes (Meyer and Rowan, 1977). I have taken part in more than one technology evaluation whose result was known on the first day.

It shows what was not visible. The metaphor I use is glasses. A nearsighted person complains that they cannot see well enough to do what needs to be done. Given glasses, they see every detail, and with it everything that needs doing, and they may well put the glasses away. Economists call it information avoidance: people avoid information that would oblige them to act, or that they expect to dislike, even when it is free (Golman, Hagmann and Loewenstein, 2017).

It changes the work. Seeing more means more to fix, and generating what was written by hand removes work that people were hired, and teams were sized, to do. On one project there was a concern that we did not have enough people to build all the integration processes we needed in time. I proposed generating the starter processes from the same metadata that already generated our codecs; the answer was that the people were already allocated. Nobody was being unreasonable: a plan was in place, and in many organizations a manager's standing is measured by the size of the team, the bureaucrat William Niskanen modeled as maximizing a budget (Niskanen, 1971).

When I told my wife about such episodes, she used to say: everybody is busy, everybody gets paid, the economy works. She is right, and it is the most important fact about them: the arrangement is stable. The runtime cannot take these costs away. What it can do is make answering a question cheap enough for one team to answer its own, for its own use, before anybody else has to agree to anything.



Confirm with my wife that she is comfortable being quoted. Check the citations against the sources: Meehl (1954), *Noise* (2021), Feldman and March (1981), Meyer and Rowan (1977), Golman, Hagmann and Loewenstein (2017), Niskanen (1971).

4.5. Failing conventionally

A traditional enterprise will most likely reject an approach like this one, and not because it does not work. It goes against the grain of how the enterprise is organized in the middle, where survival pressure is low and everybody does the same thing. The mainstream filter of Chapter 3 is one form of it. Keynes saw it in investors in 1936: "Worldly wisdom teaches that it is better for reputation to fail conventionally than to succeed unconventionally." Information technology has its own version: nobody ever got fired for buying IBM, a saying in print since at least 1978. Sociologists call the result institutional isomorphism: organizations in a field come to resemble each other, and under uncertainty they copy whoever looks successful (DiMaggio and Powell, 1983). Biologists would call it an evolutionarily stable strategy: once most of a population follows it, a rare alternative cannot displace it, because the alternative does worse while it is rare, even if a population that followed it would do better.

I cannot change the payoffs of an organization, so the runtime was designed to pay off while it is rare. Its foundation is the JDK, EMF and the OpenTelemetry API, which a platform team can approve in a meeting (Chapter 2). It needs no server and no platform, and one team can take one module without anybody else adopting anything (Chapter 1). A strategy that pays off only when everybody adopts it cannot invade; one that pays off for a single team can.

The next chapter puts the runtime on one page: its modules, how they depend on each other, and what happens when a unit of work runs.

Chapter 5. The runtime at a glance

This chapter puts the whole runtime on a few pages: the modules and what each is for, the three planes they serve, how the pieces find each other, and what happens, step by step, when one command runs. Later chapters zoom into each part; this one is the map you come back to.

5.1. The modules

The runtime is one Maven reactor in one Git repository, released as one unit with one calendar version. Table 1 lists its modules; Appendix A has the full reference with every service each one provides.

Table 1. The modules of the runtime

Path	What it is for	Depends on, beyond the foundation
`common`	Resource sets and their contributors, generic type matching and adaptation, `Flow` utilities, the pump and the transformer, the capability framework, telemetry	geantyref
`volume`	The file system SPI: volumes, content, capabilities; serving volumes over HTTP and WebDAV; volumes as EMF URI handlers	`common`
`harness`	An HTTP server that is up while code runs against it, in one trace with the code's requests	`volume`
`playwright`	Browser walk-throughs against a harness server, in its trace, with screenshots, narration and video	`harness`, Playwright
`emf-json`	EMF resources as JSON and YAML, instrumented	`common`, EMF JSON (Jackson)
`drawio`	Draw.io documents in Java: pages, layers, nodes, connections, properties, links, styles	`common`, JSON-java, jsoup, Apache Commons

Path	What it is for	Depends on, beyond the foundation
`drawio-playwright`	Draw.io pages rendered with Draw.io's own viewer, as PNG and SVG images and videos	`drawio`, `playwright`
`groovy`	A metamodel-driven Groovy DSL for authoring any EMF model	`common`, Groovy
`picocli`	picocli, bundled until a release with Nasdanika's contributions	none
`cli`	Commands as capabilities, wiring by generic type, mix-ins, the shell	`common`, `picocli`, JLine
`cli-launcher`	The launcher: one OpenTelemetry SDK per launch, telemetry saved as models, distribution building	`cli`, the OpenTelemetry SDK, the telemetry exporters
`bootstrap`	A dependency-free jar that starts a distribution from any directory	none (only `java.base`)
`models/core`	Identity, documentation as structure, provenance markers, evaluators	none
`models/meta`	A language-neutral type system: packages, classes, features, operations, with a text syntax	`models/core`
`models/jvm`	What a Java virtual machine can name: layers, modules, packages, types, members	`models/core`
`models/file`	Files and directories as a model over volumes (**SKETCH**)	`volume`
`models/telemetry`	Traces, logs and metrics as models, and change recording; exporters from the OpenTelemetry SDK	the OpenTelemetry SDK (exporters only)
`models/html`	HTML and Bootstrap as models, with generators	`models/core`
`models/typescript`	The TypeScript declaration surface as a model, with an emitter that manages imports	`models/core`, `volume`
`models/react`	JSX and React components, React Bootstrap and React Flow as models	`models/typescript`, `models/html`

Path	What it is for	Depends on, beyond the foundation
`models/diagram`	What diagram notations have in common, and its layout with ELK (`models/diagram/elk`)	`models/core`; `common` and ELK for layout
`models/presentation`	Decks and slides as diagrams (**SKETCH**)	`models/diagram`, `common`
`models/drawio`	Draw.io documents as a model, loaded from and saved to `.drawio` files and images	`models/presentation`, `drawio`
`models/svg`	A subset of SVG as a model, for generated diagrams	`common`
`bom`, `all`	A bill of materials for consumers, and an aggregate of everything	
`tests`	Tests that need several modules, or the OpenTelemetry SDK, and a CLI assembly for demos	
`doc`	This book and its examples	

Figure 1 shows the same modules as a dependency graph. The foundation is not drawn: every module may use the JDK, EMF core and the OpenTelemetry API.

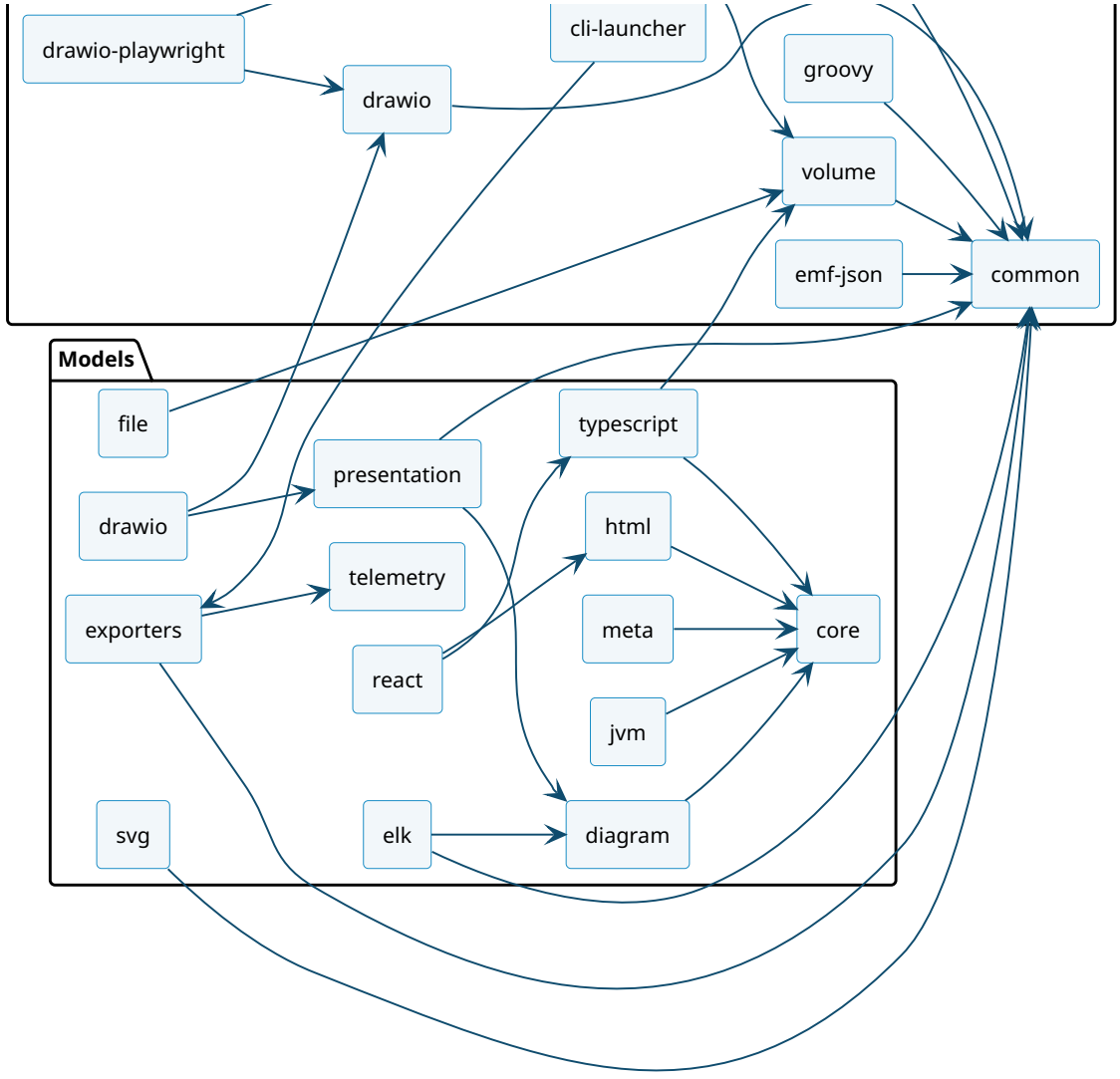


Figure 1. Module dependencies

Two things stand out. `common` sits under almost everything, which is why it is held to the strictest rule about what it may depend on. And most models do not depend on `common` at all: a model is EMF, and anything else is somebody else's concern. The exceptions need something only the runtime has: volumes, for the file and TypeScript models; telemetry, for the exporters and for layout with ELK; `DerivedSubsetEList`, for the slides of a presentation; and the Draw.io API and capabilities, for the resource factories of the Draw.io and SVG models.

5.2. Three planes

It helps to sort what the runtime does into three planes, with two seams between them.

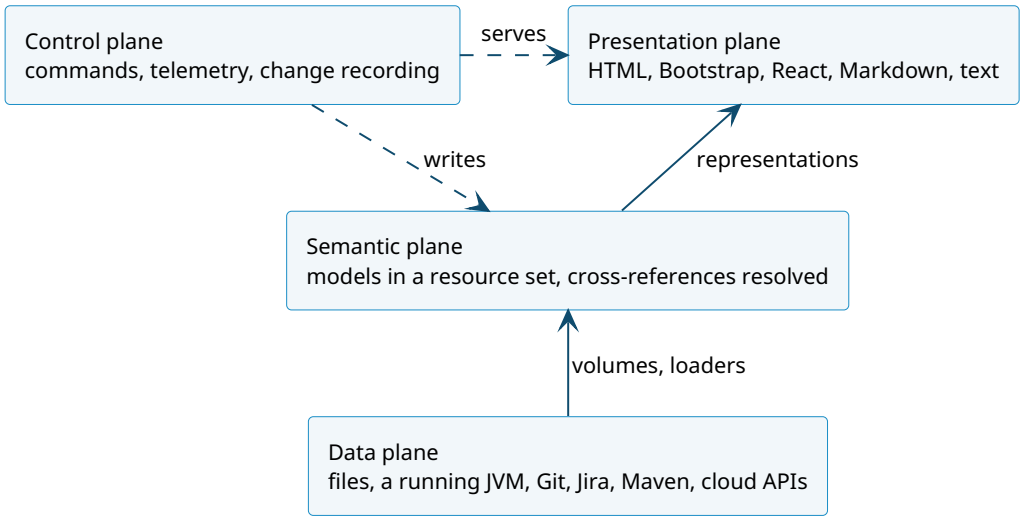


Figure 2. Three planes and the volumes between them

The **data plane** is raw data where it lives: files on disk, a running JVM, a Git repository, Jira, a Maven repository, a cloud provider's API. The **semantic plane** is models loaded from that data, typed, with cross-references resolved, in a resource set. The **presentation plane** is how people and programs see the models: HTML pages, a React UI, Markdown for an agent, plain text for a console.

Reads flow up. A volume presents data as files, and EMF loads resources from those files through a URI handler; or a loader reads an API and builds a model directly. A representation turns model objects into pages or files, and another volume presents those for serving, copying or mounting. Writes flow down, through commands, with telemetry and change recording on the way.

The planes explain why the runtime has volumes at all. A volume is the seam between the data plane and the semantic plane, and again between the semantic plane and channels such as HTTP, WebDAV or a directory. The same five-method interface serves both seams (Chapter 14).

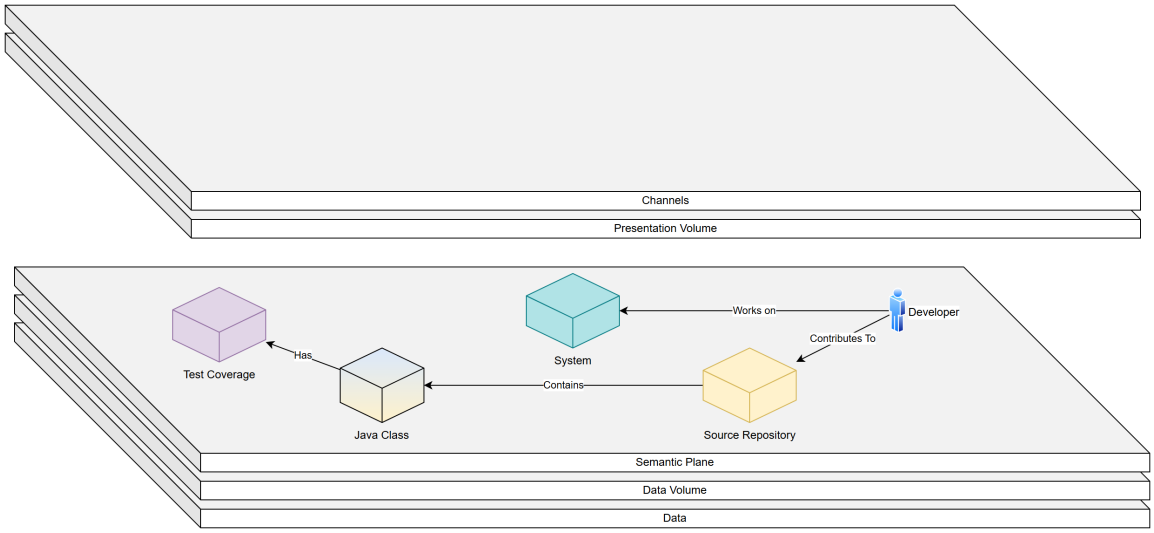


Figure 3. Another view of the planes and volumes

An example

This example is partly built. The Jira model exists, and I have used it to snapshot Jira and give the snapshots to an AI assistant, but that was before the volume module existed. A Jira volume does not exist yet; it will be built on demand, and should be straightforward.

Take a Jira story, and suppose a Jira volume presents it at `jira/MYPROJECT/issues/ISSUE-123/index.json` as JSON, and at `index.xmi` next to it as XMI. Neither file is stored anywhere: reading either one makes a REST call to Jira and materializes the issue in that format. Related objects, such as the project, its components, users and linked issues, are relative URI references, and EMF loads their resources on demand, when a reference is followed.

Because Jira is a volume, its content can be copied to a directory or a zip archive like any other files. Take a snapshot today and another in a month, load both, and compare them with EMF Compare to see what has changed.

Because the issue is an EMF object in the semantic plane, it can reference semantic elements that did not come from Jira: a Git commit, a file on disk, or an object from any other system that is also loaded as a model.

Going up, the same issue has representations in different formats: an HTML page, a Markdown or AsciiDoc document, a diagram, or any other format the presentation plane

supports. Those representations reach people and agents over different channels: a web browser, a file system mounted over WebDAV, or a directory they are copied to. For example, copy the AsciiDoc representations to a directory, then generate a PDF from them or give them to an AI assistant for analysis or enrichment. AsciiDoc carries more structure than plain text or Markdown, such as document attributes, cross-references and typed blocks, and it might be better suited to AI-assisted analysis and enrichment.

Who the planes are for

The planes say where things are, but not why the presentation plane has so many formats: HTML and Markdown, SVG and Draw.io, a PDF and a JSON file. The answer is a vocabulary older than the runtime, from ISO/IEC/IEEE 42010 and TOGAF, which the Nasdanika enterprise model puts in one diagram. Figure 4 is a lighter version of it.

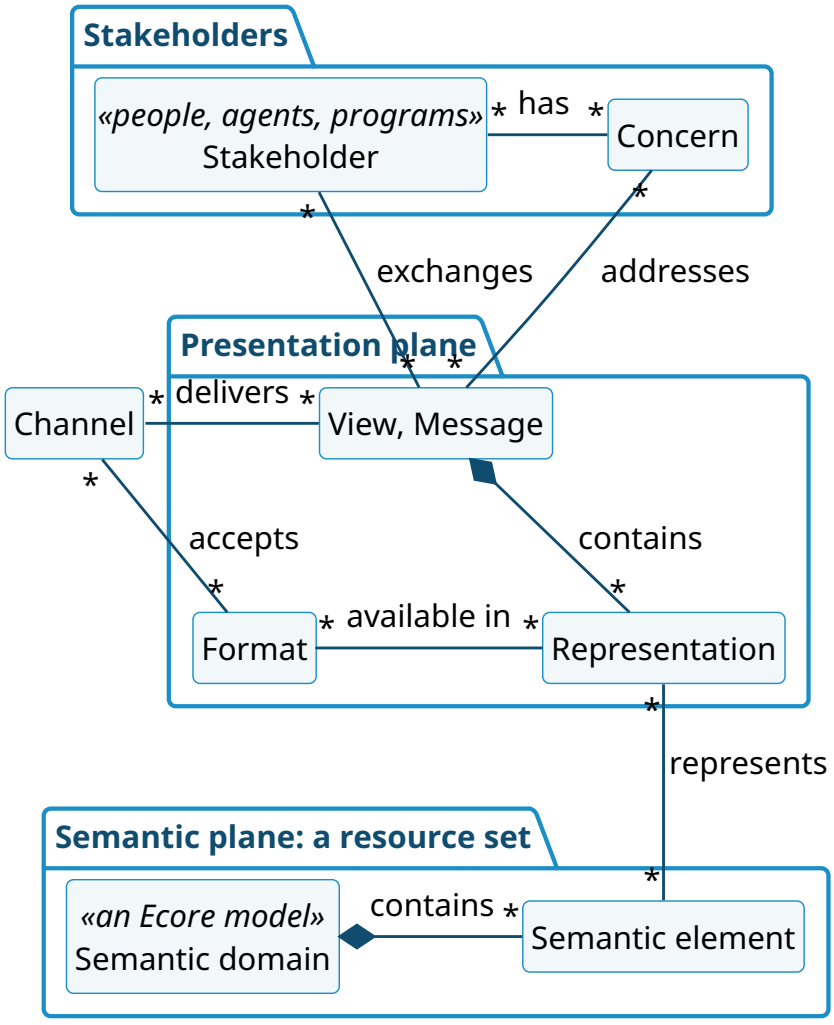


Figure 4. Stakeholders, views and the planes

A **stakeholder** is anyone with an interest in a system: a business owner, a risk officer, an engineer, and also an agent or a program. Stakeholders have **concerns**, the things they need to know or decide. A **view** addresses concerns. It is made of **representations**, such as a section, a table or a diagram, and a representation **represents** elements of the semantic plane: objects in the models of several semantic domains, many Ecore models in one resource set. A view sent to stakeholders is a **message**, delivered over **channels**: a directory, HTTP, WebDAV, MCP, a printed book. One view goes over many channels, and a channel carries many views.

Formats sit between representations and channels. A representation is available in several

formats: a table as Markdown, AsciiDoc or HTML, a diagram as SVG or Draw.io. A channel accepts some formats and not others: a browser takes HTML, an agent takes Markdown, a printer takes PDF. A human stakeholder who wants a table gets it as HTML in a browser or in a PDF; an agent gets the same table as Markdown.

The enterprise model defines an enterprise as a group of stakeholders and resources devoted to a mission (Chapter 1). Views exchanged as messages are how the mission is pursued, which gives two measures: effectiveness, whether a view addresses the concern it is for, and efficiency, what it costs to produce, deliver and understand. Formats are about efficiency. That is why the runtime has a format for every reader. SVG is for people reading a page or a book: it is static, embeddable and needs no viewer. Draw.io is for people who edit, because the picture is also its source. Markdown, or the model itself, is for agents. Each format serves some stakeholder over some channel.

The vocabulary also says what a document is: a view whose sections are representations. A document generated from a model and a document written by hand are the same kind of thing, and a Markdown file loaded as a model is a view that can be rendered again, in another format, for another stakeholder. And it describes the polymorphic UI (Chapter 24) from the outside: given a viewpoint, which names the concerns to address and the representations that address them, a format and a channel, it builds a view. Viewpoints and stakeholders can be kept in catalogs, like any other model: a Sankey chart for "where does the money go", a property table for "who owns what"; business, risk and technology stakeholders.

It is also the first model an effort needs. A project charter names the stakeholders, a communications plan says what each of them receives and how, and a quality management system, such as the one Article 17 of the EU AI Act requires, covers communication with authorities, customers and other interested parties.

The runtime's design follows this vocabulary, but the enterprise model is not part of the runtime yet. It is **PLANNED** as a nano-model of the core module (Chapter 20), so that stakeholders, concerns, formats and channels become types, and a view can be asked for by them. It is introduced here because it explains much of what follows.

5.3. How the pieces find each other

Nothing in the runtime is wired by hand, and nothing is configured in XML or YAML. Four mechanisms, from the simplest to the most capable, connect the parts.

Java services. A module declares in `module-info.java` that it provides an implementation of a service interface, and `java.util.ServiceLoader` finds every provider on the module path. Resource set configuration uses services: EPackages, URI handlers, adapter factories and `ResourceSetContributor`s (Chapter 7). So does the capability framework itself, to find capability factories.

Capabilities. A capability factory is a service that interprets requirements. A requirement is a type, possibly generic, with optional payloads: "a `Function<Library, String>` for the markdown format". Factories may raise their own requirements while satisfying one, and the loader resolves the whole graph, once per requirement, with cycle detection (Chapter 10).

Adaptation. An object can declare, with annotated fields and methods, what it can be adapted to, by generic type and with optional context. Adaptation answers "can this thing give me what I need?" without either side knowing the other's class (Chapter 8).

Wiring. A command declares, by generic type, what it needs from its parent and which sub-commands it takes. Commands contributed as capabilities are matched to parents by those declarations, and the parent is adapted to what the child needs just before it runs (Chapter 17).

Each mechanism builds on the previous ones. Commands are capabilities; capability factories are services; wiring is adaptation.

5.4. One command, end to end

To see the parts work together, follow one command through a distribution: `nsdk library save shelf.xmi`, which saves the library the `library` command supplies to a file. Figure 5 shows the main steps.

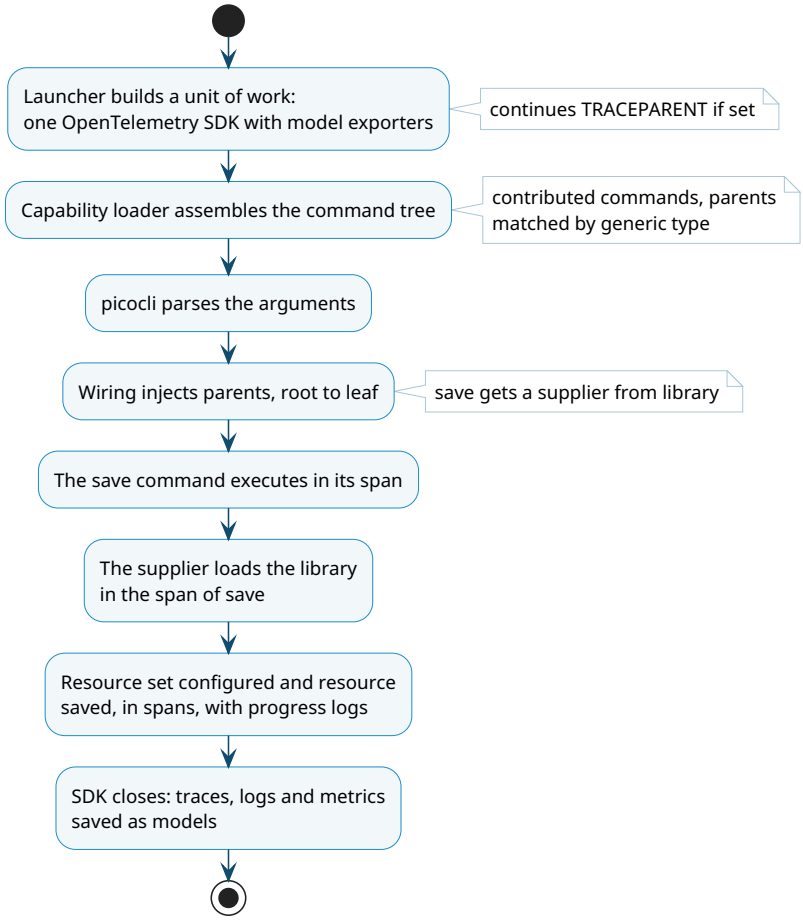


Figure 5. One command, end to end

1. **The launcher builds a unit of work.** It creates a directory for this launch's telemetry, builds an OpenTelemetry SDK whose exporters write traces, logs and metrics as models, and makes it current. If a `TRACEPARENT` environment variable is set, by a CI step for example, the launch continues that trace. Everything that follows reports to this instance and to no other (Chapter 18).
2. **The capability loader assembles the command tree.** It asks for root commands; the root command's factory produces `nsdk`, and every command factory on the module path is asked whether its command fits under `nsdk`, then under each command added so far. The `library` command fits under `nsdk` because it declares the root command as its parent. The runtime's `save` command fits under `library` because it declares that its parent must supply a collection of EMF objects, and `library` declares an adapter that does (Chapter 16). Each command arrives with its own sub-commands and mix-ins

attached, so what the loader returns is the whole tree.

3. **picocli parses the arguments** against that tree and finds `nsdk library save` with one positional parameter.
4. **Wiring injects parents.** After parsing, from the root to the executed command, each command's `@Parent` members are filled by adapting its parent. `save` receives a supplier from `library`; nothing is loaded yet (Chapter 17).
5. **The command executes in its own span.** `save` calls the supplier, which loads or builds the library inside `save's span, so the cost is attributed to the command that needed it. It creates a resource set, which is configured from services in a span with a log record per contribution, and saves the resource in a span with the byte count, while the URI handler logs progress for large streams (Chapter 11).
6. **The unit of work ends.** Closing the SDK exports what is pending, and the exporters save `traces.yml`, `logs.yml` and `metrics.yml` in the launch's telemetry directory, with log records nested in the spans they were emitted in (Chapter 13).

Nothing in that sequence was configured. The commands are present because their modules are on the module path, the wiring follows from their declared types, and the telemetry is there because the runtime's code reports through the OpenTelemetry API to whatever instance is current.

5.5. Where things are in the repository

```
runtime/
  pom.xml              the reactor; every module below is listed in it
  bom/                 versions for consumers
  common/  volume/  harness/  playwright/  emf-json/  groovy/  picocli/  cli/
  cli-launcher/  bootstrap/  drawio/  drawio-playwright/
  models/
    core/  meta/  jvm/  file/  telemetry/  html/  typescript/  react/
    diagram/  presentation/  drawio/  svg/
  tests/               multi-module tests and a demo CLI assembly
  doc/
    book/
      examples/        one Maven module per topic, built and tested with the runtime
      manuscript/      this book, in AsciiDoc, rendered to PDF, EPUB and HTML
```

Each model module keeps its Xcore source under `src/main/resources`, its generated Java under `src-gen` (committed, so that editors without Maven integration see it), and its hand-

written code under `src/main/java`.

5.6. What is available today

The capabilities in this book are at three stages, and every chapter labels them. In short, as of 2026.11.0:

AVAILABLE

Resource sets configured from services, all resource formats, generic type matching and adaptation, the flow utilities, the pump and the transformer, the capability framework, runtime telemetry with live publishing, telemetry and change recording as models, volumes over directories with HTTP and WebDAV serving, commands as capabilities with generic wiring, the launcher and the bootstrap, the Groovy DSL, the core, Meta, JVM, telemetry, HTML, Bootstrap, TypeScript and React models with their generators, Meta's text syntax, validator, glossary and agent skill, the harness and browser walk-throughs with video, the Draw.io API and rendering with Draw.io's viewer, the diagram, Draw.io and SVG models, and layout with ELK.

SKETCH

The file model; the presentation model; the reflective operations of Meta's `Object`; Meta's facets, recorded as annotations until the core model has them; the JVM model's loaders (the book shows a minimal one).

PLANNED

Everything in Chapter 26: the polymorphic UI in React, representations and views over mounted volumes, the inference and tool models, MCP, query languages, executable graphs with state in Git.

Part II: Foundations

The `common` module and the habits the rest of the runtime is built on: models without an IDE, generic types matched at run time, reactive flows on the JDK, and the capability framework that finds what code needs.

Chapter 6. Getting started

This chapter gets the runtime onto your machine, builds a first program against it, and shows that the program is observable before you have done anything to make it so. The program is small on purpose: one metamodel with one class, one instance, three files. Everything else in the book builds on the same few steps.

6.1. What you need

- **A JDK for Java 25.** The runtime is compiled with `--release 25`. Any distribution works; the examples were built with Eclipse Temurin.
- **Maven 3.9.16 or later.** The build enforces the version.
- **Git**, to get the source.
- An editor. The runtime does not need an IDE, and nothing in it is specific to one. Eclipse, IntelliJ IDEA and VS Code all import the Maven projects as they are.

6.2. Building the runtime

Until the runtime is published to Maven Central, you build it and install it into your local Maven repository:

```
git clone https://github.com/Nasdanika-SDK/runtime.git
cd runtime
mvn clean install
```

The first build downloads the dependencies and runs every module's tests, including the book's examples, and renders this book. Two properties shorten it:

```
mvn clean install -DskipTests              # compile and install, run no tests
mvn clean install -Dasciidoctor.skip=true  # everything but rendering the book
```



Update this section when 2026.11.0 is on Maven Central: readers then need only the BOM import shown below, and building from source becomes the section for contributors.

6.3. A first project

A project that uses the runtime imports its bill of materials, which manages the versions of the runtime's modules and of the libraries they use, and then depends on the modules it needs without versions. The examples share the import in their parent `pom.xml`:

Listing 1. Importing the runtime's bill of materials (`doc/book/examples/pom.xml`)

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.nasdanika.sdk.runtime</groupId>
            <artifactId>bom</artifactId>
            <version>${project.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

The first example depends on two modules: `common`, which brings EMF and the OpenTelemetry API with it, and `emf-json`, which adds the JSON and YAML formats.

Listing 2. Dependencies of the first example (`start/pom.xml`)

```
<dependencies>
    <dependency>
        <groupId>org.nasdanika.sdk.runtime</groupId>
        <artifactId>common</artifactId>
    </dependency>
    <dependency>
        <groupId>org.nasdanika.sdk.runtime</groupId>
        <artifactId>emf-json</artifactId>
    </dependency>
</dependencies>
```

Every runtime module is a named JPMS module, and so is every example. The module declaration says which runtime modules the code uses:

Listing 3. The module declaration (`start/src/main/java/module-info.java`)

```
module org.example.start {

    requires org.nasdanika.sdk.runtime.common;

    // JSON and YAML resource factories. Service binding would resolve the module
    // anyway, because it provides a service common uses; requiring it says so.
```

```
    requires org.nasdanika.sdk.runtime.emf.json;

    exports org.example.start;

}
```

The comment in the listing points at something worth knowing early. `emf-json` contributes its resource factories as a service that `common` uses. When the JVM resolves modules, it adds every observable module that provides a service a resolved module uses, a step called service binding, so `emf-json` would be in the running program even without the `requires`. I require it anyway, because a dependency that is visible in the declaration is one less surprise. Service binding comes back in Chapter 22, where it is one of the things a snapshot of a running JVM shows and no build file does.

6.4. A first model

EMF models are usually generated from Xcore, which Chapter 7 covers. The first program builds a metamodel at run time instead, which needs no generator and shows what a metamodel is: a package, containing classes, containing typed features.

Listing 4. A one-class metamodel built at run time (`start/⋯/FirstModel.java`)

```
/**
 * A one-class metamodel: a book with a title and a year.
 */
public static EPackage libraryPackage() {
    EcoreFactory ecore = EcoreFactory.eINSTANCE;

    EPackage library = ecore.createEPackage();
    library.setName("library");
    library.setNsURI("https://example.org/start/library");
    library.setNsPrefix("library");

    EClass book = ecore.createEClass();
    book.setName("Book");
    library.getEClassifiers().add(book);

    EAttribute title = ecore.createEAttribute();
    title.setName("title");
    title.setEType(EcorePackage.Literals.ESTRING);
    book.getEStructuralFeatures().add(title);

    EAttribute year = ecore.createEAttribute();
    year.setName("year");
    year.setEType(EcorePackage.Literals.EINT);
```

```
    book.getEStructuralFeatures().add(year);

    // A package lives in a resource whose URI is its namespace URI, so that instances
    // reference their class as https://example.org/start/library#//Book
    new ResourceImpl(URI.createURI(library.getNsURI())).getContents().add(library);
    return library;
}
```

The last statement puts the package into a resource whose URI is its namespace URI. That is how EMF expects packages to live, and it matters as soon as instances are saved: an instance refers to its class by the URI of the class, and a package that is in no resource produces a relative reference that cannot be resolved when the file is loaded again.

With a metamodel in hand, the program creates an instance and saves it three times:

Listing 5. Saving one book in three formats

```
/**
 * Saves one book as XMI, JSON and YAML.
 *
 * @return The saved files
 */
public static List<Path> save(EPackage library, Path directory) throws IOException {
    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure(); ①
    resourceSet.getPackageRegistry().put(library.getNsURI(), library);   ②

    EClass bookClass = (EClass) library.getEClassifier("Book");
    EObject book = EcoreUtil.create(bookClass);                           ③
    book.eSet(bookClass.getEStructuralFeature("title"),
            "Structure and Interpretation of Computer Programs");
    book.eSet(bookClass.getEStructuralFeature("year"), 1985);

    List<Path> files = new java.util.ArrayList<>();
    for (String extension: List.of("xmi", "json", "yml")) {               ④
        Path file = directory.resolve("book." + extension);
        Resource resource = resourceSet.createResource(URI.createFileURI(file.toString()));
        resource.getContents().add(EcoreUtil.copy(book));
        resource.save(null);
        files.add(file);
    }
    return files;
}
```

① A resource set configured from services: every `EPackage`, URI handler and `ResourceSetContributor` on the module path is applied to it. This is the one line that makes everything else in the runtime available to EMF.

② Packages registered as services are found by namespace URI. This one was built at run

time, so it is registered by hand.

③ `EcoreUtil.create` makes an instance of a class without generated code. Features are set and read reflectively.

④ The file extension selects the resource factory: `xmi` is the default, `json` and `yml` were contributed by `emf-json`.

The three files hold the same object:

Listing 6. book.xmi

```
<?xml version="1.0" encoding="ASCII"?>
<library:Book xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI"
    xmlns:library="https://example.org/start/library"
    title="Structure and Interpretation of Computer Programs" year="1985"/>
```

Listing 7. book.json

```
{
  "eClass" : "https://example.org/start/library#//Book",
  "title" : "Structure and Interpretation of Computer Programs",
  "year" : 1985
}
```

Listing 8. book.yml

```
---
eClass: "https://example.org/start/library#//Book"
title: "Structure and Interpretation of Computer Programs"
year: 1985
```

Loading is the mirror image: a resource set, the package registered, and `getResource` with the file's URI.

Listing 9. Loading a book and reading its title reflectively

```
/**
 * Loads a book saved by {@link #save(EPackage, Path)} and reads its title reflectively.
 */
public static String title(EPackage library, Path file) {
    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
    resourceSet.getPackageRegistry().put(library.getNsURI(), library);
    Resource resource = resourceSet.getResource(URI.createFileURI(file.toString()), true);
    EObject book = resource.getContents().getFirst();
    EStructuralFeature title = book.eClass().getEStructuralFeature("title");
    return (String) book.eGet(title);
```

```
}
```

That is a complete EMF program: a metamodel, an instance, three formats, and reflection, with no generated code and no configuration.

6.5. Watching it run

The program above contains no telemetry code, and it is observable anyway. The runtime's resource set configuration, resource factories and URI handlers report what they do through the OpenTelemetry API, to whichever `OpenTelemetry` instance the unit of work made current. To see it, make one current that publishes what it receives, and subscribe:

Listing 10. Watching the first program (`start/⋯/WatchIt.java`)

```
public static void watch(PrintStream out) throws IOException {
    PublishingTracerProvider tracerProvider = new PublishingTracerProvider(
            TracerProvider.noop(), ①
            Runnable::run,         ②
            1024);
    try (PublishingOpenTelemetry openTelemetry =
            new PublishingOpenTelemetry(OpenTelemetry.noop(), tracerProvider)) {
        openTelemetry.subscribe(new Printer(out));                    ③
        try (Scope scope = Telemetry.makeCurrent(openTelemetry)) {    ④
            Path directory = Files.createTempDirectory("watch-it");
            Telemetry.inSpan(                                          ⑤
                    Telemetry.tracer(openTelemetry),
                    "Save a book",
                    null,
                    span -> FirstModel.save(FirstModel.libraryPackage(), directory));
        }
    }
}
```

① The tracer provider the events come from wraps the no-op provider: no OpenTelemetry SDK is involved.

② `Runnable::run` delivers each event on the thread that produced it. That makes the output deterministic, which suits a console and a test. The default delivers on virtual threads instead.

③ A subscriber that prints span starts, span ends and log records, indented by nesting. It is twenty lines; the listing is in the example.

④ The instance is current until the scope closes. Everything instrumented in between reports to it.

⑤ The program's own work runs in a span named for what it does, so that the runtime's spans nest under it.

The output, with the long class and file names shortened:

```
> Save a book
  > ResourceSetContributor.configure
    Contributed uri-handler ...ClassPathURIHandlerFactory$1
    Contributed uri-handler ...ModuleURIHandlerFactory$1
    Contributed contributor ...BinaryResourceFactoryContributor
    Contributed contributor ...GzipBinaryResourceFactoryContributor
    Contributed contributor ...XMIResourceFactoryContributor
    > Contribute To Resource Set
    < Contribute To Resource Set
    Contributed contributor ...JsonResourceFactoryContributor
    ...
  < ResourceSetContributor.configure
  Created ...XMIResourceImpl for file:/tmp/watch-it/book.xmi
  > URIHandler.write
  > Resource.save
    Saved file:/tmp/watch-it/book.xmi: 230 bytes
  < Resource.save
  < URIHandler.write
  Created ...JsonResource for file:/tmp/watch-it/book.json
  > URIHandler.write
  > Resource.save
  < URIHandler.write
    Saved file:/tmp/watch-it/book.json: 145 bytes
  < Resource.save
  ...
< Save a book
Done
```

Three things in that output say a lot about how the runtime is built.

The resource set's configuration is visible: which URI handlers, factories and contributors were applied, and from which classes. When a format you expected is missing, this is where you find out that its module was not on the module path.

`URIHandler.write` and `Resource.save` are siblings, not parent and child. The stream is opened before the resource starts saving into it, and a stream is not a lexical scope, so its span is not made current. In the JSON case the stream span ends before the save span does, because the JSON resource closes the stream itself.

And nothing here needed an OpenTelemetry SDK. The SDK is what samples, batches and exports spans; the API is all instrumented code ever sees. Chapter 11 shows how to add the SDK when you want spans exported, and Chapter 13 how to export them as models.

6.6. The examples in this book

Every listing in this book is included from one of the example modules in `doc/book/examples`. They are built and tested with the runtime, before the book is rendered, so a listing that stops compiling stops the build.

Module	Chapters
`start`	Chapter 6
`library`	Chapter 7, Chapter 20; the domain model the later examples share
`generics`	Chapter 8
`flow`	Chapter 9
`capability`	Chapter 10
`telemetry`	Chapter 11, Chapter 12
`recording`	Chapter 13
`volume`	Chapter 14, Chapter 15
`cli`	Chapter 16, Chapter 17
`dsl`	Chapter 19, Chapter 20
`meta`	Chapter 21
`jvm`	Chapter 22
`web`	Chapter 23
`ui`	Chapter 23, Chapter 24
`diagram`	Chapter 25

To build and test one of them with what it depends on, from the runtime's root:

```
mvn -pl doc/book/examples/capability -am verify
```

Most examples are tests: each test method is one scenario, and its assertions say what the scenario produces. That is deliberate. A test is the shortest program that shows both what to write and what to expect, and it keeps working.

The examples use the `org.example` namespace for their packages and JPMS modules, the way your project would use its own, and share one domain: a library of books, defined as a model in the next chapter.

Chapter 7. Models: EMF without Eclipse

Everything the runtime defines is a model, and every model is EMF. This chapter explains what EMF gives you, why it no longer needs Eclipse, and how a model goes from a text file to generated Java, registered with resource sets, saved in any of six formats and loaded from a module. The example is the library model that the rest of the book uses as its domain.

7.1. Why EMF

A model is a typed graph. Its nodes are objects of classes, and its edges are references, some of which are containment: a library contains its books, a book refers to its authors. The classes, their attributes and their references are themselves described by a model, the metamodel, and EMF's metamodel of metamodels is called Ecore.

That sounds abstract until you list what a typed graph buys once a framework handles it for you, which EMF has done for more than twenty years:

- **Reflection.** Any object can be asked for its class, and any feature can be read and written by name, without generated code. A viewer, a serializer, a query engine or a diff tool written once works for every model.
- **Serialization.** XMI, and with the runtime JSON, YAML and two binary formats.
- **Resources and URIs.** Objects live in resources identified by URIs, and resources live in resource sets. A reference from one resource to another is a URI that is resolved lazily, when it is followed, which is what lets models reference each other across files, repositories and the network.
- **Integrity.** Containment means one owner. Opposite references are maintained in both directions. Multiplicities are declared and checked.
- **Notifications and change recording.** Every change to an object is a notification, and a change recorder can capture a set of changes as a model of its own, which Chapter 13 turns into an audit trail.
- **Operations.** Classes declare operations, and with operation reflection an operation can be invoked by name with a list of arguments.

The last item matters more every year. An operation with a name, typed parameters and documentation is a tool, in exactly the sense an AI agent or a command line needs. Defining an API as model operations means it is documented, invocable and discoverable by construction.

7.2. Why not Eclipse

For most of its life, EMF lived inside the Eclipse IDE. To define a model you installed Eclipse with the modeling tools, created a workspace and a plug-in project, drew or typed an Ecore model, and generated code with a wizard that wrote `plugin.xml`, `MANIFEST.MF` and a generator model next to it. The output was an OSGi bundle meant to run inside Eclipse.

None of that is EMF. EMF core is a set of plain jars on Maven Central, and the runtime uses them as plain jars: named JPMS modules on the module path, with no OSGi, no workspace and no plug-in registry. The modeling tools were the problem, and two things replaced them:

- **Xcore**, a text syntax for Ecore models with operation bodies, which reads like a small programming language and diffs like code.
- **The Xtext Maven plugin**, which generates Java from Xcore in any Maven build, on any machine, in CI, and in editors that are not Eclipse.

So a model is a source file in a Maven module, generated by the build, and committed with the code. I described the journey of getting there in "The Architectural Stalker", about liberating useful artifacts from the Eclipse zone; this chapter is the result.

7.3. Ecore in one page

Concept	Ecore	In the library model
A namespace for related types	`EPackage`, with a namespace URI and a prefix	`library`, `example.org/library`
A type of object	`EClass`, possibly abstract, with supertypes	`Library`, `Author`, `Book`

Concept	Ecore	In the library model
A value-typed property	`EAttribute`, typed by an `EDataType`	`Book.year`, an `int`
A link to other objects	`EReference`: containment or cross reference, single or many, with an optional opposite	`Library.books` (containment), `Book.authors` (cross reference, opposite `Author.books`)
A fixed set of values	`EEnum` with literals	`Genre`
Behavior	`EOperation` with parameters	`Library.publishedBefore(int)`
An instance	`EObject`	a particular book
A unit of storage	`Resource`, identified by a URI	`shelf.library`
A working set of resources	`ResourceSet`, which resolves URIs between them	the resource set a command creates

7.4. A model in Xcore

Here is the library model. It extends `NamedElement` from the runtime's core model, so every library, author and book has a name, a stable identifier, documentation and provenance markers without declaring them (Chapter 20).

Listing 11. The library model, header (`library/src/main/resources/org/example/library/library.xcore`)

```
@Ecore(nsURI="https://example.org/library", nsPrefix="library")
@GenModel(
    modelDirectory="/library/src-gen",
    featureDelegation="Dynamic",
    complianceLevel="25",
    suppressGenModelAnnotations="false",
    copyrightFields="false",
    operationReflection="true",
    importOrganizing="true"
)
package org.example.library

import org.nasdanika.sdk.runtime.models.core.NamedElement
```

Listing 12. The library model, classes

```
/*
 * A collection of books and the people who wrote them.
 */
```

```
class Library extends NamedElement {

    contains Author[] authors

    contains Book[] books

    /*
     * Books published before the year.
     */
    op Book[] publishedBefore(int year) {
        val result = new org.eclipse.emf.common.util.BasicEList<Book>
        for (book : books) {
            if (book.year < year) {
                result.add(book)
            }
        }
        result
    }

}

enum Genre {
    FICTION
    SCIENCE
    HISTORY
    TECHNOLOGY
}

/*
 * A person who wrote books.
 */
class Author extends NamedElement {

    refers Book[] books opposite authors

}

/*
 * A book. The name is the title.
 */
class Book extends NamedElement {

    String isbn

    int year

    Genre genre

    refers Author[] authors opposite books

    /*
     * Cited books. No opposite: getReferrers() finds who cites a book.
     */
    refers Book[] cites
```

```
}
```

A few things in the header are worth understanding, because they are where most first attempts go wrong.

`@Ecore` gives the package its namespace URI and prefix. The namespace URI is the package's identity: it is how serialized instances name their classes, how a resource set finds the package, and, by the runtime's identity rule, where the model's documentation page lives.

`@GenModel` configures the code generator:

- `modelDirectory` says where generated Java goes, and its first segment must be the name of the Maven module's directory: `/library/src-gen` for a module in `library/`. The generator resolves that segment as an Eclipse project name, and under Maven a mismatch produces a successful build with no generated code at all. If `src-gen` comes up empty, look here first.
- `complianceLevel="25"` lets the generated code use the Java version the runtime targets.
- `featureDelegation="Dynamic"` makes generated getters and setters go through EMF's dynamic feature storage, which keeps generated classes small and lets a base class intercept feature access.
- `operationReflection="true"` generates `eInvoke`, so that operations can be called by name.

The `import` brings in `NamedElement` from the core model, which is resolved from the module's compile class path when the model is generated. Comments in `/* */` before a class or feature become its documentation in the generated model. Operation bodies are written in Xbase, a Java-like expression language; the body of `publishedBefore` compiles to the loop you would expect.

7.5. Generating with Maven

The model module declares two plugins. Their configuration is inherited from the runtime's parent `pom.xml`, so the module's own declaration is two lines each:

Listing 13. Generation plugins (`library/pom.xml`)

```
<build>
```

```
    <plugins>
        <!-- Generates Java from src/main/resources/**/*.xcore into src-gen -->
        <plugin>
            <groupId>org.eclipse.xtext</groupId>
            <artifactId>xtext-maven-plugin</artifactId>
        </plugin>
        <!-- Adds src-gen as a source root -->
        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>build-helper-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

The inherited configuration of the Xtext plugin registers three languages, in an order that matters: Ecore and GenModel support first, because Xcore models reference them, then Xcore itself. Its source root is `src/main/resources`, where the `.xcore` files live:

Listing 14. The Xtext plugin's configuration, an excerpt of the runtime's parent `pom.xml`

```
<plugin>
  <groupId>org.eclipse.xtext</groupId>
  <artifactId>xtext-maven-plugin</artifactId>
  <version>2.44.0</version>
  <executions>
    <execution>
      <id>generate-xcore</id>
      <phase>generate-sources</phase>
      <goals><goal>generate</goal></goals>
    </execution>
  </executions>
  <configuration>
    <languages>
      <language><setup>org.eclipse.xtext.ecore.EcoreSupport</setup></language>
      <language><setup>org.eclipse.emf.codegen.ecore.xtext.GenModelSupport</setup></language>
      <language><setup>org.eclipse.emf.ecore.xcore.XcoreStandaloneSetup</setup></language>
    </languages>
    <sourceRoots>
      <root>${project.basedir}/src/main/resources</root>
    </sourceRoots>
  </configuration>
</plugin>
```

The build helper plugin adds `src-gen` as a source root, so `mvn compile` generates and compiles in one step. The model's dependencies are the model it extends, the runtime's `common` module for the services the module contributes, and two small libraries that generated code and Xbase operation bodies use:

Listing 15. Dependencies of a model module (`library/pom.xml`)

```
<dependencies>
    <!-- NamedElement and the rest of the core model, which this model extends -->
    <dependency>
        <groupId>org.nasdanika.sdk.runtime.models</groupId>
        <artifactId>core</artifactId>
    </dependency>
    <dependency>
        <groupId>org.nasdanika.sdk.runtime</groupId>
        <artifactId>common</artifactId>
    </dependency>
    <!-- Used by generated code and operation bodies written in Xcore -->
    <dependency>
        <groupId>org.eclipse.xtext</groupId>
        <artifactId>org.eclipse.xtext.xbase.lib</artifactId>
    </dependency>
    <dependency>
        <groupId>org.eclipse.emf</groupId>
        <artifactId>org.eclipse.emf.ecore.xcore.lib</artifactId>
    </dependency>
    <dependency>
        <groupId>org.nasdanika.sdk.runtime</groupId>
        <artifactId>emf-json</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

The runtime commits generated sources. Generation is deterministic, the diff of generated code is a useful review of a model change, and an editor without Maven integration still sees every class. After editing a `.xcore` file, run `mvn generate-sources` in the module and refresh the editor's view of the project.

The generated API is what you would write by hand if you had the patience: an interface and an implementation class per model class, a factory and a package singleton.

Listing 16. The generated API at work (`library/src/test/java/…/TestLibrary.java`)

```
LibraryFactory factory = LibraryFactory.eINSTANCE;
Library library = factory.createLibrary();

Author evans = factory.createAuthor();
evans.setName("Eric Evans");
library.getAuthors().add(evans);          // Containment: the library owns its authors

Book ddd = factory.createBook();
ddd.setName("Domain-Driven Design");
ddd.setYear(2003);
ddd.setGenre(Genre.TECHNOLOGY);
```

```
ddd.getAuthors().add(evans);               // A cross reference
library.getBooks().add(ddd);

assertThat(evans.getBooks()).containsExactly(ddd); // The opposite is maintained
assertThat(ddd.eContainer()).isSameAs(library);
```

Two lines in that listing are EMF earning its keep. Adding the book to the library's `books` made the library its container, and adding the author to the book's `authors` added the book to the author's `books`, because the two references are declared as opposites.

7.6. Registering the model

A model module tells the runtime about its package with a Java service. The service is the `EPackage` interface, and the provider is a class with a static `provider()` method that returns the package singleton. `ServiceLoader` calls a provider method instead of a constructor when there is one, which is exactly what a singleton needs:

Listing 17. A package provider (`library/⋯/services/LibraryEPackageProvider.java`)

```
/**
 * Registers the library package with resource sets configured from services. A provider
 * method hands {@link java.util.ServiceLoader} the singleton instead of a new instance.
 */
public final class LibraryEPackageProvider {

    private LibraryEPackageProvider() {
    }

    public static EPackage provider() {
        return LibraryPackage.eINSTANCE;
    }

}
```

Listing 18. The model module's declaration (`library/src/main/java/module-info.java`)

```
import org.eclipse.emf.ecore.EPackage;
import org.example.library.services.LibraryEPackageProvider;
import org.example.library.services.LibraryResourceSetContributor;
import org.nasdanika.sdk.runtime.common.services.ResourceSetContributor;

module org.example.library {

    exports org.example.library;        // Generated interfaces
    exports org.example.library.impl;   // Generated implementations
    exports org.example.library.util;   // Generated switch and adapter factory
```

```
    exports org.example.library.services;

    requires transitive org.nasdanika.sdk.runtime.models.core;
    requires transitive org.nasdanika.sdk.runtime.common;
    requires org.eclipse.emf.ecore.xmi;

    provides EPackage with LibraryEPackageProvider;                    ①
    provides ResourceSetContributor with LibraryResourceSetContributor;  ②

}
```

① Every resource set configured from services will have the library package in its package registry, so files that mention `example.org/library` load without any setup.

② A contributor, shown next, gives the model a file extension of its own.

The module exports the generated `impl` and `util` packages as well as the interfaces. EMF's reflective machinery and the runtime's serializers work with the implementation classes, and a JPMS module that hides them makes the model unusable outside it.

7.7. Resource sets configured from services

`NasdanikaResourceSet.createAndConfigure()` is the line that connects EMF to everything on the module path. It creates a resource set and configures it from four kinds of services:

- **URI handlers** (`URIHandler`), added in front of EMF's own, so a module can teach every resource set a new URI scheme.
- **Adapter factories** (`AdapterFactory`).
- **Packages** (`EPackage`), registered by namespace URI.
- **Contributors** (`ResourceSetContributor`), which may do anything else with the resource set, typically register resource factories.

Configuration runs in a `ResourceSetContributor.configure` span with a log record per contribution, and then wraps every URI handler, the default one included, in a `TelemetryURIHandler`, so that every stream a resource set opens is a span with a byte count and progress records. That is why the first program in Chapter 6 was observable without trying.

Three variants of `createAndConfigure` exist for when the defaults do not fit. One takes a `ClassLoader`, for code that is not on the module path, such as plugins loaded at run time.

One takes an `OpenTelemetry` instance and attaches it to the resource set, so that work done outside any unit of work, proxy resolution much later for example, still reports somewhere. And one takes both.

`NasdanikaResourceSet` also supports global objects: `registerGlobal(uri, object)` makes an object resolvable by a URI that no resource contains, which is how a catalog of shared elements is offered to every model that references it.

7.8. Contributing to resource sets

A `ResourceSetContributor` is the service for everything that is not a package, a URI handler or an adapter factory. The library module uses one to give libraries their own extension, `.library`, stored as XMI and instrumented like every resource the runtime creates:

Listing 19. A resource set contributor (`library/⋯/services/LibraryResourceSetContributor.java`)

```
/**
 * Gives libraries a file extension of their own: {@code catalog.library} is XMI,
 * loaded and saved in spans like every resource the runtime creates.
 */
public class LibraryResourceSetContributor implements ResourceSetContributor {

    public static final String EXTENSION = "library";

    @Override
    public void contribute(ResourceSet resourceSet, Span span, Logger logger) { ①
        Map<String, Object> factories =
                resourceSet.getResourceFactoryRegistry().getExtensionToFactoryMap();
        factories.put(EXTENSION, new XMIResourceFactoryImpl() {

            @Override
            public Resource createResource(URI uri) {
                return ResourceTelemetry.created(resourceSet, new XMIResourceImpl(uri) { ②

                    @Override
                    public void doLoad(InputStream in, Map<?, ?> options) throws IOException {
                        ResourceTelemetry.load(this, in, s -> super.doLoad(s, options));
                    }

                    @Override
                    public void doSave(OutputStream out, Map<?, ?> options) throws IOException {
                        ResourceTelemetry.save(this, out, s -> super.doSave(s, options));
                    }

                });
            }
```

```
        });
        span.setAttribute("library.extension", EXTENSION);                    ③
        Telemetry.log(logger, Severity.DEBUG, null, "Registered ." + EXTENSION, null);
    }

}
```

① The interface's default `contribute(ResourceSet)` runs this method in a span, with the contributor's class and module as attributes, and hands it the span and a logger. A contributor that wants no span overrides `contribute(ResourceSet)` instead.

② `ResourceTelemetry` gives a resource the same spans and log records as the runtime's own: `Resource.load` and `Resource.save` with the URI, the byte count and the number of root objects.

③ The span is the contributor's own, so anything it wants to record about the contribution goes on it.

7.9. Formats

The extension of a resource's URI selects its factory, and the runtime contributes six:

Extension	Format	Contributed by
`xmi` and any unregistered extension	XMI	`common`
`ebin`	EMF binary	`common`
`egz`	EMF binary, gzipped	`common`
`json`	EMF JSON	`emf-json`
`yaml`, `yml`	EMF JSON, written as YAML	`emf-json`
`library`	XMI	the library example

The test below saves the sample library in every one of them and loads each back in a fresh resource set:

Listing 20. Every format, round-tripped (`library/src/test/java/⋯/TestLibrary.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Library library = Samples.library();
```

```
for (String extension: List.of("xmi", "library", "ebin", "egz", "json", "yml")) {
    URI uri = URI.createFileURI(directory.resolve("shelf." + extension).toString());
    Resource resource = resourceSet.createResource(uri);   // The extension picks the factory
    resource.getContents().add(EcoreUtil.copy(library));
    resource.save(null);

    Resource loaded = NasdanikaResourceSet.createAndConfigure().getResource(uri, true);
    EObject root = loaded.getContents().getFirst();
    assertThat(EcoreUtil.equals(root, library)).as(extension).isTrue();
}
```

Which one to use is a question of who reads the file. XMI is the interchange format every EMF tool understands, and it diffs reasonably. YAML is the one people edit and review. JSON is the one a browser or a JavaScript program consumes, and Chapter 23 shows where that leads. The binary formats are for size and speed, such as telemetry from a long run or a snapshot of a large system.

7.10. Loading from modules

Two URI schemes contributed by `common` load resources from the code itself:

- `classpath://path/to/resource` reads through the context class loader.
- `module://module.name/path/to/resource` reads from a named module.

The library module ships a small catalog at the root of its jar, and the test loads it by the module's name:

Listing 21. Loading a resource from a module

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
URI uri = URI.createURI("module://org.example.library/catalog.library");
Library catalog = (Library) resourceSet.getResource(uri, true).getContents().getFirst();
assertThat(catalog.getBooks()).extracting(Book::getName).contains("Domain-Driven Design");
```

JPMS encapsulation applies here. A resource in a directory that corresponds to one of the module's packages is visible to other modules only if that package is open; a resource outside any package, such as at the root of the module or under `META-INF`, is always visible. Put resources meant to be loaded by URI outside the package directories, or open the package they are in.

7.11. One model, several files

A contained object can be saved in a file of its own and still be contained, which keeps a large model reviewable file by file: each module of a JVM in its own file, for example, so that Git shows which module changed. EMF calls this containment proxies, and it is off by default. The rule for a containment whose objects may be in files of their own:

- Declare the containment `contains resolving`, and set `containmentProxies="true"` in the package's `@GenModel`; generated code needs both.
- Give it a container reference as its opposite, declared `container resolving`. The opposite makes an object's file refer back to its container's file, so that the object loaded alone still knows where it belongs. `resolving` makes reading the container reference return the container, the same as `eContainer()`, and not a proxy.

```
@GenModel(containmentProxies="true")
package org.example.tree

class Node {
    String name
    contains resolving Node[] children opposite parent
    container resolving Node parent opposite children
}
```

Saving the model then means saving every modified resource, not only the root's, and walking a model in either direction loads files as it goes. The field note "Cross-Resource Containment in EMF", at Nasdanika Field Notes, shows each of these behaviors with a test.

7.12. Operations

Operations defined in Xcore are ordinary Java methods on the generated classes, and, with operation reflection, also callable by name:

Listing 22. Calling an operation directly and reflectively

```
Library library = Samples.library();
List<Book> classics = library.publishedBefore(1900);   // An operation implemented in Xcore
assertThat(classics).extracting(Book::getName)
    .containsExactly("On the Origin of Species", "War and Peace");

// The same operation, invoked reflectively: what a tool, a CLI or an agent would do
EOperation operation = LibraryPackage.Literals.LIBRARY___PUBLISHED_BEFORE__INT;
```

```
Object result = library.eInvoke(operation, ECollections.asEList(1900));
assertThat(result).isEqualTo(classics);
```

The reflective call is what a generic tool does. A command line can offer every operation of a model as a command, a viewer can offer it as a button, and an agent can be offered it as a tool, all from the model, with the operation’s documentation as the description.

7.13. What you have now

A model is a text file in a Maven module. The build generates its Java; a service registers it with every resource set; a contributor adds whatever else the model needs; and the model is saved and loaded in the format each reader prefers. No IDE was involved.

The next chapter turns to a problem that shows up the moment you start connecting components by type: Java forgets type arguments at run time, and the runtime needs them.

Chapter 8. Generic types at run time

Components that find each other by type run into a limit of Java almost immediately. `ServiceLoader` can be asked for a `Supplier`, and then every supplier in the assembly answers, whatever it supplies. It cannot be asked for a `Supplier<Library>`, because at run time there is no such thing: type arguments are erased.

The runtime needs them anyway. A command that needs a supplier of libraries should not be offered a supplier of authors, and a requirement for "something that renders a library" should not be answered by something that renders a book. This chapter shows what survives erasure, how the runtime matches generic types with Java's own rules, and how objects declare what they can be adapted to. The capability framework (Chapter 10) and command wiring (Chapter 17) are both built on it.

8.1. Matching and wiring

Much of software development is matching and wiring: connecting something that needs to something that provides, by a rule, and then checking that the connection is right. The Java compiler does it all the time. A method call is matched to a method by its name and signature. Generics are a check on that match and nothing more: after erasure, `get` on a `List<Book>` is a call to `List.get` returning `Object`, followed by a cast, and the type argument is there to prove at compile time that the cast will succeed. Overriding is matched by signature too, and `@Override` validates the match: it changes nothing in the program, it states that the method is meant to override one, and the compiler checks that there is one.

Wiring at run time has usually had less to work with. `ServiceLoader` and most registries match on a `Class`, which is the erased type, or on a string. Matching on `java.lang.reflect.Type` instead brings run-time matching to the level the compiler works at: not "a supplier", but "a supplier of libraries", decided by the compiler's rules. The runtime's Zephyr Book *Matching and Wiring* develops the argument and documents the `common` module in depth.

8.2. What survives erasure

Objects carry no type arguments. A `new ArrayList<String>()` and a `new ArrayList<Integer>()` are the same class at run time, and nothing can recover the difference from the objects.

Declarations are different. The class file keeps the generic signature of every class, field and method, and reflection reads it: `Field.getGenericType()`, `Method.getGenericReturnType()`, `Class.getGenericInterfaces()`. A class declared as `implements Supplier<Library>` says so at run time, forever. So does a field declared as `Supplier<Library>`, and a method declared to return one.

That gives the runtime a rule it applies everywhere: **what is matched is always a declaration, never an instance**. A capability method is matched by its declared generic return type, an adapter field by its declared type, a command's parent member by its declared type. The compiler checks the declarations, and the runtime trusts them.

8.3. Types

`org.nasdanika.sdk.runtime.common.Types` is the runtime's small library of generic type operations, built on geantyref and hiding it. The examples in this chapter use a few types of their own: a catalog lists titles, and a shelf is a catalog you can also put books on.

Listing 23. The types the examples adapt to (`generics/⋯/Catalogs.java`)

```
/**
 * Something that lists book titles.
 */
public interface Catalog {

    List<String> titles();

}

/**
 * A catalog you can also put books on.
 */
public interface Shelf extends Catalog {

    void put(String title);

}

public record ReadingList(List<String> titles) implements Catalog {
```

```
}

public static final class OakShelf implements Shelf {

    private final List<String> titles = new ArrayList<>();

    @Override
    public List<String> titles() {
        return titles;
    }

    @Override
    public void put(String title) {
        titles.add(title);
    }

}
```

A generic type is captured with a type token, an anonymous subclass of `TypeToken` whose type argument reflection can read back:

Listing 24. Type tokens (`generics/src/test/java/⋯/TestGenerics.java`)

```
static final Type SHELVES = new TypeToken<Supplier<Shelf>>() {}.getType();
static final Type CATALOGS = new TypeToken<Supplier<Catalog>>() {}.getType();
static final Type ANY_CATALOGS =
        new TypeToken<Supplier<? extends Catalog>>() {}.getType();
```

`Types.isAssignable(target, source)` answers the question the compiler answers for an assignment: may a value declared as `source` be used where `target` is required?

Listing 25. Assignability, by Java's rules for generics

```
// ? extends Catalog accepts Shelf
assertThat(Types.isAssignable(ANY_CATALOGS, SHELVES)).isTrue();
// Supplier<Shelf> is not a Supplier<Catalog>
assertThat(Types.isAssignable(CATALOGS, SHELVES)).isFalse();
// A raw target accepts any parameterization
assertThat(Types.isAssignable(Supplier.class, SHELVES)).isTrue();
// A raw source promises nothing about shelves
assertThat(Types.isAssignable(SHELVES, Supplier.class)).isFalse();
```

The first three lines are Java's rules. `Supplier<Shelf>` can be used where `Supplier<? extends Catalog>` is required, because a shelf is a catalog and the wildcard allows subtypes. It cannot be used where `Supplier<Catalog>` is required, because Java generics are invariant: `Supplier<Shelf>` is not a subtype of `Supplier<Catalog>`, even though `Shelf` is a subtype of

`Catalog`. A wildcard is how a requirement says that subtypes are welcome, and it is the requirement's choice, not the provider's. And anything parameterized can be used where the raw type is required.

The fourth line is where the runtime is deliberately stricter than Java. Java lets a raw `Supplier` be used where `Supplier<Shelf>` is required, with an unchecked warning. For matching that would let something declared as a raw supplier answer a requirement for shelves, about which it has said nothing, so `Types` refuses. The one exception is a target whose type arguments are all unbounded wildcards, such as `Supplier<?>`, which Java itself accepts from a raw type without a warning.

Two more operations read type arguments from declarations. `getTypeArgument` finds the argument a subtype fixes for a type parameter of a supertype, however many classes lie in between:

Listing 26. The type argument a declaration fixes

```
final class ShelfSupplier implements Supplier<Shelf> {

    @Override
    public Shelf get() {
        return new OakShelf();
    }

}
Type argument = Types.getTypeArgument(
        ShelfSupplier.class,
        Supplier.class.getTypeParameters()[0]);
assertThat(argument).isEqualTo(Shelf.class); // What survives erasure: the declaration
```

`getElementType` does the same for containers, replacing a wildcard by its bound: an element of `List<? extends Shelf>` is a `Shelf`, an element of `List<?>` is an `Object`.

Finally, `isGround` says whether a type can be required or provided at all. A type variable means something only inside the scope that declares it, so `T`, or `List<T>`, cannot be matched against anything. This is the trap of type tokens created inside generic methods: `new TypeToken<T>() {}` captures the variable `T`, not what `T` stands for at the call site, because erasure leaves nothing else to capture. The capability framework refuses such types with an error instead of matching them wrongly.

8.4. Adaptation

Matching declared types answers "is this declaration compatible with that one". Adaptation answers a question objects are asked all the time in a system assembled from parts: "can you give me what I need?"

An object declares what it can be adapted to with `@Adapter` on fields and methods. The declared generic type of the member is what it provides:

Listing 27. An adaptable object (`generics/⋯/Branch.java`)

```
/**
 * A library branch. What it can be adapted to is declared by its adapter members,
 * and their generic types say exactly what each one provides.
 */
public class Branch implements Adaptable {

    private final Map<String, String> titles = Map.of(
            "978-0321125217", "Domain-Driven Design",
            "978-0201633610", "Design Patterns");

    @Adapter
    private final Supplier<Shelf> shelves = OakShelf::new;                  ①

    @Adapter
    private final Supplier<Catalog> catalogs =                              ②
            () -> new ReadingList(List.copyOf(titles.values()));

    @Adapter
    public String title(String isbn) {                                      ③
        return titles.get(isbn); // null declines: "not this time"
    }

}
```

① A field: the branch can be adapted to `Supplier<Shelf>`.

② Another field of the same raw type, told apart by its type argument.

③ A method with a parameter takes context. It answers a request for a `String` with one `String` of context, and returns null to decline.

Asking is `adaptTo(type, context⋯)`:

Listing 28. Adapting

```
Branch branch = new Branch();
```

```
Supplier<Shelf> shelves = branch.adaptTo(SHELVES);
Supplier<Catalog> catalogs = branch.adaptTo(CATALOGS);
Supplier<? extends Catalog> any = branch.adaptTo(ANY_CATALOGS); // Both members qualify

assertThat(shelves.get()).isInstanceOf(Shelf.class);
assertThat(catalogs.get().titles()).contains("Design Patterns");
assertThat(any.get()).isInstanceOf(Catalog.class);

// Context is matched to the parameters of adapter methods
String title = branch.adaptTo(String.class, "978-0321125217");
assertThat(title).isEqualTo("Domain-Driven Design");
assertThat(branch.<String>adaptTo(String.class, "000")).isNull(); // Declined
Type strings = new TypeToken<Supplier<String>>() {}.getType();
assertThat(branch.<Object>adaptTo(strings)).isNull();
```

The rules, in order:

- With no context, the object itself is returned if its own declared type matches, generic supertypes included: a class that implements `Supplier<Shelf>` adapts to `Supplier<Shelf>` without any annotation.
- Otherwise adapter fields and adapter methods without parameters are tried. With context, adapter methods whose parameters accept the context values are tried: as many parameters as values, each value an instance of its parameter's class, or null for a parameter that is not primitive.
- Of the candidates whose declared type is assignable to the requested one, the more specific are tried first, fields before methods, then by name. The first non-null value wins; null means "not this time".



> "More specific first" is computed by `Types.specificity`, which counts the supertypes of the **erased** type. `Supplier<Shelf>` and `Supplier<Catalog>` both erase to `Supplier`, so they tie, and the member name decides. In the example, a request for `Supplier<? extends Catalog>` gets the `catalogs` field, not the more specific `shelves`. The runtime's `TestAdaptable` passes its equivalent assertion only because "volumes" sorts before "writables". The same ordering is used by `ReflectiveCapabilityFactory` and `CommandWiring`. A fix: break ties between members whose types are both assignable to the request by comparing them with `Types.isAssignable`, so that a member assignable to the other comes first.

Objects that do not implement `Adaptable` can still be adapted through their annotations,

with `Adapters.adapt`:

Listing 29. Adapting a plain object (`generics/⋯/Archive.java` and the test)

```
/**
 * A plain object, not an Adaptable: the annotations are enough for Adapters.adapt().
 */
public class Archive {

    @Adapter
    public Supplier<Catalog> catalog() {
        return () -> new ReadingList(List.of("On the Origin of Species"));
    }

}

Supplier<Catalog> catalog = Adapters.adapt(new Archive(), CATALOGS);
assertThat(catalog.get().titles()).containsExactly("On the Origin of Species");
```

Implementing `Adaptable` is needed only to adapt in ways an annotation cannot express: computed, delegated or cached. The override can fall back to the declared adapters:

Listing 30. Adaptation in code, with a fallback to the declarations

```
/**
 * Adaptation that annotations cannot express goes into an override, which falls back
 * to the declarations.
 */
public class Reserve implements Adaptable {

    @Adapter
    private final Supplier<Shelf> shelves = Catalogs.OakShelf::new;

    @Override
    public <T> T adaptTo(Type type, Object... context) {
        if (type == String.class && context.length == 1
                && context[0] instanceof Integer floor) {
            @SuppressWarnings("unchecked")
            T location = (T) ("Reserve, floor " + floor); // Computed, not declared
            return location;
        }
        return Adaptable.super.adaptTo(type, context);   // The declared adapters
    }

}

Reserve reserve = new Reserve();
assertThat(reserve.<String>adaptTo(String.class, 3)).isEqualTo("Reserve, floor 3");
Supplier<Shelf> shelves = reserve.adaptTo(SHELVES);
assertThat(shelves.get().titles()).isEqualTo(List.of());
```

8.5. Asking before there is an instance

Sometimes the question comes before the object exists. The command line builds its tree of commands before any command has parsed an option, and it needs to know whether a command could work under a given parent without creating either. `Adapters.isAdaptable(class, type, contextTypes…)` answers from declarations only:

Listing 31. Asking a class instead of an object

```
// Before there is an instance: can a Branch be asked for these?
assertThat(Adapters.isAdaptable(Branch.class, SHELVES)).isTrue();
assertThat(Adapters.isAdaptable(Branch.class, String.class, String.class)).isTrue();
assertThat(Adapters.isAdaptable(Branch.class, String.class, Integer.class)).isFalse();
```

True means an instance can be asked and has a declared way to answer, not that it will: an adapter may still decline at run time. A class whose adaptation lives only in an `adaptTo` override is not adaptable by this check; an `@Adapter` method that delegates to the override makes it so.

8.6. Access

Adapters are read and invoked reflectively by `common`, so JPMS has to let it. Public members of an exported package are accessible; private members, like the fields of `Branch`, need their package to be open to `common`:

Listing 32. Opening a package to the runtime (`generics/src/main/java/module-info.java`)

```
module org.example.generics {

    requires transitive org.nasdanika.sdk.runtime.common;

    exports org.example.generics;

    // Adapter fields and methods are read reflectively by common
    opens org.example.generics to org.nasdanika.sdk.runtime.common;

}
```

A qualified `opens … to` keeps the package closed to everybody else. Command modules usually open their packages without qualification anyway, because picocli needs the same access (Chapter 16).

8.7. Where this is used

Three parts of the runtime are built on these few operations:

- **The capability framework** matches a requirement's type against each capability method's declared return type with `isAssignable`, so `Function<Library, String>` and `Function<Book, String>` are different requirements (Chapter 10).
- **Command wiring** matches a command's `@Parent` and `@SubCommand` members against what other commands can be adapted to, first by declarations with `isAdaptable`, then by instances with `adapt` (Chapter 17).
- **Adaptation** itself is how a command group offers its sub-commands what they need without knowing them.

In all three, no string names a type, and the compiler checks every declaration that takes part.

8.8. A trained eye

There is a hypothesis in evolutionary biology that describes how this design works on the people who use it. The *sensory bias hypothesis* holds that a receiver's preferences can evolve first in a context unrelated to mating, such as finding food, and be exploited later by senders signaling to mates: a butterfly whose eyes were tuned to the colors of flowers, and mates that evolve wings of those colors.

Java developers have a trained eye for generic types. It was trained by collections, from Java 5 on, to read `List<Book>`, to know why a `List<Novel>` is not a `List<Book>`, and to reach for `? extends` when only reading. That skill evolved for type-safe collections, not for wiring, and the runtime piggybacks on it: a requirement for `Supplier<? extends Catalog>` asks for exactly what a Java developer reads in it, so there is no new notation to learn, no qualifiers, no string identifiers and no wiring files.

8.9. The JVM, not Java

What the runtime matches on is in the class file: the `Signature` attribute, part of the JVM specification. Kotlin, Scala and Groovy write it for the types their programs declare, so

their classes are matched like Java's. This is a property of the JVM, not of the Java language.

With all due respect to two excellent platforms, TypeScript and Python do not have it. TypeScript's type system is in many ways richer than Java's, but it is checked at compile time and erased completely; the decorator metadata it can emit records constructors, `Array` for a `Book[]`, not type arguments. Python keeps annotations at run time, generic arguments included, and FastAPI and Pydantic match on them well, but the language does not check them; a type checker does, if somebody runs it, so a match on an annotation is a match on an unchecked claim. .NET, to be fair, goes further than the JVM: its generics are reified, and even instances know their type arguments.



The butterfly is an illustration of the hypothesis, not a documented case. Documented cases include platyfish and swordtails (Basolo, 1990) and guppies, whose females' attraction to orange fruit may explain the males' orange spots (Rodd and colleagues, *Proceedings of the Royal Society B*, 2002). Verify the citations, or keep the wording that says the butterfly is an illustration.

Chapter 9. Flows, pumps and transformations

Most code in the runtime is plain blocking Java on virtual threads, and should be. Some of it is genuinely asynchronous: resolving capabilities that depend on each other, wiring a graph of objects that reference each other, delivering telemetry events to subscribers while the work they describe continues. For that code the runtime uses `java.util.concurrent.Flow`, a handful of operators, and one engine, the pump, that makes asynchronous resolution deterministic and able to say what went wrong.

This chapter is the most technical in the foundations. Architects can read the first section of each topic and skip the rest; the capability framework in the next chapter is built on what is here, and makes more sense with it.

9.1. Flow, not Reactor

The JDK has had the reactive streams interfaces since Java 9: `Flow.Publisher`, `Flow.Subscriber`, `Flow.Subscription` and `Flow.Processor`, plus one implementation, `SubmissionPublisher`. It does not have operators: no `map`, no `filter`, no `retry`. Reactive libraries such as Reactor fill that gap, and bring a large API, their own threading model and their own dependencies with them.

The runtime does not use them, for the reasons in Chapter 2, and takes a narrower position: write the few operators that are needed, and do not write a Reactor. On virtual threads, blocking is cheap, so most processing is better done on the pull side, with `java.util.stream`. The push-side operators exist for results that must stay a publisher: fed to another subscriber, subscribed to by several consumers, or delivered as soon as each item is ready.

9.2. Flows

`org.nasdanika.sdk.runtime.common.flow.Flows` holds sources, operators and terminals, named in the style of `Collectors` and `Executors`.

Kind	Members
Sources	`of`, `fromIterable`, `fromStream`, `fromFuture`, `empty`, `error`
Operators	`map`, `mapNotNull`, `filter`, `peek`, `onTerminate`, `take`, `concat`, `concatMap`
Terminals	`toList` and `first`, which return a `CompletableFuture`; `forEach`; `toStream`, the bridge to the pull side

Every operator follows three rules. It is lazy: it subscribes upstream once per downstream subscriber, at subscription time. Demand is accounted: `filter` requests one more item for every item it drops, and `take` cancels upstream after the last one. And an exception in a function you pass cancels upstream and signals `onError` downstream, so a failure never escapes as a thrown exception from somewhere unexpected.

Listing 33. Operators and a terminal (`flow/src/test/java/⋯/TestFlow.java`)

```
Flow.Publisher<String> titles =
        Flows.of("Design Patterns", "Domain-Driven Design", "War and Peace");
Flow.Publisher<String> d = Flows.filter(titles, title -> title.startsWith("D"));
Flow.Publisher<Integer> lengths = Flows.map(d, String::length);
CompletableFuture<List<Integer>> result = Flows.toList(lengths); // Nothing ran until toList
assertThat(result.join()).containsExactly(15, 20);
```

Listing 34. The pull side

```
// On a virtual thread blocking is cheap, so the pull side is java.util.stream
try (Stream<String> stream = Flows.toStream(Flows.of("a", "bb", "ccc"))) {
    assertThat(stream.mapToInt(String::length).sum()).isEqualTo(6);
}
```

Absent on purpose: `flatMap`, `merge`, `zip`, schedulers and time operators. They are where reactive libraries get large and hard to reason about, and on virtual threads they are ordinary blocking code. `concatMap` is present because sequential composition is deterministic, and the capability framework needs it.

9.3. The pump

Asynchronous resolution has two classic failure modes. Work that depends on other work deadlocks when the dependency graph has a cycle, and the program hangs with no explanation. Or a pass counter gives up after some number of iterations, finds the same

condition late, and says nothing about what was stuck.

`org.nasdanika.sdk.runtime.common.flow.Pump` replaces both with one idea: **block by pumping**. A pump is a serial work queue drained by one thread at a time. Its blocking operations, `join` and `settle`, do not sleep: the waiting thread runs queued work itself until what it waits for is done. `ForkJoinTask.join` works this way, as do SWT's `readAndDispatch` loop and Kotlin's `runBlocking`.

Listing 35. Blocking by pumping

```
Pump pump = new Pump();                    // No executor: nothing runs until somebody pumps
CompletableFuture<String> answer = new CompletableFuture<>();
pump.execute("ask", () -> pump.execute("answer", () -> answer.complete("42")));
assertThat(pump.join(answer)).isEqualTo("42"); // The calling thread drains the queue
```

A pump created without an executor runs nothing until somebody pumps, which makes it deterministic: the same work runs in the same order every time, on the calling thread. That is the mode for tests and for resolutions that settle before returning. A pump created with an executor, such as `Pump.Options.VIRTUAL_THREADS`, drains work enqueued while nobody pumps, so publishers on it make progress without a blocking caller.

Idle and stalls

The pump knows when it is idle: no task can run, and no work it is tracking is in flight. What happens then while something is still waiting is a policy, not a guess:

Policy	Meaning
`COMPLETE`	A closed world: every source feeds this pump, so nothing more can arrive. Waiters are released, a pumped publisher completes, and whatever is still unsatisfied is reported as a stall. This is the default, and what capability resolution uses.
`WAIT`	An open world: change reports, the network, webhooks. The pump waits for new work for a quiet period, then behaves as `COMPLETE`.
`FAIL`	A closed world where going idle early is a bug, for tests: the pump fails at once with a stall report.

A stall is reported as a `StallException` that lists everything that was waiting and what it was

waiting for:

Listing 36. A stall, reported

```
Pump pump = new Pump();
CompletableFuture<String> nobodyCompletesThis = new CompletableFuture<>();
pump.when(nobodyCompletesThis, "waiting for a reply that never comes", (value, error) -> {});
assertThatThrownBy(pump::settle)
    .isInstanceOf(StallException.class)
    .hasMessageContaining("waiting for a reply that never comes");
```

Work that leaves the pump

The pump can only reason about work it knows of. Work that leaves it, an HTTP call or a model turn on another thread, must be tracked, or the pump would consider itself idle while the work is still out:

Listing 37. Tracking work done elsewhere

```
Pump pump = new Pump();
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    // Work that leaves the pump, like a remote call
    CompletableFuture<String> call = CompletableFuture.supplyAsync(() -> "response", executor);
    CompletableFuture<String> tracked = pump.track(call, "remote call"); // In flight
    assertThat(pump.join(tracked)).isEqualTo("response");
}
```

`track` counts the stage as in flight until it completes, and returns a stage that completes on the pump, so whatever is composed on it runs in order with everything else. Code running inside a pump task can find its pump with `Pump.current()`, which is how a capability factory tracks the calls it makes.

Items that cannot proceed yet can yield. `when(stage, description, action)` parks an action until a stage completes, with a reason that appears in stall reports. `submit(description, step)` is a blind yield for a step that returns false while it cannot proceed; it is retried at a lower priority, and priorities age so that nothing starves.

Context and rules

A task runs in the OpenTelemetry context that was current when it was enqueued. Spans started by tasks are children of the span that enqueued them, whichever thread happens

to drain the queue, and the unit of work's `OpenTelemetry` instance travels with them (Chapter 11).

Two rules keep the pump honest. A blocking terminal on the pump's own draining thread is refused, because a nested pump is re-entrancy, the classic event-loop bug. And an exception escaping a task fails the pump: new work is dropped and terminals rethrow it. Operators route exceptions to `onError`, so tasks rarely throw.

9.4. The transformer

A transformation turns sources into targets, and the interesting ones have dependencies: the target of one source references the targets of others. A model loaded from a diagram, a graph of modules that read each other, and a set of capabilities that depend on each other all have this shape, and the references can be cyclic.

`Transformer` maps each source to a publisher of its targets. A factory receives the source and a context, and the context's `get(otherSource)` returns the targets of another source in the same transformation. Within one transformation each source is created once, its targets are replayed to everyone who asks, and all work runs through one pump, so it is deterministic and never recursive: requesting a source enqueues its creation instead of running it inline.

The example turns module descriptions, which name the modules they read, into nodes that reference each other:

Listing 38. Module descriptions and nodes (`flow/⋯/ModuleGraph.java`)

```
/**
 * What a module says about itself: its name and the names of the modules it reads.
 */
public record Description(String name, List<String> reads) {
}

/**
 * A node of the graph, with references to the nodes it reads.
 */
public static final class Node {

    private final String name;
    private final List<Node> reads = new ArrayList<>();
```

```
    Node(String name) {
        this.name = name;
    }

    public String name() {
        return name;
    }

    public List<Node> reads() {
        return reads;
    }

    @Override
    public String toString() {
        return name + " -> " + reads.stream().map(Node::name).toList();
    }

}
```

Listing 39. Wiring a graph by subscription

```
public static Map<String, List<Node>> build(Map<String, Description> modules, String root) {
    Transformer<String, Node> transformer = new Transformer<>((name, context) -> { ①
        Node node = new Node(name);
        for (String read: modules.get(name).reads()) {
            Flows.forEach(context.get(read), node.reads()::add);                    ②
        }
        return Flows.of(node);                                                      ③
    });
    return transformer.transform(List.of(root));                                    ④
}
```

① The factory: a source, here a module name, and the transformation's context.

② Wiring is subscription. The node subscribes to the targets of each module it reads and adds them as they arrive.

③ The node is emitted at once, without waiting for its references. That is what lets cyclic graphs wire without deadlock.

④ `transform` runs the transformation on a new pump and settles it. The result holds the targets of every source resolved, including those requested by factories, in the order they were requested.

Listing 40. A cycle, wired

```
Map<String, Description> descriptions = Map.of(
        "app", new Description("app", List.of("model", "common")),
        "model", new Description("model", List.of("common", "app")),   // A cycle back to app
        "common", new Description("common", List.of()));
```

```
Map<String, List<Node>> graph = ModuleGraph.build(descriptions, "app");

assertThat(graph.keySet()).containsExactly("app", "model", "common"); // In request order
Node app = graph.get("app").getFirst();
Node model = graph.get("model").getFirst();
Node common = graph.get("common").getFirst();
assertThat(app.reads()).containsExactlyInAnyOrder(model, common);
assertThat(model.reads()).containsExactlyInAnyOrder(common, app);     // The cycle, wired
```

`app` reads `model` and `model` reads `app`, and both nodes are created exactly once and reference each other. Note what the test does not assert: the order of a node's references. References arrive in the order their targets are produced, not the order they were requested; when `model` subscribes, the node for `app` already exists and arrives at once, while the node for `common` is still queued. If order matters, keep the position with each reference, or wait for all of them.

Waiting is allowed. A factory that needs another source's targets before it can emit its own collects them, for example with `Flows.toList(context.get(other))`, and emits when they are there. If two sources wait on each other, the pump goes idle with both waiting, and the transformation fails with the cycle named:

Listing 41. A waiting cycle, reported with its path

```
// Each source needs the other's targets before it can emit its own
Transformer<String, String> transformer = new Transformer<>((name, context) -> {
    String other = name.equals("chicken") ? "egg" : "chicken";
    CompletableFuture<List<String>> others = Flows.toList(context.get(other));
    return Flows.fromFuture(others.thenApply(targets -> name + targets));
});
assertThatThrownBy(() -> transformer.transform(List.of("chicken")))
    .isInstanceOf(StallException.class)
    .hasMessageContaining("cycle: chicken -> egg -> chicken");
```

Every transformation reports itself: `transform` runs in a `Transformer.transform` span, and each source is created in a `Transformer.create` span, from the factory call to the completion of its publisher. A source requested by a factory is created in a child span of the requesting source's span, so the span tree shows the dependency graph.

9.5. Mapping methods

Writing one factory per kind of source gets repetitive. `ReflectiveFactory` assembles a factory from methods annotated with `@Mapping`, so one class contributes many mappings. A mapping method takes the source, and optionally the context, and returns a publisher of targets, a completion stage of one target, a target, or null for none. The method matches sources that are instances of its first parameter's type, or of the type the annotation names.

The example renders a small document tree as outline lines, one method per kind of node:

Listing 42. Nodes (`flow/⋯/Outline.java`)

```
public sealed interface Part permits Chapter, Section, Paragraph, Note {
}

public record Chapter(String title, List<Part> parts) implements Part {
}

public record Section(String title, List<Part> parts) implements Part {
}

public record Paragraph(String text) implements Part {
}

public record Note(String text) implements Part {
}
```

Listing 43. Mapping methods

```
public static class Mappings {

    @Mapping
    public Flow.Publisher<String> chapter(Chapter chapter, Context<Object, String> context) {
        return Flows.concat(
                Flows.of("# " + chapter.title()),
                Flows.concatMap(Flows.fromIterable(chapter.parts()), context::get)); ①
    }

    @Mapping
    public Flow.Publisher<String> section(Section section, Context<Object, String> context) {
        return Flows.concat(
                Flows.of("## " + section.title()),
                Flows.concatMap(Flows.fromIterable(section.parts()), context::get));
    }

    @Mapping
    public String paragraph(Paragraph paragraph) {                                    ②
```

```
        return paragraph.text();
    }

    @Mapping
    public String note(Note note) {                                          ③
        return "> Note: " + note.text();
    }

}

public static List<String> render(Chapter chapter) {
    ReflectiveFactory<Object, String> factory =
            ReflectiveFactory.first(MethodHandles.lookup(), new Mappings());
    return new Transformer<>(factory).transform(List.of(chapter)).get(chapter);
}
```

① A chapter maps to its heading followed by the targets of its parts, each obtained from the context, in order.

② A paragraph maps to its text.

③ A note is a different type, so it gets its own method rather than a condition inside the paragraph method.

Listing 44. The rendered outline

```
Chapter chapter = new Chapter("Flows", List.of(
        new Paragraph("Publishers, subscribers and pumps."),
        new Section("The pump", List.of(
                new Paragraph("Block by pumping."),
                new Note("Never join on the pumping thread."))));
assertThat(Outline.render(chapter)).containsExactly(
        "# Flows",
        "Publishers, subscribers and pumps.",
        "## The pump",
        "Block by pumping.",
        "> Note: Never join on the pumping thread.");
```

Matching methods are ordered by priority (higher first), then by how specific the matched type is, then by how specific the declaring class is, then by name. `ReflectiveFactory.first` uses only the best match, which is polymorphic dispatch; `ReflectiveFactory.all` uses every match, in that order.



With `first`, the best match is chosen by type and priority before the method runs, and a method that returns null produces no target; it does not pass the source on to the next method. A high-priority method that matches every paragraph and returns null for most of them makes most paragraphs

> disappear. Distinguish cases by type, as the note does, or use `all` and let several methods contribute.

9.6. One engine

The pump, the transformer and the capability loader were written as separate pieces of code in older Nasdanika versions, and each hand-built the same four things: a memoizing registry, a queue drained on the caller's thread, a loop guard, and a collector of results. In the runtime they are one engine. The transformer runs on the pump, and the capability loader, the subject of the next chapter, is a transformer whose sources are requirements and whose targets are providers of capabilities. Everything in this chapter about memoization, cycles, stall reports, context propagation and spans applies to capability resolution unchanged.

Chapter 10. The capability framework

`ServiceLoader` answers one question: give me every implementation of this interface. That is enough surprisingly often, and the runtime uses it whenever it is (Chapter 2). It is not enough when the question carries more than a type.

"A renderer for libraries, in Markdown." "A contents handler for a file named `shelf.csv`." "The sub-commands that can work under this command." Each of these is a requirement that the providers have to interpret, some providers will decline it, several may answer, and answering may require resolving other requirements first. That is what the capability framework is for.

10.1. Concepts

Five words carry the whole framework, and the rest of the book uses them in exactly these senses.

Requirement

What is asked for: an object that capability factories interpret. Usually a `Requirement`, a type with optional payloads, but any object can be one.

Capability factory

A service that says whether it can handle a requirement, and if so creates providers for it. It may raise requirements of its own while doing so.

Capability provider

A publisher of the capabilities that satisfy a requirement. A provider is subscribed lazily, and may carry information about what it provides, so that a caller can choose between providers without subscribing.

Capability

What a provider publishes: the object the requirement asked for.

Capability loader

Resolves a requirement against a set of factories: asks each whether it can handle it,

collects their providers, and resolves the requirements the factories raise, once each, on a pump.

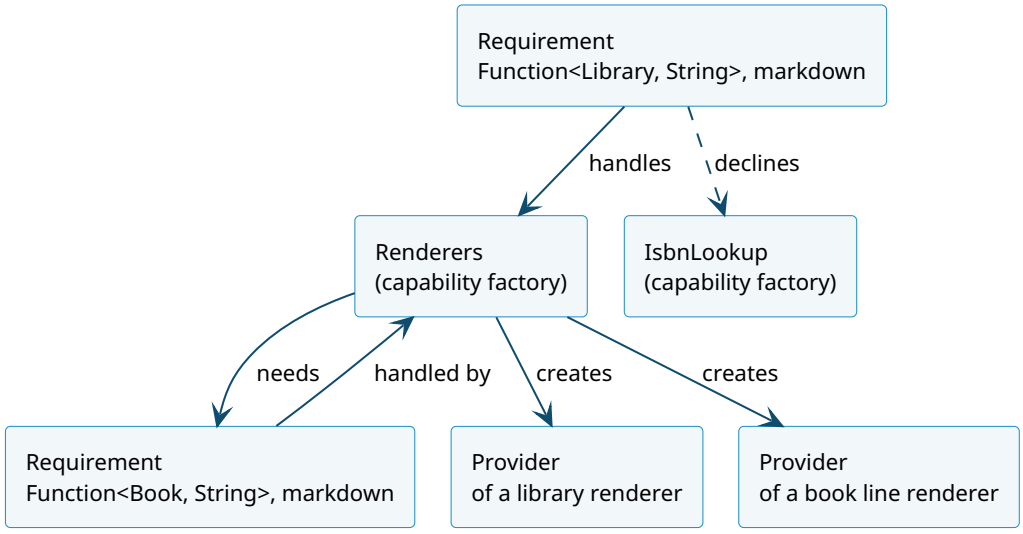


Figure 6. Resolution of a requirement that depends on another

The loader's `load` returns a `Flow.Publisher` of providers, and blocking conveniences, `loadAll` and `loadOne`, flatten them into capabilities. Resolution runs on a pump (Chapter 9): every requirement is resolved once per resolution however many factories ask for it, a cycle is reported with its path instead of hanging, and each step is a span.

10.2. Requirements

A `Requirement` is a record of a type and a list of payloads. The type says what is wanted and may be generic; the payloads select and parameterize.

```
// Anything that is a Library
Requirement.of(Library.class)

// A supplier of libraries, not of books
Requirement.of(new TypeToken<Supplier<Library>>() {})

// A renderer of libraries to text, for the markdown format
Requirement.of(new TypeToken<Function<Library, String>>() {}, new Format("markdown"))
```

With no payloads, a requirement is a plain lookup by type, which is what `ServiceLoader`

does, but generic, lazy and asynchronous. The type must be ground: a class, or a parameterized type whose arguments are classes or wildcards, never a type variable (Chapter 8).

Because a requirement is a cache key within a resolution, payloads should have value equality. Records do; the examples use them for formats and ISBNs. A type obtained from reflection, from a `TypeToken` or built some other way is equal to another describing the same type, so it does not matter where a type came from.

The example keeps its generic requirement types in one place:

Listing 45. Generic types used in requirements (`capability/⋯/Requirements.java`)

```
/**
 * Generic types used in requirements. A class literal cannot say
 * Function<Library, String>; a type token can.
 */
public final class Requirements {

    public static final Type LIBRARY_SUPPLIER =
            new TypeToken<Supplier<Library>>() {}.getType();

    public static final Type LIBRARY_RENDERER =
            new TypeToken<Function<Library, String>>() {}.getType();

    public static final Type BOOK_RENDERER =
            new TypeToken<Function<Book, String>>() {}.getType();

    private Requirements() {
    }

}
```

Any other object can be a requirement too, and is handed to factories as it is. `IsbnLookup`, below, handles requirements that are `Isbn` records.

10.3. Factories with capability methods

Most factories are written as methods. A class extending `ReflectiveCapabilityFactory` contributes one capability per method annotated with `@Capability`, behind a single entry in `module-info.java`:

Listing 46. Four capabilities in one class (capability/⋯/Renderers.java)

```
/**
 * One class, one entry in module-info, four capabilities. What each method provides
 * is its generic return type; what it needs are its parameters.
 */
public class Renderers extends ReflectiveCapabilityFactory {

    public Renderers() {
        super(MethodHandles.lookup());                                    ①
    }

    @Capability
    public Supplier<Library> sample() {                                   ②
        return Samples::library;
    }

    @Capability
    public Function<Book, String> markdownLine(Format format) {           ③
        if (!"markdown".equals(format.name())) {
            return null;                                                  ④
        }
        return book -> "* _" + book.getName() + "_ (" + book.getYear() + ")";
    }

    @Capability
    public CompletionStage<Function<Library, String>> markdown(
            Format format,
            Loader loader,                                                ⑤
            Span span) {
        if (!"markdown".equals(format.name())) {
            return null;
        }
        span.setAttribute("format", format.name());
        CompletableFuture<List<Function<Book, String>>> lines =
                loader.loadAll(Requirement.of(BOOK_RENDERER, format));    ⑥
        return lines.thenApply(renderers -> library -> "# " + library.getName() + "\n\n"
                + library.getBooks().stream().map(renderers.getFirst()).collect(joining("\n")));
    }

    @Capability
    public Function<Library, String> csv(Format format) {
        if (!"csv".equals(format.name())) {
            return null;
        }
        return library -> new CsvLibraryHandler().write(library);
    }

}
```

① A method handle lookup from the declaring module, so the methods are invoked through method handles and need not be in an exported package.

② No parameters: answers `Requirement.of(new TypeToken<Supplier<Library>>() {})`, and nothing else, because the generic return type is what is matched.

③ One parameter that is not injected takes a payload: this method is offered requirements of a compatible type with exactly one payload that is a `Format`.

④ Returning null declines. The method saw the requirement and it was not for markdown.

⑤ `Loader` and `Span` are injected by type. The method's own span is a child of the resolution's.

⑥ A dependency, resolved within the same resolution. The method does not block: it returns a stage that completes when the dependency is resolved.

What a method provides is its generic return type, matched with `Types.isAssignable`, so type arguments count. A method may return its capability directly, or wrapped in a way that says how it is delivered:

Return type	Meaning
`X`	One capability, now
`CompletionStage<X>`	One capability, later
`Flow.Publisher<X>`	Any number of capabilities, as they are produced
`CapabilityProvider<X>`, `Flow.Publisher<CapabilityProvider<X>>`	Providers, for methods that want to control them
`X[]`, `List<X>` and other iterables	The container is matched first; if it does not satisfy the requirement, its elements are offered as separate capabilities

Delivery wrappers are always unwrapped: a requirement for `Object` gets the values, not the publisher. Containers are values that hold others, so a requirement for `List<Book>` gets the list from a method returning one, while a requirement for `Book` gets its elements.

What a method takes is its parameters. `Loader`, `Span`, `Logger`, `Meter`, `OpenTelemetry` and `Type` (the required type, for a method that serves a range of them) are injected by their exact type, in any position. Every other parameter takes a payload, in order: a method matches when it has as many payload parameters as the requirement has payloads, and each payload is an instance of its parameter's class. A requirement that is neither a `Requirement` nor a type is

read as one payload with `Object` as the type, so a method taking an `Isbn` answers `loadAll(new Isbn(…))`.

When several methods match, all of them contribute, the most specific provided type first. Choosing between the capabilities is the caller's business, as with any requirement several factories answer.

10.4. Factories against the interface

A factory can also implement `CapabilityFactory` directly. That is the right choice when the factory interprets its requirements in a way no method signature can express, or simply for small factories with one job:

Listing 47. A factory written against the interface (`capability/…/IsbnLookup.java`)

```
/**
 * A factory written against the interface: it interprets requirements of its own type
 * and depends on another capability, the libraries, which it requests through the loader.
 */
public class IsbnLookup implements CapabilityFactory<Isbn, Book> {

    @Override
    public boolean canHandle(Object requirement) {                          ①
        return requirement instanceof Isbn;
    }

    @Override
    public Flow.Publisher<CapabilityProvider<Book>> create(
            Isbn isbn,
            Loader loader,
            Span span,
            Logger logger,
            Meter meter) {                                                  ②
        span.setAttribute("isbn", isbn.value());
        CompletableFuture<List<Supplier<Library>>> libraries =
                loader.loadAll(Requirement.of(LIBRARY_SUPPLIER));           ③
        return Flows.fromFuture(libraries.thenApply(suppliers -> CapabilityProvider.of(
                suppliers.stream()
                    .map(Supplier::get)
                    .flatMap(library -> library.getBooks().stream())
                    .filter(book -> isbn.value().equals(book.getIsbn()))
                    .toArray(Book[]::new))));                               ④
    }

    @Override
    public String toString() {
        return "IsbnLookup";
```

```
    }

}
```

① `canHandle` is called for every requirement in a resolution, so it must be cheap.

② `create` is called once per requirement per resolution, on the resolution's pump, in a span. The interface's default `create(requirement, loader)` creates that span and calls this method with it, a logger, and a meter for measuring what providing the capability costs.

③ Factories raise dependencies through the loader they are given, never through a loader of their own, so that the dependency is resolved within the same resolution, memoized, and checked for cycles.

④ The providers are published when the dependency completes.

10.5. Registering and loading

A factory is a Java service. The example's module declares all three of its factories, and the `ResourceSetContributor` discussed below:

Listing 48. Registering capability factories (`capability/src/main/java/module-info.java`)

```
import org.example.capability.CsvContributor;
import org.example.capability.CsvHandlers;
import org.example.capability.IsbnLookup;
import org.example.capability.Renderers;
import org.nasdanika.sdk.runtime.common.capability.CapabilityFactory;
import org.nasdanika.sdk.runtime.common.services.ResourceSetContributor;

module org.example.capability {

    requires transitive org.example.library;

    exports org.example.capability;

    // One entry per class, however many capabilities it has
    provides CapabilityFactory with
        Renderers,
        IsbnLookup,
        CsvHandlers;

    provides ResourceSetContributor with CsvContributor;

}
```

`CapabilityLoader.getInstance()` is the shared loader over every factory the service loader

finds. It is created on first use, and it is stateless: memoization lives in a resolution, not in the loader, so sharing it is safe.

Listing 49. Loading by generic type (capability/src/test/java/⋯/TestCapabilities.java)

```
CapabilityLoader loader = CapabilityLoader.getInstance(); // Factories registered as services
Supplier<Library> sample = loader.loadOne(Requirement.of(LIBRARY_SUPPLIER));
assertThat(sample.get().getName()).isEqualTo("A Modeler's Shelf");
```

Listing 50. Loading by type and payload

```
Function<Library, String> markdown = CapabilityLoader.getInstance()
        .loadOne(Requirement.of(LIBRARY_RENDERER, new Format("markdown")));
String text = markdown.apply(Samples.library());
assertThat(text)
    .startsWith("# A Modeler's Shelf")
    .contains("* _Domain-Driven Design_ (2003)");
```

Listing 51. A plain object as the requirement

```
List<Book> books = CapabilityLoader.getInstance().loadAll(new Isbn("978-0321125217"));
assertThat(books).extracting(Book::getName).containsExactly("Domain-Driven Design");
```

`loadAll` and `loadOne` are the blocking conveniences, for the edge of a program. Each creates a pump and pumps it on the calling thread until the result is complete; `loadOne` stops at the first capability and abandons work still queued. Inside a larger asynchronous computation, call `load(requirement, pump)` with the computation's pump instead, so that resolution is ordered and settled with the rest of the work. `load(requirement)` without a pump resolves on virtual threads, for subscribers that do not pump.

10.6. Composing sources and loaders

A loader resolves against a **source** of factories, and the two can be composed separately, with different effects.

Composing **sources** puts factories into one resolution. A factory from one source can then satisfy a dependency raised by a factory from another. That is the way to put a local, task-specific factory in front of everything registered globally, for a test double or a one-off override:

Listing 52. A local source in front of the registered factories

```
/**
 * A test double, in front of everything registered: its sample wins with loadOne.
 */
public static class EmptyLibrary {

    @Capability
    public Supplier<Library> empty() {
        return () -> {
            Library library = LibraryFactory.eINSTANCE.createLibrary();
            library.setName("Empty");
            return library;
        };
    }

}

@Test
void localInFrontOfGlobal() {
    var factory = new ReflectiveCapabilityFactory(MethodHandles.lookup(), new EmptyLibrary());
    CapabilityFactorySource local = CapabilityFactorySource.of(factory);
    CapabilityFactorySource global = CapabilityFactorySource.serviceLoader();
    CapabilityFactorySource both = CapabilityFactorySource.concat(local, global);
    CapabilityLoader loader = CapabilityLoader.of(both);
    Supplier<Library> first = loader.loadOne(Requirement.of(LIBRARY_SUPPLIER));
    assertThat(first.get().getName()).isEqualTo("Empty");
    List<Supplier<Library>> all = loader.loadAll(Requirement.of(LIBRARY_SUPPLIER));
    assertThat(all).hasSize(2); // The registered sample is still there
}
```

Filtering a source is stronger than filtering results. A factory that is not in the filtered set cannot satisfy a dependency raised by another factory either, so a filtered source is a closure, not a check on the surface. That is how a trust level is enforced structurally:

Listing 53. Only factories from trusted modules

```
// Only factories from our own modules: a factory outside the set cannot satisfy
// a dependency raised inside it either
CapabilityFactorySource trusted = CapabilityFactorySource.serviceLoader()
        .filtered(f -> f.getClass().getModule().getName().startsWith("org.example."));
CapabilityLoader loader = CapabilityLoader.of(trusted);
assertThat(loader.<Book>loadAll(new Isbn("978-0201633610"))).hasSize(1);
```

Composing **loaders** keeps resolutions separate. `CapabilityLoader.concat` lets every loader contribute, in order; `fallback` consults the next loader only if the previous one produced nothing; `cached` memoizes across calls, which a loader deliberately does not do on its own, and should be used only where the same requirements recur and the answers are stable.

Sources are closeable. Closing one closes the factories in it that are `AutoCloseable`, which is how a factory that holds resources, a connection pool for example, is disposed of. A source owns its factories; a loader borrowing a shared source does not, which is why `close` is on the source.

10.7. When nothing comes back

A resolution that produces nothing has two very different causes: no factory claimed the requirement, or a factory claimed it and produced nothing. They have different fixes, and the explaining listener tells them apart:

Listing 54. Explaining a resolution

```
List<String> lines = new ArrayList<>();
CapabilityLoader loader = CapabilityLoader
        .of(new Renderers(), new IsbnLookup())
        .withListener(ResolutionListener.explaining(lines::add));
loader.loadAll(Requirement.of(LIBRARY_RENDERER, new Format("markdown")));
lines.forEach(System.out::println);
```

For the markdown renderer it prints the following, shortened here: package names are removed, the factory's list of methods is elided, and `F<A>` stands for `Function<A, String>` with the markdown format as its payload.

```
requested F<Library>
  Renderers[...] handles F<Library>
  IsbnLookup declines F<Library>
  Renderers[...] needs F<Book> for F<Library>
requested F<Book> by F<Library>
  Renderers[...] produced 1 provider(s) for F<Library>
completed F<Library> with 1 provider(s) in 1389 us
  Renderers[...] handles F<Book>
  IsbnLookup declines F<Book>
  Renderers[...] produced 1 provider(s) for F<Book>
completed F<Book> with 1 provider(s) in 649 us
```

The order of the lines is worth a second look. The library renderer's provider was produced before the book renderer it depends on was resolved, because the method returned a stage at once; the provider delivers its capability when the stage completes.

Listeners receive a sealed hierarchy of `ResolutionEvent` records (`Requested`, `CacheHit`,

`FactoryConsidered`, `DependencyRequested`, `Resolved`, `Failed`, `Completed`), so a listener of your own pattern-matches on them. `FactoryNodeCollector` is one that rebuilds the dependency graph of a resolution, for troubleshooting or for drawing:

Listing 55. Collecting the dependency graph

```
FactoryNodeCollector collector = FactoryNodeCollector.create();
CapabilityLoader.of(new Renderers())
    .withListener(collector)
    .loadAll(Requirement.of(LIBRARY_RENDERER, new Format("markdown")));
// The markdown renderer needed a book renderer
assertThat(collector.getFactoryNodes())
    .anySatisfy(node -> assertThat(node.dependencies()).hasSize(1));
```

A requirement cycle fails the resolution with a `StallException` naming the path, for example `cycle: chicken -> egg -> chicken`, exactly as in a transformation. And every resolution is visible in telemetry: `CapabilityLoader.loadAll` or `loadOne`, a `Transformer.create` span per requirement, a `CapabilityFactory.create` span per factory that handles it, and a `CapabilityFactory.method` span per capability method invoked, with the requirement, the factory and the number of providers as attributes.

10.8. A case study: file names as pipelines

The runtime uses capabilities for one job that shows the framework at its best: deciding how to load a resource from its name.

A `ContentsHandlingResource` splits the last segment of its URI on dots and reads the parts as qualifiers, right to left. `reading-group.library.groovy` has the qualifiers `groovy` and `library`. The resource asks the capability loader for a `ResourceContentsHandler` with a requirement that carries the resource, the expected contents type and the qualifiers, starting with the leftmost qualifier, so that the most specific handler wins. The handler for `library` needs a compiled script, so it asks for a handler of the `groovy` qualifier to its right, and the two together turn a Groovy script into library objects (Chapter 19).

Adding a file format is therefore three small pieces. The example adds CSV for libraries. First the handler, which reads and writes the format:

Listing 56. A resource contents handler (`capability/⋯/CsvLibraryHandler.java`)

```
/**
 * Libraries as CSV: an optional "# name" line, a header, then one book per line with
 * its authors separated by semicolons. Loading builds the model; saving writes it back.
 */
public class CsvLibraryHandler implements ResourceContentsHandler<EObject[]> {

    static final String HEADER = "title,year,genre,isbn,authors";

    @Override
    public Order getOrder() {
        return Order.of(0);
    }

    @Override
    public EObject[] load(InputStream in, Map<?, ?> options) throws IOException {
        LibraryFactory factory = LibraryFactory.eINSTANCE;
        Library library = factory.createLibrary();
        Map<String, Author> authors = new LinkedHashMap<>();
        var reader = new BufferedReader(new InputStreamReader(in, StandardCharsets.UTF_8));
        for (String line = reader.readLine(); line != null; line = reader.readLine()) {
            if (line.startsWith("# ")) {
                library.setName(line.substring(2).trim());
            } else if (!line.isBlank() && !line.equals(HEADER)) {
                List<String> cells = split(line);
                Book book = factory.createBook();
                book.setName(cells.get(0));
                book.setYear(Integer.parseInt(cells.get(1)));
                book.setGenre(Genre.get(cells.get(2)));
                book.setIsbn(cells.get(3).isEmpty() ? null : cells.get(3));
                for (String name: cells.get(4).split(";")) {
                    book.getAuthors().add(authors.computeIfAbsent(name.trim(), n -> {
                        Author author = factory.createAuthor();
                        author.setName(n);
                        return author;
                    }));
                }
                library.getBooks().add(book);
            }
        }
        library.getAuthors().addAll(authors.values());
        return new EObject[] { library };
    }

    @Override
    public void save(EObject[] roots, OutputStream out, Map<?, ?> options) throws IOException {
        out.write(write((Library) roots[0]).getBytes(StandardCharsets.UTF_8));
    }
```

Then a capability method that offers the handler for the `csv` qualifier and declines everything else:

Listing 57. Offering the handler as a capability (`capability/⋯/CsvHandlers.java`)

```
/**
 * Offers the CSV handler for resources whose name has a {@code csv} qualifier at the
 * position being resolved, such as {@code shelf.csv}. It reads the bytes of the resource
 * itself, so it does not chain to a handler on its right.
 */
public class CsvHandlers extends ReflectiveCapabilityFactory {

    public CsvHandlers() {
        super(MethodHandles.lookup());
    }

    @Capability
    public ResourceContentsHandler<EObject[]> csv(ResourceContentsHandler.Requirement r) {
        String qualifier = r.getQualifiers()[r.getQualifierIndex()];
        boolean mine = r.getContentsType() == EObject[].class
                && "csv".equalsIgnoreCase(qualifier);
        return mine ? new CsvLibraryHandler() : null;
    }

}
```

And a resource set contributor that registers a factory of contents-handling resources for the `csv` extension:

Listing 58. Registering the extension (`capability/⋯/CsvContributor.java`)

```
/**
 * Registers a factory for the csv extension whose resources find their contents handler
 * as a capability. The registration is a plain service (a lookup by extension); the
 * handler is a capability (a requirement the providers interpret).
 */
public class CsvContributor implements ResourceSetContributor {

    @Override
    public void contribute(ResourceSet resourceSet, Span span, Logger logger) {
        CapabilityLoader loader = CapabilityLoader.getInstance();
        var factory = new ContentsHandlingResourceFactory(loader, resourceSet);
        resourceSet.getResourceFactoryRegistry().getExtensionToFactoryMap().put("csv", factory);
    }

}
```

The split follows the principle of services before capabilities. Registering a factory for an extension is a lookup by type: every contributor, applied to every resource set, so it is a service. Choosing a handler is a requirement that providers interpret, a resource with a name and a contents type, so it is a capability.

With the three pieces on the module path, a CSV file is a model:

Listing 59. A CSV file, loaded, edited and saved as a model

```
Path file = directory.resolve("shelf.csv");
Files.writeString(file, """
        # Two classics
        title,year,genre,isbn,authors
        Domain-Driven Design,2003,TECHNOLOGY,978-0321125217,Eric Evans
        "Design Patterns, Elements",1994,TECHNOLOGY,,Erich Gamma; Richard Helm
        """);

URI uri = URI.createFileURI(file.toString());
Resource resource = NasdanikaResourceSet.createAndConfigure().getResource(uri, true);
Library library = (Library) resource.getContents().getFirst();
assertThat(library.getName()).isEqualTo("Two classics");
assertThat(library.getAuthors()).hasSize(3);
assertThat(library.getBooks().get(1).getName()).isEqualTo("Design Patterns, Elements");

library.getBooks().getFirst().setYear(2004);   // Edit the model, save the CSV
resource.save(null);
assertThat(Files.readString(file)).contains("Domain-Driven Design,2004,TECHNOLOGY");
```

The mechanism chains. A handler for one qualifier can ask for the handler of the qualifier to its right and transform what that one produces, which is how `library` builds on `groovy`. Each qualifier is a step, and the file name declares the pipeline.

10.9. When to use which

Use	When
A Java service	The lookup is by type alone, such as every contributor or every `EPackage`.
A capability	There is a requirement for providers to interpret, providers may decline, or resolving it requires resolving something else.
Adaptation	You already have an object and need it in another shape (Chapter 8).

The capability framework is how the command line is assembled, which is where the next part of the book is heading after a detour through telemetry. Every resolution above was a span, and the next chapter explains why that was free.

Part III: Observability

Telemetry as part of the code: how the runtime instruments itself, how to watch a unit of work while it runs, and how telemetry and recorded changes become models kept next to the models they describe.

Chapter 11. Telemetry in the runtime

Telemetry usually arrives late in a project. The code works, it goes to production, something misbehaves, and somebody adds logging, then tracing, then a vendor's agent that instruments what it can guess at. By then the code was written without the question "how will anyone know what this did?", and it shows.

In the runtime, telemetry is part of the code from the first line, and the first program in Chapter 6 showed what that buys: a program with no telemetry code of its own was observable, because everything it called reported what it did. This chapter explains the conventions behind that, how to instrument your own code the same way, and where the telemetry goes.

11.1. Two kinds of telemetry

Mainstream observability is built for the firehose. It collects everything cheaply, samples, keeps it for days or weeks, and answers "is it healthy right now?". That is the right tool for operations, and the runtime does not compete with it.

There is another kind, which most organizations do not have at all: telemetry that is collected deliberately, kept indefinitely, and resolved against models of the systems and the work, so that years later it can answer "who changed what, in the course of what work?". I call it strategic telemetry, and its rule is that every signal is precious. It is cheap at the scale it operates on, and what is expensive is not being able to answer.

Both kinds start from the same instrumentation. Code reports through the OpenTelemetry API, and whoever runs the code decides whether the signals go to an operational backend, to a console, into a model that is kept forever, or to all three.

11.2. The conventions

Four conventions make the runtime's telemetry consistent. They were stated in Chapter 2; here is what each means in code.

Spans for operations, log records for what happens in them. An operation that has a

duration is a span: loading a resource, resolving a requirement, executing a command. Everything that happens within an operation is a log record emitted through the OpenTelemetry Logs API and correlated with the current span: a message, a warning, a measurement, progress. Span events, `Span.addEvent`, are not used, because OpenTelemetry is moving events onto the Logs API, where an event is a log record with an event name.

One OpenTelemetry per unit of work. A command, a request and a test are units of work. Each builds or is handed its own `OpenTelemetry` instance and makes it current, the way it would make a span current. Instrumented code gets the instance from the context, never from `GlobalOpenTelemetry`, which the runtime does not use. Two commands running in one JVM can therefore send their telemetry to different places, and a test can capture exactly the telemetry of the code it tests.

The context carries everything. The current instance travels in the OpenTelemetry `Context` next to the current span, so it crosses threads wherever the context does: the pump propagates it to its tasks, and content written on another thread runs in the context of the code that asked for it.

Watching is subscribing. A console, a progress bar, a test or a browser watching a unit of work subscribes to the same spans and log records an exporter receives, through a publishing wrapper (Chapter 12).

11.3. The Telemetry class

`org.nasdanika.sdk.runtime.common.telemetry.Telemetry` is where instrumented code gets its instance, and where the runtime's attribute keys and event names are defined.

Member	Purpose
`current()`	The instance in the current context, or the no-op instance
`makeCurrent(openTelemetry)`, `with(context, openTelemetry)`	Make an instance current, as a scope to close
`of(notifier)`	The current instance, or the one attached to a resource set, for EMF code that runs outside a unit of work
`attach(resourceSet, openTelemetry)`	Attach an instance to a resource set

Member	Purpose
`tracer`, `logger`, `meter`	The runtime's tracer, logger and meter of an instance
`inSpan(tracer, name, attributes, body)`	Run a body in a new current span; a failure is recorded on the span and rethrown
`log(logger, severity, eventName, body, attributes)`	Emit a log record in the current context, if the logger is enabled for the severity
`recordFailure(span, failure)`	Record an exception and set the error status
`w3cPropagators()`, `extract`, `extractFromEnvironment`	Continue a trace started by another process

The keys and event names are listed in Appendix B.

11.4. Instrumenting your code

The telemetry example catalogs books, one at a time, and reports what it does. It starts with a few constants:

Listing 60. Instrumentation scope and attribute keys (`telemetry/⋯/Cataloging.java`)

```
/**
 * The instrumentation scope is the module name, by the runtime's identity rule.
 */
static final String SCOPE = Cataloging.class.getModule().getName();

static final AttributeKey<String> TITLE = AttributeKey.stringKey("library.book.title");
static final AttributeKey<Long> BOOKS = AttributeKey.longKey("library.books");
```

The instrumentation scope identifies who produced a span or a log record. By the runtime's identity rule it is the JPMS module name, which the code reads from its own class rather than repeating it in a string. Attribute keys are typed, `AttributeKey<String>` and `AttributeKey<Long>`, and named in the dotted lowercase style of OpenTelemetry's semantic conventions, with a prefix that says whose they are.

The operation itself:

Listing 61. Work in a span, with log records, progress and a metric

```
public static int catalog(List<String> titles) {
    OpenTelemetry openTelemetry = Telemetry.current();                ①
```

```
    Tracer tracer = openTelemetry.getTracer(SCOPE);
    Logger logger = openTelemetry.getLogsBridge().get(SCOPE);
    LongCounter cataloged = openTelemetry.getMeter(SCOPE)
            .counterBuilder("library.books.cataloged")
            .build();
    long total = titles.size();
    Attributes attributes = Attributes.of(BOOKS, total);

    return Telemetry.inSpan(tracer, "Catalog books", attributes, span -> { ②
        int done = 0;
        for (String title: titles) {
            try {
                catalogOne(tracer, title);
                cataloged.add(1);                                          ③
            } catch (IllegalArgumentException e) {
                String message = "Skipped: " + e.getMessage();
                Telemetry.log(logger, Severity.WARN, null, message, null);    ④
            }
            ++done;
            Telemetry.log(                                                 ⑤
                    logger,
                    Severity.INFO,
                    PROGRESS_EVENT,
                    "Processed " + done + " of " + total,
                    Attributes.of(PROGRESS_WORKED, (long) done, PROGRESS_TOTAL, total));
        }
        return done;
    });
}
```

① The instance of the current unit of work. The code does not know or care whether it is an SDK exporting to a backend, a publishing wrapper feeding a console, or the no-op instance.

② `Telemetry.inSpan` starts a span, makes it current, runs the body with the span, records a failure if the body throws, and ends the span. It returns what the body returns.

③ A counter from the instance's meter. Metrics are aggregated by the SDK and exported periodically.

④ A warning is a log record in the current span. The span does not fail: skipping one book is part of the operation, not a failure of it.

⑤ Progress is a log record with the event name `nasdanika.progress` and two attributes, the work done and the total. Anything watching the operation can draw a progress bar from it.

When the work is not a lexical block, or when you want control over the span, start one yourself:

Listing 62. A span started and ended by hand

```
static void catalogOne(Tracer tracer, String title) {
    Span span = tracer.spanBuilder("Catalog a book")
            .setAttribute(TITLE, title)                                  ①
            .startSpan();
    try (Scope scope = span.makeCurrent()) {                             ②
        if (title.isBlank()) {
            throw new IllegalArgumentException("a book needs a title");
        }
        span.setStatus(StatusCode.OK);
    } catch (RuntimeException e) {
        Telemetry.recordFailure(span, e);                                ③
        throw e;
    } finally {
        span.end();                                                      ④
    }
}
```

① Attributes known up front go on the builder.

② Making the span current is what makes spans and log records created inside it its children.

③ A failure is recorded on the span with its exception, and the span gets the error status.

④ A span must end, whatever happens, or it is never exported.

11.5. Units of work

Instrumented code only reports. The unit of work decides where the reports go, by making an instance current:

Listing 63. A unit of work, watched live (`telemetry/⋯/UnitOfWork.java`)

```
public static void main(String[] args) throws InterruptedException {
    SpanTree tree = new SpanTree(System.out);
    try (var openTelemetry = new PublishingOpenTelemetry(OpenTelemetry.noop())) { ①
        openTelemetry.subscribe(tree);                                   ②
        try (Scope scope = Telemetry.makeCurrent(openTelemetry)) {       ③
            Cataloging.catalog(List.of("Domain-Driven Design", " ", "Design Patterns"));
        }
    }                                                                    ④
    tree.await();                                                        ⑤
}
```

① A publishing wrapper around the no-op instance: no SDK, and everything is published to subscribers.

② A subscriber that prints the spans as a tree, shown in Chapter 12.

③ The instance is current for the unit of work, and only for it.

④ Closing the wrapper completes the subscribers.

⑤ Delivery is asynchronous by default, so the program waits for the subscriber to finish.

It prints:

```
+ Catalog books {library.books=3}
  + Catalog a book {library.book.title=Domain-Driven Design}
  - Catalog a book (0 ms)
  [1/3] Processed 1 of 3
  + Catalog a book {library.book.title= }
    ! a book needs a title
  - Catalog a book (0 ms)
  WARN Skipped: a book needs a title
  [2/3] Processed 2 of 3
  + Catalog a book {library.book.title=Design Patterns}
  - Catalog a book (0 ms)
  [3/3] Processed 3 of 3
- Catalog books (5 ms)
```

In a real assembly the unit of work is created once, in one place. The launcher creates one per command, with an OpenTelemetry SDK whose exporters save the telemetry as models (Chapter 18). The HTTP handler creates one per request (Chapter 15). A test creates one per test, usually a publishing wrapper with deterministic delivery, so that it can assert on what the code reported.

11.6. What the runtime instruments

Most of the telemetry of a typical program comes from the runtime itself.

Spans	Produced by
`ResourceSetContributor.configure`, with a log record per contribution	Configuring a resource set from services
`Contribute To Resource Set`	Each contributor that uses the interface's default
`Resource.load`, `Resource.save`, `Resource.unload`, with URI, bytes and root objects	Every resource the runtime's factories create, and yours if you use `ResourceTelemetry`

Spans	Produced by
`URIHandler.read`, `URIHandler.write`, `URIHandler.delete`, with progress every second	Every stream a configured resource set opens
`Content.writeTo`, `Content.pipe`, and client spans named after the HTTP method	Transfers of volume content (Chapter 14)
Server spans named `<method> <route>`	Requests through the volume HTTP handlers (Chapter 15)
`Transformer.transform`, `Transformer.create`	Transformations (Chapter 9)
`CapabilityLoader.loadAll`, `CapabilityLoader.loadOne`, `CapabilityFactory.create`, `CapabilityFactory.method`, `Create Capability`	Capability resolution (Chapter 10)
One span per executed command, named after the command	The command line (Chapter 16)
`Evaluating script`, `Loading <Type>`, `Setting <feature>`, with script line numbers	The Groovy DSL (Chapter 19)



Two span names are phrases, `Contribute To Resource Set` and `Create Capability`, while the rest follow `Type.operation`. Renaming them to `ResourceSetContributor.contribute` and `CapabilityFactory.create` would make the names predictable, but `CapabilityFactory.create` is already the name of the loader's span around the same call, so the factory's own span would need another name, or could be dropped.

11.7. Crossing threads and processes

Within a JVM, the OpenTelemetry context lives in a thread local, and the runtime carries it wherever work moves. A pump task runs in the context that was current when it was enqueued. A stage tracked or awaited on a pump completes in the caller's context. Content produced by a writer on a virtual thread runs the writer in the context of the reader that started it. Spans therefore nest the way the work nests, whichever thread does it, and the

unit of work's instance travels along.

Across processes, context travels in W3C trace context headers. `Telemetry.w3cPropagators()` returns the propagators for W3C trace context and baggage; an instance built with them injects `traceparent` into the requests the volume HTTP client makes, and the volume HTTP handler extracts it, so one trace spans both sides (Chapter 15).

A process can also be invoked by something that has a trace of its own: a build step, a function call, a workflow engine. The runtime continues it from a map of carrier values, such as request parameters, or from the environment variables `TRACEPARENT`, `TRACESTATE` and `BAGGAGE`:

Listing 64. Continuing a trace started elsewhere (`telemetry/⋯/Propagation.java`)

```
/**
 * Runs the work as part of the trace whose context the carrier holds, for example
 * traceparent from request parameters or TRACEPARENT from the environment.
 */
public static <T> T continueTrace(
        OpenTelemetry openTelemetry,
        Map<String, String> carrier,
        Supplier<T> work) {
    Context remote = Telemetry.extract(openTelemetry, Context.current(), carrier); ①
    try (Scope scope = Telemetry.with(remote, openTelemetry).makeCurrent()) {     ②
        return work.get();
    }
}

/**
 * The launcher's way: TRACEPARENT, TRACESTATE and BAGGAGE environment variables.
 */
public static <T> T continueFromEnvironment(OpenTelemetry openTelemetry, Supplier<T> work) {
    Context remote = Telemetry.extractFromEnvironment(openTelemetry, Context.current());
    try (Scope scope = Telemetry.with(remote, openTelemetry).makeCurrent()) {
        return work.get();
    }
}
```

① Extraction reads `traceparent`, or `TRACEPARENT` as environment variables are named, with the instance's propagators. If the carrier holds nothing valid, the context is returned unchanged.

② The extracted context and the instance become current together, so spans started inside continue the remote trace.

Listing 65. A span in the continued trace

```
var propagating = OpenTelemetry.propagating(Telemetry.w3cPropagators());   ①
var carrier = Map.of("traceparent", "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01");
String traceId = Propagation.continueTrace(propagating, carrier, () -> {
    Tracer tracer = Telemetry.tracer(Telemetry.current());
    Span span = tracer.spanBuilder("Handle request").startSpan();
    try {
        return span.getSpanContext().getTraceId();
    } finally {
        span.end();
    }
});
assertThat(traceId).isEqualTo("4bf92f3577b34da6a3ce929d0e0e4736");          ②
```

① The no-op instance with propagators: enough to extract a context, with no SDK.

② The span started inside belongs to the trace from the carrier.

The launcher does this for every command it runs, so `nsdk` started by a CI job with `TRACEPARENT` set adds its spans to the job's trace.

11.8. Where telemetry goes

Instrumented code never decides, which is the point. The options are the unit of work's:

- **Nowhere.** With no instance current, `Telemetry.current()` is the no-op instance, and instrumentation costs almost nothing.
- **To subscribers.** A `PublishingOpenTelemetry` wrapper publishes spans and log records as they happen, with or without an SDK behind it (Chapter 12).
- **To an operational backend.** An OpenTelemetry SDK with an OTLP exporter sends everything to a collector, as any OpenTelemetry application would.
- **Into models.** An SDK with the runtime's model exporters saves traces, logs and metrics as EMF resources, next to the models they describe (Chapter 13).

These combine. A launcher can export to a backend and to models at once, and still publish to a console.

Chapter 12. Watching work live

Exported span data arrives when a span ends, often in a batch some seconds later. That is fine for looking at what happened, and useless for a person watching a long operation, who wants to know what is happening now and how far along it is. It is also awkward for tests, which want to see what the code under test reported, in order, without an exporter and a collector in between.

The runtime publishes telemetry as it happens. This chapter shows how, and how to subscribe.

12.1. Publishing OpenTelemetry

`PublishingOpenTelemetry` wraps any `OpenTelemetry` instance, the no-op instance included, and is a `Flow.Publisher<SpanEvent>`. Its tracers publish an event for every change to the spans they start: started, attribute set, status set, exception recorded, renamed, ended. Its loggers publish every log record they emit, together with the span that was current. Meters and propagators are the delegate's.

Delivery has three properties that make it safe to leave on:

- **It never blocks the instrumented code.** Events are offered to a bounded buffer per subscriber; a subscriber that cannot keep up loses events, and the losses are counted in `getDroppedCount()`.
- **It costs nothing when nobody listens.** With no subscribers, events are not even created.
- **It does not need an SDK.** Over the no-op instance, spans record nothing, and events are still published. Tracers and loggers report themselves as enabled while anybody is subscribed, so instrumentation that checks before doing work does not go quiet on a progress bar because no SDK is installed.

Events are delivered asynchronously, in order per subscriber, on virtual threads by default. For tests and consoles, a tracer provider created with `Runnable::run` delivers each event on the thread that produced it:

Listing 66. Deterministic delivery for tests (`telemetry/src/test/java/⋯/TestTelemetry.java`)

```
/**
 * Delivers events on the instrumented thread, in order: deterministic, as tests want.
 */
static PublishingOpenTelemetry deterministic(OpenTelemetry delegate) {
    var provider = new PublishingTracerProvider(TracerProvider.noop(), Runnable::run, 1024);
    return new PublishingOpenTelemetry(delegate, provider);
}
```

12.2. Span events

`SpanEvent` is a sealed interface with one record per kind of event:

Event	When
`Started`	A span started, with its name, kind and initial attributes
`AttributeSet`	An attribute was set on a live span
`StatusSet`	The status was set
`ExceptionRecorded`	An exception was recorded
`Renamed`	The span was renamed
`Ended`	The span ended
`EventAdded`	`Span.addEvent` was called. The runtime itself does not do this; it emits log records instead
`LogEmitted`	A log record was emitted, with the publishing span that was current, if any

A subscriber pattern-matches on them, and its `switch` decides what a new kind of event does to it. With a `default` case, as in the tree below, it ignores the new kind. With a case for every kind and no `default`, it stops compiling when a kind is added, so the compiler shows what needs handling; compiled earlier and run against a newer runtime, it throws `MatchException` when the new kind arrives.

Every event has the span it concerns and a timestamp. A `PublishingSpan` knows its publishing parent, so a subscriber can compute nesting, and `isWithin(span)` tells whether a span is a descendant of another.

12.3. A live tree

The subscriber the previous chapter used prints spans as an indented tree, with errors, log records and progress:

Listing 67. Printing spans as they run (`telemetry/⋯/SpanTree.java`)

```
/**
 * Prints spans as an indented tree while they run: starts, errors, log records with
 * progress, and ends with durations. A console is one subscriber; a browser, a test or
 * a file is another.
 */
public class SpanTree implements Flow.Subscriber<SpanEvent> {

    private final PrintStream out;
    private final Map<PublishingSpan, Instant> started = new ConcurrentHashMap<>();
    private final CountDownLatch completed = new CountDownLatch(1);

    public SpanTree(PrintStream out) {
        this.out = out;
    }

    @Override
    public void onSubscribe(Flow.Subscription subscription) {
        subscription.request(Long.MAX_VALUE);
    }

    @Override
    public void onNext(SpanEvent event) {
        switch (event) {                                                    ①
            case SpanEvent.Started s -> {
                started.put(s.span(), s.timestamp());
                out.println(indent(s.span()) + "+ " + s.name() + format(s.attributes()));
            }
            case SpanEvent.StatusSet s when s.code() == StatusCode.ERROR ->
                out.println(indent(s.span()) + "  ! " + s.description());
            case SpanEvent.LogEmitted l ->
                out.println(indent(l.span()) + "  " + describe(l.record()));
            case SpanEvent.Ended e -> {
                Duration duration = Duration.between(started.remove(e.span()), e.timestamp());
                String name = e.span().getName() + " (" + duration.toMillis() + " ms)";
                out.println(indent(e.span()) + "- " + name);
            }
            default -> {
                // Attributes set later, exceptions and renames are not printed
            }
        }
    }

    private static String describe(LogRecord record) {
        if (Telemetry.PROGRESS_EVENT.equals(record.eventName())) {          ②
            Long worked = record.attributes().get(Telemetry.PROGRESS_WORKED);
```

```
            Long total = record.attributes().get(Telemetry.PROGRESS_TOTAL);
            return "[" + worked + "/" + total + "] " + record.bodyAsString();
        }
        return record.severity() + " " + record.bodyAsString();
    }
```

① One case per kind of event, with a guard for error statuses. Kinds the tree does not show fall through to `default`.

② Progress is recognized by its event name and drawn from its two attributes.

The same twenty lines could update a progress bar, write lines to a file, or push events to a browser. A console is one subscriber among others, not a second logging API.

12.4. Following one operation

A busy unit of work has many concurrent operations. `within(span)` filters the events to one span and the publishing spans started under it, which is what a progress view for one operation needs:

Listing 68. Subscribing to one operation

```
try (PublishingOpenTelemetry openTelemetry = deterministic(OpenTelemetry.noop());
        Scope scope = Telemetry.makeCurrent(openTelemetry)) {
    Tracer tracer = openTelemetry.getTracer("org.example.telemetry");
    Span unrelated = tracer.spanBuilder("Unrelated").startSpan();
    Span operation = tracer.spanBuilder("Operation").startSpan();

    Started started = new Started();
    openTelemetry.within((PublishingSpan) operation).subscribe(started);   ①

    try (Scope operationScope = operation.makeCurrent()) {
        tracer.spanBuilder("Step").startSpan().end();                      // Seen
    }
    Context elsewhere = Context.current().with(unrelated);
    tracer.spanBuilder("Elsewhere").setParent(elsewhere).startSpan().end(); // Not seen
    operation.end();
    unrelated.end();
    assertThat(started.names).containsExactly("Step");
}
```

① Only events of `operation` and its descendants are delivered. `Unrelated` and the span started under it are filtered out.

Nesting follows the context, not the tracer: a span started by any tracer of the same

provider while `operation` is current is its child, and is seen.

12.5. Testing with telemetry

Because telemetry is published without an SDK, a test can assert on what the code reported as easily as on what it returned:

Listing 69. Asserting on reported telemetry

```
var bytes = new ByteArrayOutputStream();
try (PublishingOpenTelemetry openTelemetry = deterministic(OpenTelemetry.noop())) {
    openTelemetry.subscribe(new SpanTree(new PrintStream(bytes, true, StandardCharsets.UTF_8)));
    try (Scope scope = Telemetry.makeCurrent(openTelemetry)) {
        List<String> titles = List.of("Domain-Driven Design", " ", "Design Patterns");
        assertThat(Cataloging.catalog(titles)).isEqualTo(3);
    }
}
String tree = bytes.toString(StandardCharsets.UTF_8).replaceAll("\\(\\d+ ms\\)", "(n ms)");
assertThat(tree)
    .contains("+ Catalog books {library.books=3}")
    .contains("  ! a book needs a title")
    .contains("  [2/3] Processed 2 of 3")
    .contains("- Catalog books (n ms)");
```

This changes what is worth testing. "The import reports progress for every book" and "a book without a title is a warning, not a failure" are now properties a test can check, and keep checking.

12.6. What comes next

The runtime's direction for live observation is a browser. A command, or a test, starts the JDK HTTP server on a random port, subscribes to its unit of work, and streams span events to a React UI with server-sent events, so that a long operation is watched as a tree of spans with progress bars, live, and the same UI opens recorded telemetry afterwards (PLANNED, Chapter 26). Everything it needs on the Java side exists today: publishing, `within`, and the model the next chapter describes.

Chapter 13. Telemetry as a model

Telemetry sent to a backend becomes the backend's: it is stored in its format, queried with its language, and forgotten on its schedule. The runtime can also keep telemetry as models, in files next to the models it describes, in the same formats, versioned in the same repositories. Then a trace is something you load, navigate, query, diff and commit, and a change to a model can be recorded in the span that made it.

This chapter covers the telemetry model, the exporters that fill it from the OpenTelemetry SDK, and change recording.

13.1. Why keep telemetry as a model

"Somebody changed the year of a book" is a log line. "In the course of the work named 'Correct the year', the attribute `year` of the book 'Domain-Driven Design' changed to 2004" is structure: the work, the object and the attribute are elements you can navigate to, and the statement can be checked years later.

That is the difference between operational and strategic telemetry made concrete. Its uses are the ones where an answer has to survive the people who knew it:

- **Audit and compliance.** Every change to an element during an audit period, who made it, and in the course of what work, is a query instead of an archaeology project.
- **Agents.** An agent's run is a trace whose spans are its tool calls, and the changes it made to models are in the spans that made them. Reviewing an agent's work means reading that trace, and reverting it means applying the changes in reverse.
- **Evidence.** Telemetry kept with the build or the release it describes is evidence of what ran, the right end of the line in Chapter 2.
- **Replay.** Spans carry timing and nesting, so an execution can be played back over a diagram of the system that ran it, which is planned for the polymorphic UI in React (Chapter 26).

13.2. The telemetry model

The model lives in `models/telemetry`, in four parts, following OpenTelemetry's own data model (OTLP) so that nothing is lost on the way in:

Package	Contents
`telemetry`	`Resource` (the entity that produced the signals), `InstrumentationScope`, and typed attribute values, `KeyValue` and `AnyValue`
`traces`	`TracesData`, `ResourceSpans`, `ScopeSpans`, `Span`, `SpanEvent`, `SpanLink`, `SpanStatus`, and the change recording classes
`logs`	`LogsData`, `ResourceLogs`, `ScopeLogs`, `LogRecord`
`metrics`	`MetricsData`, `ResourceMetrics`, `ScopeMetrics`, `Gauge`, `Sum`, `Histogram`, `ExponentialHistogram`, `Summary`, their data points, and `Exemplar`

On top of OTLP, the model adds what makes telemetry navigable as a graph:

- **Spans nest.** A span's children are contained by it, within an instrumentation scope, so a trace reads as a tree. Across scopes, spans reference their parent by id.
- **Log records live in their spans.** A log record emitted in a span is contained by that span, not stored in a separate stream to be joined later.
- **Changes live in their spans.** A span contains the description of the model changes recorded while it was current.
- **References resolve.** Span links and the change classes reference spans as objects, not only by id, and an exemplar references the span that produced a measurement.

13.3. Exporting to the model

The model exporters are OpenTelemetry SDK exporters: `ModelSpanExporter`, `ModelLogRecordExporter` and `ModelMetricExporter`, in the module `org.nasdanika.sdk.runtime.models.telemetry:exporters`. Each writes into an EMF resource and saves it when the SDK flushes or shuts down.

Only the assembly that runs the unit of work needs the SDK. Instrumented code, the runtime's included, needs only the API:

Listing 70. Dependencies of an assembly that records telemetry (`recording/pom.xml`)

```
<dependencies>
    <dependency>
        <groupId>${project.groupId}</groupId>
        <artifactId>library</artifactId>
        <version>${project.version}</version>
    </dependency>
    <!-- Exporters to the telemetry model, and the model itself -->
    <dependency>
        <groupId>org.nasdanika.sdk.runtime.models.telemetry</groupId>
        <artifactId>exporters</artifactId>
    </dependency>
    <!-- The SDK: only the assembly that runs the unit of work needs it -->
    <dependency>
        <groupId>io.opentelemetry</groupId>
        <artifactId>opentelemetry-sdk</artifactId>
    </dependency>
    <!-- YAML for the telemetry files -->
    <dependency>
        <groupId>org.nasdanika.sdk.runtime</groupId>
        <artifactId>emf-json</artifactId>
    </dependency>
</dependencies>
```

The SDK is built as usual, with the model exporters plugged in:

Listing 71. An SDK that exports to models (`recording/⋯/Recorder.java`)

```
/**
 * The SDK owns the exporters, and the exporters own the telemetry resources: shutting
 * the SDK down exports what is pending and saves them. Log records emitted in a span go
 * into that span.
 */
static OpenTelemetrySdk openTelemetry(
        ModelSpanExporter spans,
        Resource logs,
        Resource metrics) {

    var service = io.opentelemetry.sdk.resources.Resource.create(
            Attributes.of(AttributeKey.stringKey("service.name"), "librarian"));
    var resource = io.opentelemetry.sdk.resources.Resource.getDefault().merge(service);

    var logRecords = new ModelLogRecordExporter(logs, spans);
    var measurements = new ModelMetricExporter(metrics);
    return OpenTelemetrySdk.builder()
            .setPropagators(Telemetry.w3cPropagators())
            .setTracerProvider(SdkTracerProvider.builder()
                    .setResource(resource)
                    .addSpanProcessor(BatchSpanProcessor.builder(spans).build())
                    .build())
            .setLoggerProvider(SdkLoggerProvider.builder()
```

```
                    .setResource(resource)
                    .addLogRecordProcessor(BatchLogRecordProcessor.builder(logRecords).build())
                    .build())
            .setMeterProvider(SdkMeterProvider.builder()
                    .setResource(resource)
                    .registerMetricReader(PeriodicMetricReader.builder(measurements).build())
                    .build())
            .build();
}
```

The log record exporter is given the span exporter, so that log records emitted in exported spans are added to those spans; log records emitted outside any span go to the logs resource. The W3C propagators let the unit of work continue a trace started elsewhere (Chapter 11).

13.4. Recording changes

Change recording is EMF's, extended so that recorded changes know their spans. `Changes.record(target, body)` records the changes the body makes to a resource set, a resource or an object, and hands the description to the sink that is current in the OpenTelemetry context. The span exporter is such a sink: it puts each description into the span it was recorded in.

Listing 72. A unit of work whose telemetry and changes are saved as models

```
public static void curate(Path directory) throws IOException {
    NasdanikaResourceSet telemetry = NasdanikaResourceSet.createAndConfigure(); ①
    Resource traces = telemetry.createResource(uri(directory, "traces.yml"));
    Resource logs = telemetry.createResource(uri(directory, "logs.yml"));
    Resource metrics = telemetry.createResource(uri(directory, "metrics.yml"));
    ModelSpanExporter spans = new ModelSpanExporter(traces);

    try (OpenTelemetrySdk openTelemetry = openTelemetry(spans, logs, metrics);
            Scope telemetryScope = Telemetry.makeCurrent(openTelemetry);        ②
            Scope changesScope = Changes.makeCurrent(spans)) {                  ③

        Tracer tracer = openTelemetry.getTracer(Recorder.class.getModule().getName());
        Telemetry.inSpan(tracer, "Curate the shelf", null, span -> {
            NasdanikaResourceSet models = NasdanikaResourceSet.createAndConfigure();
            Resource shelf = models.createResource(uri(directory, "shelf.library"));
            Changes.record(models, () -> {                                      ④
                shelf.getContents().add(Samples.library());
                shelf.save(null);
                return null;
            });
```

```
            Book ddd = book((Library) shelf.getContents().getFirst(), "Domain-Driven Design");
            return Telemetry.inSpan(tracer, "Correct the year", null, edit -> {
                return Changes.record(models, () -> {                          ⑤
                    ddd.setYear(2004);
                    shelf.save(null);
                    return null;
                });
            });
        });
    }                                                                          ⑥
}
```

① The telemetry resources live in a resource set of their own, created before the instance is made current, so that saving them at shutdown is not itself traced.

② The SDK is the unit of work's instance. Everything below reports to it.

③ The span exporter becomes the sink of recorded changes for the unit of work.

④ Creating the library and saving it is recorded in the current span, "Curate the shelf".

⑤ Correcting the year is recorded in its own span, so the change lands there.

⑥ Closing the SDK exports what is pending and saves `traces.yml`, `logs.yml` and `metrics.yml`.

The recorded description is turned around before it is stored. EMF's change recorder produces a description that undoes the changes; the runtime stores one that describes them forward, detached from the model:

- Objects added in the body are copied into the description, as objects to attach.
- Objects removed in the body are moved into the description, so their state when they were removed is kept for audit.
- Other objects are referenced by proxies with their URIs, so the description can be saved in a different resource set and resolves to the model wherever the model is saved.

Nothing is recorded without a current sink or outside a sampled span, so the same code runs unchanged in a unit of work that does not record.

13.5. Reading telemetry back

Recorded telemetry is a model, so reading it is ordinary EMF:

Listing 73. Reading traces and their changes (`recording/⋯/TraceReader.java`)

```
public static List<String> read(Path directory) {
    TracesData traces = (TracesData) NasdanikaResourceSet.createAndConfigure()
            .getResource(Recorder.uri(directory, "traces.yml"), true)
            .getContents()
            .getFirst();

    // Spans nest within an instrumentation scope; across scopes they have parent ids
    Map<String, Span> spans = new LinkedHashMap<>();
    traces.getResourceSpans().stream()
        .flatMap(resourceSpans -> resourceSpans.getScopeSpans().stream())
        .flatMap(scopeSpans -> scopeSpans.getSpans().stream())
        .forEach(span -> collect(span, spans));
    Map<String, List<Span>> children = new LinkedHashMap<>();
    List<Span> roots = new ArrayList<>();
    for (Span span: spans.values()) {
        if (span.getParentSpanId() == null) {
            roots.add(span);
        } else {
            children.computeIfAbsent(span.getParentSpanId(), id -> new ArrayList<>()).add(span);
        }
    }

    List<String> lines = new ArrayList<>();
    roots.forEach(root -> print(root, children, "", lines));
    return lines;
}

static void print(
        Span span,
        Map<String, List<Span>> children,
        String indent,
        List<String> out) {

    out.add(indent + span.getName() + bytes(span));
    ChangeDescription changes = span.getChangeDescription();                    ①
    if (changes != null) {
        for (EObject created: changes.getObjectsToAttach()) {
            out.add(indent + "  * created " + label(created));
        }
        changes.getObjectChanges().forEach(entry -> {
            for (FeatureChange change: entry.getValue()) {
                String feature = label(entry.getKey()) + "." + change.getFeatureName();
                out.add(indent + "  * " + feature + " = " + change.getValue());
            }
        });
    }
    children.getOrDefault(span.getSpanId(), List.of()).stream()
        .sorted(Comparator.comparingLong(Span::getStartTimeUnixNano))
        .forEach(child -> print(child, children, indent + "  ", out));
}
```

① The change description is a feature of the span. Its object changes are keyed by the changed objects, which resolve through their proxies to the saved model.

For the unit of work above, it prints:

```
Curate the shelf
  * created Library 'A Modeler's Shelf'
  ResourceSetContributor.configure
    Contribute To Resource Set
    Contribute To Resource Set
    Contribute To Resource Set
    Contribute To Resource Set
  URIHandler.write (2059 bytes)
  Resource.save (2059 bytes)
  Correct the year
    * Book 'Domain-Driven Design'.year = 2004
    URIHandler.write (2059 bytes)
    Resource.save (2059 bytes)
```

The runtime's spans, from another instrumentation scope, appear in the tree by their parent ids: the resource set configuration and the saves happened inside "Curate the shelf" and "Correct the year", and the tree says so. Because the trace is a model, the same questions can be asked by a query language instead of a loop, which is on the roadmap (Chapter 26).

13.6. Recording a unit of work

`Recording`, in the exporters module, packages what this chapter has shown into one object: an OpenTelemetry SDK with the three model exporters, the log record exporter linked to the span exporter, and W3C propagators, recording to a traces, a logs and a metrics resource that are saved when it is closed. `Recording.builder().serviceName("import").directory(dir, "yml").build()` records to `traces.yml`, `logs.yml` and `metrics.yml` in a directory; `makeCurrent()` makes its instance the unit of work's; after `close()`, `getTraces()` returns the model. The launcher records every command with it, and tests record with it what they and the servers they start did (Chapter 15).

13.7. Formats and where files go

Telemetry resources use the formats every model uses, selected by extension: `yml` or `json` to read, `xmi` to interchange, `ebin` and `egz` for size. The launcher saves the telemetry of every command it runs in a directory per launch:

```
<home>/telemetry/2026/10/16-14-03-12-345/
  traces.yml
  logs.yml
  metrics.yml
```

The format is chosen with the system property `nsdk.telemetry.format` or the environment variable `NSDK_TELEMETRY_FORMAT`, with the values `yml` (the default), `json`, `xml`, `binary` and `compressed` (Chapter 18).

13.8. Every signal is precious

Keeping telemetry forever sounds expensive until you count. A command line run, an agent session or a build step produces hundreds to thousands of spans, which compress to kilobytes. Strategic telemetry is not the firehose; it is the record of work that mattered, and it costs less than the meeting held to reconstruct it.

The discipline that keeps it that way is deciding what is a unit of work worth keeping. A command that changes models, a release, an agent run, a migration: yes. Every request a server handles: no, that is operational telemetry, and it belongs in an operational backend, from the same instrumentation.

Part IV: Files and Serving

Volumes, the runtime's small file abstraction, and what they make possible: models loaded from anywhere, content written on demand, and anything presented as files over HTTP and WebDAV.

Chapter 14. Volumes and content

Models live in files, and files live in more places than a disk: a zip, a Git repository reached over a REST API, an in-memory sandbox, a remote system with a file-shaped API. The opposite direction is just as useful: a model, a running JVM or an issue tracker presented as files, so that a browser, a mounted drive or an agent with file tools can read it.

The runtime has one small abstraction for both directions, the volume, and one for the bytes of a file, the content. This chapter covers both, and how models are loaded from volumes and saved to them. Chapter 15 serves volumes over HTTP and WebDAV.

14.1. Why not NIO

The JDK has a file system abstraction, `java.nio.file`, and it is a pleasure to consume. It is expensive to implement. A `FileSystemProvider` needs a `Path` class with `resolve`, `relativize`, `normalize` and `toUri`, channels, attribute views, a provider registry keyed by URI scheme, and care wherever paths of two file systems meet.

The runtime implements file systems often: over models, over Git, over REST APIs, as overlays. So it owns a small interface for implementing them, and uses NIO for consuming: any NIO file system, the default one, a zip or an in-memory one, becomes a volume through one adapter.

This is where prior art converged. Go's `io/fs` is almost the same design, Python's fsspec has string paths and dozens of backends, and Hadoop's `FileSystem`, Apache Commons VFS and Eclipse EFS made the same decision in Java. Commons VFS is also the reminder to keep the core small.

14.2. The volume SPI

A volume has five methods:

Method	Contract
`stat(path)`	The entry at a path, or empty when nothing is there

Method	Contract
`list(directory)`	The children of a directory, in a stable order. Blocking, which is fine on virtual threads
`content(path)`	The content at a path, lazily: no I/O happens until it is opened
`caseSensitive()`	Declared, because some clients ignore case: Windows Explorer over WebDAV, macOS by default
`capability(type)`	An optional capability, or empty

Paths follow Go's rules and are validated once, at the boundary: slash-separated, relative to the root, with no leading or trailing slash and no empty, `.` or `..` segments. The root is `.`. There is no `Path` class: a path is a string, and URIs appear only at the edges.

An entry is a record of a path, a kind (`FILE`, `DIRECTORY` or `LINK`), a size, a hash and a modification time. The hash matters most. A content hash, or a Git blob id, gives HTTP ETags and conditional requests, suppression of writes that change nothing, incremental copies and blob caching. A volume that knows its hashes cheaply should report them.

Here is a complete read-only volume: a library presented as Markdown files, with an index, a page per book and a page per author, linked to each other.

Listing 74. A library as files (`volume/⋯/CatalogVolume.java`)

```
/**
 * A library presented as Markdown files: an index, a page per book and a page per author,
 * linked to each other. A browser, a WebDAV client or an agent with file tools reads it
 * like any folder.
 *
 * <p>
 * A read-only volume is the core of the SPI: stat, list, content, case sensitivity, and no
 * capabilities.
 */
public class CatalogVolume implements Volume {

    private final Map<String, byte[]> files = new TreeMap<>();
    private final Instant modified = Instant.now();

    public CatalogVolume(Library library) {
        files.put("index.md", bytes(index(library)));
        for (Book book: library.getBooks()) {
            files.put("books/" + slug(book.getName()) + ".md", bytes(book(book)));
        }
        for (Author author: library.getAuthors()) {
            files.put("authors/" + slug(author.getName()) + ".md", bytes(author(author)));
```

```
        }
    }

    @Override
    public Optional<Entry> stat(String path) {
        if (".".equals(path) || "books".equals(path) || "authors".equals(path)) {
            return Optional.of(new Entry(path, Entry.Kind.DIRECTORY, -1, null, modified));
        }
        byte[] bytes = files.get(path);
        if (bytes == null) {
            return Optional.empty();
        }
        Entry file = new Entry(path, Entry.Kind.FILE, bytes.length, hash(bytes), modified);
        return Optional.of(file);
    }

    @Override
    public List<Entry> list(String directory) throws IOException {
        if (".".equals(directory)) {
            return List.of(
                    stat("authors").orElseThrow(),
                    stat("books").orElseThrow(),
                    stat("index.md").orElseThrow());
        }
        if (!"books".equals(directory) && !"authors".equals(directory)) {
            throw new NoSuchFileException(directory);
        }
        List<Entry> entries = new ArrayList<>();
        for (String path: files.keySet()) {
            if (path.startsWith(directory + "/")) {
                entries.add(stat(path).orElseThrow());
            }
        }
        return entries;
    }

    @Override
    public Content content(String path) {
        byte[] bytes = files.get(path);
        if (bytes == null) {
            return () -> {
                throw new NoSuchFileException(path); // Lazy: opening fails, asking does not
            };
        }
        return Content.of(bytes);
    }

    @Override
    public boolean caseSensitive() {
        return true;
    }

    @Override
    public <T> Optional<T> capability(Class<T> type) {
```

```
        return Optional.empty(); // Not writable, not watchable, not searchable
    }
```

Reading it is reading files:

Listing 75. Reading a volume

```
Volume catalog = new CatalogVolume(Samples.library());
assertThat(catalog.list("."))
    .extracting(Volume.Entry::path)
    .containsExactly("authors", "books", "index.md");
assertThat(catalog.stat("books/domain-driven-design.md")).hasValueSatisfying(entry -> {
    assertThat(entry.kind()).isEqualTo(Volume.Entry.Kind.FILE);
    assertThat(entry.hash()).isNotNull();                                    ①
});
try (InputStream in = catalog.content("books/design-patterns.md").openStream()) {
    assertThat(new String(in.readAllBytes(), StandardCharsets.UTF_8))
        .contains("* Cited by: [Domain-Driven Design](domain-driven-design.md)");
}
```

① The hash is known without reading the file. The HTTP handler turns it into an ETag.

Book pages link to their authors and to the books that cite them. A Markdown viewer renders the catalog as a small site, and an agent with file tools navigates the model by following links without knowing that EMF exists. That is the point of presenting a model as files: every tool that understands files understands the model, at the level of detail the presentation chooses.

14.3. Capabilities

Everything beyond reading is optional, and discovered by asking for it:

Capability	Adds	Status
`Writable`	`write(path, content)`, `delete`, `move` (atomic where the backend supports it) and `createDirectory`	**AVAILABLE**
`Watchable`	`changes()`, a `Flow.Publisher` of change batches: how a backend reports changes underneath it	**SKETCH**
`Transactional`	`begin()`, then `commit(message)` or `rollback()`	**SKETCH**
`Searchable`	`glob(pattern)` and `grep(regex, glob)`, pushed down to a backend that can do better than a walk	**SKETCH**

Capability	Adds	Status
`Linked`	`readLink(path)`: in a model presented as files, non-containment references are links	**SKETCH**

The interfaces of all five are in the runtime. "Available" means that a volume in the runtime implements the capability; the others are declared so that the volumes to come, and the code that will use them, agree on the shape.

A read-only volume implements the core and answers empty to every capability. A volume over a Git repository will implement the core, `Writable`, `Transactional` and `Watchable`, and nothing forces it to implement the rest.

14.4. Volumes over directories

`Volume.of(path)` is a writable volume over a directory of any NIO file system: the default one, a zip file system, an in-memory one. It validates paths so that they cannot escape the root, and probes the file system for case sensitivity instead of guessing it from the operating system. It keeps file system semantics: writing into a missing directory fails, and so does deleting a non-empty one.

`GenericVolume<C>` is the same SPI with the type of content as a parameter, and `Volume` is the flavor whose content is bytes. `adapt` turns a volume of one content type into a volume of another, which is how the TypeScript generator of the runtime writes text without converting it to bytes itself:

Listing 76. A text volume over a directory

```
GenericVolume<String> text = Volume.of(directory).adapt(
        content -> {                                                    ①
            try (InputStream in = content.openStream()) {
                return new String(in.readAllBytes(), StandardCharsets.UTF_8);
            } catch (IOException e) {
                throw new UncheckedIOException(e);
            }
        },
        string -> Content.of(string, StandardCharsets.UTF_8));          ②

@SuppressWarnings("unchecked")
GenericVolume.Writable<String> writable = text.capability(GenericVolume.Writable.class)
        .orElseThrow();
writable.write("hello.md", "# Hello");
```

```
assertThat(text.content("hello.md")).isEqualTo("# Hello");
assertThat(directory.resolve("hello.md")).hasContent("# Hello");
```

① Reading maps bytes to text. The mapping runs when content is asked for, so a text volume reads eagerly.

② Writing maps text to bytes, encoded when the target volume reads the content.

14.5. Content

A file's state is a `Content`: a re-readable source of bytes, in the spirit of Guava's `ByteSource` and of the JDK HTTP client's `BodyPublishers.ofInputStream(Supplier)`. An `InputStream` is one-shot and has unclear ownership. A content can be read more than once, for a hash, then for a write, then by each subscriber, and owns nothing until it is opened.

- **Getting content does no I/O.** Opening it does.
- **Assigning content does not move bytes.** Bytes move when a consumer reads the content: a commit, an HTTP response, a hash. An I/O failure surfaces when the bytes move, not at assignment.
- **Consumers push when they can.** They call `writeTo(out)` for a response, a file, a digest or an upload; `openStream()` only when an input stream is required; and `publisher()` for the JDK HTTP client's `BodyPublishers.fromPublisher`.

Factory	Content
`Content.of(bytes)`	In memory, with a known length
`Content.of(text, charset)`	Text, encoded when read
`Content.of(supplier)`	From a stream supplier, called on every open
`Content.of(uri)`	At a URI, requested on every open: HTTP and HTTPS through the JDK HTTP client, `file` through NIO, other schemes through `URL`. A 404 is a `NoSuchFileException`
`Content.once(stream)`	A stream someone handed over. Not repeatable
`Content.ofWriter(writer)`	Produced by a closure that writes to an output stream, when the content is read

A content may also know its length and its hash without reading, and says whether it is

repeatable. A consumer that needs to read a one-shot content twice buffers it first.

Content produced by a writer

Much content in the runtime is produced by code that writes to an output stream: saving a resource, generating a page, encoding text. `Content.ofWriter` turns such a closure into content without running it:

- `writeTo(out)` runs the writer on the consumer's stream, with no pipe and no thread. This is the arrangement of JAX-RS's `StreamingOutput`.
- `openStream()` returns a lazy pipe. The writer starts on a virtual thread at the first read and writes bounded chunks into a queue that the reader drains. A failure of the writer is rethrown to the reader, and a reader that closes early cancels the writer.
- Each read runs the writer again, so the content is repeatable as long as the writer is.

The pipe is deliberately not `PipedInputStream`, which detects a dead writer or reader from thread liveness and has a 1 KB buffer.

Copying a model to another volume takes three lines, and the model is serialized straight into the target's stream:

Listing 77. A model as content (`volume/⋯/Shelves.java`)

```
/**
 * Copies a model as content produced by a writer: nothing is serialized until the target
 * volume reads the content, and it is serialized straight into the target's stream.
 */
public static void copy(Resource resource, Volume target, String path) throws IOException {
    Content content = Content.ofWriter(out -> resource.save(out, null));      ①
    Volume.Writable writable = target.capability(Volume.Writable.class)       ②
            .orElseThrow(() -> new IOException("Read-only volume"));
    writable.write(path, content);                                             ③
}
```

① Nothing is serialized here. The content is a promise to save the resource when it is read.

② Writing is a capability: a read-only volume refuses it by not having it.

③ The volume reads the content, which runs the save on the volume's own stream.

14.6. Models in volumes

A path is where the bytes of a model are; a URI is what the model is. `VolumeURIHandler` bridges the two for EMF: URIs below a base URI are paths in a volume, so a resource set loads and saves resources through the volume.

Listing 78. A resource set over a volume (`volume/⋯/Shelves.java`)

```
public static final URI BASE = URI.createURI("library://shelves/");

/**
 * A resource set whose library:// URIs are paths in the volume.
 */
public static ResourceSet resourceSet(Volume volume) {
    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
    URIConverter converter = resourceSet.getURIConverter();
    converter.getURIHandlers().add(0, new VolumeURIHandler(volume, BASE));    ①
    TelemetryURIHandler.wrapAll(converter, () -> Telemetry.of(resourceSet)); ②
    return resourceSet;
}

public static void save(Library library, Volume volume, String path) throws IOException {
    URI uri = BASE.appendSegments(path.split("/"));                           ③
    Resource resource = resourceSet(volume).createResource(uri);
    resource.getContents().add(library);
    resource.save(null);                                                      ④
}

public static Library load(Volume volume, String path) {
    URI uri = BASE.appendSegments(path.split("/"));
    Resource resource = resourceSet(volume).getResource(uri, true);
    return (Library) resource.getContents().getFirst();
}
```

① URIs below `library://shelves/` are paths in the volume. The handler goes first, before EMF's default handlers.

② A configured resource set has every handler instrumented, but this one was added later, so it is wrapped too. Reads and writes become `URIHandler.read` and `URIHandler.write` spans.

③ `library://shelves/2026/shelf.library` is the path `2026/shelf.library`. Segments are decoded, and a query and a fragment are ignored, so the URI of an object maps to the path of its resource.

④ The bytes are buffered and written as one content when the stream is closed, so a failed save writes nothing. Missing parent directories are created first.

Listing 79. Saving and loading through a volume

```
Volume volume = Volume.of(directory);                                  ①
Shelves.save(Samples.library(), volume, "2026/shelf.library");         ②
assertThat(directory.resolve("2026").resolve("shelf.library")).exists(); ③
Library loaded = Shelves.load(volume, "2026/shelf.library");
assertThat(loaded.getBooks()).hasSize(8);
```

① A writable volume over a temporary directory.

② The resource's URI is `library://shelves/2026/shelf.library`, whatever the directory is.

③ The handler created the missing directory `2026` before writing.

The model never mentions where it is stored. Moving models from a working tree to an in-memory volume, a Git overlay or a volume over a remote system changes one handler, not the models and not the references between them.

The handler answers EMF's attribute queries from `stat`: the time stamp, the length, whether a path is a directory, whether it is read-only (no `Writable` capability), and the hash.

14.7. Telemetry of transfers

Moving bytes is work, and it is reported to the unit of work's OpenTelemetry instance:

Span	When
`Content.writeTo`	Content pushed to a stream, with the byte count, unless the content is in memory
`Content.pipe`	Content produced by a writer and read as a stream: from the first read to the end of the writer. The writer runs in it, so the spans the writer starts are its children
`GET` (client)	Content at an HTTP URI, from the request to the close of the body, with the HTTP semantic convention attributes and the trace context in the request headers
`Content.read`	Content at another URI, from opening to closing
`URIHandler.read`, `URIHandler.write`	A stream opened through an instrumented URI handler, from opening to closing

All of them emit progress log records at most once a second while bytes move, so a long

download shows in a live tree (Chapter 12) while it happens. Crossing to the writer's virtual thread carries the OpenTelemetry instance and the parent span along, as the rules of Chapter 11 require. The URL recorded on a client span has no user info and no query string, which may carry credentials.

14.8. What comes next

The volume is the bottom layer of a larger design **SKETCH**:

- **The file model.** Files and directories as an Xcore model over volumes: `File`, `Directory`, `TextFile` and types specialized by content type, with operations such as `view`, `replace` and `insert` that are also an agent's file tools. Views decode content on get and encode it lazily on set, so two views of one file cannot diverge. The model and its generated code are in the runtime, in `models/file`; its behavior is being attached.
- **More volumes.** An in-memory volume for tests and sandboxes. An overlay that keeps writes in memory over a read-only base until commit. A Git volume whose commit is a Git commit, locally or over the GitHub and GitLab APIs without a clone. A model volume that presents a resource set as folders, files and links.
- **A reactive file system.** Processors, deterministic or agentic, react to file changes and change files in turn until nothing changes, and then one commit records the run. The hash in every entry is what lets such loops converge.

Chapter 26 places these in the roadmap.

Chapter 15. Serving over HTTP and WebDAV

Anything presented as a volume can be served: to a browser over HTTP, and to anything that mounts drives over WebDAV. A model becomes a folder in Finder or Explorer, a preview of a generated site is one URL away, and an agent reads a running system with the file tools it already has.

The runtime serves volumes with the HTTP server the JDK has shipped for years, `com.sun.net.httpserver` in the module `jdk.httpserver`. With virtual threads it is enough for local tools, previews and agents, with no servlet container and no framework.

15.1. Serving a volume

`VolumeHttpHandler` serves a volume read-only, in the manner of the JDK's `SimpleFileServer`:

- Files are sent with a media type from their extension, a `Last-Modified` header from the entry, and an ETag from the entry's hash. A conditional request whose `If-None-Match` matches gets a `304`.
- Directories are generated HTML listings. A directory requested without a trailing slash is redirected to it, so relative links in pages and listings resolve.
- The request path below the context path is the volume path. Paths with empty, `.` or `..` segments are refused.
- `GET` and `HEAD` are served; other methods get a `405`.

`WebDavHandler` extends it into a read-only WebDAV (RFC 4918) class 1 share:

- `OPTIONS` answers `DAV: 1` and the allowed methods.
- `PROPFIND` answers a `207 Multi-Status` with the resource type, display name, content length, content type, ETag and modification time, as far as the volume knows them. `Depth: infinity` is answered as `1`, so that a client cannot make the server walk a large volume.
- `GET` and `HEAD` work as in `VolumeHttpHandler`, listings included, so the share also browses in a web browser.

- Methods that write or lock get a `405`. Writing belongs behind the volume's `Writable` capability, and is not implemented yet.

That is enough to mount a share in macOS Finder ("Connect to Server"), on Windows (`net use`), on Linux (davfs2) or with rclone, and to read it with any tool. Clients need the content length of files to read them reliably, so a volume served over WebDAV should know its file sizes.

Serving the catalog of Chapter 14 both ways takes a few lines:

Listing 80. Serving a volume (`volume/⋯/Serve.java`)

```
public static HttpServer serve(Library library, OpenTelemetry openTelemetry, int port)
        throws IOException {
    Volume catalog = new CatalogVolume(library);
    var address = new InetSocketAddress(InetAddress.getLoopbackAddress(), port);
    HttpServer server = HttpServer.create(address, 0);
    server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());              ①

    HttpHandler files = new VolumeHttpHandler(catalog);
    server.createContext("/catalog", new TelemetryHandler(files, openTelemetry)); ②

    HttpHandler share = new WebDavHandler(catalog);
    server.createContext("/dav", new TelemetryHandler(share, openTelemetry));     ③

    server.start();
    return server;
}
```

① A virtual thread per request. Handlers block, and blocking is cheap.

② Browsers read the catalog at `/catalog/`. Each request is a unit of work with a server span.

③ WebDAV clients mount the same volume at `/dav/`.

The server binds to the loopback address. The handlers do not authenticate, so serving beyond the local machine needs what the JDK server provides for it, an `HttpsServer` and an `Authenticator` on the context, or a proxy in front.

Running `Serve` prints the two URLs. Opening the first in a browser shows the listing of the catalog's root; following `index.md` shows the Markdown of the library.

15.2. Reading over HTTP

On the client side, content at a URL is a `Content` like any other:

Listing 81. Content at a URL

```
Content index = Content.of(URI.create(root + "/catalog/index.md")); ①
try (InputStream in = index.openStream()) {
    String markdown = new String(in.readAllBytes(), StandardCharsets.UTF_8);
    assertThat(markdown).startsWith("# A Modeler's Shelf");
}
```

① Nothing is requested until the content is opened, and every open requests it again.

A WebDAV listing is a `PROPFIND` request, which the JDK HTTP client sends like any other method:

Listing 82. Listing a directory over WebDAV

```
HttpResponse<String> listing = HttpClient.newHttpClient().send(
        HttpRequest.newBuilder(URI.create(root + "/dav/books/"))
            .method("PROPFIND", HttpRequest.BodyPublishers.noBody())
            .header("Depth", "1")
            .build(),
        HttpResponse.BodyHandlers.ofString());
assertThat(listing.statusCode()).isEqualTo(207); // Multi-Status
assertThat(listing.body()).contains("domain-driven-design.md");
```

15.3. One trace across client and server

`TelemetryHandler` wraps a handler, or is added as a filter of a context, and makes each request a unit of work:

- The trace context is extracted from the request headers with the propagators of the OpenTelemetry instance, so the client's span is the parent of the server's.
- The server span is named `<method> <route>`, the route being the context path, and carries the HTTP semantic convention attributes: method, route, path, scheme, server and client address, protocol version, user agent, response status and response body size.
- The instance and the span are current while the wrapped handler runs, so whatever it does, content transfers, resource loads, its own log records, reports to the request.
- A failure of the handler is recorded on the span. If no response was sent yet, the client

gets a `500`; if one was, the connection is dropped instead of completing a response that would look successful.

- A failure or a `5xx` sets the error status. A `4xx` is the client's error, not the server's, and leaves the status unset.

On the client side, `Content.of(uri)` starts a client span and propagates its context in the request headers. Together they make one trace:

Listing 83. A trace that crosses HTTP

```
InMemorySpanExporter exporter = InMemorySpanExporter.create();
var tracerProvider = SdkTracerProvider.builder()
        .addSpanProcessor(SimpleSpanProcessor.create(exporter))
        .build();
try (OpenTelemetrySdk sdk = OpenTelemetrySdk.builder()
        .setPropagators(Telemetry.w3cPropagators())                    ①
        .setTracerProvider(tracerProvider)
        .build()) {
    HttpServer server = Serve.serve(Samples.library(), sdk, 0);        ②
    String index = "http://127.0.0.1:" + server.getAddress().getPort() + "/catalog/index.md";
    try (Scope scope = Telemetry.makeCurrent(sdk);                     ③
            InputStream in = Content.of(URI.create(index)).openStream()) {
        in.readAllBytes();
    } finally {
        server.stop(0);
    }
    SpanData client = await(exporter, SpanKind.CLIENT);
    SpanData handled = await(exporter, SpanKind.SERVER);
    assertThat(handled.getTraceId()).isEqualTo(client.getTraceId());   ④
    assertThat(handled.getParentSpanId()).isEqualTo(client.getSpanId());
    assertThat(handled.getName()).isEqualTo("GET /catalog");
}
```

① W3C trace context propagation, on both sides: the client injects, the server extracts.

② The server's instance. In a real deployment it is another process with its own instance; here both share one, so the test can see both spans.

③ The client's unit of work. Opening the content is a `GET` client span.

④ The server span is a child of the client span, in the same trace.

This is the cross-process rule of Chapter 11 at work: the unit of work spans two processes, each with its own instance, and the trace connects them. When both sides save their telemetry as models (Chapter 13), the two halves of the trace are two files that reference the same trace id, and a query can join them.

15.4. A server for the length of a test

In a test the two sides are one process, and the setup above, a server, an executor, a filter on every context, a client that propagates, is the same every time. The `harness` module packages it in one method, shaped like `Telemetry.inSpan`:

- `Harness.inServer(name, routes, body)` starts a JDK HTTP server on the loopback address and a free port, on virtual threads; lets `routes` create its contexts, each instrumented with `TelemetryHandler` as it is created; runs `body` in a span named `name`; and stops the server when the body returns or throws.
- The body gets a `Server`: its base URI, a `get` and a `send` whose requests are client spans that propagate the trace, and the propagation headers for clients the harness does not own, such as a browser.
- With a `Recording` (Chapter 13), the body's span, its client spans, the server's spans and whatever the handlers logged are one trace, saved as one model.

The runtime's own tests serve generated applications this way, and the `playwright` module drives a browser against a harness server in the same trace (Chapter 23). The Zephyr Book *Harness* covers the module in depth.

15.5. What comes next

Serving is the first of several surfaces over volumes, and the others are planned **PLANNED**:

- **Writable WebDAV**, behind the `Writable` capability, so that editing a mounted model is editing the model.
- **VS Code**, through its `FileSystemProvider` API, which consumes the change batches of a `Watchable` volume directly.
- **FUSE**, for a native mount on Linux and macOS.
- **The polymorphic UI in React**: a browser application that renders any model, and live telemetry, from the same server (Chapter 26).

Part V: The Command Line

Commands contributed by modules, found as capabilities and wired to their parents by generic type, and the launcher that runs them as units of work with their telemetry saved as models.

Chapter 16. Commands as capabilities

The command line is the runtime's first user interface, for people, for build steps and for agents. An agent that can run a command and read a file can use everything the command line offers, with no protocol to implement.

Three decisions shape it:

- **Commands are contributed.** A module that adds a command declares a factory in its module descriptor. Nothing lists all the commands: the command tree is whatever the module path contributes.
- **The tree is resolved.** Commands find their parents and their sub-commands by capability resolution (Chapter 10), matched by the generic types they declare (Chapter 17).
- **A command execution is a unit of work.** Every command runs in a span, and the launcher saves the telemetry of each launch as models (Chapter 18).

16.1. Picocli

The commands are picocli commands: annotated classes with options, parameters, mix-ins, generated help and completion. Picocli is mature, small and dependency-free, and a picocli command is a plain object that the runtime can create, inspect and wire. Created by Remko Popma, it expresses the domain of command lines in Java's own constructs, annotations on typed fields, so well that a Java developer reads a command without learning anything new; the runtime adds to it what a contributed command line needs, assembly through capabilities and wiring by generic types. The Zephyr Book *Declarative Command Lines* covers the `cli` module in depth.

The runtime bundles picocli in its own module, `org.nasdanika.sdk.runtime.picocli`, until a picocli release includes the changes the runtime contributed. Commands import `picocli.CommandLine` as usual.

16.2. A first command

Listing 84. A command (cli/⋯/HelloCommand.java)

```
@Command(
        name = "hello",
        description = "Says hello",
        mixinStandardHelpOptions = true)
public class HelloCommand extends CommandBase {                          ①

    @Parent
    private RootCommand root;                                            ②

    @Option(names = "--name", defaultValue = "World", description = "Whom to greet")
    private String name;

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) { ③
        span.setAttribute("greeting.name", name);
        Telemetry.log(logger, Severity.INFO, null, "Greeting " + name, null);
        spec.commandLine().getOut().println("Hello, " + name + "!");
        return 0;
    }

}
```

① `CommandBase` runs the command in a span named after it, with the command's qualified name, class, module and module version as attributes.

② The command goes under the root command. How a declaration decides the parent is the subject of Chapter 17.

③ The execution gets the span and the logger and meter of the unit of work. Whatever the command does reports to them.

A command is contributed by a factory:

Listing 85. Its factory (cli/⋯/HelloCommandFactory.java)

```
/**
 * Contributes the hello command. The factory is asked once per candidate parent; it creates the
 * command where the declarations of the command and the parent match.
 */
public class HelloCommandFactory extends SubCommandCapabilityFactory<HelloCommand> {

    @Override
    protected Class<HelloCommand> getCommandType() {
        return HelloCommand.class;                                       ①
    }
```

```
    @Override
    protected Flow.Publisher<HelloCommand> doCreateCommand(
            List<CommandLine> parentPath,
            Loader loader,
            Span span,
            Logger logger,
            Meter meter) {
        return Flows.of(new HelloCommand());                    ②
    }

}
```

① The command's class. The base factory matches it against each candidate parent before creating anything.

② A new command for each parent it matches. A factory can also create several commands, or none, depending on the parent path and on what the loader resolves.

The module declares the factory, and opens the command package:

Listing 86. The module of the example commands (`cli/⋯/module-info.java`)

```
import org.example.cli.BooksCommandFactory;
import org.example.cli.CsvExporterFactory;
import org.example.cli.ExportCommandFactory;
import org.example.cli.HelloCommandFactory;
import org.example.cli.LibraryCommandFactory;
import org.example.cli.LimitMixInFactory;
import org.example.cli.MarkdownExporterFactory;
import org.nasdanika.sdk.runtime.common.capability.CapabilityFactory;

module org.example.cli {

    requires transitive org.nasdanika.sdk.runtime.cli;
    requires transitive org.example.library;

    exports org.example.cli;

    opens org.example.cli; ①

    provides CapabilityFactory with ②
        HelloCommandFactory,
        LibraryCommandFactory,
        BooksCommandFactory,
        ExportCommandFactory,
        MarkdownExporterFactory,
        CsvExporterFactory,
        LimitMixInFactory;

}
```

① Picocli sets option fields reflectively, and the wiring injects parents and sub-commands, so the package is open.

② Each factory is a capability factory, found by the service loader in the module layer.

16.3. How the tree is built

The command tree is built by resolving one requirement: a `CommandLine` with a `SubCommandRequirement` payload whose parent path is empty. The runtime's `RootCommandFactory` answers it with the root command, `nsdk`.

Every factory that creates a command then asks for the command's own sub-commands, with the command appended to the parent path, and for its mix-ins. The tree grows within the same resolution until no factory matches a new parent path. A factory emits its command line only when the command's sub-commands and mix-ins are attached, so a command line always arrives with its subtree, and a blocking load returns the whole tree. Two limits stop a runaway contribution: a parent path of at most 15 commands and at most 2,000 commands in a tree, set with the system properties `org.nasdanika.sdk.runtime.cli.maxCommandPath` and `org.nasdanika.sdk.runtime.cli.maxCommands`.

Listing 87. Assembling the command line (`cli/⋯/Assembly.java`)

```
/**
 * The root command line with every command and mix-in contributed on the module path.
 */
public static CommandLine root(CapabilityLoader loader) {
    Requirement requirement = Requirement.of(
            CommandLine.class,
            new SubCommandRequirement(List.of(), new AtomicInteger())); ①
    List<CommandLine> roots = loader.loadAll(requirement);              ②
    return roots.getFirst();
}

public static void main(String[] args) {
    System.exit(root(CapabilityLoader.getInstance()).execute(args));
}
```

① The requirement for root commands: an empty parent path and a counter shared by the whole tree.

② A blocking load, pumping the resolution on the calling thread until it completes. The root it returns carries the whole tree.

With the example module on the module path, the tree is:

```
nsdk
  launcher
  hello
  library
    books
    export
      csv
      markdown
    save
```

`launcher` is the root command's own sub-command, declared statically and therefore first (Chapter 18), and `save` is the runtime's, which found its way under `library` by itself (Chapter 17). Running a command is picocli's `execute`:

Listing 88. Running a command

```
Run run = run("hello", "--name", "Reader");
assertThat(run.exitCode()).isZero();
assertThat(run.out()).isEqualTo("Hello, Reader!\n");
```

The test prints the tree by walking the command lines:

```
static void tree(CommandLine commandLine, String indent, List<String> lines) {
    lines.add(indent + commandLine.getCommandName());
    commandLine.getSubcommands().values().stream()
        .filter(sub -> !sub.getCommandSpec().usageMessage().hidden())
        .forEach(sub -> tree(sub, indent + "  ", lines));
}
```

16.4. Groups

A group is a command that does nothing itself. It organizes sub-commands under a name and gives them something to work on, from its options.

Listing 89. A group (`cli/⋯/LibraryCommand.java`)

```
/**
 * A group: it does nothing itself, and gives its sub-commands a library to work on. What
 * it gives is declared by the generic return types of its adapters.
 */
@Command(
        name = "library",
```

```
        description = "Works with a library of books",
        mixinStandardHelpOptions = true)
public class LibraryCommand extends CommandGroup {

    @Parent
    private RootCommand root;

    @Option(names = "--file", description = "Library file, the sample library if not set")
    private File file;                                                          ①

    @Adapter
    public ExecutionSupplier<Library> library() {                               ②
        return (span, logger, meter) -> load(span);
    }

    @Adapter
    public ExecutionSupplier<List<EObject>> contents() {                        ③
        return (span, logger, meter) -> List.of(load(span));
    }

    private Library load(Span span) {
        if (file == null) {
            span.setAttribute("library.source", "sample");
            return Samples.library();
        }
        span.setAttribute("library.source", file.getPath());
        URI uri = URI.createFileURI(file.getAbsolutePath());
        ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
        return (Library) resourceSet.getResource(uri, true).getContents().getFirst(); ④
    }

}
```

① The group's option, parsed before any of its sub-commands run.

② What the group gives: a library, produced when a sub-command asks for it. The generic return type is the declaration that commands match.

③ The same library as a list of objects, for commands that work on any model.

④ Loading through a configured resource set, so the file can be in any format the runtime reads.

`CommandGroup` fails with "Missing required subcommand" when it is run alone, which picocli reports with the usage help. `nsdk library --file shelf.yml books` parses `--file` for the group and runs `books`, which asks the group for its library.

16.5. Mix-ins

A picocli mix-in contributes options to a command. In the runtime, mix-ins are contributed like commands, by factories, and a command asks for them by type:

Listing 90. A mix-in (`cli/⋯/LimitMixIn.java`)

```
/**
 * An option contributed to commands that ask for it with @MixIns, or that a mix-in declares it
 * wants with a @Parent member.
 */
public class LimitMixIn {

    @Option(names = "--limit", description = "At most this many items")
    private Integer limit;

    public <T> List<T> apply(List<T> items) {
        return limit == null || items.size() <= limit ? items : items.subList(0, limit);
    }

}
```

Listing 91. Its factory (`cli/⋯/LimitMixInFactory.java`)

```
public class LimitMixInFactory extends MixInCapabilityFactory<LimitMixIn> {

    @Override
    protected String getName() {
        return "limit";
    }

    @Override
    protected Class<LimitMixIn> getMixInType() {
        return LimitMixIn.class;
    }

    @Override
    protected Flow.Publisher<LimitMixIn> doCreateMixIn(
            List<CommandLine> commandPath,
            Loader loader,
            Span span,
            Logger logger,
            Meter meter) {
        return Flows.of(new LimitMixIn());
    }

}
```

A mix-in is added to a command when the mix-in takes the command through a `@Parent`

member, when the command accepts it through a `@MixIns` field or method, by generic type, as `@SubCommand` accepts sub-commands, or when the command's class is annotated with `@MixIns` listing a type of the mix-in. The `books` command of Chapter 17 asks for `LimitMixIn` and finds it among its mix-ins at execution, so `nsdk library books --limit 2` prints two books.

The runtime's mix-ins add common options without a factory, as picocli `@Mixin` fields:

Mix-in	Options
`ResourceSetMixIn`	`--extension-resource-factory`, `--protocol-resource-factory` and `--content-type-resource-factory`, which map an extension, a protocol or a content type to a resource factory class, and `createResourceSet()`
`PropertiesMixIn`	`-p` for one property and `-P` for a properties resource, in YAML, JSON or properties format, loaded in order

16.6. Overriding

Several modules may contribute a command with the same name under the same parent. The runtime keeps one when it can tell which one overrides the others:

- A command whose class is annotated with `@Overrides` overrides commands of the listed types.
- A command implementing `Overrider` decides, for each other command, whether it overrides it.
- A command whose class is a subclass of another command's class overrides it.

Commands that do not override each other are all kept, and picocli reports the duplicate name. The same rules apply to mix-ins with the same name.

16.7. Commands in the runtime

Command	What it does
`nsdk`	The root command, with version information from the module descriptors

Command	What it does
`launcher`	Generates a Java command line, or an options file, from directories of modules and jars (Chapter 18)
`save`	Saves its parent's model contents to a file, in any format the runtime writes (Chapter 17)
`ecore`	A group that narrows its parent's objects to Ecore model elements
`shell`	Added by the launcher: an interactive shell over the root command, with `exit`

16.8. Commands and telemetry

Each executed command is a span named after the command, a child of the launcher's root span. Spans the command starts, resource loads and saves, capability resolution, content transfers, are its children, and log records it emits are in it.

A parent's work done for a child, loading the library in the example, runs in the child's span when the child asks for it through an `ExecutionSupplier`, so the cost is attributed to the command that needed it (Chapter 17). The launcher saves all of it as models in a directory per launch (Chapter 18), which makes every command run an inspectable record.

Chapter 17. Wiring commands with generic types

Commands contributed by different modules have to find each other. The runtime's `save` command should work under any command that has model contents to save; a `books` command should work under anything that supplies a library; an `export` command should run whatever exporters other modules contribute. None of these modules knows the others.

The runtime wires commands by declarations: a command declares, with generic types, what it needs from its parent and what it accepts as sub-commands, and a parent declares what it offers. Matching the declarations decides where commands go. Injecting adapted values just before execution connects them.

17.1. What a command needs from its parent

A `@Parent` member, a field or a method taking one parameter, declares what a command needs from its parent:

Listing 92. A command that needs a library (`cli/⋯/BooksCommand.java`)

```
@Command(
        name = "books",
        description = "Lists books",
        mixinStandardHelpOptions = true)
@MixIns(LimitMixIn.class)                                                    ①
public class BooksCommand extends CommandBase {

    @Parent
    private ExecutionSupplier<? extends Library> library;                    ②

    @Option(names = "--before", description = "Only books published before this year")
    private Integer before;

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) throws Exception {
        Library theLibrary = library.get(span, logger, meter);               ③
        List<Book> books = before == null
                ? theLibrary.getBooks()
                : theLibrary.publishedBefore(before);
        for (Book book: limit(books)) {
            spec.commandLine().getOut().println(book.getName() + " (" + book.getYear() + ")");
```

```
        }
        span.setAttribute("library.books", (long) books.size());
        return 0;
    }

    private <T> List<T> limit(List<T> items) {
        for (CommandSpec mixIn: spec.mixins().values()) {                    ④
            if (mixIn.userObject() instanceof LimitMixIn limit) {
                return limit.apply(items);
            }
        }
        return items;
    }

}
```

① The command asks for the `LimitMixIn` of Chapter 16.

② What the command needs from its parent: something that produces a library while the command executes. `? extends` widens what answers: a parent producing a subclass of `Library` matches too.

③ The library is produced now, inside this command's span, with its logger and meter. If the command does not run, the library is never loaded.

④ The mix-in, found among the command's mix-ins.

The parent declares what it offers with `@Adapter` methods, whose generic return types are the declarations matched. The `library` group of Chapter 16 offers `ExecutionSupplier<Library>` and `ExecutionSupplier<List<EObject>>`, and `ExecutionSupplier<Library>` is assignable to `ExecutionSupplier<? extends Library>`, so `books` goes under `library`.

A parent member is used twice:

- **Matching**, before the command exists. A factory creates the command under a parent whose class is adaptable to the type of any of the command's parent members (Chapter 8). Declarations decide, because the parent's options are not parsed while the tree is built.
- **Injection**, after parsing and before execution. The parent is adapted to the member types, most specific first, and the first value obtained is injected. A command has one parent, so several parent members are alternative ways to take it, and one is injected. A command whose parent provides none of them fails to execute, with a message naming what it needed.

17.2. ExecutionSupplier

What a parent hands a child is often not cheap: a model to load, a service to call, a repository to clone. `ExecutionSupplier<T>` is the shape for such values: a function from the executing command's span, logger and meter to a value.

```
@FunctionalInterface
public interface ExecutionSupplier<T> {

    T get(Span span, Logger logger, Meter meter) throws Exception;

}
```

The work happens only if the command runs, and it happens inside the command's span, so its cost, its log records and its failures are attributed to the command that needed it. A cheap value is better asked for directly: adapting the parent is then all it takes.

17.3. The runtime's save command

The `save` command is in the runtime's `cli` module. It has never heard of libraries:

```
@Parent
private ExecutionSupplier<? extends Collection<? extends EObject>> contentsSupplier;

@Parent
private EObjectSupplier<? extends EObject> eObjectSupplier;
```

`ExecutionSupplier<List<EObject>>`, the second adapter of the `library` group, is assignable to the first member's type, so `save` goes under `library` and saves the library:

Listing 93. The runtime's command under the example's group

```
Path file = directory.resolve("shelf.xmi");
assertThat(run("library", "save", file.toString()).exitCode()).isZero();
assertThat(Files.readString(file)).contains("Domain-Driven Design");

// And the saved file is a library a later command can load
assertThat(run("library", "--file", file.toString(), "books", "--before", "1900").out())
    .contains("War and Peace (1869)");
```

The second member is the older shape, a parent implementing `EObjectSupplier`. Both work,

and only one is injected.

17.4. What a command accepts as sub-commands

A `@SubCommand` member declares what a command accepts as sub-commands. A member typed as an array, a `List`, a `Collection`, an `Iterable` or a `Set` takes many sub-commands, and its element type is what is accepted; any other member takes one.

Listing 94. A command that runs what it is given (`cli/⋯/ExportCommand.java`)

```
@Command(
        name = "export",
        description = "Exports the library. Without a format, lists the formats",
        mixinStandardHelpOptions = true)
public class ExportCommand extends CommandBase {

    @Parent
    private ExecutionSupplier<? extends Library> library;                    ①

    @SubCommand
    private List<Exporter> exporters;                                        ②

    /**
     * For the exporters, which take this command as their parent. Not an adapter: an
     * adapter would offer the library to every command that needs one, and they would
     * all appear under export.
     */
    public ExecutionSupplier<? extends Library> library() {                  ③
        return library;
    }

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        PrintWriter out = spec.commandLine().getOut();
        exporters.stream().map(Exporter::format).sorted().forEach(out::println);
        return 0;
    }

}
```

① The export command needs a library from its parent, like `books`.

② It accepts every sub-command adaptable to `Exporter`. Without a sub-command on the command line, it lists their formats.

③ It hands its library down to its exporters with a plain method, deliberately not an adapter.

Sub-command members are also used twice. A sub-command is created under a command whose sub-command members accept its class. After parsing, every member gets the sub-commands adaptable to what it accepts: all of them for a many-valued member, possibly none, and the first for a single-valued one. Sub-commands that are not on the command line have not had their options parsed, so a command uses them for what they are, not for their options.

An exporter takes its parent by class:

Listing 95. An exporter (`cli/⋯/MarkdownExporter.java`)

```
@Command(
        name = "markdown",
        description = "Exports the library as Markdown",
        mixinStandardHelpOptions = true)
public class MarkdownExporter extends CommandBase implements Exporter {

    @Parent
    private ExportCommand parent; // A parent typed by its class

    @Override
    public String format() {
        return "markdown";
    }

    @Override
    public String export(Library library) {
        StringBuilder markdown = new StringBuilder("# ").append(library.getName());
        markdown.append("\n\n");
        for (Book book: library.getBooks()) {
            markdown.append("* _").append(book.getName()).append("_ (")
                .append(book.getYear()).append(")\n");
        }
        return markdown.toString();
    }

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) throws Exception {
        Library library = parent.library().get(span, logger, meter);
        spec.commandLine().getOut().print(export(library));
        return 0;
    }

}
```

`nsdk library export` prints `csv` and `markdown`, and `nsdk library export markdown` prints the library as Markdown. A module that contributes a `JsonExporter` adds a format without

touching the export command.

17.5. Why export hides its library

Had `ExportCommand.library()` been an `@Adapter`, every command that needs a library, `books` included, would have appeared under `export` as well as under `library`. Adapters are offers to every command that might match. A plain method is an offer only to commands that take the export command by class, which is what the exporters do.

The test of the example asserts it: `books` is under `library`, and not under `export`.



> Offer with `@Adapter` what any child may use. Hand down with plain methods what only children written for this parent should use.

17.6. When wiring happens

`WiringExecutionStrategy` injects `@Parent` and `@SubCommand` members of the commands on the command line, and the `@Parent` members of their mix-ins, and then executes as its delegate does, picocli's `RunLast` by default.

- **After parsing.** A parent's options are parsed by then. Adapting the parent while the tree is being built would see their defaults.
- **From the root to the executed command.** A parent's own members are in place before it is adapted for its child, so a value can flow down several levels.
- **Not for help.** Help and version requests skip wiring, so asking for help never fails on a parent that cannot provide.

`SubCommandCapabilityFactory` sets the strategy on the root commands it creates. A command line assembled otherwise needs `commandLine.setExecutionStrategy(new WiringExecutionStrategy())`.

17.7. Rules

- **Members are found once per class**, on the class and its superclasses, and must be accessible to the `cli` module: in a package opened to it, as picocli needs command packages to be anyway. An inaccessible member fails with a message that says which package to open.
- **A parent member is a field or an instance method taking one parameter**, never static. The same holds for sub-command members.
- **Ask with wildcards.** `? extends` at every level that is only read from, so that more parents answer.
- **Ask for an `ExecutionSupplier`** for whatever is expensive to produce.
- **Most specific first.** Of alternative parent members, the one with the most specific type is tried first, which is subject to the tie described in the editor's note of Chapter 8.

17.8. Why declarations

Wiring by declarations, rather than by names or by registration, has three consequences that matter:

- **The tree is known before anything runs.** Help lists exactly the commands that will work, because a command is only placed where its declared needs are met.
- **The compiler checks the contract.** A parent's adapter returns a type, a child's member has a type, and both are checked where they are written. Nothing is matched by a string.
- **Modules stay independent.** The runtime's `save` and the example's `library` have no dependency on each other, and an assistant generating a new command needs to know only the type it consumes.

Chapter 18. Distributions and launching

A command line is only useful when it can be installed and run. This chapter describes what a distribution of the runtime's command line is, how it starts from any directory on any operating system, and how each launch becomes a unit of work whose telemetry is saved as models.

18.1. What a distribution is

The `cli-launcher` module builds a distribution of the runtime's command line, and the same machinery builds a distribution of yours. A distribution is a directory, also packaged as `nsdk.zip`:

```
dist/
  lib/              ①
  modules           ②
  options           ③
  nsdk  nsdk.bat    ④
  nsdk-debug  nsdk-debug.bat
```

① The jars, in the Maven repository layout: about fifty for the runtime's own command line, the bootstrap jar among them.

② The names of the modules in the launcher's module layer, recorded at build time.

③ The Java options of the launch: the module path, the class path and the main module, with paths relative to the distribution root.

④ Wrapper scripts. The debug variants start the JVM with a debugger listening on port 8998.

The wrapper scripts are one line each, broken here to fit the page:

```
#!/bin/bash
exec java -jar \
  "$(dirname "$0")/lib/org/nasdanika/sdk/runtime/bootstrap/2026.10.0/bootstrap-2026.10.0.jar" \
  "$@"
```

A Windows variant of the distribution calls a JDK bundled under `jdk/` instead of the `java` on the path, for machines where installing Java is a project of its own.

18.2. Module path and class path

The runtime is a set of named modules, and so are its direct dependencies: EMF, OpenTelemetry, Jackson. Some of the libraries further down are not modules, or are automatic modules whose names and requirements are guesses.

The launch puts on the module path the named modules and the modules they require, and everything else on the class path, where it is part of the unnamed module. The split is computed once, at build time, from the launcher's module layer, by the `launcher` command, and written to `options`:

```
-p "lib/.../jackson-core-2.22.3.jar;lib/.../opentelemetry-api-1.67.0.jar;..."
--add-modules ALL-SYSTEM
-classpath "lib/.../guava-33.6.0-jre.jar;..."
-m org.nasdanika.sdk.runtime.cli.launcher/org.nasdanika.sdk.runtime.cli.launcher.Launcher
```

The `launcher` command can also be run by hand, for an assembly of your own: it scans directories of jars (`lib` by default) and prints, or writes to a file, a Java command line or just its options. Root modules can be given with patterns, `. for the modules in a package and .*` for those below it, and the Java command can carry extra options such as system properties.

18.3. Starting from any directory

Java resolves the relative paths of an argument file against the current directory, and scripts cannot expand their own location inside one. The bootstrap closes that gap. It is a small jar with no dependencies, and it:

1. Finds the distribution root from its own location, walking up to the nearest `lib` directory.
2. Reads `options` and rewrites every relative `lib/⋯` entry to an absolute path under the root.
3. Writes a temporary argument file with the absolute paths. A file, not a command line, because a distribution with many jars exceeds the Windows limit of 32 KB for a command line.
4. Launches a child JVM with that file and the user's arguments, with standard input,

output and error inherited and the exit code propagated.

The child runs in the user's working directory, so relative file arguments of commands work as expected.

The bootstrap chooses the Java executable in this order: the `BOOTSTRAP_JAVA` environment variable, a `java` property in an optional `bootstrap.conf` in the distribution root, the executable that launched the bootstrap, `java.home`, a bundled `jdk/bin/java`, and finally `java` on the path. `BOOTSTRAP_HOME` forces the distribution root, and `BOOTSTRAP_JAVA_OPTIONS` adds JVM options to one invocation, which is how the debug scripts work. `bootstrap.conf` can also override the options file, the main module and class, the module path and the class path, and add JVM options.

18.4. A launch is a unit of work

`Launcher` is the main class of the distribution. It follows the rule of Chapter 11, one OpenTelemetry instance per unit of work, with the launch as the unit:

1. It finds the home directory: the `nsdk.home` system property, the `NSDK_HOME` environment variable, or the parent of the `lib` directory it was loaded from.
2. It creates a telemetry directory for the launch, `<home>/telemetry/<year>/<month>/<day>-<hour>-<minute>-<second>-<millisecond>`, and three telemetry resources in it, in the format of `nsdk.telemetry.format` or `NSDK_TELEMETRY_FORMAT` (Chapter 13).
3. It builds an OpenTelemetry SDK whose exporters write to those resources, with the service name `launcher`, its version, and the working directory as resource attributes, and W3C propagators.
4. It makes the instance current, continuing the trace in the `TRACEPARENT`, `TRACESTATE` and `BAGGAGE` environment variables if they are set.
5. It resolves the command tree (Chapter 16), adds the `shell` command to the root, and executes the command line in a span named "Executing the root command".
6. It closes the SDK, which exports what is pending and saves `traces`, `logs` and `metrics`, and exits with the command's exit code.

The environment variables connect a launch to whatever started it. A build step, a scheduler or another program that sets `TRACEPARENT` makes the launch a part of its trace, so

a pipeline of command runs is one trace across processes, each saved where it ran.

18.5. Building a distribution

A distribution is built by a Maven module with three pieces:

- **Dependencies.** The bootstrap jar, `org.nasdanika.sdk.runtime:bootstrap`, and the modules of the assembly: the runtime's `all` aggregate and the modules that contribute commands.
- **Copying.** `maven-dependency-plugin:copy-dependencies` with `useRepositoryLayout` copies the runtime dependencies into `target/dist/lib` in the `prepare-package` phase.
- **Generating.** An integration test runs `DistributionBuilder`, which copies the module's own jar, records the module layer, computes `options` and writes the scripts. `maven-assembly-plugin` packages the directory.

The runtime's own integration test is the template:

```
@Test
public void generateLauncher() throws IOException {
    new DistributionBuilder(new File("target/dist")).build();
}

@Test
public void generateWindowsLauncher() throws IOException {
    // The Windows-specific distribution ships a bundled JDK under jdk/.
    new DistributionBuilder(new File("target/dist-win")) {

        @Override
        protected String getBatJava() {
            return "@\"%~dp0jdk\\bin\\java.exe\"";
        }

    }.build();
}
```

`DistributionBuilder` launches `Launcher` from the launcher module. An assembly with its own commands overrides `getModuleName` to return its own module, whose `requires` and `provides` bring in its commands and their dependencies, while the main class stays `Launcher`. The script name, the debug options, the path separator of the options file and the Java invocation are protected methods as well.

18.6. Running it

With the distribution unpacked and its directory on the path:

```
nsdk --help
nsdk shell
```

A distribution assembled with the example module of Chapter 16 also runs its commands, for example `nsdk library --file shelf.yml books --before 2000`.

`nsdk shell` starts an interactive shell over the root command, with history and line editing from JLine; `exit` leaves it. Every command run in the shell is executed by the same root command line, in the same launch, so the telemetry of a shell session is one trace.

After each launch, the telemetry directory holds the record of the run: the command's span with the spans of everything it did, log records in the spans that emitted them, and recorded model changes where a command recorded them (Chapter 13).

Part VI: Models in the Runtime

The models that ship with the runtime: a Groovy DSL to author any of them, the core model of identity, documentation and provenance, Meta, the JVM model, the models that make the web a generation target, user interfaces assembled from views chosen by the class of what they show, and diagrams as sources of models and as drawings of them.

Chapter 19. Authoring models with the Groovy DSL

XMI, JSON and YAML are good formats for saving models and poor ones for writing them. Xcore is the language for metamodels (Chapter 7), and there is no equivalent for instances: a library, an architecture, a backlog.

The runtime's `groovy` module fills the gap with a Groovy DSL that is driven by the metamodel. Nothing in it is written for a particular class or feature: every keyword, every nesting and every attribute is read reflectively from the packages the DSL is bound to. Bind it to a different metamodel and you get a different DSL, for free.

Groovy brings what a home-made language would have to build: nesting, variables, loops, string interpolation, and code where code is the clearest way to say something.

This chapter shows the DSL at the level of the rest of this book: what it is for, its rules, and how it is bound to a metamodel. The Groovy DSL Zephyr Book (The Nasdanika Zephyr Books) is its manual, with every call form, selector and option, the patterns for saving scripts, and the handlers you write for a metamodel of your own.

19.1. A library as a script

Listing 96. A library as a Groovy script (`dsl/src/test/resources/reading-group.library.groovy`)

```
library {
    name 'Reading Group'

    def evans = author { name 'Eric Evans' }         ①
    def fowler = author { name 'Martin Fowler' }

    book {                                           ②
        name 'Domain-Driven Design'                  ③
        year 2003                                    ④
        genre 'TECHNOLOGY'
        authors evans                                ⑤
        cites "../books[name='Patterns of Enterprise Application Architecture']" ⑥
        documentation '''
            Named the *Ubiquitous Language* and the *Published Language*.
            '''                                      ⑦
    }
```

```
    book {
        name 'Patterns of Enterprise Application Architecture'
        year 2002
        genre 'TECHNOLOGY'
        authors fowler
    }
}
```

① **Type dispatch.** `author` names the class `Author`, so the call creates an author and puts it into the one containment feature of the library that accepts authors. The call returns the object, so it can be kept in a variable.

② A book, into `Library.books` by the same rule.

③ **Feature dispatch.** `name` names a feature of the book, inherited from the core model's `NamedElement`, so the call sets it. Feature dispatch is tried before type dispatch.

④ Values are coerced to the feature's type: `2003` to `int`, `'TECHNOLOGY'` to the enumeration literal.

⑤ A reference to an object the script holds.

⑥ A reference by path, to a book defined further down. Paths are resolved after the whole script has run, so forward references work.

⑦ The core model's documentation feature, in Markdown.

The file name selects the language. It is read as qualifiers from right to left (Chapter 10): `groovy` compiles the text to a script, and `library` binds the DSL to the library metamodel and evaluates it. Loading the script is loading a resource:

Listing 97. Loading a script

```
Library library = (Library) NasdanikaResourceSet.createAndConfigure()
        .getResource(uri("reading-group.library.groovy"), true) ①
        .getContents()
        .getFirst();

Book ddd = library.getBooks().getFirst();
assertThat(ddd.getAuthors()).extracting(a -> a.getName()).containsExactly("Eric Evans");
assertThat(ddd.getCites()).containsExactly(library.getBooks().get(1));
assertThat(ddd.getDocumentation()).contains("Published Language");
```

① A configured resource set has the Groovy resource factory, and the qualifiers select the handlers.

The model is an ordinary model: it can be saved in any format, validated, transformed,

served or generated from. The script is the source; the model is what it means.

19.2. The rules of the language

Entry points. At the top level, every concrete class of the bound packages whose uncapitalized simple name is unambiguous is a keyword: `library`, `ePackage`.

Dispatch. Inside a builder closure, a call whose name is a feature of the current object sets the feature; otherwise a call whose name is a class creates an object of that class in the one containment feature that accepts it. When a class fits more than one containment feature, or the feature is typed by an abstract class, the feature is named and the class given explicitly: `eStructuralFeatures('EReference') { … }`.

Built-in keywords. `ref`, `dsl`, `onSave`, `loadResource`, `copy` and `eObject` at the top level, and `global`, `proxy`, `loadResource`, `copy` and `eObject` in a closure, win over both kinds of dispatch.

Setting features. `year 2003`, `year(2003)` and `year = 2003` all set the feature. One form is a trap: `upperBound -1` is the expression `upperBound - 1` in Groovy, which reads the feature, subtracts one and discards the result. Whenever a value starts with a minus or a plus sign, use the assignment or the parenthesized form.

References take one of three values:

- An object the script holds, in a variable.
- A path in the style of EMF URI fragments, resolved after the script has run: `/` anchors it at the root, `..` goes to the container, and predicates select by attribute, as in `../books[name='Patterns of Enterprise Application Architecture']`. A path containing `#` is a URI, resolved against the script's own URI, possibly in another resource.
- A closure, evaluated after the script with the referencing object as its delegate, which returns an object or a path.

Names of classes can be simple (`Book`), uncapitalized (`book`), qualified by a package path (`library.Book`) or a namespace URI with a fragment (`<nsURI>#Book`). When a simple name matches several classes and one is a subclass of another, the more specific class wins, which is what makes refinements of a metamodel work in the same script as the metamodel itself. A name that is still ambiguous is an error that lists the qualified

alternatives.

Results. The contents of the resource are the script's last expression, if it is an object or a collection of objects, and otherwise the roots created by entry points.

19.3. Metamodels too

The runtime binds the DSL to Ecore itself, under the qualifier `ecore`, so a metamodel can be a script as well:

Listing 98. A metamodel as a script (`dsl/src/test/resources/catalog.ecore.groovy`)

```
ePackage {
    name 'catalog'
    nsURI 'https://example.org/catalog'
    nsPrefix 'catalog'

    def entry = eClass {                                          ①
        name 'Entry'
        eAttribute { name 'title'; eType 'EString'; lowerBound 1 }  ②
    }

    eClass {
        name 'Catalog'
        eStructuralFeatures('EReference') {                       ③
            name 'entries'
            eType entry
            containment true
            upperBound = -1                                       ④
        }
    }
}
```

① A class, kept in a variable for the reference below.

② Several features on one line, separated by semicolons. `eType 'EString'` is resolved as a class name.

③ `eStructuralFeatures` accepts attributes and references, so the class is named.

④ The assignment form, because the value is negative.

Listing 99. Loading a metamodel script

```
EPackage catalog = (EPackage) NasdanikaResourceSet.createAndConfigure()
        .getResource(uri("catalog.ecore.groovy"), true)
        .getContents()
        .getFirst();
```

```
EClass entry = (EClass) catalog.getEClassifier("Entry");
EClass catalogClass = (EClass) catalog.getEClassifier("Catalog");
assertThat(catalogClass.getEStructuralFeature("entries").getEType()).isSameAs(entry);
assertThat(catalogClass.getEStructuralFeature("entries").isMany()).isTrue();
```

Xcore remains the way to define a metamodel that has generated code. An Ecore script is the quick way to sketch one, or to derive one from something else.

19.4. Binding the DSL to a metamodel

Binding the DSL to a metamodel takes a handler that names the packages:

Listing 100. The library DSL (dsl/⋯/LibraryDslHandler.java)

```
/**
 * The whole DSL for libraries: the metamodel is the language.
 */
public class LibraryDslHandler extends DslResourceContentsHandler {

    public LibraryDslHandler(
            Resource resource,
            ResourceContentsHandler<CompiledScript> sourceHandler) {
        super(resource, sourceHandler, LibraryPackage.eINSTANCE);
    }

}
```

and a capability factory that decides which qualifier it serves and connects it to the source handler on its right:

Listing 101. The qualifier (dsl/⋯/LibraryDsl.java)

```
/**
 * Serves files named *.library.groovy: the library qualifier, with the groovy source to its
 * right.
 */
public class LibraryDsl extends ReflectiveCapabilityFactory {

    public LibraryDsl() {
        super(MethodHandles.lookup());
    }

    @Capability
    public Flow.Publisher<ResourceContentsHandler<EObject[]>> library(
            ResourceContentsHandler.Requirement requirement,
            Loader loader) {
        int index = requirement.getQualifierIndex();
```

```
        if (!EObject[].class.equals(requirement.getContentsType())
                || index <= 0 // A source must be to the right
                || !"library".equalsIgnoreCase(requirement.getQualifiers()[index])) {
            return null;                                                          ①
        }

        Resource resource = requirement.getResource();
        var source = ResourceContentsHandler.createRequirement(                  ②
                resource,
                CompiledScript.class,
                requirement.getQualifiers(),
                index - 1);
        CompletableFuture<List<ResourceContentsHandler<CompiledScript>>> sources =
                loader.loadAll(Requirement.of(ResourceContentsHandler.class, source));

        CompletableFuture<List<ResourceContentsHandler<EObject[]>>> handlers = sources
                .thenApply(list -> list.stream().map(s -> dsl(resource, s)).toList()); ③
        return Flows.concatMap(Flows.fromFuture(handlers), Flows::fromIterable);
    }

    private static ResourceContentsHandler<EObject[]> dsl(
            Resource resource,
            ResourceContentsHandler<CompiledScript> source) {
        return new LibraryDslHandler(resource, source);
    }

}
```

① Not a library script: some other factory's business.

② The handler of the qualifier to the right, `groovy`, which compiles the source.

③ One library handler per source handler.

Listing 102. The module (`dsl/⋯/module-info.java`)

```
import org.example.dsl.LibraryDsl;
import org.nasdanika.sdk.runtime.common.capability.CapabilityFactory;

module org.example.dsl {

    requires transitive org.example.library;
    requires transitive org.nasdanika.sdk.runtime.groovy;

    exports org.example.dsl;

    provides CapabilityFactory with LibraryDsl;

}
```

A handler can do more than name packages: provide bindings to the script, a short form

that creates an element from a plain value for a containment reference, and source locations for the elements it creates.

19.5. Scripts that derive models

A script is not limited to transcribing a model:

- `loadResource 'payments.drawio'` loads another resource through the script's resource set, relative to the script, with every handler and factory the set has. A script can iterate a diagram, a spreadsheet or another model and emit a model from it, which turns a transcription into a derivation. The Draw.io Zephyr Book shows the diagram's side of this (The Nasdanika Zephyr Books).
- `copy '<uri>#<fragment>' { … }` deep-copies an element as a prototype and then populates the copy like a new element: structure from a diagram, details from code. A copy is a snapshot, not a live link.
- `global '<uri>'` registers the current element under a URI, so that other resources in the same resource set can reference it.

A DSL resource is read-only by default, because the script, not the model, is the source of truth and there is no general way to write a model back as a script. An `onSave` callback in the script decides what saving means when it should mean something: writing the model elsewhere, or rewriting the script.

19.6. Errors and telemetry

Failures are reported with the script's URI and line, including failures of references resolved after the script has run, which report the line where the reference was written.

Loading a script is traced, and the spans nest the way the script does. This test collects the spans of loading the library script, with the script lines from the `code.line.number` attribute:

Listing 103. Collecting the spans of loading a script

```
List<String> spans = Collections.synchronizedList(new ArrayList<>());
var provider = new PublishingTracerProvider(TracerProvider.noop(), Runnable::run, 4096);
try (var openTelemetry = new PublishingOpenTelemetry(OpenTelemetry.noop(), provider);
        Scope scope = Telemetry.makeCurrent(openTelemetry)) {
```

```
    openTelemetry.subscribe(new DslSpans(spans));                          ①
    NasdanikaResourceSet.createAndConfigure()
        .getResource(uri("reading-group.library.groovy"), true);
}
assertThat(spans).containsSubsequence(
        "      Setting books, line 7",
        "        Loading Book, line 7",                                    ②
        "          Setting cites, line 12",
        "      Resolving cites, line 12");                                 ③
```

① A subscriber that records started spans with their line numbers, indented by nesting.

② The first book, from line 7, created in `Library.books`.

③ Its forward reference, resolved after the script ran, reported with the line where it was written.

The complete tree:

```
Resource.load
  Compiling script
  Evaluating script
    Loading Library, line 1
      Setting name, line 2
      Setting authors, line 4
        Loading Author, line 4
          Setting name, line 4
      Setting authors, line 5
        Loading Author, line 5
          Setting name, line 5
      Setting books, line 7
        Loading Book, line 7
          Setting name, line 8
          Setting year, line 9
          Setting genre, line 10
          Setting authors, line 11
          Setting cites, line 12
          Setting documentation, line 13
      Setting books, line 18
        Loading Book, line 18
          Setting name, line 19
          Setting year, line 20
          Setting genre, line 21
          Setting authors, line 22
    Resolving references
      Resolving cites, line 12
```

Spans carry locations, never loaded values: the script URI, the line, and the identities of classes and features as `<nsURI>#<name>`. A long script that fails, or that is slow, says where.

19.7. What comes next

The DSL is built on the JDK’s scripting API, `javax.script`, so the same pipeline can serve other JVM languages SKETCH. Kotlin scripts are designed, with the same qualifiers, so `reading-group.library.kts` would load like the Groovy script. Validation rules expressed in the DSL, and a path from a sketched metamodel script to Xcore and generated code, are part of the same design.

Chapter 20. The core model

Every model needs the same few things: elements that can be identified, named and documented, that know where they came from, and that can be found by whatever refers to them. Written once, these are a foundation; written in every model, they are a source of small, incompatible differences.

The runtime's core model, in `models/core`, is that foundation. The library of this book extends it, and so do the runtime's own models and the models of the tower to come (Chapter 26). In one sentence: model elements that are named, documented, identifiable, referrable, and that know where they came from.

20.1. The element hierarchy

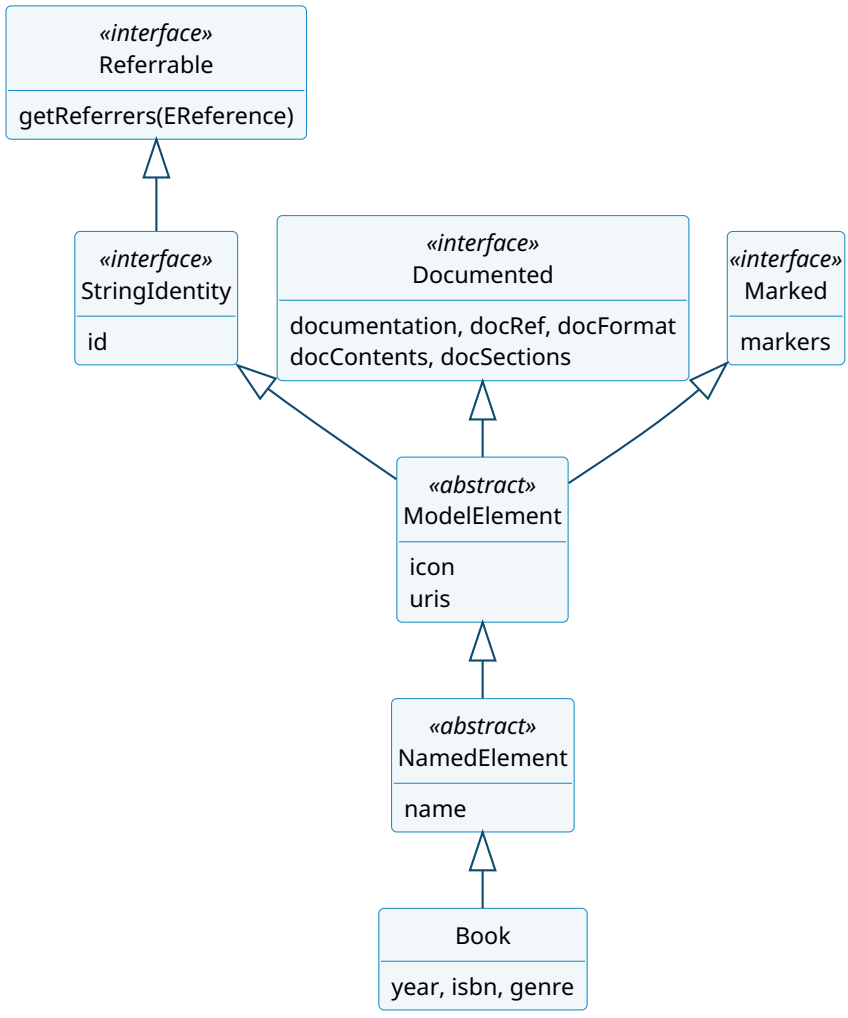


Figure 7. The core hierarchy, and the library's `Book`

`ModelElement` combines identity, documentation and provenance, and adds an icon and alias URIs. `NamedElement` adds a name. A model extends `NamedElement`, or `ModelElement` for elements without a name, and its elements get the rest by inheritance, along with everything built on top: documentation generation, sites, the polymorphic UI.

20.2. Identity and references

`StringIdentity` gives an element an `id`: a stable, human-readable identifier for URIs and cross-references. A name is for people and may change; an id is for references and should not.

`ModelElement.uris` are alias URIs, which may be absolute or opaque, such as `actors://john-smith`. They act as merge anchors: several teams may maintain elements with the same URI, for example `myorg://capabilities/developer-productivity`, and when their models are assembled into one, those elements are one logical element.

`Referrable.getReferrers(reference)` is inverse navigation without opposite references. It finds the objects in the element's containment tree, resource or resource set that reference the element through a given reference:

Listing 104. Who refers to this element

```
Library library = Samples.library();
Book patterns = book(library, "Design Patterns");

// Book.cites has no opposite. Who cites Design Patterns?
assertThat(patterns.getReferrers(LibraryPackage.Literals.BOOK__CITES))
    .extracting(referrer -> ((Book) referrer).getName())
    .containsExactlyInAnyOrder("Domain-Driven Design", "EMF: Eclipse Modeling Framework");
```

An opposite reference is the right choice when both directions are part of the model's meaning, and it has a cost: both ends must be in memory and kept consistent, which fails across resources loaded on demand. `getReferrers` costs a walk of what is loaded, and nothing when it is not called. The catalog volume of Chapter 14 uses it for the "Cited by" line of each book.

20.3. Documentation

`Documented` gives every element documentation that can be as short or as long as it needs to be:

Listing 105. Documenting an element

```
Book ddd = book(Samples.library(), "Domain-Driven Design");
ddd.setDocumentation("""
        Introduced the *Ubiquitous Language* and the *Published Language*:
        a vocabulary shared by a team, and its documented form shared with others.
        """);                                   ①
ddd.setDocRef("ddd/review.md");                 ②
ddd.setId("ddd");                               ③
ddd.setIcon("fas fa-book");                     ④
```

① Inline documentation, in Markdown unless `docFormat` says otherwise.

② Long-form documentation in a resource, resolved relative to the element's resource, with its format derived from the extension.

③ An identity for URIs and references.

④ An icon: a URL if it contains a slash, a CSS class otherwise, such as a Font Awesome icon.

Beyond the documentation and its reference, `docContents` adds a list of contents, inline or by reference, each with a content type, and `docSections` adds a tree of titled sections. Documentation can be written where it belongs, a sentence inline, a chapter in a Markdown file next to the model, and still be assembled into one site.

Micro-wikis

Documentation anchored to model elements grows into a way of sharing knowledge of its own: the micro-wiki. Every element of a model anchors its own documentation space, a Markdown page referenced by `docRef`, with sections and contents beside it.

A wiki has depth and is easy to contribute to, and it has no structure: pages multiply, links rot, and nobody knows which page is the one about a given thing. A model has structure, and it is a poor place for long prose. A micro-wiki takes the depth and the ease of contribution of the wiki, and gives it what it never had: the model as its site map. Navigation, cross-references and diagrams are generated from the model, and the pages are where people write.

Two properties make micro-wikis worth the name:

- **Documentation cannot drift away from what it describes**, because it lives at it. Rename an element and its page moves with it; remove the element and the page goes too, in the same change.
- **Generated stubs say which file documents what.** A documentation stub per element tells people, and assistants, which file to edit when they learn something about that element. An assistant asked to map a set of guidelines onto a model edits the stub of each element with the paragraphs that govern it, and the published site shows, for every element, the rules that apply to it.

Micro-models decompose meaning (Chapter 2); micro-wikis decompose the documentation of that meaning, along the same lines. The core model provides the structure today.

Generating micro-wiki sites, with stubs and a site map, was part of the previous generation of Nasdanika's tools and is planned for the runtime PLANNED.

20.4. Provenance

`Marked` elements carry markers that record where an element, or one feature of it, came from:

Listing 106. Recording where an element came from

```
Book ddd = book(Samples.library(), "Domain-Driven Design");
Marker marker = CoreFactory.eINSTANCE.createMarker();
marker.setLocation("https://example.org/catalogs/2026/books.csv"); ①
marker.setPosition("42");                                          ②
marker.setDate(new Date());
marker.setComment("Imported from the acquisitions spreadsheet");
ddd.getMarkers().add(marker);
```

① The location of the source, typically the URI of the file the element was loaded from.

② A position in the source: a line, a cell, a page.

A marker can name the feature it is about, carry a digest of its source, a hash of a file or of a model, and have child markers, for a value computed from other values that have markers of their own. `GitMarker` adds the path in a repository, its remotes, the branch, the head commit and the refs on it: with the position, enough to build a permalink to the exact line in the exact commit.

Markers make what is generated from models traceable. Every element on a generated page can point back to the file, the line and the version it came from, and a derived value can be traced through its inputs.

20.5. Attributes

`Attributed` is a mixin for elements that carry `attributes`: string keys and string values. They hold what a source format carries and a model does not name, such as the properties of a Draw.io shape or the block attributes of Markdown and AsciiDoc, and conventions of one's own, such as layout options of a diagram element. The diagram model's elements are `Attributed`, and so are, through them, the Draw.io model's (Chapter 25).

They are called attributes, not properties, so that a model with properties of its own can mix in `Attributed` without a clash.

20.6. Evaluators

An evaluator is a documented, marked expression or script, evaluated against bindings and coerced to a result type. It is the core model's way of putting computed values and conditions into models: a rule, a derived property, a filter.

Listing 107. A rule in the model

```
ScriptEvaluator classic = CoreFactory.eINSTANCE.createScriptEvaluator();
classic.setLanguage("groovy");                                          ①
classic.setScript("book.year < 1980 && book.authors.size() == 1");
classic.setDocumentation("A classic: older than 1980, with a single author.");

Library library = Samples.library();
for (Book book: library.getBooks()) {
    Boolean result = classic.evaluate(Boolean.class, Map.of("book", book)); ②
    if (result) {
        System.out.println(book.getName());
    }
}
```

① A JSR-223 script. The language names the engine, here Groovy, which the `dsl` example module has on its module path.

② The bindings are the script's variables, and the evaluator itself is bound as `self`.

It prints "The Art of Computer Programming", "On the Origin of Species" and "War and Peace".

Evaluator	Source	Status
`ScriptEvaluator`	A script in any JSR-223 language, inline or by reference. A referenced script selects its engine by extension when no language is given	**AVAILABLE**
`GroovyEvaluator`	A Groovy script, inline or by reference	**PLANNED**
`SpelEvaluator`	A Spring Expression Language expression	**PLANNED**
`XPathEvaluator`	An XPath expression over the model	**PLANNED**

The planned evaluators are declared in the model, so that models can contain them today, and throw `UnsupportedOperationException` when evaluated. Their implementations are to be loaded as services, which keeps the core model free of the engines' dependencies.

20.7. Why a shared core matters

The benefits of the core model compound with the number of models that extend it.

- **One generator for many models.** A documentation generator that knows `NamedElement`, `Documented` and `Marked` documents any model in the tower, including ones written after it.
- **One UI for many models.** The polymorphic UI's views of the core model show names, icons, documentation and provenance for any element, whatever its class.
- **Merging across teams.** Alias URIs let independently maintained models be assembled into one.
- **Traceability by default.** Markers are there to fill, by loaders, by importers, by agents recording where a change came from.

The library of this book got all of it by writing `extends NamedElement`.

Chapter 21. Meta: a language-neutral type system

A class with named, typed, possibly multi-valued features is one of the most stable ideas in computing, and every ecosystem spells it its own way: Ecore, Java, XML Schema, JSON Schema, a database schema, TypeScript, a table on a wiki page, a spreadsheet. The differences are real and small, and most are about the host language, not about the domain.

The cost is not the spelling. It is that one definition, of a customer, of a book, of a service, lives in many formats at once and nothing relates them. Each is a projection of a domain model that was never written down.

Meta, in `models/meta`, is a place to write it down once and project. It is a micro-model of typed domain models, about twenty classifiers, positioned directly on the core model (Chapter 20). In the terms of Domain-Driven Design it is a Published Language: a documented, shared vocabulary that bounded contexts translate into and out of, with the part that was out of reach when the term was coined: the vocabulary is machine-checkable, because the types and the documentation are generated from it.

21.1. What it models

Area	Classifiers
Base	`ModelElement`, on the core model's `NamedElement`, and `Annotation`
Structure	`Package`, `Classifier`, `Class`, `DataType`, `Enum`, `EnumLiteral`
Members	`TypedElement`, `Feature`, `Attribute`, `Reference`, `Operation`, `Parameter`
Types	`TypeParameter`, `TypeReference`, `ClassifierReference`, `TypeParameterReference`, `PrimitiveTypeReference`, `UnionTypeReference`, `IntersectionTypeReference`, `WildcardTypeReference`, `OpaqueTypeReference`, `PrimitiveKind`
Instances	`Object`, `IncomingReference`

`Attribute.identifying` marks the attributes that identify an instance, which is how a reference cites a target, and how a Domain-Driven Design entity is told from a value

object. `Reference.containment` separates tree edges from graph edges; `opposite` and `keys` carry bidirectionality and identity within a list. A `Package` is the unit of publication: one package becomes one Maven artifact or one npm package.

The model is metacircular: a package containing classes describes `Package` and `Class`, so a runtime can bootstrap from it alone.

21.2. What it leaves out

The rule is that what a class-based type system cannot express is not a class in Meta. That keeps it at about twenty classifiers, against Ecore's fifty-three:

- No `volatile`, `transient`, `unsettable` or `resolveProxies`: implementation concerns of EMF with no meaning in a browser.
- No feature maps, no factories, no generic type algebra. Unions, intersections and wildcards are carried, so that a Java or TypeScript signature can be written down without loss, but nothing computes subtyping or substitution with them.
- No subpackages, which neither TypeScript modules nor npm packages need. A loader flattens and records the original path.
- No interface flag. Whether a class becomes an interface, a type alias or a class is the projection's decision.

What a source expresses and Meta does not goes into an `Annotation`, keyed by a source URI, the bargain Ecore's `EAnnotation` makes. Loaders record what they cannot map instead of dropping it, so a round trip to the source stays possible.

21.3. Documentation as structure

Most formats carry documentation as one string: an Ecore annotation detail, a JSON Schema `description`, a database column remark. XML Schema, OpenAPI and OWL do better. None of them treats documentation as structure: a string cannot be divided into sections, point to a resource elsewhere, declare its format, or say who owns it.

Meta elements are core model elements, so they have all of that, and markers that say where each element came from: this class from that sheet of that spreadsheet, at that

commit; this feature from that schema element. When two sources disagree about what a term means, the model shows both, and where each came from.

21.4. A vocabulary

Here is the library's vocabulary in Meta, written with small builders:

Listing 108. Builders (meta/⋯/Vocabulary.java)

```
static Class type(Package vocabulary, String name, String definition) {
    Class type = META.createClass();
    type.setName(name);
    type.setDocumentation(definition);                    ①
    vocabulary.getClassifiers().add(type);
    return type;
}

static Attribute attribute(
        Class owner,
        String name,
        PrimitiveKind kind,
        boolean identifying,
        String definition) {
    Attribute attribute = META.createAttribute();
    attribute.setName(name);
    attribute.setDocumentation(definition);
    attribute.setIdentifying(identifying);                ②
    attribute.setLowerBound(identifying ? 1 : 0);
    PrimitiveTypeReference type = META.createPrimitiveTypeReference();
    type.setKind(kind);
    attribute.setType(type);
    owner.getFeatures().add(attribute);
    return attribute;
}

static Reference reference(Class owner, String name, Class target, String definition) {
    Reference reference = META.createReference();
    reference.setName(name);
    reference.setDocumentation(definition);
    reference.setUpperBound(-1);
    ClassifierReference type = META.createClassifierReference();
    type.setTarget(target);
    reference.setType(type);
    owner.getFeatures().add(reference);
    return reference;
}
```

① Documentation is the core model's structure, not a string attribute of Meta.

② Identity of instances, as for entities in Domain-Driven Design.

Listing 109. The vocabulary (`meta/⋯/Vocabulary.java`)

```
public static Package library() {
    Package vocabulary = META.createPackage();
    vocabulary.setName("library");
    vocabulary.setNamespace("https://example.org/vocabulary/library");
    vocabulary.setDocumentation("The words a library uses, and what they mean here.");

    Class person = type(vocabulary, "Person",
            "Someone who wrote, edited or translated a book.");
    attribute(person, "name", PrimitiveKind.STRING, true,
            "The name as printed on the title page.");

    Class book = type(vocabulary, "Book",
            "A published work, identified by its ISBN.");
    attribute(book, "isbn", PrimitiveKind.STRING, true,
            "International Standard Book Number, 13 digits.");
    attribute(book, "title", PrimitiveKind.STRING, false,
            "The title as printed, subtitle excluded.");
    attribute(book, "published", PrimitiveKind.DATE, false,
            "The date of the first edition.");
    reference(book, "authors", person,
            "The people credited as authors, in the order printed.");
    return vocabulary;
}
```

A vocabulary is a model, so it is saved, versioned, diffed and reviewed like any other:

Listing 110. Saving a vocabulary

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
URI uri = URI.createFileURI(directory.resolve("library.meta.yml").toString());
Resource resource = resourceSet.createResource(uri);
resource.getContents().add(Vocabulary.library());
resource.save(null);                                                    ①
```

① A YAML file in a repository. A change to what a term means is a pull request whose diff shows which features moved.

21.5. The text syntax

A vocabulary built in Java is a program that builds a vocabulary, which is how a harvester or a projection makes one, and not how a person should write one. A vocabulary written in Meta's text syntax is the vocabulary itself: a file a developer reads top to bottom, diffs in

a pull request, and edits in any editor.

The syntax writes metamodels the way Xcore writes Ecore models (Chapter 7). Files have the extension `.nmeta`, one file is one package, and they are parsed by the ANTLR runtime alone, with no Xtext and no Eclipse. The grammar, `models/meta/model/src/main/grammar/Meta.g4`, documents every construct and how it maps to the model.

Here is the library again:

Listing 111. The library's vocabulary as text (`meta/src/main/meta/library.nmeta`)

```
/** The words a library uses, and what they mean here. */
package library namespace "https://example.org/vocabulary/library"

/** Someone who wrote, edited or translated a book. */
class Person {
    /** The name as printed on the title page. */
    id string[1] name
}

/** A published work, identified by its ISBN. */
class Book {
    /** International Standard Book Number, 13 digits. */
    id string[1] isbn

    /** The title as printed, subtitle excluded. */
    string title

    /** The date of the first edition. */
    date published

    /** The people credited as authors, in the order printed. */
    refers Person[*] authors
}
```

A documentation comment, `/** ... */`, is the Markdown documentation of the element that follows it, and other comments are ignored.

A second file shows the rest of the notation. Lending is a vocabulary of its own, kept by the circulation desk, which refers to the library's books without owning them:

Listing 112. A package and its imports (`meta/src/main/meta/lending.nmeta`)

```
/** What the circulation desk lends, to whom and for how long. */
package lending namespace "https://example.org/vocabulary/lending" version "2026.10.0"
```

```
import "https://example.org/vocabulary/library" as library
import "https://runtime.sdk.nasdanika.org/models/occupancy" as occupancy
annotation "urn:org.nasdanika" as Nasdanika
```

- The namespace URI is the package's identity: what other packages import, and what a reference to one of its classes resolves against. The prefix and the version are optional.
- `import` names another package by its namespace, under an alias. `library.Book` is then the class `Book` of that package. A namespace is a name, not an address: nothing is fetched from it.
- `annotation` declares an alias for an annotation source, so that `@Nasdanika(icon = "bi-person")` is an annotation with the source `urn:org.nasdanika` and a detail `icon`.

Listing 113. Data types and enumerations (`meta/src/main/meta/lending.nmeta`)

```
/** An amount of money, a decimal string so that no cent is lost. */
type Amount as string wraps java.math.BigDecimal

/** Where a loan stands. */
enum LoanStatus {
    ACTIVE,
    RETURNED,
    WRITTEN_OFF as "written-off"
}
```

A data type has a name of its own and says how it crosses the wire: an `Amount` is a string, so that no projection turns money into a floating-point number. `wraps` records the type it stands for in a source language, as provenance. An enumeration's literals may have a serialized form, `as "written-off"`, and an ordinal, `= 3`; without one, a literal's ordinal is one more than the previous literal's.

Listing 114. Classes (`meta/src/main/meta/lending.nmeta`)

```
/** Someone entitled to borrow, with a card to show for it. */
@Nasdanika(icon = "bi-person")
class Member {
    /** Identifies the member, and is printed on the card. */
    id string[1] cardNumber

    /** The loans made to the member, addressed by their identifiers. */
    contains Loan[*] loans opposite member keys loanId
}

/** A book in a member's hands, until it comes back. */
```

```
class Loan {
    /** Identifies the loan among the member's loans. */
    id string[1] loanId

    /** The member who borrowed the book. */
    refers Member[1] member opposite loans

    /** The book on loan. Defined by the library's vocabulary, not here. */
    refers library.Book[1] book

    /** The day the book is due back. */
    date[1] due

    /** Where the loan stands. */
    LoanStatus[1] status = "ACTIVE"

    /** The fine so far, computed rather than stored. Not every jurisdiction allows one. */
    @occupancy.Presence { jurisdiction: "CA-ON" }
    derived readonly Amount fine

    /** The fine as it would stand at the end of a day. */
    op Amount fineOn(date day)
}
```

A feature is a type followed by a name, as in Xcore:

- With `refers` or `contains` it is a reference, and `contains` makes the target part of this object: a loan belongs to its member, a book does not.
- Otherwise it is an attribute, of a primitive kind, such as `string` and `date`, or of a data type or enumeration.
- `id` marks an attribute that identifies its instance.
- `opposite` pairs two references, `keys` names the attributes that identify a contained object within its list, and `= "ACTIVE"` is a default value.
- `derived` and `readonly` say that a feature is computed rather than stored, and `op` declares the signature of an operation. Meta carries no bodies.

The multiplicity follows the type:

Written	Bounds	Meaning
`string name`	0..1	Optional, single-valued
`string[1] name`	1..1	Required
`string[?] name`	0..1	Optional, said explicitly

Written	Bounds	Meaning
`Loan[*] loans`, `Loan[] loans`	0..*	Any number
`Author[+] authors`	1..*	At least one
`string[2..5] names`, `string[2..*] names`	2..5, 2..*	A range

Types also combine. `string | number` is a union, `A & B` an intersection, `Repository<Loan, string>` a class with type arguments, `? extends T` a wildcard, and text in backticks an opaque type, for what nothing else covers, such as TypeScript's `Record<string, unknown>`. They are rarely needed in a business vocabulary, and they are what lets a Java or TypeScript signature be written down without loss.

An annotation is a bag of strings keyed by its source. A facet is typed: `@occupancy.Presence { jurisdiction: "CA-ON" }` says where the fine applies, with a class of an occupancy model as the type of what it says. A facet is checked against its type when the type is available, and a property the class has no feature for is a warning. The occupancy model is **PLANNED**, so here the facet is read without checking, and the warning says so. Until the core model has facets, a facet is recorded as an annotation whose source is the URI of its type **SKETCH**.

21.6. Reading and writing text

A vocabulary in text loads like any other model:

Listing 115. A resource set for vocabularies (`meta/⋯/TestMeta.java`)

```
static ResourceSet vocabularies() {
    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();    ①
    resourceSet.getURIConverter().getURIMap().put(                          ②
            URI.createURI("https://example.org/vocabulary/library"),
            URI.createFileURI(TEXT.resolve("library.nmeta").toString()));
    return resourceSet;
}
```

① The module's resource set contributor registers the text syntax for `.nmeta` files.

② The library is another team's vocabulary. The URI map says where its namespace is found here: a file next to this one.

Listing 116. Loading the lending vocabulary (`meta/⋯/TestMeta.java`)

```
Resource resource = vocabularies().getResource(LENDING, true);
Package lending = (Package) resource.getContents().getFirst();

Class loan = (Class) classifier(lending, "Loan");
ClassifierReference book = (ClassifierReference) feature(loan, "book").getType();
assertThat(book.getTarget().getName()).isEqualTo("Book");                ①
assertThat(book.getTarget().eResource().getURI().lastSegment())
    .isEqualTo("library.nmeta");

Reference loans = (Reference) feature((Class) classifier(lending, "Member"), "loans");
assertThat(loans.isContainment()).isTrue();                              ②
assertThat(loans.getOpposite()).isSameAs(feature(loan, "member"));
assertThat(loans.getKeys()).extracting(Feature::getName).containsExactly("loanId");

assertThat(resource.getWarnings()).extracting(Resource.Diagnostic::getMessage)  ③
    .containsExactly("Facet type occupancy.Presence is not available, "
            + "its properties are not checked");
```

① `library.Book` resolves to a class of another resource, loaded on demand from where the URI map says.

② The target is part of this object, its opposite is the member, and `loanId` identifies it within the list.

③ The occupancy facet's type is not available. That is a warning, not an error.

A name qualified with an alias becomes a classifier reference with the imported namespace and the class name. Its target is set when the package of that namespace is available: loaded in the resource set already, or loadable because the URI converter maps the namespace to a location. Otherwise the reference keeps the namespace and the name, and the package is read on its own, which is what lets a team publish its vocabulary without the vocabularies it refers to.

The reader collects what is wrong in the resource's `getErrors()` and `getWarnings()`, each with its file, line and column, and loading fails with the first error, as EMF's XML resources do. Every package, classifier, feature, operation, parameter and enum literal it reads gets a core model marker with the file's URI and the `line:column` where its declaration starts. When a generated page or a projected schema says something surprising, the marker says which line of which file it came from.

A vocabulary read from text is the same model as one built in code, so it is saved in any format a resource set supports, and the projections below work on it unchanged. The

other direction is `MetaWriter`, which writes a package as text, and which the resource uses to save a `.nmeta` file:

Listing 117. Writing a vocabulary as text (`meta/⋯/TestMeta.java`)

```
Package library = Vocabulary.library();                                    ①
String text = MetaWriter.write(library);

URI uri = URI.createFileURI(directory.resolve("library.nmeta").toString());
Resource resource = vocabularies().createResource(uri);
resource.getContents().add(library);
resource.save(null);                                                       ②
```

① The vocabulary built with the builders above. The text is `library.nmeta`, which the test checks.

② The same text, as a file.

The text is canonical, not a copy of a source: comments other than documentation and the layout of the source are not kept, and the imports and annotation aliases are derived from the namespaces and sources the package uses. Reading what was written and writing it again gives the same text. That makes the text syntax a target as well as a source: a vocabulary harvested from a database or authored in a spreadsheet can be written as text, reviewed as text, and compared with last month's as text.

The Zephyr Book *Meta Model* (The Nasdanika Zephyr Books) writes a bank's whole vocabulary in the text syntax, from the first line of a file to the glossary the business reads.

21.7. Projections

A vocabulary is worth writing down once because of what is projected from it. The runtime has the first projection below, a glossary; the others are planned, and the example sketches the second to show the idea.

The first is for people: a glossary that cannot drift from the types, because both come from one source.

Listing 118. A glossary (`meta/⋯/TestMeta.java`)

```
String glossary = MetaGlossary.write(Vocabulary.library());                ①
```

```
List<Path> written = MetaGlossary.generate(                    ②
        Path.of("src/main/meta"),
        directory.resolve("glossary"));
```

① One package, as Markdown.

② A glossary for each `.nmeta` file under a directory.

It is for people who do not read the notation, so its Rule column states what the structure says, and the types are in plain words. For the library vocabulary, the section on books reads:

Listing 119. The books of the library's glossary (`meta/src/test/resources/library.md`)

```
## Book

A published work, identified by its ISBN.

| Term | Type | Rule | Meaning |
|---|---|---|---|
| isbn | text | required, identifies it | International Standard Book Number, 13 digits. |
| title | text | optional | The title as printed, subtitle excluded. |
| published | date | optional | The date of the first edition. |
| authors | Person | any number | The people credited as authors, in the order printed. |
```

The second is for tools: a dynamic Ecore package, so that every EMF tool works on a vocabulary without generated Java.

Listing 120. An Ecore projection (`meta/⋯/Projections.java`)

```
/**
 * A sketch of the projection to Ecore: classes, attributes and references, enough for EMF to
 * create, validate and serialize instances without generated code.
 */
public static EPackage ecore(Package vocabulary) {
    EcoreFactory ecore = EcoreFactory.eINSTANCE;
    EPackage ePackage = ecore.createEPackage();
    ePackage.setName(vocabulary.getName());
    ePackage.setNsURI(vocabulary.getNamespace());
    ePackage.setNsPrefix(vocabulary.getName());

    Map<Class, EClass> classes = new HashMap<>();
    for (Classifier classifier: vocabulary.getClassifiers()) {
        if (classifier instanceof Class type) {
            EClass eClass = ecore.createEClass();
            eClass.setName(type.getName());
            ePackage.getEClassifiers().add(eClass);
            classes.put(type, eClass);
        }
```

```
    }
    classes.forEach((type, eClass) -> {
        for (Feature feature: type.getFeatures()) {
            if (feature instanceof Attribute attribute
                    && attribute.getType() instanceof PrimitiveTypeReference primitive) {
                EAttribute eAttribute = ecore.createEAttribute();
                eAttribute.setName(attribute.getName());
                eAttribute.setEType(dataType(primitive.getKind()));
                eAttribute.setLowerBound(attribute.getLowerBound());
                eAttribute.setUpperBound(attribute.getUpperBound());
                eAttribute.setID(attribute.isIdentifying());
                eClass.getEStructuralFeatures().add(eAttribute);
            } else if (feature instanceof Reference reference
                    && reference.getType() instanceof ClassifierReference target) {
                EReference eReference = ecore.createEReference();
                eReference.setName(reference.getName());
                eReference.setEType(classes.get((Class) target.getTarget()));
                eReference.setContainment(reference.isContainment());
                eReference.setLowerBound(reference.getLowerBound());
                eReference.setUpperBound(reference.getUpperBound());
                eClass.getEStructuralFeatures().add(eReference);
            }
        }
    });
    new ResourceImpl(URI.createURI(ePackage.getNsURI())).getContents().add(ePackage);
    return ePackage;
}

private static EDataType dataType(PrimitiveKind kind) {
    return switch (kind) {
        case NUMBER -> EcorePackage.Literals.EDOUBLE;
        case BOOLEAN -> EcorePackage.Literals.EBOOLEAN;
        case DATE, DATE_TIME -> EcorePackage.Literals.EDATE;
        default -> EcorePackage.Literals.ESTRING;
    };
}
```

Listing 121. Instances of a projected vocabulary

```
EPackage library = Projections.ecore(Vocabulary.library());
EClass bookClass = (EClass) library.getEClassifier("Book");
EObject book = EcoreUtil.create(bookClass);                    ①
book.eSet(bookClass.getEStructuralFeature("isbn"), "978-0321125217");
book.eSet(bookClass.getEStructuralFeature("title"), "Domain-Driven Design");

ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
resourceSet.getPackageRegistry().put(library.getNsURI(), library); ②
URI uri = URI.createFileURI(directory.resolve("books.json").toString());
Resource resource = resourceSet.createResource(uri);
resource.getContents().add(book);
resource.save(null);
```

① An instance of a class that has no Java class.

② The package is registered with the resource set, so its instances can be saved and loaded in any format.

The Ecore projection is the quiet one, and possibly the most useful: a vocabulary captured in a spreadsheet can be validated, compared, diagrammed and serialized by tools that already exist.

21.8. Sources and targets

The text syntax works today, in both directions, and so do YAML and JSON. The design names the other surfaces through which a vocabulary comes in and goes out PLANNED:

Direction	Formats
Authored in	Excel, for the people who already keep the vocabulary there; Draw.io diagrams, where the diagram is the source; Markdown, where the prose matters most; the Groovy DSL (Chapter 19)
Harvested from	Ecore, Java, XML Schema, JSON Schema, database metadata: most of a domain model is already written down somewhere, and asking people to retype it is how modeling efforts die
Projected to	TypeScript packages with runtime type information (Chapter 23), JSON Schema, for validation and for the tool schemas of agents, Ecore, documentation sites with diagrams, and on demand SQL, Java or Python

The rule that makes many formats affordable is that every format reads into Meta and every output is projected from Meta. Nothing converts from one format to another directly, so a new input format works with every output there is, and a new output with every input.

The incumbent for a shared vocabulary is a spreadsheet and a wiki table, which win because they are free and ask nobody to adopt anything. Meta treats them as inputs, not as competitors.

21.9. AI-assisted domain design

Until those inputs are here, there is another way in for the people who know what the words mean. A librarian knows what makes a loan overdue, can spot a wrong definition of it in a second, and has no reason to learn a grammar. An assistant can write the notation for them: they describe the domain in their own words, or hand over a glossary, a spreadsheet extract or meeting notes, the assistant drafts a `.nmeta` file, and they correct what it says about their business.

An assistant also writes a plausible definition of a term it has never heard as easily as a correct one, and that gets past a reader who does not know the domain. So review only works where the reviewer can judge what is reviewed, and this way of working gives each check to whoever can:

- **The validator judges the structure**, which the subject-matter expert cannot and does not have to: the syntax, that every type exists, that opposites and keys exist, that imports within the project resolve. It says nothing about whether a definition is right.
- **The subject-matter expert judges the meaning**, which neither the assistant nor the validator can. The definitions are prose. The rules are in the structure, often guessed, so the assistant reads them back as questions, the guessed ones first: `refers Member[1] member` asks whether a loan can exist without a member, and `contains Loan[*] loans` whether a loan exists apart from its member. The generated glossary states the same rules for the whole vocabulary, in its Rule column.
- **A developer or an architect** sets the project up once, and reviews the pull request for how the vocabulary fits with others.

The validator is what lets an assistant correct its own drafts before anyone reads them. `MetaValidator` reads the `.nmeta` files of a directory together, and reports problems in the format of compilers:

Listing 122. Validating a directory (`meta/⋯/TestMeta.java`)

```
List<Problem> problems = MetaValidator.validate(TEXT, TEXT);            ①
assertThat(problems).extracting(Problem::toString).containsExactly(     ②
        "lending.nmeta:51:6: warning: Facet type occupancy.Presence "
        + "is not available, its properties are not checked");
assertThat(MetaValidator.hasErrors(problems)).isFalse();
```

① The directory to search, and the directory the reported file names are relative to. `lending.nmeta` imports `library.nmeta`, and no URI map is needed.

② `file:line:column: severity: message`, which a person can read, an assistant can act on, and an editor can show.

What teaches an assistant all this is an agent skill: a directory of instructions it loads when a task calls for them. The skill `nasdanika-meta` is in the runtime's repository, at `models/meta/model/src/main/skill/nasdanika-meta`, and holds the workflow, with the read-back as a step of it, the syntax, tested examples, and the conventions a language reference would not give: a definition for a business reader for every term, a multiplicity that says the business rule, and terms other teams own imported rather than copied. It comes with a starter project, whose whole build is one test that runs the validator and writes the glossary to `target/glossary`, so `mvn -q test` is what the assistant runs on a laptop and what stops a broken vocabulary from being merged. The runtime's tests check what an assistant learns from the skill, and do not test an assistant: what it writes is checked by the validator and by the people who own the words.

The first half of the corresponding chapter of the Zephyr Book *Meta Model* (The Nasdanika Zephyr Books) needs no code, and can be handed to a subject-matter expert: how a session goes, how to read a vocabulary without learning the notation, and what to watch for.

21.10. Instances whose type is a reference

`Object` is the reflective instance of a Meta class: no generated interface, no implementation class, no factory. It knows its type by reference, and answers about itself through a dozen operations: `get`, whole or by range, `set`, `add`, `remove`, `unset`, `size`, `invoke`, its container and containment feature, and its incoming references, filtered, paged and counted. An agent gets an interaction surface that can be taught in a paragraph.

Because the type is a reference, an object can exist before it is classified, and be classified later. That is the shape elicitation has: a thing is named before anyone knows what kind of thing it is, an agent proposes a classification, and a person corrects it. Values without a typed home wait in annotations, and classification promotes what fits into features:

Change	Example	Safety
Specialization	`Person` to `Author`	Safe: every feature stays in scope
Generalization	`Author` to `Person`	Loses a distinction
Reclassification	`Person` to `Publisher`	Arbitrary: names may coincide while meanings do not

Values are never silently dropped: a value whose feature falls out of scope moves to an annotation, with its feature name and the type that declared it. Identity is intrinsic, not derived from an identifying attribute, so it survives retyping, and a marker records who classified an object and when, which separates confirmed facts from unreviewed machine guesses.

The obvious implementation is dynamic EMF behind this interface, through the Ecore projection. In the runtime as of this edition, the operations of `Object` are declared and have no implementation **SKETCH**.

21.11. Federated by design

There is no server and no central repository. A vocabulary is files in Git, or an artifact published to Maven Central or npm, and packages reference each other by namespace URI, as `lending.nmeta` refers to the library. A team owns its vocabulary and refers to another team's without coordinating a release, and gets what a catalog cannot give:

- **Branching.** A proposed change to a term, reviewed as a pull request.
- **History.** The same history the code has, with the same tools.
- **Ownership at the edges.** Nobody administers the whole graph.
- **Immutable snapshots.** What a term meant last year is an install, not an investigation.

Chapter 22. The JVM model

A running Java virtual machine knows a great deal about itself. `ModuleLayer.boot()` gives the module graph, `Configuration` reports what resolution actually produced, and reflection has exposed types and members since Java 1.1. The information was never the problem. Getting an answer meant writing a program, so only questions worth a program were ever asked, and that is a small set.

I have written such programs more than once (Prior art, first-hand). The largest, in 2012, was for a monolith of about fifteen thousand classes, in which two hundred packages formed a single dependency cycle; on our diagrams those packages were red, and we called them the black hole. Its model of the code held about two million objects on a 32-bit JVM. The questions it answered were worth it, because the monolith had to be taken apart. Most questions are smaller than that, and they went unasked.

The JVM model, in `models/jvm`, is a model of what a JVM can name: layers, class loaders, modules, packages, types, methods, fields and parameters. With the JVM as a model, a question costs what it costs to ask it, and the answer can be saved, compared and published like any other model.

22.1. What your assembly became

There is a gap between what a build declares and what runs, and nothing reports it. A dependency tree describes what could be on the path, test scope, optional dependencies and libraries nothing loads included. The resolved module graph is shorter and truer.

Its clearest instance is readability: modules that read something they never required. Usually that is correct and by design, because `requires transitive` propagates readability and service binding pulls providers into the resolved graph that no dependency declaration mentions. It is a property of resolution, not of the build, so it is absent from every POM and every bill of materials, and no dependency tool can report it.

Non-modular code is as visible as modular code. Code on the class path lives in a class loader’s unnamed module, which is a value with packages and types in it, not an absence, so the model is useful for an application that has never heard of the module system.

22.2. Identity

Elements are named the way the JVM names them: binary names and method descriptors, such as `java.util.List`, `[I` and `get(I)Ljava/lang/Object;`.

That is not a local convention. It is the form in the constant pool, in stack traces, in `jdeps` output, in JFR events and in profilers and coverage tools, and OpenTelemetry's code attributes carry its two halves. A build scan, a coverage report, a profile and a span name the same method the same way, and models produced independently merge on URIs with no translation:

Element	URI
Module	`jvm:module/<module name>`
Package	`jvm:package/<binary package name>`
Type	`jvm:type/<binary name>`
Method	`jvm:method/<owner binary name>#<name><descriptor>`
Field	`jvm:field/<owner binary name>#<name>:<descriptor>`
Parameter	`jvm:parameter/<owner method>#<index>`

Joining static structure to runtime observation is then mechanical, not a project. A JaCoCo report already keys classes by binary name and methods by name and descriptor, so coverage attaches to these elements with no mapping layer.

22.3. What it models

Area	Types
Root	`JavaVirtualMachine`: whether it is live, the VM's name, version, vendor and specification version; it contains modules, layers and class loaders
Runtime structure	`Layer`, with its parents and modules; `ClassLoader`, with its parent and unnamed module
Modules	`Module`, with its version, modifiers, main class, `uses` and `reads`, the resolved readability; `Requires`, `Exports` and `Opens` with their targets; `Provides` with the service and its providers

Area	Types
Types	`Package`; `Type`, with its kind, access level, binary name, descriptor, signature, supertype, interfaces and class file version
Members	`Method`, constructors and static initializers included; `Field`, with its constant value; `Parameter`

Three decisions shape it:

- **Modules are contained by the root, not by a layer.** A named module belongs to a layer and an unnamed module to a class loader, and one containment cannot express both, so layers and loaders reference modules.
- **Constructors are methods**, named `<init>`, and static initializers are `<clinit>`, as in bytecode, where the identity scheme comes from.
- **Access flags are booleans**, not a packed integer, so that a query language without bitwise operators can ask about them.

Generic types are carried as data: the class file's `Signature` attribute, verbatim, on types and members. Identity uses erased descriptors, because that is how the JVM tells overloads apart. Annotations, source positions and instructions are out of scope for the first version; annotations are the most likely next addition.

The JVM model extends the core model, not Meta (Chapter 21). Bridge methods, synthetic accessors and overloads that differ only in erased parameter types are realities of the JVM that Meta deliberately does not hold, so the JVM model projects to Meta instead of extending it.

22.4. A snapshot of a running JVM

The runtime's live loader is planned **PLANNED**. A minimal one fits in a page, and shows the shape:

Listing 123. A snapshot of a module layer (`jvm/⋯/Snapshot.java`)

```
public static JavaVirtualMachine of(ModuleLayer layer) {
    JvmFactory factory = JvmFactory.eINSTANCE;
    JavaVirtualMachine vm = factory.createJavaVirtualMachine();
    vm.setLive(true);                                                    ①
```

```
    vm.setVmName(System.getProperty("java.vm.name"));
    vm.setVmVersion(System.getProperty("java.vm.version"));
    vm.setSpecificationVersion(System.getProperty("java.specification.version"));

    Layer boot = factory.createLayer();
    boot.setName("boot");
    boot.setBoot(true);
    vm.getLayers().add(boot);

    Map<String, Module> modules = new TreeMap<>();
    for (java.lang.Module running: layer.modules()) {                    ②
        ModuleDescriptor descriptor = running.getDescriptor();
        Module module = factory.createModule();
        module.setName(running.getName());
        module.setId("jvm:module/" + running.getName());                 ③
        module.setNamed(true);
        descriptor.rawVersion().ifPresent(module::setVersion);
        if (descriptor.isAutomatic()) {
            module.getModifiers().add(ModuleModifier.AUTOMATIC);
        }
        module.getUses().addAll(new TreeSet<>(descriptor.uses()));
        for (ModuleDescriptor.Provides service: descriptor.provides()) {
            Provides provides = factory.createProvides();
            provides.setServiceName(service.service());
            provides.getProviderNames().addAll(service.providers());
            module.getProvides().add(provides);
        }
        modules.put(module.getName(), module);
    }
    vm.getModules().addAll(modules.values());
    boot.getModules().addAll(modules.values());

    Configuration configuration = layer.configuration();
    for (Module module: modules.values()) {
        ModuleDescriptor descriptor = layer.findModule(module.getName())
                .orElseThrow()
                .getDescriptor();
        for (ModuleDescriptor.Requires declared: descriptor.requires()) {
            Requires requires = factory.createRequires();
            requires.setName(declared.name());
            for (ModuleDescriptor.Requires.Modifier modifier: declared.modifiers()) {
                requires.getModifiers().add(RequiresModifier.valueOf(modifier.name()));
            }
            requires.setModule(modules.get(declared.name()));            ④
            module.getRequires().add(requires);
        }
        configuration.findModule(module.getName()).ifPresent(resolved -> {  ⑤
            for (ResolvedModule read: resolved.reads()) {
                Module target = modules.get(read.name());
                if (target != null) {
                    module.getReads().add(target);
                }
            }
        });
```

```
    }
    return vm;
}
```

① Observed in a running VM, not scanned from jars.

② What each module declares, from its descriptor.

③ The JVM's own naming, as the identity of the element.

④ A required module that was not resolved stays unresolved: the declaration is kept, with no target.

⑤ What resolution made of the declarations: the modules each module reads.

A snapshot is a model, so it is saved as evidence of what ran:

Listing 124. Saving a snapshot and asking it a question

```
JavaVirtualMachine vm = Snapshot.of(ModuleLayer.boot());

URI uri = URI.createFileURI(directory.resolve("jvm.yml").toString());
Resource resource = NasdanikaResourceSet.createAndConfigure().createResource(uri);
resource.getContents().add(vm);
resource.save(null);                                                    ①

Map<String, List<String>> implicit = Snapshot.readsWithoutRequires(vm);
assertThat(implicit.get("org.example.jvm")).contains("org.nasdanika.sdk.runtime.models.core");
```

① A YAML file, next to the build or the release it describes.

The question is ordinary code over the model:

Listing 125. Modules that read what they never required (`jvm/⋯/Snapshot.java`)

```
/**
 * Modules that read something they never required: by requires transitive, usually.
 */
public static Map<String, List<String>> readsWithoutRequires(JavaVirtualMachine vm) {
    Map<String, List<String>> answer = new TreeMap<>();
    for (Module module: vm.getModules()) {
        Set<Module> required = module.getRequires().stream()
                .map(Requires::getModule)
                .filter(Objects::nonNull)
                .collect(Collectors.toSet());
        List<String> implicit = module.getReads().stream()
                .filter(read -> !required.contains(read))
                .map(Module::getName)
                .sorted()
                .toList();
```

```
        if (!implicit.isEmpty()) {
            answer.put(module.getName(), implicit);
        }
    }
    return answer;
}
```

For the test's own module layer, the answer includes:

Module	Reads without requiring
`org.nasdanika.sdk.runtime.models.core`	`emfjson.jackson`, `io.opentelemetry.api`, `io.opentelemetry.context`, `org.eclipse.emf.ecore.xmi`
`org.nasdanika.sdk.runtime.common`	`emfjson.jackson`
`org.example.jvm`	`emfjson.jackson`, `io.opentelemetry.api`, `io.opentelemetry.context`, `org.eclipse.emf.common`, `org.eclipse.emf.ecore`, `org.eclipse.emf.ecore.xmi`, `org.nasdanika.sdk.runtime.models.core`

Some of this is `requires transitive` at work: the example requires the JVM model, which requires the core model and EMF transitively. The first row is more interesting. The core model requires only EMF and `java.scripting`, and yet it reads OpenTelemetry and a JSON library. EMF, OpenTelemetry and `emfjson-jackson` are automatic modules, jars without a module descriptor, and the module system gives a module that reads one automatic module readability of all of them. No build file says so, and the snapshot shows it in the first query anyone runs.

22.5. Loading

A live loader comes first, before any scanner of jars, because it is smaller and produces a real graph to ask questions of: it walks the boot layer and its parents, reads each module descriptor, takes readability from the configuration, and reflects over loaded types. Where reflection does not reach, as for a field's constant value, it reads the loaded type's class file through the JDK's class file API, which is final since Java 24 and needs no dependency.

A build-time scanner over class files follows, and adds what a running VM cannot show: types that were never loaded. It has to record which release of a multi-release jar it modeled, and take automatic module names from manifests and file names.

22.6. Applications

- **Ask a running process what it is.** Logs, metrics, traces and profiles describe behavior over time; none describes composition. The JVM model is the missing view.
- **Deployed versus running.** The difference between the dependency tree and the resolved graph, computed rather than argued.
- **Reports that are expensive today.** Which class file versions are present, which modules are automatic, which packages are opened for deep reflection and to whom.
- **A join point for telemetry.** A stack trace, a JFR event and a span resolve to model elements without a mapping, so a slow span resolves to a method, in a module, from an artifact, owned by a team.
- **Coverage that knows what runs.** With a live loader recording what was loaded and coverage attached from builds, two lists appear that a coverage tool alone cannot produce: covered but never loaded, and loaded but not covered.
- **The JVM as files.** A snapshot presented as a volume (Chapter 14) and served over WebDAV (Chapter 15) is a running system that an agent reads with the file tools it already has.



> Documentation of JVM concepts is meant to be generated, and the specifications and the JDK's Javadoc are under licenses that do not permit republishing derived text under the runtime's license. Concepts are not copyrightable and expression is: generated documentation must be original prose, and an assistant describing `requires transitive` must not reproduce Javadoc it remembers.

Chapter 23. From models to the web

A model that the people who own it cannot read is a private notation. The web is where they read, so the runtime treats web pages and web applications as generation targets, and generates them the way it does everything else: from models, into models, and only then into text.

The whole browser side is generated or served from Java: HTML and Bootstrap for pages, TypeScript and React for applications. A Java developer builds them with Java APIs and models, with no npm, no Node.js and no build step, and with little knowledge of how React works. What runs in the browser can be deployed as static files with no JVM behind it, and still be an application, because React provides the dynamic behavior.

23.1. Pages are models too

Generating HTML by concatenating strings is easy to start and hard to keep correct: escaping, nesting, attribute order, the one closing tag that went missing. The runtime's generators build a model of the page and let a generator write the text.

Model	What it covers
`models/html`	HTML elements, attributes, text and comments, with a generator that writes HTML text
`models/html/bootstrap`	Bootstrap 5 semantics: documents with themes, containers, grids, and components for content, navigation, overlays and forms
`models/typescript`	The declaration surface of TypeScript: packages, modules, declarations and type references, with a generator that writes sources and manages imports
`models/react`	React components, hooks, contexts and JSX, on top of the TypeScript model
`models/react/bootstrap`, `models/react/flow`	Bootstrap components in React, and React Flow diagrams with a simple layered layout

The Bootstrap model describes semantics, not markup: a modal has a title, a size and whether it is centered and scrollable. How that becomes elements, classes, ARIA and data

attributes, and ids that Bootstrap's JavaScript connects, is the business of `BootstrapStructure`, used by the HTML generator for static pages and by the React generator for components. One model of a page renders both ways.

23.2. A catalog site

The example generates a small site from the library: an index with a table of books, and a page per book with a breadcrumb and its facts.

Listing 126. Generating the site (`web/⋯/CatalogSite.java`)

```
public static void generate(Library library, Volume volume) throws IOException {
    Volume.Writable site = volume.capability(Volume.Writable.class)
            .orElseThrow(() -> new IOException("Read-only volume"));
    String name = library.getName();
    site.write("index.html", html(page(name, heading(name), table(library)))); ①
    if (volume.stat("books").isEmpty()) {
        site.createDirectory("books");                                          ②
    }
    for (Book book: library.getBooks()) {
        site.write("books/" + slug(book.getName()) + ".html", html(page(
                book.getName(),
                breadcrumb(library, book),
                heading(book.getName()),
                facts(book))));
    }
}

private static Content html(BootstrapDocument document) {
    String html = BootstrapStructure.generate(document);                        ③
    return Content.of(html, StandardCharsets.UTF_8);
}
```

① Each page is a model of a Bootstrap document, turned into content and written to the volume.

② File system semantics: a directory is created before files are written into it.

③ The generator interprets the page model and writes HTML.

A page is a document with a theme, a head and a body:

Listing 127. A page (`web/⋯/CatalogSite.java`)

```
static BootstrapDocument page(String title, Node... contents) {
    BootstrapDocument document = BOOTSTRAP.createBootstrapDocument();
    document.setTheme(Theme.FLATLY);                                            ①
```

```
    document.setLang("en");

    Head head = HTML.createHead();
    head.setTitle(title);
    document.setHead(head);

    Container container = BOOTSTRAP.createContainer();
    container.getClasses().add("my-4");
    container.getContents().addAll(List.of(contents));
    Body body = HTML.createBody();
    body.getContents().add(container);
    document.setBody(body);
    return document;
}
```

① A Bootswatch theme, applied by the generator.

The table is a Bootstrap table with badges, built from the library:

Listing 128. A table of books (web/⋯/CatalogSite.java)

```
static Table table(Library library) {
    Table table = CONTENT.createTable();
    table.setStriped(true);
    table.setHover(true);

    TableHead head = HTML.createTableHead();
    head.getRows().add(row(true, text("Title"), text("Year"), text("Genre"), text("Authors")));
    table.setHead(head);

    TableBody body = HTML.createTableBody();
    for (Book book: library.getBooks()) {
        Anchor link = HTML.createAnchor();
        link.setHref("books/" + slug(book.getName()) + ".html");
        link.getContents().add(text(book.getName()));

        Badge genre = CONTENT.createBadge();
        genre.setBg(book.getGenre() == Genre.TECHNOLOGY ? Variant.PRIMARY : Variant.SECONDARY);
        genre.setContent(book.getGenre().getLiteral());

        String authors = book.getAuthors().stream()
                .map(Author::getName)
                .collect(Collectors.joining(", "));
        body.getRows().add(row(
                false,
                link,
                text(String.valueOf(book.getYear())),
                genre,
                text(authors)));
    }
    table.getBodies().add(body);
    return table;
```

```
}
```

The site is written to any writable volume (Chapter 14):

```
CatalogSite.generate(Samples.library(), Volume.of(directory));
```

A directory today, an in-memory volume for a test, a Git overlay for a publishing pipeline, or a volume served over HTTP for a preview (Chapter 15): the generator does not know and does not need to.

Writing the page as a model has consequences beyond correctness. The page model can be checked before it is written, by a test or an agent; it can carry markers that point back to the model elements each part was generated from (Chapter 20); and it can be transformed, for example by adding a navigation panel to every page of a site after the pages were generated.

23.3. Navigation

Documentation sites of models need navigation that works for people, crawlers and agents alike. The Bootstrap module's `NavigationTree` builds a side panel as nested `details` and `summary` elements, which work without JavaScript, so crawlers and agents read the tree as plain links. Branches on the path to the current page are open, and the current item is marked with `aria-current="page"`. For large models, a local tree renders the branches off the path as links without their children, so a page of a model with thousands of elements stays small.

23.4. Applications without npm

A React application is usually a Node.js project: a package manifest, hundreds of packages in `node_modules`, a bundler and a build. None of it is needed to run React in a browser. Browsers load ES modules natively, an import map in the page resolves bare specifiers such as `react` to URLs, and a CDN such as esm.sh serves npm packages as ES modules. What remains is the application's own code, and the runtime generates that from a model.

The example renders the library's books as a table with a filter:

Listing 129. A component as a model (`ui/⋯/BookTable.java`)

```
/**
 * A table of books with a filter. The module is not a TSX module, so the generator writes
 * createElement calls that a browser runs as they are.
 */
public static Module module() {
    ComponentDeclaration table = REACT.createComponentDeclaration();
    table.setName("BookTable");
    table.setExported(true);
    table.getParameters().add(parameter("props"));
    table.getHooks().add(state("rows", "setRows", "null"));                          ①
    table.getHooks().add(state("query", "setQuery", "\"\""));
    table.getHooks().add(effect(                                                     ②
            "fetch(props.dataUrl).then(response => response.json()).then(setRows);",
            "props.dataUrl"));
    table.getHooks().add(memo("shown",                                               ③
            "(rows || []).filter(row => matches(row, query))",
            "rows", "query"));

    Input search = HTML.createInput();                                               ④
    search.setType("search");
    search.setPlaceholder("Filter by title");
    search.getClasses().addAll(List.of("form-control", "mb-3"));
    JsxNode filter = intrinsic(search,
            attribute("value", "query"),
            event("onChange", "event => setQuery(event.target.value)"));

    JsxNode spinner = element(external("Spinner", "react-bootstrap"),          ⑤
            literal("animation", "border"));
    JsxNode books = children(
            element(external("Table", "react-bootstrap"), attribute("striped", "true")),
            intrinsic(head("Title", "Year", "Authors")),
            children(
                    intrinsic(HTML.createTableBody()),
                    iterator("shown", "row", "row.title",                                ⑥
                            intrinsic(row("row.title", "row.year", "row.authors")))));
    table.setRender(children(tag("div"), filter, conditional("!rows", spinner, books)));

    Module module = TS.createModule();
    module.setName("books");
    module.setPath("modules/books");
    module.setHeader("// Generated from a React model. Do not edit.");
    module.getImports().add(Jsx.namedImport("react-dom/client", "createRoot"));
    module.getDeclarations().add(matches());
    module.getDeclarations().add(table);
    module.getDeclarations().add(mount());
    return module;
}
```

① State: the rows once they are loaded, and the text of the filter.

② An effect that loads the rows from a URL when the component mounts.

③ The rows that match the filter, recomputed only when the rows or the filter change. The test itself is a function of the module, `matches`.

④ An input of the HTML model, used as a JSX intrinsic, with its value and its change handler as JSX attributes.

⑤ React Bootstrap components, referenced by name and module. The generator imports them.

⑥ A row per matching book, keyed by title.

`Jsx` is a small helper of the example: each of its methods creates one object of the React, TypeScript or HTML model and sets what the call passes. The generator writes this module:

Listing 130. The generated module (`modules/books.js`)

```
// Generated from a React model. Do not edit.

import { createRoot } from "react-dom/client";
import { Spinner, Table } from "react-bootstrap";
import { createElement, useState, useEffect, useMemo } from "react";

function matches(row, query) {
  return row.title.toLowerCase().includes(query.toLowerCase());
}

export function BookTable(props) {
  const [rows, setRows] = useState(null);
  const [query, setQuery] = useState("");
  useEffect(() => {
    fetch(props.dataUrl).then(response => response.json()).then(setRows);
  }, [props.dataUrl]);
  const shown = useMemo(() => ((rows || []).filter(row => matches(row, query))), [rows, query]);
  return createElement("div", null,
    createElement("input", {
      className: "form-control mb-3",
      type: "search",
      placeholder: "Filter by title",
      value: query,
      onChange: event => setQuery(event.target.value)
    }),
    !rows
      ? createElement(Spinner, { animation: "border" })
      : createElement(Table, { striped: true },
        createElement("thead", null,
          createElement("tr", null,
            createElement("th", null, "Title"),
            createElement("th", null, "Year"),
            createElement("th", null, "Authors"))),
```

```
        createElement("tbody", null,
          shown.map(row => createElement("tr", { key: row.title },
            createElement("td", null, row.title),
            createElement("td", null, row.year),
            createElement("td", null, row.authors))))));
}

export function mount(element, dataUrl) {
  createRoot(element).render(createElement(BookTable, { dataUrl }));
}
```

The imports are computed from the model: `createElement` and the hooks from `react`, `Spinner` and `Table` from `react-bootstrap`. Only `createRoot`, which is used in opaque code, was declared by hand. A module without type annotations is plain JavaScript, so the browser runs what the generator wrote, as it is. The test of the example compares the generated module with the listing above, so the listing is what the generator writes.

The page is a model too:

Listing 131. The page (`ui/⋯/BookTable.java`)

```
/**
 * The page: Bootstrap's stylesheet, an import map resolving React and React Bootstrap to a
 * CDN, a mount point and a module script. Relative URLs only, so the page works at any
 * address, static or served.
 */
public static Document page() {
    ImportMap importMap = HTML.createImportMap();                                    ①
    importMap.getImports().put("react", ESM + "react@" + REACT_VERSION);
    importMap.getImports().put("react/jsx-runtime",
            ESM + "react@" + REACT_VERSION + "/jsx-runtime");
    importMap.getImports().put("react-dom",
            ESM + "react-dom@" + REACT_VERSION + "?external=react");
    importMap.getImports().put("react-dom/client",
            ESM + "react-dom@" + REACT_VERSION + "/client?external=react");
    importMap.getImports().put("react-bootstrap",
            ESM + "react-bootstrap@2.10.10?external=react,react-dom");

    Head head = HTML.createHead();
    head.setTitle("Books");
    head.getMetas().add(viewport());
    head.getStylesheets().add(stylesheet(BOOTSTRAP_CSS));
    head.setImportMap(importMap);

    Div root = HTML.createDiv();
    root.setId("root");
    Script script = HTML.createScript();                                             ②
    script.setModule(true);
    script.setContent("""
```

```
            import { mount } from "./%s";
            mount(document.getElementById("root"), "%s");
            """.formatted(MODULE, DATA));

    Main main = HTML.createMain();
    main.getClasses().addAll(List.of("container", "py-4"));
    main.getContents().add(heading("Books"));
    main.getContents().add(root);
    Body body = HTML.createBody();
    body.getContents().add(main);
    body.getContents().add(script);

    Document page = HTML.createDocument();
    page.setLang("en");
    page.setHead(head);
    page.setBody(body);
    return page;
}
```

① The import map resolves bare specifiers to the CDN. It is the only place where the CDN is named.

② A module script that mounts the component, with the URL of the data.

The generated page, without the stylesheet link and the import map, which are long lines in the file:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Books</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
  </head>
  <body>
    <main class="container py-4">
      <h1>Books</h1>
      <div id="root"></div>
    </main>
    <script type="module">
import { mount } from "./modules/books.js";
mount(document.getElementById("root"), "data/books.json");
</script>
  </body>
</html>
```

23.5. Static or served

The application is three files, written to any writable volume (Chapter 14):

Listing 132. Writing the application (`ui/⋯/BookTable.java`)

```
/**
 * Writes the application to a writable volume: a directory to copy to a static host, or
 * a volume that the runtime serves. The same bytes either way.
 */
public static void write(Library library, Volume volume) throws IOException {
    Volume.Writable site = volume.capability(Volume.Writable.class)
            .orElseThrow(() -> new IOException("Read-only volume"));
    String html = new Generator().generate(page()) + "\n";
    String javaScript = new ReactGenerator().generate(module());
    site.write(PAGE, Content.of(html, UTF_8));
    createDirectory(volume, site, "modules");
    site.write(MODULE, Content.of(javaScript, UTF_8));
    createDirectory(volume, site, "data");
    site.write(DATA, Content.of(data(library).toString(2), UTF_8));
}
```

Written to a directory, it is a static site:

Listing 133. A static deployment

```
BookTable.write(Samples.library(), Volume.of(directory));                    ①

assertThat(directory.resolve("index.html")).content()
    .contains("<script type=\"importmap\">");
assertThat(directory.resolve("modules/books.js")).content()
    .contains("export function BookTable(props)");
JSONArray books = new JSONArray(Files.readString(directory.resolve("data/books.json")));
assertThat(books.length()).isEqualTo(8);
```

① Any directory: copy it to any static host.

When a browser opens the page, React loads from the CDN, the component fetches `data/books.json`, and filtering happens in the browser. No JVM runs anywhere. ES modules do not load from `file:` URLs, so even on a laptop the files need an HTTP server, such as the JDK's `jwebserver`, or the runtime's own handler:

Listing 134. The same files, served

```
Volume site = Volume.of(directory);
BookTable.write(Samples.library(), site);
```

```
var address = new InetSocketAddress(InetAddress.getLoopbackAddress(), 0);
HttpServer server = HttpServer.create(address, 0);
server.createContext("/", new VolumeHttpHandler(site));                    ①
server.start();
try (HttpClient client = HttpClient.newHttpClient()) {
    URI base = URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/");
    assertThat(contentType(client, base.resolve(BookTable.PAGE))).startsWith("text/html");
    assertThat(contentType(client, base.resolve(BookTable.MODULE)))
        .startsWith("text/javascript");                                    ②
    assertThat(contentType(client, base.resolve(BookTable.DATA)))
        .startsWith("application/json");
} finally {
    server.stop(0);
}
```

① The volume the application was written to, served by the handler of Chapter 15.

② Media types come from the extensions, as browsers require for modules.

The bytes are the same either way, so a preview is the deployment. Served with data computed for each request, the same kind of application shows live data: the runtime's own tests serve a table of the properties of the running JVM that way.

23.6. The TypeScript and React models

The TypeScript model covers the declaration surface: packages, modules, interfaces, classes, functions, type aliases, and the type references between them. Bodies are opaque code blocks, which carry references to the declarations they use. Its generator writes sources to a volume of text (Chapter 14) and owns import management, the hardest part of emitting TypeScript correctly: it walks the type references reachable from each module, resolves each to the module that declares it, computes the module specifier, decides whether an import can be `import type`, and aliases names that would collide. The model is write-only by design; nothing in the runtime parses TypeScript.

The React model adds what the TypeScript model leaves opaque: components, hooks, contexts and JSX. `ReactGenerator` writes JSX into `.tsx` modules compiled with the automatic runtime, or, as in the example, `createElement` calls into plain JavaScript modules that a browser runs without a build step. HTML elements used in JSX are interpreted by the same structure the HTML generator uses, with attributes mapped to React props, so a page model and a component model mean the same markup. The React Bootstrap and React Flow models describe those libraries' components as annotated classes, so using a

component library takes no generator code.

Generating through models, not templates, keeps three things that templates lose:

- **Imports**, computed from references instead of maintained by hand.
- **Provenance**, from a source element through every intermediate model to the generated code.
- **Structure**: generated code that can be diffed, analyzed and reported on as a model, not compared as text.

23.7. Walking through an application

A generated application is checked the way a user checks it: in a browser. The `playwright` module drives one with Playwright for Java, against a server of the `harness` module (Chapter 15), in the server's trace:

- `Browsing.inBrowser(server, session → ⋯)` launches Chromium, an installed Chrome or Edge or Playwright's own, with relative URLs resolving against the server, and runs the session in a `Browser` span.
- `session.step(name, () → ⋯)` runs a step in a span, with screenshots before and after it, or of the failure, recorded as log records in the step with the image files' paths.
- Every request the browser sends to the server carries the step's span, so the server's spans are children of the step that caused them; requests to content delivery networks get no trace context.
- `session.narrate(text)`, or a step's narration, records what a person would say about the step as a log record, in order with the screenshots: the script of a video, the text of a guide.
- Console messages and uncaught page errors are log records too.
- With a video file in the options, the session is recorded as a video, and each step, screenshot and narration line carries its offset into it, so the video and the trace explain each other: in the video, a list changes at some second; the trace says which step was running then and which request the server was handling.

The runtime's React model tests walk through a generated application, a table of the

running JVM's properties: generated from models, served by a harness server, opened and reloaded in Chrome, and recorded with a `Recording` as one model in which each step has its screenshots and the server's spans for the page, the module and the data. A recorded walk-through is a user guide and a demo video in waiting: steps, pictures, and what the server did, regenerated with every build; with the steps narrated, the trace is also the script (Chapter 26). A session needs no server: the Draw.io rendering module runs Draw.io's viewer in one, a page per step, to render diagrams as images and videos (Chapter 25). The Zephyr Book *Walk-throughs with Playwright* covers the module in depth.

23.8. JavaScript as another bytecode

With the TypeScript and React models in place, JavaScript is a generation target: the format a machine executes, written by a tool from a source people work in, as bytecode is written by a compiler. The idea is not new. Scott Hanselman called JavaScript the assembly language of the web in 2011, Google Web Toolkit compiled Java to JavaScript in 2006, and J2CL, TeaVM, Kotlin/JS and Scala.js do it today. The runtime's route is different, generation from models into readable modules rather than compilation of Java code, but the destination is the same: people work on the JVM, and the browser gets JavaScript.

That raises a question of direction, and it deserves a careful answer. On the JVM, with these generators, a team can do most of what it would do with Node.js to build and serve a browser application: generate modules and pages, serve them, test them, and deploy them as static files. Not all of it: bundlers, server-side rendering of React, and the npm tool chain of linters and component test runners run on Node.js, and only some of them run on the JVM, through GraalJS. The other direction works as text, since a Node.js program can write Java source as easily as Java writes JavaScript. What it cannot do is be the JVM: it has no checked generic declarations to match at run time (Chapter 8), and it cannot load the JVM's libraries, EMF, JDBC drivers and the rest of the enterprise ecosystem, into its own process. So the asymmetry is real but narrower than "the JVM can do everything Node.js can": the JVM can generate and host most of a browser application, while Node.js can generate Java but cannot host what Java runs on.



Verify Hanselman's 2011 post, "JavaScript is Assembly Language for the Web", before publishing.

23.9. What comes next

The next chapter assembles user interfaces from views chosen by the class of what they show: the polymorphic UI (Chapter 24). On the roadmap PLANNED:

- **Vendored modules**, copied into the application's volume with an import map pointing at them, for offline and closed networks.
- **Documentation sites for models**: a page per element with its documentation, its neighbors, provenance links and a diagram of its neighborhood, for any model built on the core model.
- **The polymorphic UI in React**: a browser application that renders any model, and live telemetry, from the runtime's HTTP server. Meta's projection to TypeScript gives it runtime type information (Chapter 21).



The module `models/typescript/ecore`, which generates TypeScript packages from EMF packages, is commented out of the build and still uses the previous generation's package `org.nasdanika.models.typescript.ecore` and module `org.nasdanika.common`.

Chapter 24. Polymorphic UI

A model with fifty classes should not need fifty screens. A polymorphic user interface renders each element with the view of its class, the way a method call dispatches on the class of its receiver. The benefit is the same as for methods: the general case is written once, and the specific cases only where they earn their place.

This chapter builds one from pieces the runtime already has: the capability framework finds the views, the class hierarchy of the model orders them, and the HTML and Bootstrap models of Chapter 23 are what they produce.

A user interface, in this book, is wherever a person or an agent exchanges information with a system, even in one direction: a page, a PDF, Markdown or AsciiDoc that an agent reads, a tool it calls. With more than one format, the dispatch has two dimensions, the class of the element and the format asked for, which makes it double dispatch, the job the visitor pattern does for code. This chapter works in one format, HTML.

24.1. Where the idea comes from

The idea is old, and its history is the best argument for it.

JFace. An Eclipse tree or table viewer does not know what it displays. A content provider tells it the children of an element; a label provider tells it the text and the icon. One viewer shows anything for which those two can answer.

EMF.Edit. For every class of a model, EMF generates an item provider that is both the content and the label provider of its instances, along with a property source for the property sheet. Adapter factories find the item provider of an object's class, and a reflective item provider answers for classes without one. That is how a tree editor, generated in minutes, edits any Ecore model.

Naked Objects. Richard Pawson argued in the early 2000s that a user interface should be generated by reflection over the domain object model, and that a domain model from which no decent interface can be generated is probably the problem. I saw it presented at a conference at the time, and the idea stayed with me. It lives on as Apache Causeway, and the reflective interface has been rediscovered roughly once per platform generation since:

Django's admin, JSON Forms, the metadata-driven screens of the large business platforms.

Three things are different here:

- **The model is the source, and code is one of its outputs.** Naked Objects derives the model from the classes; here the classes, the documentation, the schemas and the user interface all come from the model.
- **Views are contributions.** A module adds views by declaring them, and the capability framework finds them, the way EMF.Edit finds item providers through adapter factories, with no registry to maintain.
- **The output is a page model**, rendered to static HTML or to React components that run in a browser with no JVM behind them (Chapter 23).

24.2. Dispatch on the class of an element

Method dispatch looks for the most specific implementation along a class hierarchy. A model's class hierarchy is explicit and reflective, and it has multiple inheritance: a library's `Book` extends the core model's `NamedElement`, which extends `ModelElement`, which mixes in identity, documentation and provenance. The order in which to ask is the class itself, then its superclasses, nearest first:

Listing 135. The lineage of a book

```
assertThat(Util.lineage(LibraryPackage.Literals.BOOK)).containsExactly(
        LibraryPackage.Literals.BOOK,
        CorePackage.Literals.NAMED_ELEMENT,
        CorePackage.Literals.MODEL_ELEMENT,
        CorePackage.Literals.STRING_IDENTITY,
        CorePackage.Literals.REFERRABLE,
        CorePackage.Literals.DOCUMENTED,
        CorePackage.Literals.MARKED);
```

The lineage is breadth-first, and superclasses at the same distance keep their declaration order, so a mix-in declared first is asked first. The HTML model uses the same walk on a small scale: a Bootstrap element finds its tag name in the nearest class, along that order, whose annotation names one.

24.3. Views are capabilities

A view is asked for with a requirement naming exactly one class:

Listing 136. The requirement (ui/⋯/ViewOf.java)

```
/**
 * A requirement for the view of exactly one class. Factories answer it only for their own
 * class, so a resolver can ask from the most specific class to the most general, and the
 * first answer is the most specific view.
 */
public record ViewOf(EClass eClass) {
}
```

A view is a capability factory that answers it for its own class and provides itself:

Listing 137. A view (ui/⋯/View.java)

```
/**
 * The view of the instances of one class, contributed as a capability factory. It answers
 * the requirement for a view of its class, and the capability it provides is itself.
 */
public abstract class View<T extends EObject> implements CapabilityFactory<ViewOf, View<?>> {

    private final EClass eClass;

    protected View(EClass eClass) {
        this.eClass = eClass;
    }

    /**
     * The page model of an element. Views of the elements it shows come from the resolver.
     */
    protected abstract Node render(T element, Views views);

    @SuppressWarnings("unchecked")
    public Node apply(EObject element, Views views) {
        return render((T) element, views);
    }

    @Override
    public boolean canHandle(Object requirement) {
        return requirement instanceof ViewOf viewOf && viewOf.eClass() == eClass; ①
    }

    @Override
    public Flow.Publisher<CapabilityProvider<View<?>>> create(
            ViewOf requirement,
            Loader loader,
            Span span,
            Logger logger,
```

```
            Meter meter) {
        return Flows.of(CapabilityProvider.<View<?>>of(this));                  ②
    }
```

① Only the requirement for this exact class. A view of `NamedElement` does not answer for `Book`; the resolver asks for `NamedElement` itself when `Book` has no view.

② The factory is its own capability.

Asking for exact classes is deliberate. A requirement matching every superclass at once, such as the generic type `Function<? super Book, Node>`, would leave the choice between the matching views to the order in which capabilities are returned, and that order compares erased types, as the editor's note of Chapter 8 explains. With one class per requirement, the first answer along the lineage is the most specific view, by construction.

24.4. The resolver

Listing 138. Polymorphic rendering (`ui/⋯/Views.java`)

```
/**
 * Renders model elements polymorphically: with the view of an element's class, or of its
 * nearest superclass that has one, or with the reflective view of last resort.
 */
public class Views {

    private final CapabilityLoader loader;
    private final Map<EClass, Optional<View<?>>> resolved = new HashMap<>();

    public Views(CapabilityLoader loader) {
        this.loader = loader;
    }

    public Node render(EObject element) {
        for (EClass eClass: Util.lineage(element.eClass())) {                   ①
            Optional<View<?>> view = resolved.computeIfAbsent(                  ②
                    eClass,
                    key -> Optional.ofNullable(loader.<View<?>>loadOne(new ViewOf(key))));
            if (view.isPresent()) {
                return view.get().apply(element, this);                         ③
            }
        }
        return Properties.render(element, this);                                ④
    }

}
```

① The element's class first, then its superclasses, nearest first.

② One resolution per class, remembered: the view of a class does not change while a page is rendered.

③ The first view found renders the element. A view renders what the element shows, books in a library, by calling the resolver again.

④ No view anywhere along the lineage: the reflective view of last resort.

24.5. Views of the library

A library is rendered by its own view, which does not know what a book or an author looks like:

Listing 139. The library (`ui/⋯/LibraryView.java`)

```
/**
 * A library: its name, a card per book, and its authors. What a book or an author looks like
 * is not this view's business: it asks the resolver.
 */
public class LibraryView extends View<Library> {

    public LibraryView() {
        super(LibraryPackage.Literals.LIBRARY);
    }

    @Override
    protected Node render(Library library, Views views) {
        Div page = HTML.createDiv();
        page.getContents().add(heading(1, library.getName()));

        Div books = HTML.createDiv();
        books.getClasses().addAll(List.of("d-grid", "gap-3", "mb-4"));
        for (Book book: library.getBooks()) {
            books.getContents().add(views.render(book));                    ①
        }
        page.getContents().add(books);

        page.getContents().add(heading(2, "Authors"));
        for (Author author: library.getAuthors()) {
            page.getContents().add(views.render(author));                   ②
        }
        return page;
    }
```

① A book's view comes from the resolver.

② So does an author's. The library model has no view of its own for authors, so the view of `NamedElement` renders them.

Listing 140. A book (ui/⋯/BookCard.java)

```
/**
 * A book as a Bootstrap card.
 */
public class BookCard extends View<Book> {

    public BookCard() {
        super(LibraryPackage.Literals.BOOK);
    }

    @Override
    protected Node render(Book book, Views views) {
        CardTitle title = CONTENT.createCardTitle();
        title.setContent(book.getName());
        CardSubtitle subtitle = CONTENT.createCardSubtitle();
        subtitle.setContent(book.getYear() + ", " + book.getGenre().getLiteral().toLowerCase());
        CardText authors = CONTENT.createCardText();
        authors.setContent(book.getAuthors().stream()
                .map(Author::getName)
                .collect(Collectors.joining(", ")));

        CardBody body = CONTENT.createCardBody();
        body.setTitle(title);
        body.setSubtitle(subtitle);
        body.getTexts().add(authors);
        for (Marker marker: book.getMarkers()) {
            body.getContents().add(views.render(marker));                    ①
        }

        Card card = CONTENT.createCard();
        card.getBodies().add(body);
        return card;
    }

}
```

① A marker of the core model has no view at all; the reflective view renders it, inside the card.

The view of `NamedElement` is written once for the core model, and renders every named element of every model built on it, until a module contributes something more specific:

Listing 141. Anything with a name (ui/⋯/NamedElementView.java)

```
/**
 * Anything with a name, in any model built on the core model: the name, and the
 * documentation if there is any. It renders every named element without a view of its own.
 */
public class NamedElementView extends View<NamedElement> {
```

```
    public NamedElementView() {
        super(CorePackage.Literals.NAMED_ELEMENT);
    }

    @Override
    protected Node render(NamedElement element, Views views) {
        Strong name = HTML.createStrong();
        name.getContents().add(text(element.getName()));
        Paragraph paragraph = HTML.createParagraph();
        paragraph.getClasses().add("named-element");
        paragraph.getContents().add(name);
        if (element.getDocumentation() != null) {
            paragraph.getContents().add(text(": " + element.getDocumentation()));
        }
        return paragraph;
    }

}
```

24.6. The reflective view of last resort

The last resort reads what the metamodel says about the element: its class, and the features it has values for. It is the equivalent of EMF.Edit's reflective item provider and property sheet:

Listing 142. The reflective view (ui/⋯/Properties.java)

```
/**
 * A table of the element's class and of the features it has values for.
 */
static Node render(EObject element, Views views) {
    TableBody body = HTML.createTableBody();
    body.getRows().add(row(text("Class"), text(element.eClass().getName())));
    for (EStructuralFeature feature: element.eClass().getEAllStructuralFeatures()) {
        if (element.eIsSet(feature)) {
            body.getRows().add(row(text(feature.getName()), value(element, feature, views)));
        }
    }
    Table table = CONTENT.createTable();
    table.setSize(Size.SM);
    table.getClasses().add("properties");
    table.getBodies().add(body);
    return table;
}

private static Node value(EObject element, EStructuralFeature feature, Views views) {
    Object value = element.eGet(feature);
    List<?> values = value instanceof Collection<?> many ? List.copyOf(many) : List.of(value);
    if (feature instanceof EReference reference && reference.isContainment()) {
```

```
        Div contents = HTML.createDiv();
        for (Object child: values) {
            contents.getContents().add(views.render((EObject) child));       ①
        }
        return contents;
    }
    return text(values.stream().map(Properties::label).collect(Collectors.joining(", ")));
}
```

① Contained objects are rendered polymorphically too, so a reflective view can contain specific ones, and the other way around.

24.7. Contributing views

Views are declared like any capability factory:

Listing 143. The module (`ui/⋯/module-info.java`)

```
import org.example.ui.BookCard;
import org.example.ui.LibraryView;
import org.example.ui.NamedElementView;
import org.nasdanika.sdk.runtime.common.capability.CapabilityFactory;

module org.example.ui {

    requires transitive org.example.library;
    requires transitive org.nasdanika.sdk.runtime.models.html.bootstrap;
    requires transitive org.nasdanika.sdk.runtime.models.react;
    requires transitive org.nasdanika.sdk.runtime.volume;
    requires org.json;

    exports org.example.ui;

    provides CapabilityFactory with ①
        LibraryView,
        BookCard,
        NamedElementView;

}
```

① Declaration is registration (Chapter 2). Another module can contribute a view of `Author`, or a better view of `Book`, without changing this one or the pages that use it.

24.8. Putting it together

Listing 144. Rendering a library polymorphically

```
Library library = Samples.library();
Marker marker = CoreFactory.eINSTANCE.createMarker();
marker.setLocation("acquisitions.csv");
marker.setPosition("42");
book(library, "Domain-Driven Design").getMarkers().add(marker);

Views views = new Views(CapabilityLoader.getInstance());                    ①
String html = BootstrapStructure.generate(views.render(library));

assertThat(html)
    .contains("<h1>A Modeler's Shelf</h1>")                                 ②
    .contains("<h5 class=\"card-title\">Domain-Driven Design</h5>")          ③
    .contains("<p class=\"named-element\"><strong>Eric Evans</strong></p>") ④
    .contains("<th>location</th>")                                          ⑤
    .contains("acquisitions.csv");
```

① A resolver over the capability loader of the module path, which finds the views the module declares.

② The library: its own view.

③ The books: their own view, a Bootstrap card.

④ The authors: the view of `NamedElement`, the nearest class along their lineage that has one.

⑤ The marker: the reflective view.

The book carrying the marker renders as:

```
<div class="card">
  <div class="card-body">
    <h5 class="card-title">Domain-Driven Design</h5>
    <h6 class="card-subtitle mb-2 text-body-secondary">2003, technology</h6>
    <p class="card-text">Eric Evans</p>
    <table class="table table-sm properties">
      <tbody>
        <tr>
          <th>Class</th>
          <td>Marker</td>
        </tr>
        <tr>
          <th>location</th>
          <td>acquisitions.csv</td>
        </tr>
        <tr>
          <th>position</th>
          <td>42</td>
```

```
        </tr>
      </tbody>
    </table>
  </div>
</div>
```

Objects of classes without generated code are rendered too, because the lineage is that of the class, not of a Java type:

Listing 145. A dynamic object

```
EClass note = EcoreFactory.eINSTANCE.createEClass();                    ①
note.setName("Note");
EAttribute text = EcoreFactory.eINSTANCE.createEAttribute();
text.setName("text");
text.setEType(EcorePackage.Literals.ESTRING);
note.getEStructuralFeatures().add(text);
EPackage notes = EcoreFactory.eINSTANCE.createEPackage();
notes.setName("notes");
notes.setNsURI("https://example.org/notes");
notes.getEClassifiers().add(note);

EObject reminder = EcoreUtil.create(note);
reminder.eSet(text, "Return the book by Friday");

String html = BootstrapStructure.generate(
        new Views(CapabilityLoader.getInstance()).render(reminder));
assertThat(html)
    .contains("<td>Note</td>")                                          ②
    .contains("<td>Return the book by Friday</td>");
```

① A class defined at run time, as a vocabulary projected from Meta would be (Chapter 21).

② No view along its lineage, so the reflective view shows it.

24.9. Mix-ins and sections

Because the lineage walks every superclass, a view can be contributed for a mix-in. A view of `Documented` renders the documentation of anything documented that has no nearer view; a view of `Marked` renders provenance.

The resolver above stops at the first view, which suits a card or a row. A page can instead collect a view from every level of the lineage, nearest first: the fields of the class itself, then the documentation section contributed for `Documented`, then the provenance section for `Marked`. Each class and each mix-in contributes what it knows, and a new mix-in brings its

section to every page of every class that uses it. This is multiple inheritance applied to user interfaces, and it is a small change to the resolver: ask for all the views at each level instead of the first one.

24.10. Static pages, and the browser

The pages in this chapter are static HTML, generated once and served by anything. The same views can render inside React components: an `Intrinsic` places an element of the HTML model in a component, and `JsxContent` places JSX inside an HTML element, so the page model of a view becomes part of an application that runs in a browser (Chapter 23).

Every resolution is a capability resolution, so it is traced: the telemetry of a page generation says which view rendered which element.

24.11. What comes next

The pieces of this chapter are available in the runtime; the example assembles them. The roadmap turns them into a product PLANNED:

- **The polymorphic UI in a terminal.** The same dispatch with views that draw in a terminal, so that any model element gets a tree and a property sheet where developers already work.
- **The polymorphic UI in React.** A browser application that renders any model this way: a property sheet for every object, with categories from the annotations Xcore models already carry, and views resolved by class through the capability framework.
- **More formats.** The same dispatch for Markdown and AsciiDoc, which agents read, and later for MCP and query languages. A normative document, such as a standard or a policy, then comes out as a PDF for people and AsciiDoc for agents from one source, so the two cannot disagree.
- **Telemetry and JVM browsing**, with the generic views first and specific views, such as a timeline for spans, where the generic ones are visibly weak.
- **Documentation sites**, where every element of every model gets a page assembled from the views and sections along its lineage, and from its micro-wiki (Chapter 20).

Chapter 25. Diagrams

Most diagrams in an organization are drawn once, looked at a few times, and left to drift away from what they describe. They do not have to be. A diagram drawn in a tool people already use is a structured document: shapes with names, properties, tooltips and links, joined by connections and nested in containers. The runtime treats it as one, in both directions: a diagram is a source that models are read from, and a target that models are drawn into.

I once moved a software delivery process of about a hundred activities from a drawing into a diagram editor backed by a model of what is done, by whom, in which phase and with what (Prior art, first-hand). The drawing and the editor showed the same boxes. Only the editor could say which activities a role performs or what a phase needs, and that difference is what this chapter is about.

This chapter introduces seven modules that do it, and an example that generates a diagram of the library's citations, lays it out, saves it as a Draw.io file, reads it back as a diagram, checks it against the library, and draws it as SVG.

25.1. A continuum, not a choice

Between a picture and a model there is a continuum. At one end, a diagram is a picture: boxes and lines that mean something to the people who drew them. Add names and tooltips, and it is documentation. Add properties, and programs can read it: a property names the system a shape stands for, its owner, the class of model element it is. At the far end, a diagram is the source of a model, and the people who own the domain keep it in a tool they already know, while programs check it, generate from it and keep it in step with what it describes.

Each diagram should sit on the continuum where its structure pays for itself, and move right when it needs to. That is the principle of more structure with less effort applied to pictures (Chapter 2): nobody has to learn a modeling tool to contribute, and nothing has to be redrawn when a diagram starts to matter.

The other direction matters as much. A model that nobody can see is a private notation

(Chapter 23), and a diagram generated from a model is the quickest way to show it, provided that it is laid out well enough to read without anyone moving a box.

25.2. What a good diagram is worth

A diagram is a message, and most of its cost is the reader's time. An icon for a well-known element, a database, a queue, a person, is recognized before its label is read. A layout that keeps related things together and lines from crossing is understood in seconds; the same boxes placed at random take minutes, and some readers give up. Multiplied by every reader and every reading, a good diagram is one of the cheapest efficiencies an enterprise has (Chapter 1).

Many people say diagrams are not worth the trouble. Some of them are the fox in Aesop's fable, who could not reach the grapes and decided they were sour. Good diagrams have been expensive to draw and more expensive to keep current, so dismissing them was cheaper than making them. Generated from models and laid out automatically, they are no longer out of reach.

25.3. Two layers, seven modules

The Draw.io API and its rendering. `drawio` reads, navigates, changes and writes Draw.io documents in plain Java: documents, pages, layers, nodes and connections, properties and placeholders, selectors, links across pages and documents, and styles. It reads and writes diagrams embedded in PNG images, which is how a figure can be its own source: a reader sees a picture, Draw.io opens the same file for editing, and a program loads it. `drawio-playwright` renders pages with Draw.io's own viewer in a browser driven by the `playwright` module, in a session that needs no server (Chapter 23): PNG and SVG images with the page embedded, pages after a program has clicked their shapes to run custom actions, and videos of pages shown in sequence, each rendering in a span. It is a module of its own so that `drawio` stays light: Playwright and a browser come only with it.

The models. They form one family, and most of its value comes from the supertypes:

- **The diagram model**, `models/diagram`, has what every notation has: elements with ids, names, documentation and attributes (Chapter 20); nodes with bounds and ports;

connections with waypoints; layers; nesting; and the elements of other models that a shape depicts, its semantic elements.

- **Layout with ELK**, `models/diagram/elk`, places the nodes and routes the connections of any diagram of the model with the Eclipse Layout Kernel: its layered, tree, force and radial algorithms, with options set in code or as attributes of the elements.
- **The presentation model**, `models/presentation` `SKETCH`, extends the diagram model with decks: a presentation is a diagram of slides, and a slide is a diagram of shapes, text, images, tables and connectors.
- **The Draw.io model**, `models/drawio`, extends both: a document is a presentation, a page is a slide, and its layers, nodes and connections are their diagram namesakes. A `.drawio` file, or a PNG or SVG image that Draw.io exported with the diagram in it, loads into any resource set as a model and saves back.
- **The SVG model**, `models/svg`, is a subset of SVG as EMF, with SVG's own namespace, so that a resource of it is an SVG file. It is the vector target for generated diagrams, the same in HTML, PDF and EPUB.

So a program written against the diagram model reads a Draw.io page without knowing it is one, and ELK lays out a Draw.io page because it lays out diagrams. The example shows both.

25.4. A diagram generated from a model

The example's diagram shows which books of the library cite which: a node per book that cites or is cited, and a connection per citation. It is built as a Draw.io model from the start, because a Draw.io page is a diagram, so nothing has to be converted later:

Listing 146. The citations as a Draw.io document (`diagram/⋯/Citations.java`)

```
/**
 * A Draw.io document with one page of the library's citations.
 */
public static Document diagram(Library library) {
    DrawioFactory factory = DrawioFactory.eINSTANCE;
    Document document = factory.createDocument();
    Page page = factory.createPage();
    page.setId("citations");
    page.setName("Citations");
    page.getAttributes().put("elk.direction", "DOWN");                ①
    document.getChildren().add(page);
```

```
    Layer layer = factory.createLayer();
    page.getChildren().add(layer);

    Map<Book, org.nasdanika.sdk.runtime.models.drawio.Node> nodes = new LinkedHashMap<>();
    for (Book book : library.getBooks()) {
        if (book.getCites().isEmpty() && cited(library, book).isEmpty()) {
            continue;
        }
        var node = factory.createNode();
        node.setId("book-" + library.getBooks().indexOf(book));
        node.setName(book.getName());
        node.setDocumentation(book.getYear() + ", " + authors(book));      ②
        node.getAttributes().put(BOOK, book.getName());                    ③
        node.getSemanticElements().add(book);                              ④
        node.getStyle().put("rounded", "1");
        node.getStyle().put("fillColor", "#dae8fc");
        node.getStyle().put("strokeColor", "#6c8ebf");
        layer.getChildren().add(node);
        nodes.put(book, node);
    }
    for (Book book : nodes.keySet()) {
        for (Book cited : book.getCites()) {
            var connection = factory.createConnection();
            connection.setId(nodes.get(book).getId() + "-cites-" + nodes.get(cited).getId());
            connection.setSource(nodes.get(book));
            connection.setTarget(nodes.get(cited));
            connection.getStyle().put("endArrow", "open");
            layer.getChildren().add(connection);
        }
    }
    return document;
}
```

① A layout option of ELK, as an attribute of the page. Attributes of elements override the options set in code, so a diagram can carry its own layout preferences.

② The documentation of an element is its tooltip in Draw.io.

③ An attribute that names the book. It is written to the file as a property of the shape, so it survives a round trip through Draw.io, where people can see and change it.

④ The book the node depicts, as a semantic element: a reference to an object of another model. It lives in memory and is not written to the Draw.io file; the attribute is how the reference is restored after loading.

The nodes have no bounds and the connections no routes. That is the layout's job:

Listing 147. Laid out and saved (`diagram/⋯/TestCitations.java`)

```
Document document = Citations.diagram(library);
```

```
Page page = document.getPages().getFirst();
new ElkLayout().layout(page);                                          ①

ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
Resource drawio = resourceSet.createResource(uri("citations.drawio"));    ②
drawio.getContents().add(document);
drawio.save(null);
```

① ELK places the nodes, sizes them to fit their names, routes the connections, and writes bounds and waypoints back into the diagram, in an `ELK Layout` span with the numbers of nodes and edges.

② The Draw.io model contributes a resource factory for `.drawio` files to every resource set, so saving a document is saving a resource (Chapter 7).

The file opens in Draw.io as a diagram anyone can edit, with the book of each shape among its properties.

25.5. Read back, and checked

A person rearranges the diagram, adds a citation, or reverses an arrow by mistake. The program reads the file back and checks it against the library. The check is written against the diagram model alone, so it works on a Draw.io page, and would work on any notation that maps to the diagram model:

Listing 148. Mapping nodes to books and checking connections (`diagram/⋯/Citations.java`)

```
/**
 * Maps the nodes of a diagram to the books they depict, by the book attribute, and returns
 * the citations the diagram shows that the library does not have.
 */
public static List<String> check(DiagramElement diagram, Library library) {
    Map<String, Book> books = new LinkedHashMap<>();
    library.getBooks().forEach(book -> books.put(book.getName(), book));
    List<String> problems = new ArrayList<>();
    for (EObject element : contents(diagram)) {
        if (element instanceof Node node && node.getAttributes().containsKey(BOOK)) {
            Book book = books.get(node.getAttributes().get(BOOK));
            if (book == null) {
                problems.add("No book " + node.getAttributes().get(BOOK));
            } else {
                node.getSemanticElements().add(book);                  ①
            }
        }
    }
    for (EObject element : contents(diagram)) {
```

```
        if (element instanceof Connection connection) {
            Book source = book(connection.getSource());
            Book target = book(connection.getTarget());
            if (source != null && target != null && !source.getCites().contains(target)) {
                problems.add(source.getName() + " does not cite " + target.getName()); ②
            }
        }
    }
    return problems;
}
```

① A node becomes a depiction of a book again: the attribute names it, and the book becomes the node's semantic element.

② A connection between two books must be a citation in the library. The check reports what the diagram shows and the model does not have.

Listing 149. Reading the file back as a diagram (`diagram/⋯/TestCitations.java`)

```
Resource loaded = NasdanikaResourceSet.createAndConfigure()
    .getResource(uri("citations.drawio"), true);
DiagramElement diagram = (DiagramElement) loaded.getContents().getFirst(); ①
assertThat(Citations.check(diagram, library)).isEmpty();

Connection connection = all(diagram, Connection.class).getFirst();
Connectable source = connection.getSource();
connection.setSource(connection.getTarget());                              ②
connection.setTarget(source);
assertThat(Citations.check(diagram, library)).containsExactly(
        connection.getSource().getName() + " does not cite " + source.getName()); ③
```

① The loaded Draw.io document, used as a `DiagramElement` of the diagram model.

② What a person might do in Draw.io: reverse an arrow.

③ The check names the citation that does not exist.

The direction can also be turned around. A diagram drawn first, with properties that name classes and features, becomes the source of a model; the Zephyr Book *Diagram Model* does that with the figure of its own model, drawn in Draw.io, mapped to the metamodel it depicts and checked against it.

25.6. Drawn as SVG

The same laid-out page draws as SVG through the SVG model. The drawing is written against the diagram model too, and short, because ELK has done the hard part:

Listing 150. A diagram drawn as SVG (diagram/⋯/SvgDrawing.java)

```
public static Svg draw(DiagramElement diagram) {
    SvgFactory factory = SvgFactory.eINSTANCE;
    Svg svg = factory.createSvg();
    svg.setFontFamily("Noto Sans, Helvetica, Arial, sans-serif");
    svg.setFontSize("12");
    svg.add(arrowhead());
    double width = 0;
    double height = 0;
    for (EObject element : (Iterable<EObject>) diagram::eAllContents) {
        if (element instanceof Node node && !(node instanceof Label)) {          ①
            Bounds b = node.getBounds();
            width = Math.max(width, b.getX() + b.getWidth());
            height = Math.max(height, b.getY() + b.getHeight());
            G g = svg.add(factory.createG());
            if (node.getDocumentation() != null) {
                g.add(factory.createTitle()).setValue(node.getDocumentation());     ②
            }
            Rect rect = g.add(factory.createRect());
            rect.setX(round(MARGIN + b.getX()));
            rect.setY(round(MARGIN + b.getY()));
            rect.setWidth(round(b.getWidth()));
            rect.setHeight(round(b.getHeight()));
            rect.setRx(6);
            rect.setFill("#dae8fc");
            rect.setStroke("#6c8ebf");
            Text text = g.add(factory.createText());
            text.setX(number(MARGIN + b.getX() + b.getWidth() / 2));
            text.setY(number(MARGIN + b.getY() + b.getHeight() / 2 + 4));
            text.setTextAnchor("middle");
            text.addText(node.getName());
        } else if (element instanceof Connection connection) {                   ③
            Polyline line = svg.add(factory.createPolyline());
            line.setPoints(connection.getWaypoints()
                .stream()
                .map(p -> number(MARGIN + p.getX()) + "," + number(MARGIN + p.getY()))
                .collect(Collectors.joining(" ")));
            line.setFill("none");
            line.setStroke("#333333");
            line.setMarkerEnd("url(#arrow)");
        }
    }
    svg.setWidth(number(width + 2 * MARGIN));
    svg.setHeight(number(height + 2 * MARGIN));
    svg.setViewBox("0 0 " + svg.getWidth() + " " + svg.getHeight());
    return svg;
```

```
}
```

① Every node has bounds, from ELK: a rounded rectangle with the name in the middle. A label of a connection is a node too, and is not drawn here.

② The documentation becomes an SVG `title`, which browsers show as a tooltip.

③ The waypoints of a connection, from ELK, become a polyline with an arrowhead.

Listing 151. Saving the drawing (`diagram/⋯/TestCitations.java`)

```
Svg svg = SvgDrawing.draw(page);
Resource image = resourceSet.createResource(uri("citations.svg"));       ①
image.getContents().add(svg);
image.save(null);
```

① The SVG model contributes a resource factory for `.svg` files, so an SVG drawing is saved, and loaded, like any other model.

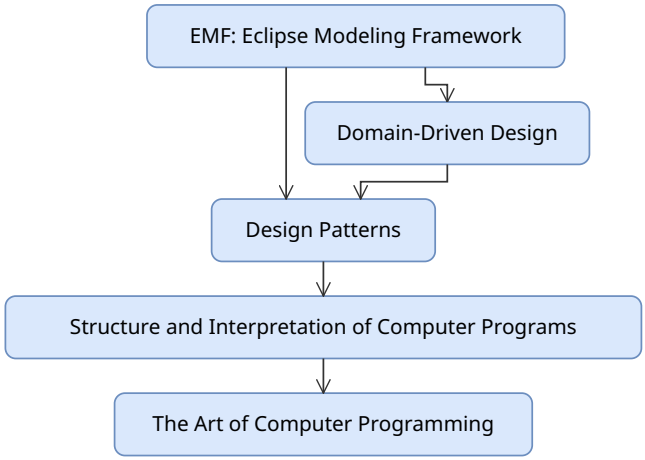


Figure 8. The library's citations, generated, laid out with ELK and drawn as SVG by the example

The figure is a copy of the drawing, kept with the example's test resources. The test loads it back as an SVG model and fails if it no longer shows the books the diagram does, so a figure that no longer matches its example stops the build, like a listing. Opened on its own in a browser, the SVG file shows each book's year and authors as a tooltip.

25.7. The Zephyr Books

Each of these modules is documented in depth in a Zephyr Book (The Nasdanika Zephyr Books):

- **Draw.io**, for `drawio` and `drawio-playwright`: the visual communication continuum, the API, and rendering with Draw.io's viewer.
- **Draw.io Model**, for `models/drawio`: loading, saving and generating Draw.io documents as models, with a diagram of the running JVM's modules generated, laid out with ELK and rendered by Draw.io's viewer.
- **Diagram Model**, for `models/diagram`: diagrams as data, stored as YAML, converted from Draw.io and mapped to the models they depict.
- **Diagram Layout with ELK**, for `models/diagram/elk`: layout options, diagrams of Ecore models, and rendering with React Flow and Draw.io.
- **SVG Model**, for `models/svg`: SVG as a model, and diagrams laid out with ELK drawn as SVG, every shape a link and every documented shape a tooltip.
- **Presentation Model**, for `models/presentation`: decks, slides, masters, shapes, text, tables, connectors and speaker notes as a model on the diagram model, because a slide is a diagram.

25.8. What comes next

On the roadmap PLANNED, as the Zephyr Books describe it:

- **Mapping to models by properties**, as Nasdanika core's semantic mapping does: element properties that name classes and features, so that a diagram loaded into a resource set is a model without code like this chapter's.
- **Merging generated and arranged diagrams**: a diagram generated from a model and arranged by a person, regenerated when the model changes, with the changes merged into the person's layout by id.
- **Custom actions and animations as objects**, so that an animation is generated like a diagram, checked before it renders, and recorded as a video by `drawio-playwright`.
- **Documented diagrams**: a documentation site generated from a diagram, a page per documented element, with the diagram embedded in the viewer and linked to the

pages.

- **Executable diagrams**: behavior bound to elements, wired along connections by the capability framework, with telemetry that records what the diagram did, the road to the executable graphs of Chapter 26.
- **Figures from Draw.io files**: an Asciidoctor extension that renders a `.drawio` file referenced from a book as the book is built.

Models for Visio, PowerPoint and Excel are postponed.

Part VII: Looking Ahead

Where the runtime is going: what comes next, what is designed, and the North Star that orders the work.

Chapter 26. Roadmap and vision

The previous chapters describe what the runtime does in this edition. This one describes where it is going: the North Star that orders the work, the picture that the pieces fit into, and the increments planned next. Plans change as they meet code, and the runtime's site and repository are the current record; this chapter is the plan as of November 2026, distilled from the roadmap described in Section 3.8.

26.1. The North Star

The North Star is agentic work on models: executable graphs whose activities include AI agents, operating on bounded, typed models, with transactional safety and state saved to Git, runnable on a laptop, in a container or in a CI pipeline as the same artifact.

Each part of that sentence is a constraint:

- **Executable graphs.** Work is a graph of activities with transitions between them, drawn as a diagram and executed as it is drawn.
- **Activities include AI agents.** An agent is an activity like any other, with inputs, outputs, tools and a record of what it did.
- **Bounded, typed models.** Agents work on models with declared types and documented terms, not on unstructured text, so what they produce can be checked.
- **Transactional safety.** An agent's changes are a change set, reviewed before it is applied and reversible after.
- **State saved to Git.** History, review and rollback come from the tool every developer already has.
- **The same artifact everywhere.** No server to install and no platform to adopt: what runs on a laptop runs in a pipeline.

Executing a drawing is the oldest idea in this book. My first executable model, in the early 2000s, was XML that described mainframe screens and the flows between them, run by an interpreter; for years after that I built integrations that were drawn and deployed as drawn; and the last time I modeled an integration code base, the model became an engine that executed its processes directly (Prior art, first-hand). What is new is that some

activities are agents, and the transactional, Git-backed state around them.

The minimum credible demonstration is one agentic workflow with Git-backed state and transactional rollback, runnable locally and in CI. Everything in this book is on the way there, and useful before it.

The work is ordered by a rule: an increment is admitted when it states the agentic capability it adds, the demonstration that shows it, what it pulls into the runtime, and that it builds green from a clean clone. Code that no increment needs does not move.

26.2. Three planes

The picture the pieces fit into has three planes, separated by volumes:

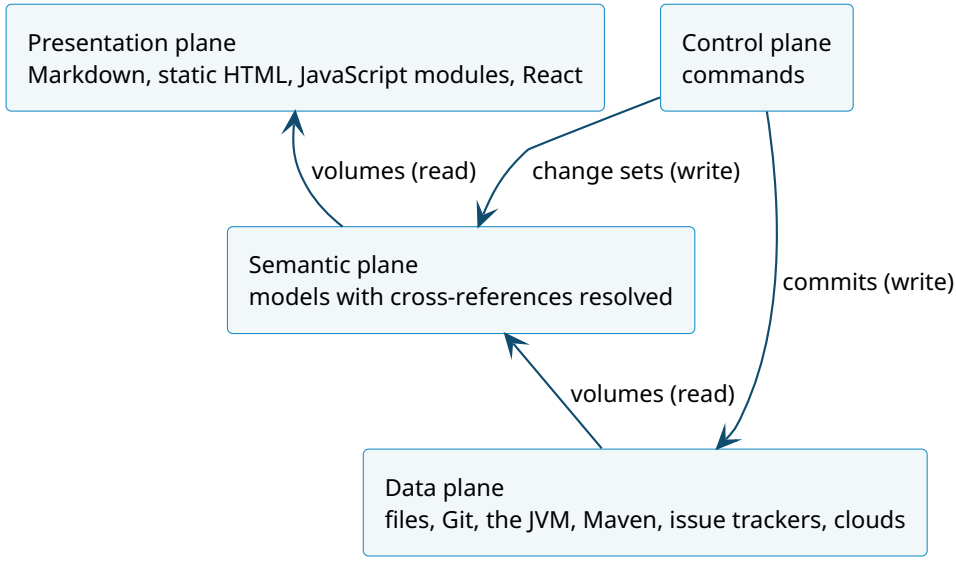


Figure 9. Three planes, separated by volumes

- **The data plane** is where data sits: files, repositories, a running JVM, a Maven repository, an issue tracker, a cloud's resource graph.
- **The semantic plane** is models loaded from the data, with references resolved across sources.
- **The presentation plane** is what people and agents read: Markdown, static HTML, JavaScript modules and React applications generated from models.

Reads flow up through volumes, which pull what is asked for. Writes flow down through commands, the control plane, as recorded change sets and commits.

Layer by layer, a stack built this way has data source volumes that serve one resource each, a mount that composes them under paths (`jvm`, `maven`, `issues`), a resource set that resolves references across the mount, and representations chosen by the type of the source object and the requested view, whose output is again a volume: served over HTTP, mounted over WebDAV, or copied into a zip for static hosting. A model object becomes a folder with a static page, an application page, its JSON and its JavaScript module.

26.3. Next: browsing and serving

PLANNED

- **The polymorphic UI in a terminal.** The same dispatch (Chapter 24), with views for the terminal beside those for HTML, so that the reflective view gives any model element a tree and a property sheet where developers already work. How this divides into a format and a delivery channel is still to be refined: a terminal draws, and the same command piped to a file or read by an agent gets Markdown or text.
- **The polymorphic UI in React.** A browser application over the JavaScript model, which is read-only facades over EMF JSON, with a property sheet per object, categories from the annotations Xcore models already carry, and custom views resolved by capability, with the class of the object as the requirement. It reaches the people a terminal does not.
- **Telemetry and JVM browsing** in the React UI, the generic views first, and custom ones where they are visibly needed: spans want a timeline, not a property sheet.
- **Live telemetry in the browser.** A subscriber that builds the telemetry model from span events as they happen, exposed as a watchable volume, delivered to the browser with server-sent events, so a running command and a recorded one look the same.
- **`http`, `https` and `webdav` commands** over the handlers of Chapter 15, with safe defaults and token authentication.
- **The runtime's site from its models**: pages generated from the models themselves, beside the module pages and the Zephyr Books (The Nasdanika Zephyr Books).
- **Narrated videos from walk-throughs**: browser walk-throughs already record

narration as log records in their steps, beside the screenshots, so the trace is a storyboard, and can record a silent video of the session with each step's offset into it (Chapter 23); planned is a narrated video rendered from the storyboard, each screenshot held for as long as its narration takes to speak, the narration spoken by a text-to-speech service, with subtitles; and a video model between the trace and the rendering, for people to trim and annotate (Chapter 23).

- **Documents from walk-throughs**: user guides in AsciiDoc and Markdown with the narration as their text, slide decks through the presentation model, and a PowerPoint model when it is built, with the narration in the speaker notes, and sequence diagrams through the diagram model and ELK, generated from the same traces (Chapter 23, Chapter 25).

26.4. Next: more volumes

PLANNED

- **A JVM volume**: the JVM model of Chapter 22 served one module per resource, references as proxies resolved on demand.
- **An Azure volume** over the plain REST API, where resource ids are already paths and the resource graph reports changes.
- **A Git volume** that commits without a clone, over the GitHub and GitLab APIs, and an overlay that keeps writes in memory over a read-only base until they are committed.
- **A caching volume**, and a coalescing processor that folds queued changes into the net change per path.
- **Writes through change sets**: a client submits a recorded change set to a command inbox, the command side of a CQRS split, and raw bytes remain plain `PUT` and `DELETE`.

26.5. Next: the command line

PLANNED

- `nsdk install`: extend an installation from Maven coordinates, with profiles for rollback, so new models and commands arrive without rebuilding the distribution.
- **Command lines as models**: a Groovy DSL for long command pipelines, and forms

generated from command definitions, sub-commands as wizards and mix-ins as sections, with an assistant that proposes a command and a person who keeps or undoes it.

- **Nasdanika Commander**: a two-panel file manager in the terminal, over mounted volumes, with the classic function keys and a key bar. View, edit, copy, move and delete across a directory, a running JVM or an organization's own volumes, where WebDAV would be more than is needed. Contributed commands and syntax highlighting arrive through the capability framework.
- **Narrated video as a model**: a video model whose narration and screen events are regenerated as build artifacts. The diagram models it builds on are in this edition (Chapter 25); the presentation model is a sketch, and Visio, PowerPoint and Excel models are postponed.

26.6. The agentic increments

The increments toward the North Star, in their planned order PLANNED:

Capability	Demonstration
Conversations with tool calling, recorded as models	An agent loop against Anthropic and OpenAI models with file tools, view, create, replace, insert, glob and grep, over an in-memory volume: a seeded folder and a task in, a diff and a recorded session out
A sandbox	The agent writes to an overlay over a read-only base; the changes are reviewed as a diff, then committed or discarded
Model choice and portable sessions	A saved session is reloaded and continued with another provider, selected by requirement
Declarative tools	Tools from a tool model, with input schemas generated from Meta, and the same tools over the Model Context Protocol from the runtime's own server
Provenance	Every element an agent produced says which turn and which model produced it, in its markers
Model and graph queries	XPath over models, then a read-only Cypher tool over the graph: the query side

Capability	Demonstration
Reviewable writes	A command tool that takes changes, returns a plan, applies and records them: the command side
Executable agentic graphs	A diagram's transitions as `Flow` publishers and subscribers, one workflow with one agentic activity running until it settles
Git-backed state and rollback	Durable inboxes, one commit per settled step, a CI trigger, and a reply approved in a pull request, then replayed over the diagram from history

The last two rest on one idea: a commit is a message. Transitions of an executable graph are publishers and subscribers, and payloads are models that serialize to JSON. When an activity sends an event, the event is committed; the commit triggers a pipeline that runs `nsdk`, which loads the graph, finds the event and delivers it to the receiving activity. When the run settles, its changes are committed and pushed, which may trigger another run elsewhere. Execution is recorded as commits, so it can be inspected, replayed over the diagram, and reverted.

The reactive file system of Chapter 14 is the runner: processors, deterministic or agentic, react to file changes until nothing changes, and one commit records the run.

26.7. Humans at both ends of the line

Draw software development as a line from intent to operation. AI takes the middle, writing and testing code, and pushes people out to both ends.

To the left is intent: the big picture, the vocabulary, the specification. Meta (Chapter 21) and the core model's documentation are tools for that end.

To the right is assurance: knowing what the system actually did. Telemetry per unit of work saved as models (Chapter 13), snapshots of running systems (Chapter 22), recorded changes in the spans that made them, and live observation (Chapter 12) are tools for that end. The runtime is built for both ends of the line, and for the agents in the middle to be checked from both.

26.8. An architecture method on Git

The same pieces compose into a lightweight architecture method, in the spirit of TOGAF without its ceremony:

- **Git is the architecture repository**, with models in it, reviewed as pull requests.
- **Meta models the business concepts** and the data, the vocabulary everything else uses.
- **Diagrams model processes and application architecture**, as sources, not pictures, and processes may later be executed.
- **Technology stacks are managed as models**, with coordinates for variants by region, tier or audience.
- **Agents are grounded** in the models, code is generated from them, external systems are wired in through volumes, and snapshots are taken with a copy.

It is the answer to the question the first chapter asked: more structure, for less effort than the tools that make structure expensive.

26.9. The model tower

The runtime's models are the lower floors of a tower of small models, each extending the ones below instead of reinventing them: the core model, then Meta, the JVM model, telemetry, the file model and the diagram models. Above them sit floors for work and roles, ownership and governance, accounting, kinds that configure, and occupancy, where elements are situated in coordinates such as time, jurisdiction and tier. Each floor enters the runtime when an increment needs it.

26.10. Following along

The runtime is developed in the open at github.com/Nasdanika-SDK/runtime, with its documentation at runtime.sdk.nasdanika.org. The stories behind the design are published at medium.com/nasdanika, and new capabilities are shown as short videos. This book is built with the runtime and versioned with it: when the runtime changes, the next edition's examples are compiled and tested against it.

Appendix A: Module reference

Every module of the runtime has version 2026.11.0. Import the BOM, `org.nasdanika.sdk.runtime:bom`, and leave versions out of the dependencies (Chapter 6). For each module: its directory in the repository, its Maven coordinates, its Java module name, and what it contains. A module with a Nasdanika Zephyr Book says so; the book is on the module's page and attached to its artifact with the classifier `book` (The Nasdanika Zephyr Books).

A.1. Runtime modules

`common`

Maven `org.nasdanika.sdk.runtime:common`

Module `org.nasdanika.sdk.runtime.common`

`NasdanikaResourceSet` and resource set contributors (Chapter 7); `Types`, `TypeToken`, `Adaptable` and `Adapters` (Chapter 8); `Flows`, `Pump`, `Transformer` and `ReflectiveFactory` (Chapter 9); the capability framework (Chapter 10); `Telemetry`, the publishing OpenTelemetry and the instrumented URI handler (Chapter 11, Chapter 12); `DerivedSubsetEList`, a modifiable view of the elements of one many-valued feature that belong to a derived subset of it, such as the slides among a presentation's children. Its Zephyr Book is *Matching and Wiring*.

`emf-json`

Maven `org.nasdanika.sdk.runtime:emf-json`

Module `org.nasdanika.sdk.runtime.emf.json`

JSON and YAML resources for EMF, selected by extension (Chapter 7).

`volume`

Maven `org.nasdanika.sdk.runtime:volume`

Module `org.nasdanika.sdk.runtime.volume`

The volume SPI, `Content`, `Volume.of(Path)`, `VolumeURIHandler` (Chapter 14), and the HTTP, WebDAV and telemetry handlers (Chapter 15). Its Zephyr Book is *Volumes and Content*.

`harness`

Maven `org.nasdanika.sdk.runtime:harness`

Module `org.nasdanika.sdk.runtime.harness`

`Harness.inServer`: an HTTP server that is up while code runs against it, with the code's requests and the server's work in one trace (Chapter 15). Its Zephyr Book is *Harness*.

`playwright`

Maven `org.nasdanika.sdk.runtime:playwright`

Module `org.nasdanika.sdk.runtime.playwright`

`Browsing.inBrowser`: browser sessions with Playwright against a harness server, steps as spans and screenshots as log records in them, and an optional video of the session with each step's offset into it (Chapter 23). Its Zephyr Book is *Walk-throughs with Playwright*.

`drawio`

Maven `org.nasdanika.sdk.runtime:drawio`

Module `org.nasdanika.sdk.runtime.drawio`

Draw.io documents in Java: pages, layers, nodes and connections, properties and placeholders, selectors, links across pages and documents, styles, and diagrams embedded in PNG images (Chapter 25). Its Zephyr Book is *Draw.io*.

`drawio-playwright`

Maven `org.nasdanika.sdk.runtime:drawio-playwright`

Module `org.nasdanika.sdk.runtime.drawio.playwright`

`Rendering` and `Viewer`: Draw.io pages rendered with Draw.io's own viewer in a browser driven by Playwright, as PNG and SVG images with the page embedded, after a program has clicked their shapes, and as videos of pages shown in sequence (Chapter 25). A module of its own so that `drawio` stays light; it is documented in the *Draw.io* Zephyr Book.

`groovy`

Maven `org.nasdanika.sdk.runtime:groovy`

Module `org.nasdanika.sdk.runtime.groovy`

The Groovy DSL for any metamodel, its Ecore binding and the Groovy resource factory (Chapter 19). Its Zephyr Book is *Groovy DSL*.

`picocli`

Maven `org.nasdanika.sdk.runtime:picocli`

Module `org.nasdanika.sdk.runtime.picocli`

Picocli, bundled until a picocli release includes the runtime's contributions (Chapter 16).

`cli`

Maven `org.nasdanika.sdk.runtime:cli`

Module `org.nasdanika.sdk.runtime.cli`

`CommandBase`, `CommandGroup`, the command and mix-in factories, `@Parent`, `@SubCommand`, `ExecutionSupplier` and `WiringExecutionStrategy`; the root, `save`, `ecore`, `shell` and `launcher` commands; resource set and properties mix-ins (Chapter 16, Chapter 17). Its Zephyr Book is *Declarative Command Lines*.

`cli-launcher`

Maven `org.nasdanika.sdk.runtime:cli-launcher`

Module `org.nasdanika.sdk.runtime.cli.launcher`

`Launcher`, which runs a launch as a unit of work with telemetry saved as models, and `DistributionBuilder`; builds the command line distribution (Chapter 18).

`bootstrap`

Maven `org.nasdanika.sdk.runtime:bootstrap`

Not a named module: it is run with `java -jar`.

The dependency-free bootstrap that relaunches a distribution from any directory (Chapter 18).

`bom`, `all`

Maven `org.nasdanika.sdk.runtime:bom` and `org.nasdanika.sdk.runtime:all`, both POMs

The bill of materials of the runtime and its dependencies, and an aggregate of the runtime's modules and models for assemblies, which the launcher's distributions are built from (Chapter 18). The models have a bill of materials and an aggregate of their own, `org.nasdanika.sdk.runtime.models:bom` and `org.nasdanika.sdk.runtime.models:all`, which these two import and include.

A.2. Models

The models follow the identity rule of Chapter 2: the nsURI is `runtime.sdk.nasdanika.org/` followed by the model's path.

`models/core`

Maven `org.nasdanika.sdk.runtime.models:core`
Module `org.nasdanika.sdk.runtime.models.core`
Packages `models/core` and `models/core/kind`. Identity, documentation, provenance, attributes and evaluators (Chapter 20).

`models/telemetry/model`

Maven `org.nasdanika.sdk.runtime.models.telemetry:model`
Module `org.nasdanika.sdk.runtime.models.telemetry`
Packages `models/telemetry`, `models/telemetry/traces`, `models/telemetry/logs` and `models/telemetry/metrics`, with change recording (Chapter 13). It is documented in the *Telemetry Model* Zephyr Book, with its exporters.

`models/telemetry/exporters`

Maven `org.nasdanika.sdk.runtime.models.telemetry:exporters`
Module `org.nasdanika.sdk.runtime.models.telemetry.exporters`
OpenTelemetry SDK exporters that write spans, log records and metrics into telemetry models, and `Recording`, an SDK that records a unit of work to them (Chapter 13). Its Zephyr Book is *Telemetry Model*.

`models/file/model`

Maven `org.nasdanika.sdk.runtime.models.file:model`
Module `org.nasdanika.sdk.runtime.models.file`
Package `models/file`. Files and directories as a model over volumes **SKETCH** (Chapter 14).

`models/meta/model`

Maven `org.nasdanika.sdk.runtime.models.meta:model`
Module `org.nasdanika.sdk.runtime.models.meta`
Package `models/meta`. The language-neutral type system, its text syntax, validator and glossary, and the agent skill `nasdanika-meta` (Chapter 21). Its Zephyr Book is *Meta Model*.

`models/jvm/model`

Maven `org.nasdanika.sdk.runtime.models.jvm:model`

Module `org.nasdanika.sdk.runtime.models.jvm`

Package `models/jvm`. What a JVM can name (Chapter 22). Its Zephyr Book is *JVM Model*.

`models/html/model`

Maven `org.nasdanika.sdk.runtime.models.html:model`

Module `org.nasdanika.sdk.runtime.models.html`

Package `models/html`, with the HTML generator (Chapter 23). It is documented in the *HTML and Bootstrap Models* Zephyr Book.

`models/html/bootstrap`

Maven `org.nasdanika.sdk.runtime.models.html:bootstrap`

Module `org.nasdanika.sdk.runtime.models.html.bootstrap`

Packages `models/html/bootstrap`, `⋯/forms` and `⋯/components/content`, `navigation` and `overlays`, with `BootstrapStructure` and `NavigationTree` (Chapter 23). Its Zephyr Book is *HTML and Bootstrap Models*.

`models/typescript/model`

Maven `org.nasdanika.sdk.runtime.models.typescript:model`

Module `org.nasdanika.sdk.runtime.models.typescript`

Package `models/typescript`, with the generator and its import management (Chapter 23). It is documented in the *TypeScript and React Models* Zephyr Book.

`models/react/model`, `models/react/bootstrap`, `models/react/flow`

Maven `org.nasdanika.sdk.runtime.models.react:model`, `:bootstrap` and `:flow`

Modules `org.nasdanika.sdk.runtime.models.react`

`org.nasdanika.sdk.runtime.models.react.bootstrap`

`org.nasdanika.sdk.runtime.models.react.flow`

React components, hooks and JSX with `ReactGenerator`; Bootstrap in React; React Flow with a layered layout (Chapter 23). Their Zephyr Book is *TypeScript and React Models*, in `models/react/model`.

`models/diagram/model`

Maven `org.nasdanika.sdk.runtime.models.diagram:model`

Module `org.nasdanika.sdk.runtime.models.diagram`

Package `models/diagram`. What diagram notations have in common: diagrams, nodes with bounds and ports, connections with waypoints, layers, nesting, attributes, and the elements of other models a shape depicts (Chapter 25). Its Zephyr Book is *Diagram Model*.

`models/diagram/elk`

Maven `org.nasdanika.sdk.runtime.models.diagram:elk`

Module `org.nasdanika.sdk.runtime.models.diagram.elk`

Layout of the diagram model with the Eclipse Layout Kernel: layered, tree, force and radial algorithms, in spans (Chapter 25). Its Zephyr Book is *Diagram Layout with ELK*.

`models/presentation/model`

Maven `org.nasdanika.sdk.runtime.models.presentation:model`

Module `org.nasdanika.sdk.runtime.models.presentation`

Package `models/presentation`. Decks and slides on the diagram model: a presentation is a diagram of slides, and a slide is a diagram of shapes, text, images, tables and connectors **SKETCH** (Chapter 25). Its Zephyr Book is *Presentation Model*.

`models/drawio/model`

Maven `org.nasdanika.sdk.runtime.models.drawio:model`

Module `org.nasdanika.sdk.runtime.models.drawio`

Package `models/drawio`. Draw.io documents as a model on the presentation and diagram models, loaded from and saved to `.drawio` files and the PNG and SVG images Draw.io exports, through a resource factory contributed to every resource set (Chapter 25). Its Zephyr Book is *Draw.io Model*.

`models/svg/model`

Maven `org.nasdanika.sdk.runtime.models.svg:model`

Module `org.nasdanika.sdk.runtime.models.svg`

A subset of SVG as a model, with SVG's own namespace URI, `www.w3.org/2000/svg`, so that a resource of it is an SVG file: the elements and presentation attributes generated diagrams need, written and read by EMF (Chapter 25). Its Zephyr Book is *SVG Model*.

A.3. Book examples

The examples of this book are modules too, in `doc/book/examples`, with the group `org.nasdanika.sdk.runtime.doc.book.examples` and Java modules named `org.example.<example>`. They are built and tested with the runtime and never deployed. Chapter 6 lists them by chapter.

Appendix B: Telemetry reference

The runtime's own spans and log records use the instrumentation scope `org.nasdanika.sdk.runtime`. Attribute keys and event names are constants of `Telemetry`, so instrumentation and the code that reads telemetry share them. Chapter 11 describes the conventions; this appendix lists the names.

B.1. Spans

Span	Started by
`Resource.load`, `Resource.save`, `Resource.unload`	Resources of a configured resource set
`URIHandler.read`, `URIHandler.write`, `URIHandler.delete`	Instrumented URI handlers, from opening a stream to closing it
`ResourceSetContributor.configure`, `Contribute To Resource Set`	Configuring a resource set from services
`Transformer.transform`, `Transformer.create`	Transformations, and each mapping of a source
`PumpedPublisher.onNext`	Delivery of an item to a subscriber in another span's context
`CapabilityLoader.loadAll`, `CapabilityLoader.loadOne`	The blocking conveniences of the capability loader
`CapabilityFactory.create`, `CapabilityFactory.method`, `Create Capability`	A factory handling a requirement, a capability method invoked, a capability created
`Content.writeTo`, `Content.pipe`, `Content.read`	Content transfers of the volume module
`GET` (client)	Content at an HTTP URI
`<method> <route>` (server)	Requests through `TelemetryHandler`
The name of a command	`CommandBase`, for each executed command
`Executing the root command`	`Launcher`, the root span of a launch
`Compiling script`, `Evaluating script`, `Saving script`	Groovy DSL resources

Span	Started by
`Loading <Type>`, `Setting <feature>`, `Resolving references`, `Resolving <feature>`, `Loading resource`	Evaluation of a DSL script

B.2. Attributes

Key	Type	Meaning
`nasdanika.resource.uri`	string	The URI of a resource or a stream
`nasdanika.resource.type`	string	The class of a resource
`nasdanika.resource.contents`	long	The number of root objects loaded or saved
`nasdanika.io.bytes`	long	Bytes read or written, so far or in total
`nasdanika.uri.handler`	string	The URI handler that served a stream
`nasdanika.content`	string	The description of a content
`nasdanika.content.cancelled`	boolean	The reader of piped content closed it before the writer finished
`nasdanika.contribution.kind`	string	What was contributed to a resource set: `epackage`, `uri-handler`, `adapter-factory` or `contributor`
`nasdanika.contribution`	string	The contribution: an nsURI or a class name
`nasdanika.contribution.count`	long	The number of contributions
`nasdanika.transformer.source`	string	The source of a mapping
`nasdanika.transformer.sources`, `nasdanika.transformer.targets`	long	The numbers of sources and targets of a transformation
`nasdanika.capability.requirement`	string	The requirement being resolved
`nasdanika.capability.factory`	string	The factory handling it
`nasdanika.capability.providers`	long	The number of providers produced
`nasdanika.capability.method`	string	The factory method that produced capabilities
`nasdanika.flow.description`	string	The description of a flow stage

Key	Type	Meaning
`nasdanika.progress.worked`, `nasdanika.progress.total`	long	Progress, on progress log records
`qualifiedName`, `commandClass`, `commandModule`, `commandModuleVersion`	string	The executed command, on command spans
`nasdanika.dsl.type`	string	The class an element of a script was created as
`nasdanika.dsl.type.superseded`	string array	The less specific classes that the class superseded
`nasdanika.dsl.feature`, `nasdanika.dsl.feature.class`, `nasdanika.dsl.feature.kind`	string	The feature being set, its class, and `attribute`, `containment` or `reference`
`nasdanika.dsl.deferred`	boolean	A reference resolved after the script ran
`nasdanika.dsl.references`	long	The number of deferred references
`nasdanika.dsl.prototype.uri`, `nasdanika.dsl.target.uri`	string	The prototype of a `copy`, the target of a `loadResource`
`nasdanika.dsl.global.uri`	string	The URI a script registered a global object under
`code.line.number`	long	The line of a script

HTTP spans carry the semantic convention attributes `http.request.method`, `http.route`, `http.response.status_code`, `http.response.body.size`, `url.full` (client, without user info and query), `url.path`, `url.scheme`, `server.address`, `server.port`, `client.address`, `network.protocol.version` and `user_agent.original`.

B.3. Events

Log records that are events have an event name:

Event	Emitted when
`nasdanika.progress`	Work progresses, with the worked and total amounts; at most once a second for transfers
`nasdanika.resource_set.contribution`	Something is contributed to a resource set

Event	Emitted when
`nasdanika.resource.created`, `nasdanika.resource.loaded`, `nasdanika.resource.saved`	A resource is created, loaded or saved
`nasdanika.dsl.global`	A DSL script registers a global object

B.4. Configuration

Setting	Effect
`nsdk.home`, `NSDK_HOME`	The home directory of a distribution, where telemetry is saved
`nsdk.telemetry.format`, `NSDK_TELEMETRY_FORMAT`	The format of saved telemetry: `yml` (the default), `json`, `xml`, `binary` or `compressed`
`TRACEPARENT`, `TRACESTATE`, `BAGGAGE`	A trace to continue: a launch becomes part of the trace of whatever started it
`org.nasdanika.sdk.runtime.cli.maxCommandPath`, `org.nasdanika.sdk.runtime.cli.maxCommands`	Limits of the command tree: 15 and 2,000 by default
`BOOTSTRAP_HOME`, `BOOTSTRAP_JAVA`, `BOOTSTRAP_JAVA_OPTIONS`	The distribution root, the Java executable and extra JVM options of the bootstrap

Appendix C: Rules of thumb for people and assistants

This appendix collects the rules the chapters explain, as short imperatives. It is written to be pasted into the instructions of a coding assistant working on a Nasdanika project, and to be read by the people who review its work. Each rule names the chapter that explains it.

C.1. Foundations

1. Depend on the JDK, EMF core and the OpenTelemetry API, and nothing else, in code meant to be reused. Anything that needs more goes into its own module (Chapter 2).
2. Use the JDK where it can do the job: `java.util.concurrent.Flow`, not Reactor; the JDK HTTP client and server; virtual threads; the service loader (Chapter 2).
3. Derive identities from the module's path: the Java module `org.nasdanika.sdk.runtime.<path>`, dropping a trailing `model`, and the nsURI `https://runtime.sdk.nasdanika.org/<path>` (Chapter 2).
4. Register by declaring: `provides` in `module-info.java`, never registration code (Chapter 2).
5. Register by type with Java services, such as EPackages and resource set contributors. Use capabilities only for lookups that have a requirement object (Chapter 2, Chapter 10).
6. Say what is stuck. A failure names what it needed, what it found and what was waiting for what (Chapter 2).

C.2. Models

1. Write metamodels in Xcore. Give the GenModel's `modelDirectory` the Maven module's directory as its first segment (Chapter 7).
2. Contribute each package through an EPackage provider in the module's `services` package, declared in `module-info.java` (Chapter 7).
3. Create resource sets with `NasdanikaResourceSet.createAndConfigure()`, so that every contributed package, factory and handler is there and every load and save is traced (Chapter 7).

4. Choose the format with the extension: `xmi` to interchange, `yml` and `json` to read and review, `ebin` and `egz` for size (Chapter 7).
5. Extend the core model's `NamedElement`, or `ModelElement` for elements without names, and get identity, documentation and provenance with it (Chapter 20).
6. Prefer `getReferrers` to an opposite reference when the inverse is not part of the model's meaning (Chapter 20).
7. Fill markers. A loader, an importer and an agent record where an element came from (Chapter 20).
8. Keep a vocabulary in Meta when it has to be projected to more than one language (Chapter 21).
9. Give a model a module of its own, a micro-model, only for a dependency the others do not want, a different set of consumers or a different maturity; otherwise add a nano-model, an `.xcore` file, to the module it changes with (Chapter 2).
10. Keep the core concepts of a model on one diagram that a newcomer understands in a quarter of an hour. A model that needs two diagrams is two models (Chapter 2).
11. Anchor long documentation to the element it describes, a page per element with `docRef`: a micro-wiki (Chapter 20).

C.3. Types, flows and capabilities

1. Ask with wildcards, `? extends`, at every level that is only read from (Chapter 8, Chapter 17).
2. Use ground types in requirements: classes, or parameterized types whose arguments are classes or wildcards (Chapter 10).
3. Remember that objects carry no type arguments: what matches is always what was declared, a return type or a generic supertype (Chapter 8).
4. Drive asynchronous work that must be ordered and settled with a `Pump` (Chapter 9).
5. Emit only complete results from a capability factory: attach what a result needs before emitting it, so that a blocking load returns it whole (Chapter 10, Chapter 16).
6. Do not return null from a `first` mapping to skip a source; distinguish sources by type, or use `all` (Chapter 9).
7. Explain a resolution with a listener before guessing why a capability is missing

(Chapter 10).

C.4. Telemetry

1. Create one OpenTelemetry instance per unit of work and make it current. Never use `GlobalOpenTelemetry` (Chapter 11).
2. Depend on the OpenTelemetry API in instrumented code; only the assembly that runs a unit of work depends on the SDK (Chapter 11, Chapter 13).
3. Make an operation a span named `Type.operation`, and emit log records with the Logs API inside it. Do not use span events (Chapter 11).
4. Put locations and identities in attributes: URIs, lines, names of types. Never put loaded values or credentials there (Chapter 11, Chapter 19).
5. Report progress with progress log records, at most once a second (Chapter 11).
6. Carry the context across threads; the pump does it for its subscribers (Chapter 11).
7. Record changes to models with `Changes.record` inside the span that makes them, so the change lands in the span (Chapter 13).
8. Keep the telemetry of units of work worth keeping, as models, next to what they describe (Chapter 13).

C.5. Files and serving

1. Use string paths, slash-separated and relative to the root, and keep URIs at the edges (Chapter 14).
2. Report hashes in entries when they are cheap to know (Chapter 14).
3. Push content with `writeTo` when you can; pull with `openStream` only when an input stream is required (Chapter 14).
4. Produce generated content with `Content.ofWriter`, so it is written when read, into the reader's stream (Chapter 14).
5. Ask a volume for capabilities instead of assuming them (Chapter 14).
6. Keep model URIs independent of where the bytes are: map them to volumes with `VolumeURIHandler` (Chapter 14).
7. Bind servers to the loopback address unless authentication is in front of them (Chapter

15).

C.6. Commands

1. Extend `CommandBase` and implement `execute(span, logger, meter)`; write output to `spec.commandLine().getOut()` (Chapter 16).
2. Contribute a command with a factory and `provides CapabilityFactory`, and open the command's package (Chapter 16).
3. Declare what a command needs from its parent with `@Parent` and a generic type, and what it accepts with `@SubCommand` (Chapter 17).
4. Ask for an `ExecutionSupplier` for anything expensive, so the work happens in the command's span and only if it runs (Chapter 17).
5. Offer with `@Adapter` what any child may use; hand down with plain methods what only children written for the parent should use (Chapter 17).
6. Never let help fail. Wiring is skipped for help; keep constructors and option defaults free of work (Chapter 17).

C.7. User interfaces

1. Build pages and components as models of HTML, Bootstrap and React, and let generators write the text (Chapter 23).
2. Deliver applications as static files where you can, and serve the same bytes for previews and live data (Chapter 23).
3. Resolve the view of an element by its class, nearest first along its lineage, with a reflective view of last resort (Chapter 24).
4. Ask for the view of an exact class at each level; do not leave the choice between views to the order of capabilities (Chapter 24).
5. Write the general view once, and specific views only where they earn their place (Chapter 24).

C.8. The Groovy DSL

1. Set a negative number with `upperBound = -1` or `upperBound(-1)`, never `upperBound -1` (Chapter 19).
2. Reference forward with paths; they are resolved after the script has run (Chapter 19).
3. Prefix a path with `./` when a local name clashes with a class name (Chapter 19).

C.9. Working with an assistant

1. Record the binding of decisions: what was decided, by whom or by what, in the course of what work, and on what evidence, at design time and at run time (Chapter 3).
2. Ask whether a task should exist before asking an assistant to do it faster (Chapter 4).
3. Keep ownership with the people who will live with the result: they choose the language and the stack, and they review what an assistant binds (Chapter 4).
4. Measure a system by what it does, not by its lines of code, services or boxes on a diagram (Chapter 4).
5. Give each check to whoever can judge it. When an assistant drafts a vocabulary, the validator checks the structure and the people who own the words check the meaning; a draft that validates has not been reviewed (Section 21.9).

C.10. For assistants

1. Ground answers in models and their documentation, and cite elements by their URIs.
2. Check an API against the module reference (Appendix A) and the listings of this book and of the Zephyr Books, which compile and are tested, before using it. Do not invent methods.
3. Go to a module's Nasdanika Zephyr Book for what this book only outlines, and to the module's test sources for the complete code behind a listing (The Nasdanika Zephyr Books).
4. Respect the status labels. Present **SKETCH** and **PLANNED** features as designs, never as available.
5. Propose changes as recorded change sets or diffs that a person reviews; do not apply them silently.

6. Say which decisions you bound and why, so that the people who own the result can review them (Chapter 3).
7. Write original prose when documenting the JDK and other licensed specifications; never reproduce their text (Chapter 22).
8. Prefer the surfaces the runtime gives you, commands and files, to new integrations.
9. When you learn something about a model element, write it into the element's documentation page, its micro-wiki stub, so the next person or assistant finds it there (Chapter 20).
10. Spend effort at the right altitude: change the model or the generator rather than the code it generates (Chapter 3).
11. When you draft a vocabulary, read its rules back to the people who own the words as plain statements, and say which you guessed (Section 21.9).

Appendix D: Glossary

Adaptable

An object that can be asked for a view of itself as another type, through `adapt(type)`. Its `@Adapter` methods declare, with their generic return types, what it can be adapted to (Chapter 8).

Adapter method

A method annotated with `@Adapter` whose return type declares a type the object can be adapted to. Command groups offer their sub-commands values this way (Chapter 17).

Agent skill

A directory of instructions that an AI assistant loads when a task calls for them. Meta's, `nasdanika-meta`, teaches an assistant to write and validate vocabularies (Section 21.9).

Assembly

A set of modules on a module path, run together: a test, a command line distribution, an application. What an assembly became at run time is what the JVM model snapshots (Chapter 22).

Attributes

String keys and values on an `Attributed` element of the core model, for what a source format carries and a model does not name, such as the properties of a Draw.io shape (Chapter 20).

Automatic module

A jar without a module descriptor on the module path. Its name comes from its manifest or file name, it reads every module, and a module that reads one automatic module reads all of them (Chapter 22).

Bootstrap

The dependency-free jar that starts a distribution from any directory, by rewriting the distribution's options to absolute paths and relaunching Java (Chapter 18).

Capability

Something a factory provides in answer to a requirement: an object, a function, a service (Chapter 10).

Capability factory

A class that answers requirements, declared as a service. A reflective factory answers with methods annotated `@Capability`, matched by their declared return types (Chapter 10).

Capability loader

The object that resolves a requirement against a source of factories, on a pump, and returns a publisher of providers (Chapter 10).

Change description

A record of changes to models, stored forward and detached from the models, contained by the span in which the changes were recorded (Chapter 13).

Command group

A command that does nothing itself and gives its sub-commands something to work on (Chapter 16).

Content

The state of a file in a volume: a re-readable source of bytes that does no I/O until it is opened (Chapter 14).

Core model

The model of identity, documentation, provenance and evaluators that the other models extend (Chapter 20).

Decision binding

Fixing an open question of a design so that it can be executed, at design time, build time, deployment or run time, by a person, a tool or an assistant. Software development is the process of binding decisions, and with assistants binding some of them, the record of who or what bound each one matters as much as the result (Chapter 3).

Distribution

A directory, or an archive of one, with the jars of a command line, its options file, the bootstrap and wrapper scripts (Chapter 18).

Ecore

The metamodel of EMF: the language of packages, classes, attributes and references in which EMF models are defined (Chapter 7).

EMF

The Eclipse Modeling Framework: a Java framework that generates code from a model and provides reflection, serialization, notifications and references across files. It does not need the Eclipse IDE (Chapter 7).

EPackage provider

A service that contributes a generated EMF package, so that a configured resource set knows it (Chapter 7).

ExecutionSupplier

A value that a parent produces for a child while the child executes, in the child's span (Chapter 17).

Explaining listener

A listener that tells, line by line, which factories a capability loader asked about a requirement, which declined, and what each produced (Chapter 10).

Gradient ascent

Ordering the work by taking, at each point, the step with the most value per unit of time and effort, then measuring again from where the work stands; the best next action is picked from a roadmap of interconnected decisions that an assistant keeps consistent, with the North Star as the guard against climbing the wrong hill (Section 3.8).

Import map

A script in a web page that maps bare module specifiers, such as `react`, to URLs, so that a browser loads libraries as ES modules from a CDN with no build step (Chapter 23).

JPMS

The Java Platform Module System, part of Java since version 9. A module declares in `module-info.java` what it requires, what it exports and which services it provides (Chapter 6).

Marker

A record of where an element, or one of its features, came from: a location, a position, a date, a digest, or a commit (Chapter 20).

Meta

The language-neutral type system of the runtime: packages, classes, features, operations and type references, to write a vocabulary once and project it to many languages (Chapter 21).

Meta text syntax

Meta's own notation, in `.nmeta` files: metamodels written the way Xcore writes Ecore models (Section 21.5).

Metamodel

A model of models: what a model may contain, such as a package of classes with typed features. A library of books is a model; the classes `Library` and `Book` are its metamodel (Chapter 7).

Micro-model

A small model with a single concern, explicit dependencies on other models and no knowledge of its consumers, in a module of its own (Chapter 2).

Micro-wiki

Documentation anchored to model elements: a page per element, with the model as the site map, so that documentation cannot drift from what it describes and everyone knows which file to edit (Chapter 20).

Mix-in

A set of options contributed to a command, by a factory or by a picocli `@Mixin` field (Chapter 16).

Model tower

The stack of small models, each extending the ones below: the core model at the bottom, then Meta, the JVM model, telemetry, the diagram models and the floors above (Chapter 26).

Nano-model

One of several small models in one module, because they are reused and changed together, such as the four telemetry models (Chapter 2).

Nasdanika Body of Knowledge (NBOK)

The books, stories and example code about Nasdanika, laid out by one convention and built so that every listing is tested code; every book in it ends with the same appendix on how it is organized (Appendix F).

Nasdanika Zephyr Book

An in-depth technical manual for one module of the runtime, or a small group of modules used together, in the manner of IBM Redbooks, with every listing included from the module's tests. It is on the module's page and attached to the module's artifact with the classifier `book`, as PDF and HTML (The Nasdanika Zephyr Books).

nsURI

The namespace URI of an EMF package: its identity, under `runtime.sdk.nasdanika.org/` for the runtime's models (Chapter 2).

OpenTelemetry

A vendor-neutral standard and API for traces, log records and metrics. Code is instrumented once, and whoever runs it decides where the signals go (Chapter 11).

Planes

The data plane where data sits, the semantic plane of models with resolved references, and the presentation plane of what people and agents read, separated by volumes, with commands writing down (Chapter 26).

Polymorphic UI

A user interface that renders each element with the view of its class, or of its nearest

superclass that has one, or with a reflective view of last resort, the way a method call dispatches on the class of its receiver. With several formats (HTML, React, Markdown, AsciiDoc), it dispatches on the format as well (Chapter 24).

Published Language

In Domain-Driven Design, a documented, shared vocabulary that bounded contexts translate into and out of. Meta is one that is machine-checkable (Chapter 21).

Pump

A serial work queue, drained by one thread at a time, that drives capability resolution, transformations and pumped publishers. A thread that waits on it runs queued work instead of sleeping, and it reports what is stuck when nothing can progress (Chapter 9).

Pure Nasdanika

Code that depends on the JDK, EMF core and the OpenTelemetry API and nothing else (Chapter 2).

Qualifier

A dot-separated part of a file name, read right to left, that selects a handler for a resource's contents: `reading-group.library.groovy` has the qualifiers `groovy` and `library` (Chapter 10, Chapter 19).

Reflective view

The view of last resort in a polymorphic user interface: what the metamodel says about an element, its class and the features it has values for (Chapter 24).

Requirement

A type, possibly generic, and payloads that select and parameterize what is required (Chapter 10).

Resource set contributor

A service that configures resource sets: packages, factories, URI handlers, adapters (Chapter 7).

Semantic elements

The objects of other models that a diagram element depicts, such as the book a node of a diagram stands for. A diagram element is a model element in its own right, so a diagram is a model even when it depicts nothing (Chapter 25).

Snapshot

A model of a running system saved as a file, such as the JVM model of a module layer: evidence of what ran (Chapter 22).

Span

An operation in a trace, with a name, a start, an end, attributes and log records; in the runtime, one per operation worth seeing (Chapter 11).

Status label

AVAILABLE for what the runtime does in this edition, **SKETCH** for what is designed and partly built, **PLANNED** for what is designed and not built.

Telemetry model

Traces, logs and metrics as EMF models, faithful to OpenTelemetry's data model, with nested spans, log records in their spans and recorded changes (Chapter 13).

Transformer

A function from sources to publishers of targets, with mappings declared by annotated methods and wiring of targets to each other (Chapter 9).

Type token

An object that captures a generic type at compile time, such as `new TypeToken<Supplier<Library>>() {}`, where erasure would lose it (Chapter 8).

UI

Wherever a person or an agent exchanges information with a system, even in one direction: a page, a PDF, Markdown or AsciiDoc that an agent reads, a tool it calls (Chapter 24).

Unit of work

A piece of work with its own OpenTelemetry instance and its own record: a command run, a request served, an agent session, a test (Chapter 11).

Volume

The runtime's file abstraction: string paths, `stat`, `list`, `content`, declared case sensitivity and optional capabilities, implemented over NIO, models or remote systems (Chapter 14).

Wiring

Matching commands to parents and sub-commands by declared generic types, and injecting adapted values before execution (Chapter 17).

XMI

XML Metadata Interchange, a standard XML format for models and the one every EMF tool understands (Chapter 7).

Xcore

A textual syntax for Ecore models with operation bodies, from which EMF generates Java (Chapter 7).

Appendix E: Building and publishing this book

This book is written in AsciiDoc and built with the runtime, in the runtime's repository. Its examples are Maven modules compiled and tested with every build, so a listing that no longer compiles, or a test that no longer passes, fails the build before it can mislead a reader.

E.1. Layout

```
doc/                       documentation parent, never deployed
  book/
    examples/              one Maven module per topic: start, library, generics, flow,
                           capability, telemetry, recording, volume, cli, dsl, meta,
                           jvm, web, ui
    manuscript/
      src/docs/asciidoc/   book.adoc, front matter, chapters, appendices, back matter, images
      src/docs/themes/     the PDF theme
      src/cover/           the cover generator
  zephyr/                  the PDF theme and the stylesheets of the Zephyr Books
```

The documentation parent sets `maven.deploy.skip`, so nothing under `doc` is deployed, as with test modules. The manuscript module does not depend on the examples through Maven; it includes their files.

E.2. Building

The book is built with the runtime:

```
mvn clean install                      # the runtime, the examples and the book
mvn clean install -Dasciidoctor.skip   # everything but the book
```

Or on its own, from `doc/book/manuscript`:

```
mvn verify                             # PDF, EPUB and HTML in target/book
mvn verify -Dbook.editor-notes=false   # without editor's notes, for publication
mvn verify -Dbook.media=prepress       # print: no cover, URLs after links, recto chapters
```

The build produces three editions, named after the runtime version:

- **PDF**, at 189 by 246 mm (7.44 by 9.69 inches), a standard trim size for print on demand, with a theme that extends Asciidoctor PDF's default: Noto Serif for text, Noto Sans for headings, and M+ 1mn for code.
- **EPUB**, for readers.
- **HTML**, one page with every image embedded: the whole book in one file, every term and every rule in it. Its markup, most of it the highlighting of listings, makes the file nearly three times the size of its text, so an assistant working in the repository is better served by the AsciiDoc sources and the examples, read in the parts it needs (Chapter 3).

Diagrams are PlantUML, rendered by Asciidoctor Diagram with PlantUML's own layout engine, so no Graphviz installation is needed. The exception is the figure of Chapter 25, an SVG file that its example generates and checks.

E.3. Conventions of the manuscript

- **Listings are included, not copied.** A listing is a tagged region of an example, included with `include::{examplesdir}/...[tag=name,indent=0]`. Callouts are comments of the form `// <1>` in the example's source.
- **Listings fit the page.** At the code font size of the PDF, a line holds 96 characters, and longer lines in tagged regions are wrapped in the source, not by the renderer.
- **Warnings fail the build.** A missing tag, a broken cross-reference or an unknown attribute stops it.
- **Status labels** mark what is not available yet: `[.sketch]#SKETCH#` and `[.planned]#PLANNED#`.
- **Editor's notes** are `IMPORTANT` blocks inside `ifdef::editor-notes[]`, for the author and the runtime's maintainers. They record inconsistencies between the runtime and its documentation found while writing, and are left out of published editions.

E.4. The cover

The cover is generated by a single-file Java program, `src/cover/Cover.java`, which draws the title, the subtitle and the logo with Java 2D, using the Noto Sans fonts bundled with Asciidoctor PDF:

```
java src/cover/Cover.java \
    <path to asciidoctorj-pdf jar> \
    src/docs/asciidoc/images/nasdanika-sdk-logo.jpg \
    src/docs/asciidoc/images/cover.png
```

E.5. Publishing

The book is published on Leanpub as a "Bring Your Own Book" title: Leanpub does not build it; each release uploads the PDF and the EPUB built here, and a sample if one is wanted. The minimum price is zero with a suggested price, so the book is free and readers can choose to pay. The text and figures are licensed under CC BY 4.0 and the examples under EPL-2.0, as the colophon states; the legal code of CC BY 4.0 is in `doc/book/manuscript/LICENSE`. A release of the book follows a release of the runtime, and has its version. The Zephyr Books with readers beyond the runtime are Leanpub titles too, released with each release of the runtime and offered with this book in a bundle (The Nasdanika Zephyr Books).

E.6. Building a Zephyr Book

A Nasdanika Zephyr Book (The Nasdanika Zephyr Books) is built with the same tool chain, in the module it documents. A module has one when it has `src/docs/asciidoc/index.adoc`: that file activates a profile in the runtime's parent POM, so a module opts in without a change to its own POM. Its listings are tagged regions of the module's tests and test resources, included as this book includes its examples, and the conventions above hold, warnings that fail the build included.

The profile renders two editions in the Zephyr look, a PDF with a theme of its own and a single HTML page with Bootswatch's Zephyr stylesheet, both from `doc/zephyr`. It attaches them to the module's artifact with the classifier `book`, and the module's test sources with the classifier `test-sources`. The release build leaves the editor's notes out, as it does for this book, because what is published to Maven Central cannot be withdrawn.

Appendix F: The Nasdanika Body of Knowledge

This book is part of the Nasdanika Body of Knowledge, NBOK for short: the books, stories and example code about Nasdanika, organized so that people and AI assistants can find what they need in it and rely on what they find. Every book in it ends with this appendix, word for word, so that whichever one you are reading, you know where the others are and how they fit together.

Bodies of knowledge such as SWEBOK for software engineering and PMBOK for project management are written guides. NBOK is built. Its listings are regions of code that is compiled and tested before a book is rendered, so a book whose code no longer does what it says does not build.

F.1. What it holds

- **The SDK book**, *Nasdanika SDK Runtime: More Structure with Less Effort*: the overview, with the why, the key capabilities and the high-level how. Read it first.
- **Nasdanika Zephyr Books**: one per module, or per small group of modules used together, each the detail for its modules, as user guide, developer guide and reference. Each stands on its own.
- **Readmes**: one page on what a module is and why. A module with a Zephyr Book may have none.
- **Nasdanika SDK Stories** PLANNED: one use case each, related to the Nasdanika SDK in some shape or form, written as a Maven module whose code, if it has any, is the source of its listings, and collected each year.
- **The International Banking System** PLANNED: the practice of architecture as code, taught as a course on one example that grows with the platform. Inspired by the C4 Model "Internet Banking System".
- **Nasdanika Field Notes**: short articles about the technologies and services Nasdanika is built and published with, such as EMF, each verified by a test where it can be, and recipes, such as publishing a book, collected in one book. Their tests use the third-party versions of the runtime's BOM, so every runtime release checks the runtime's

assumptions about its dependencies. Unlike the rest of NBOK, they are about other projects' code, and useful without Nasdanika.

A story is à la carte; the International Banking System is the course; a field note is about the ground they stand on.

F.2. How it is laid out

Every repository of NBOK follows one layout, the Maven layout with a few additions:

```
pom.xml                          the reactor: its modules, their names and descriptions
doc/                             documentation about the repository as a whole
  book/manuscript/               a book about the repository, if it has one
  book/examples/<topic>/         that book's examples, one Maven module per topic
<module>/
  pom.xml                        the module's name and description
  readme.md                      what the module is and why, on one page
  src/main/java/                 the code, with module-info.java
  src/main/resources/            resources, and Xcore models under the package path
  src/test/java/                 the tests: the source of the book's listings
  src/test/resources/            test data: the source of the book's figures
  src/docs/asciidoc/index.adoc   the module's Zephyr Book, with front/, chapters/
                                 and appendices/ beside it
```

A module has a Zephyr Book when it has `src/docs/asciidoc/index.adoc`. A story PLANNED is a module at `<year>/<topic>/` in the `stories` repository, with `story.adoc` and `index.adoc` in `src/docs/asciidoc/` beside its code, and `<year>/book/` builds the year's collection from the stories next to it. A field note is a module at `<technology>/<topic>/` in the `field-notes` repository, with `article.adoc` and `index.adoc` in `src/docs/asciidoc/` beside its tests, and `book/` builds the collection.

F.3. How it is published

- **Maven Central**, per module and version: the jar, its `sources` and `javadoc`, and, for a module with a Zephyr Book, the book as PDF and HTML with the classifier `book` and the tests with the classifier `test-sources`, the complete source of every listing. Central is permanent, so the book of every released version stays available.
- **The runtime's site**, PLANNED runtime.sdk.nasdanika.org, with the latest books.

- **Leanpub**: the SDK book, with the Zephyr Books that have readers beyond the runtime in a bundle, and the yearly collection of stories PLANNED, each free with a suggested price.
- **Zenodo**, sources published by GitHub and books uploaded manually.
- **Lulu** for print-on-demand copies of the books.

F.4. How to read it

People read the PDF or the HTML: the SDK book for the why and the big picture, then the Zephyr Book of the module at hand.

AI assistants are better served by the sources than by the renders. AsciiDoc is a fraction of the size of the HTML, whose markup and highlighting multiply it, and the sources lead to the code itself. In a repository of NBOK, an assistant finds its way as an engineer does:

1. The root `pom.xml` lists the modules, with a name and a description each.
2. A module's `readme.md`, where it has one, says what the module is.
3. A module's `src/docs/asciidoc/index.adoc` includes the book's files, and each listing's `include::` names the test file and the tagged region it comes from: follow it to the code.
4. A term is in a book's glossary appendix, and a convention in its rules or building appendix.

Two aids for assistants are planned PLANNED: each book and story also as one AsciiDoc file with its includes resolved, so that it is complete without the repository, and an index generated from the headers of the books and stories (title, description, keywords, version), so that an assistant can choose the few it needs instead of reading all of them.

F.5. Why it can be trusted, and for which version

- **Listings are code.** Every listing is a tagged region of a test or an example that is compiled and run before the book is rendered, and a warning, such as a missing tag, fails the build.
- **Every book has a version.** A book is built with a release of the runtime and carries its version, here 2026.11.0. It is true for that version; for another, read the book of that

version.

- **Status labels** mark what is not built yet: SKETCH for what is designed and partly built, PLANNED for what is designed and not built. Everything without a label is available.
- **Editor's notes**, the author's notes on what is still to be checked, are left out of published editions.
- **Stories are built twice** PLANNED: against the release each was published with, so that it reproduces, and against the next one, so that a story that no longer applies is marked before a reader finds out.

The source of this appendix is `doc/nbok/nbok-appendix.adoc` in github.com/Nasdanika-SDK/runtime, published with each release as `org.nasdanika.sdk.runtime.doc:nbok`.

About the author



I am Pavel Vlasov, the creator of Nasdanika and of the runtime this book describes. I have worked in information technology for more than thirty years, thirty of them in financial services, as a developer, an architect and an enterprise architect.

I hold a master's degree in plasma physics from the Novosibirsk State University. I am certified as a Java developer and as a Java architect, and as a TOGAF Enterprise Architecture Practitioner.

I write about models, architecture and AI-assisted development at medium.com/nasdanika. My code is on GitHub, and I am on LinkedIn.